

UNIVERSITY OF CALGARY

Improved Arithmetic in the Ideal Class Group of Imaginary Quadratic Number Fields

With an Application to Integer Factoring

by

Maxwell Sayles

A THESIS

SUBMITTED TO THE FACULTY OF GRADUATE STUDIES

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE

DEGREE OF MASTER OF SCIENCE

DEPARTMENT OF COMPUTER SCIENCE

CALGARY, ALBERTA

May, 2013

© Maxwell Sayles 2013

Abstract

This thesis proposes practical improvements to arithmetic and exponentiation in the ideal class group of imaginary quadratic number fields for the Intel x64 architecture. This is useful in tabulating the class number and class group structure, but also as an application to a novel integer factoring algorithm called “SuperSPAR”. SuperSPAR builds upon SPAR using an idea from Sutherland, namely a bounded primorial steps search, as well as by using optimized double-based representations of exponents, and general improvements to arithmetic in the ideal class group.

Since the target architecture has 64 bit machine words, we specialize two implementations of ideal class arithmetic: one using 64 bit arithmetic, and the other using 128 bit arithmetic. In each case, our results show an improvement in the average performance of ideal class arithmetic over reference implementations. Since much of the effort of ideal class arithmetic is in computing the extended greatest common divisor, this thesis studies the average performance of several implementations of this and shows an improvement for some over reference implementations. Dimitrov and Cooklev propose the double base number system, and Imbert, Jacobson, and Schmidt propose a method of ideal class cubing. Using these ideas, we explore several techniques for exponentiation using 2,3 number systems and demonstrate an improvement in the average performance of exponentiation in the ideal class group over that of binary exponentiation. Lastly, each of these improvements are applied to our implementation of the SuperSPAR factoring algorithm, and our results show an improvement in the average time to factor integers 49 bits to 62 bits in size.

Acknowledgements

I would like to thank my supervisor, Dr. Michael J. Jacobson, Jr., for all of his great advice. I am certain that without his patience I would not have finished. I would also like to thank Dr. Andrew Sutherland for his reference implementation of SuperSPAR, and his polite willingness to answer all my questions. I would like to thank Dr. Arthur Schmidt for the idea of exploring a left-to-right binary extended GCD – this may be one of the most critical sub-algorithms in our implementation of SuperSPAR. Furthermore, I would like to thank Dr. Laurent Imbert for pointing out the variety of techniques to compute 2,3 representations, and for reviewing my reading list. In addition, I would like to thank my examining committee, Dr. Renate Scheidler and Dr. Kristine E. Bauer, for taking the time to read this thesis and for their valuable comments.

Finally, I would like to thank my wife, Faye, for being by my side, and for her kind understanding, especially during “crunch time”. Thank you all.

Table of Contents

Abstract	i
Acknowledgements	ii
Table of Contents	iii
List of Tables	vi
List of Figures	vii
1 Motivation	1
1.1 Organization and Contributions of this Thesis	3
2 Ideal Arithmetic	6
2.1 Quadratic Number Fields	6
2.2 Ideals and the Class Group	8
2.3 Ideal Arithmetic	12
2.3.1 Reduced Representatives	13
2.3.2 Ideal Class Multiplication	14
2.3.3 Fast Ideal Multiplication (NUCOMP)	17
2.3.4 Fast Ideal Squaring (NUDUPL)	20
2.3.5 Fast Ideal Cubing (NUCUBE)	21
3 Exponentiation	24
3.1 Binary Exponentiation	25
3.2 Non-Adjacent Form Exponentiation	25
3.3 Double-Base Number Systems	28
3.3.1 Chained 2,3 Representations	29
3.3.2 Unchained 2,3 Representations	30
3.4 Methods for Computing 2,3 Chains/Representations	34
3.4.1 Right-to-Left Chains (from low-order to high-order)	34
3.4.2 Left-to-Right Representations (from high-order to low-order)	36
3.4.3 Pruned Tree of ± 1 Nodes	38
3.4.4 Shortest Additive 2,3 Chains	39
3.5 Summary	40
4 SuperSPAR	42
4.1 SPAR	42
4.1.1 Ambiguous Classes and the Factorization of the Discriminant	43
4.1.2 SPAR Algorithm	44
4.1.3 SPAR Complexity	45
4.2 SuperSPAR	46
4.2.1 Bounded Primorial Steps	47
4.2.2 Primorial Steps for a Set	49
4.2.3 SuperSPAR Algorithm	50
4.3 Summary	53
5 Extended Greatest Common Divisor Experiments	54
5.1 The Euclidean Algorithm	55
5.2 Right-to-Left Binary Extended GCD	56
5.2.1 Windowed Right-to-Left Binary Extended GCD	59

5.3	Left-to-Right Binary Extended GCD	61
5.4	Lehmer's Extended GCD	63
5.5	Partial Extended GCD	65
5.6	Specialized Implementations of the Extended GCD	68
5.7	Experimental Results	71
5.7.1	32-bit Extended GCDs	73
5.7.2	64-bit Extended GCDs	81
5.7.3	128-bit Extended GCDs	88
5.8	Summary	91
6	Ideal Arithmetic Experiments	93
6.1	Specialized Implementations of Ideal Arithmetic	95
6.2	Average Time for Operations	100
6.3	Summary	110
7	Exponentiation Experiments	111
7.1	Binary and Non-Adjacent Form	113
7.2	Right-to-Left 2,3 Chains	113
7.3	Windowed Right-to-Left Chains	115
7.4	Left-to-Right 2,3 Representations	116
7.5	Pruned Trees	118
7.6	Left-to-Right Best Approximations	121
7.7	Additive 2,3 Chains	123
7.8	Incremental Searching	125
7.9	Big Exponents from Small Exponents	126
7.10	Experimental Results	127
7.11	Summary	139
8	SuperSPAR Experiments	140
8.1	Coprime Finding	145
8.1.1	Coprime Sieve	146
8.1.2	Coprime Wheel	146
8.1.3	Comparison	148
8.2	Lookup Tables	149
8.2.1	Chaining with List Heads	152
8.2.2	Open Addressing	153
8.2.3	Chaining vs. Open Addressing	153
8.3	Factorization of the Order of Ideal Classes	153
8.3.1	Prime Power Bound	155
8.3.2	Difference between the Order of Two Ideal Classes	157
8.3.3	Best Multipliers	159
8.3.4	Expected Number of Ideal Classes	160
8.4	SuperSPAR Algorithm in Practice	162
8.5	Search Bounds	168
8.6	Empirical Search for Exponent and Step Count	174
8.6.1	Two-Dimensional Ternary Search of a Noisy Surface	179
8.7	Reference Implementation of SPAR	184
8.8	Comparison to Other Factoring Implementations	190

9	Conclusions and Future Work	196
9.1	Ideal Arithmetic	197
9.2	Exponentiation	198
9.3	Super ² SPAR	200
	Bibliography	202

List of Tables

6.1	Relative Average Cost of Ideal Arithmetic.	103
7.1	Best 16 bit exponentiation techniques.	128
7.2	Average time (microseconds) to exponentiate 16 bit exponents.	129
8.1	The first 12 primorials.	149
8.2	The probability that $\text{ord}([\mathfrak{a}]^{p_i^{e_i}})$ is coprime to p_i	156
8.3	The probability that $\text{ord}([\mathfrak{a}]^E)$ is coprime to E	157
8.4	Average time to factor based on power bound B	158
8.5	Probability that $[\mathfrak{v}]^{h'}$ has $ \mathcal{P} $ factors.	159
8.6	Average time to factor for a fixed number of classes per group.	161
8.7	Theoretical Number of Search Steps.	174
8.8	SuperSPAR exponentiation and search parameters.	183
8.9	Average time to factor	195

List of Figures and Illustrations

3.1	The construction of $2^{15}3^6 - 2^83^5 - 2^43^6 + 2^33^3$.	33
5.1	Reference Implementations for 32-bit Inputs.	74
5.2	Extended Euclidean GCD for 32-bit Inputs.	75
5.3	Stein's Extended GCD for 32-bit Inputs.	75
5.4	2 bit Windowed Stein's Extended GCD for 32-bit Inputs.	76
5.5	3 bit Windowed Stein's Extended GCD for 32-bit Inputs.	76
5.6	4 bit Windowed Stein's Extended GCD for 32-bit Inputs.	77
5.7	5 bit Windowed Stein's Extended GCD for 32-bit Inputs.	77
5.8	All 32-bit Implementations of Stein's Extended GCD.	78
5.9	Shallit's Left-to-Right Binary Extended GCD for 32-bit Inputs.	78
5.10	Simplified Left-to-Right Binary Extended GCD for 32-bit Inputs.	79
5.11	Lehmer's Extended GCD for 32-bit Inputs.	79
5.12	Best Extended GCDs for 32-bit Inputs.	80
5.13	Reference Implementations for 64-bit Inputs.	82
5.14	Extended Euclidean GCD for 64-bit Inputs.	82
5.15	Stein's Extended GCD for 64-bit Inputs.	83
5.16	2 bit Windowed Stein's Extended GCD for 64-bit Inputs.	83
5.17	3 bit Windowed Stein's Extended GCD for 64-bit Inputs.	84
5.18	4 bit Windowed Stein's Extended GCD for 64-bit Inputs.	84
5.19	5 bit Windowed Stein's Extended GCD for 64-bit Inputs.	85
5.20	All 64-bit Implementations of Stein's Extended GCD.	85
5.21	Shallit's Left-to-Right Binary Extended GCD for 64-bit Inputs.	86
5.22	Simplified Left-to-Right Binary Extended GCD for 64-bit Inputs.	86
5.23	Lehmer's Extended GCD for 64-bit Inputs.	87
5.24	Best Extended GCDs for 64-bit Inputs.	87
5.25	Reference Implementations for 128-bit Inputs.	88
5.26	All 128-bit Implementations of Stein's Extended GCD.	89
5.27	Lehmer's Extended GCD for 128-bit Inputs.	89
5.28	Best Extended GCDs for 128-bit Inputs.	90
6.1	Full or Approximate Square Root for 64 bit Ideal Multiplication	103
6.2	Full or Approximate Square Root for 64 bit Ideal Multiplication	104
6.3	Full or Approximate Square Root for 128 bit Ideal Multiplication	104
6.4	Full or Approximate Square Root for 128 bit Ideal Cubing	105
6.5	Average Time to Multiply Reduced Ideal Class Representatives.	105
6.6	Average Time to Square Reduced Ideal Class Representatives.	106
6.7	Average Time to Cube Reduced Ideal Class Representatives.	106
6.8	Time to compute $[\mathfrak{a}^3]$ in our 64-bit implementation.	107
6.9	Time to compute $[\mathfrak{a}^3]$ in our 128-bit implementation.	107
6.10	Time to compute $[\mathfrak{a}^3]$ in our GMP implementation.	108
6.11	Time to compute $[\mathfrak{a}^3]$ using Pari.	108

6.12	Methods of Handling a_1^2 in 128 bit Cubing.	109
7.1	Global bounds on left-to-right 2,3 representations.	117
7.2	Global Bounds on Pruned Trees.	120
7.3	Varying the Number of Best Approximations.	122
7.4	Base 2 Exponentiation (59 bit Discriminants).	130
7.5	Base 2 Exponentiation (118 bit Discriminants).	131
7.6	Right-to-Left 2,3 Chains (59 bit Discriminants).	131
7.7	Right-to-Left 2,3 Chains (118 bit Discriminants).	132
7.8	Left-to-Right 2,3 Representations (59 bit Discriminants).	133
7.9	Left-to-Right 2,3 Representations (118 bit Discriminants).	134
7.10	4 Best $(x \pm \dots)/(2^c 3^d)$ (59 bit Discriminants).	135
7.11	4 Best $(x \pm \dots)/(2^c 3^d)$ (118 bit Discriminants).	136
7.12	Use g^b for 16 bit b (59 bit Discriminants).	137
7.13	Use g^b for 16 bit b (118 bit Discriminants).	137
7.14	The best performers from each category (59 bit Discriminants).	138
7.15	The best performers from each category (118 bit Discriminants).	138
8.1	Inversion aware hashing.	151
8.2	Open addressing vs Chaining.	154
8.3	Attempts $N \equiv 1 \pmod{4}$	166
8.4	48 bit Factoring Search Space.	176
8.5	56 bit Factoring Search Space.	177
8.6	64 bit Factoring Search Space.	177
8.7	72 bit Factoring Search Space.	178
8.8	80 bit Factoring Search Space.	178
8.9	Reference SPAR	185
8.10	Bound the Number of Ideals per Group	186
8.11	Bounded Primorial Steps Search	186
8.12	Bounded Primorial Steps Search (Zoomed)	187
8.13	Bound e_i in Exponent $E = \prod p_i^{e_i}$	187
8.14	Independent Exponent E and Search Bound mP_w	188
8.15	Perform a Single Search	188
8.16	SPAR vs SuperSPAR	189
8.17	Factoring performance for 16 bits to 100 bits.	192
8.18	Factoring performance for 44 bits to 72 bits.	192
8.19	Best factoring implementations for 44 bits to 68 bits.	193
8.20	Best factoring implementations for 44 bits to 58 bits.	193
8.21	Best factoring implementations for 58 bits to 68 bits.	194

List of Algorithms

2.1	Ideal Reduction ([39, p.90]).	13
2.2	NUCOMP – Fast Ideal Multiplication ([40, pp.441-443]).	20
2.3	NUDUPL – Fast Ideal Squaring.	22
2.4	NUCUBE – Fast Ideal Cubing ([35, p.26]).	23
3.1	Computes g^n using right-to-left non-adjacent form (Reitwiesner [51]).	27
3.2	Computes g^n given n as a chained 2,3 partition (Dimitrov et al [26]).	30
3.3	Compute g^n for a 2,3 representation of n ([46, Section 3.2]).	31
3.4	2,3 strict chains from low order to high order (Ciet et al [18]).	36
3.5	Greedy left to right representation (Berthé and Imbert [16]).	38
3.6	Chain from ± 1 Pruned Tree (Doche and Habsieger [28]).	39
4.1	Primorial Steps (Sutherland [58, p.57]).	48
5.1	Extended Euclidean Algorithm.	57
5.2	Right-to-left Binary Extended GCD (Stein [57]).	60
5.3	Windowed Right-to-left Binary Extended GCD.	61
5.4	Shallit and Sorenson Left-to-Right Binary Extended GCD ([54]).	63
5.5	Lehmer’s extended GCD ([44]).	66
5.6	Conditionally swap (s, U, V) with (t, X, Y) when $s < t$	70
5.7	Return the index of the most significant set bit of x	71
7.1	Fast $n \bmod 3$ (adapted from Hacker’s Delight [60]).	115
7.2	16 bit Blocked Exponentiation [20, Algorithm 9.7].	126
8.1	Compute deltas for P_{w+1} given deltas for P_w	148
8.2	Repeatedly Square	163
8.3	Try Many Ideal Classes.	164
8.4	SuperSPAR Integer Factoring Algorithm.	167
8.5	Compute baby step bound candidates.	171
8.6	Subroutine to Search a Curve in 1-Dimension.	181
8.7	2-Dimensional Ternary Search of a Noisy Surface.	182

Chapter 1

Motivation

There are many interesting problems related to the study of algebraic number fields, and since quadratic number fields are the simplest of algebraic number fields [40, p.77], they provide a good starting point for the study of ideals and class groups. This thesis discusses practical improvements for arithmetic in the ideal class group of imaginary quadratic number fields. Quadratic number fields are a degree 2 extension over the rational numbers, with the form $\mathcal{K} = \mathbb{Q}(\sqrt{\Delta})$. When the discriminant Δ is negative, then $\mathcal{K}$ is referred to as an *imaginary* quadratic number field, and when Δ is positive, then $\mathcal{K}$ is referred to as a *real* quadratic number field.

The first published work on ideal class groups was the work of Carl Friedrich Gauss in the *Disquisitiones Arithmeticae* [31]. Gauss studied the theory of binary quadratic forms – polynomials of the form $Q(x, y) = ax^2 + bxy + cy^2$ with $a, b, c \in \mathbb{Z}$. We call $\Delta = b^2 - 4ac$ the *discriminant* of the form, and say that an integer n is represented by a form $Q(x, y)$ if there exist integers x and y such that $Q(x, y) = n$. Gauss introduced the notion of *equivalence* of forms. For a given form $Q(x, y)$, there exists a set of integers that can be represented by that form. We say that two forms are equivalent if there exists a linear integral transformation of variables from one form to another, in other words, if the set of integers represented by each form are identical [23, p.239]. This notion of equivalence leads to *equivalence classes*, i.e. all the forms equivalent to each other constitute an equivalence class. Gauss defined an operation between equivalence classes, called *composition*, under which the set of equivalence classes of binary quadratic forms with discriminant Δ form a finite Abelian group [21, p.136]. Later this would be shown to be analogous to what is called the ideal class group of a quadratic field [30].

There are also many interesting problems related to the ideal class group of imaginary quadratic number fields, such as computing the class number or group structure for a particular field (see [50]), or solving the discrete logarithm problem in the class group. A related problem to computing the discrete logarithm is that of computing the order of an ideal class. This problem is of particular relevance to this thesis, since knowing the order of an ideal class can lead to the factorization of an integer associated with the discriminant.

Many methods exist for computing the order of an element in a generic group, such as Shanks' baby-step giant-step method [55]. In this thesis, we employ a new technique of Sutherland [58] for computing the order of an element. This method is asymptotically faster than previous methods for generic groups, and is referred to as the *bounded primorial steps algorithm* [58, §4.1]. A principal component of this algorithm is to exponentiate a group element to an exponent that is the product of the w smallest prime powers – a product that is referred to as a *primorial*.

To improve exponentiation in the ideal class group, we use the results of Imbert, Jacobson, and Schmidt [35], which show that ideal class cubing can be made faster than multiplying an ideal class with its square. Using this, we are able to take advantage of double-base number systems, introduced by Dimitrov and Cooklev [24, 25], when exponentiating in the ideal class group.

With this in mind, this thesis focuses on practical improvements to arithmetic in the ideal class group of imaginary quadratic number fields with discriminants less than 128 bits in size, but also to that of exponentiating an element in this group to a primorial. For the purposes of this thesis, these improvements are applied to an extension of an integer factoring algorithm called SPAR [53]. This algorithm is based on arithmetic in the ideal class group, and works by computing the order of an ideal class. In this thesis, we adapt this algorithm to use the ideal class cubing of Imbert, Jacobson, and Schmidt [35], the bounded primorial steps algorithm of Sutherland [58], and the double base number system of Dimitrov and

Cooklev [24, 25]. We call this new algorithm “SuperSPAR”. We optimize the performance of SuperSPAR to minimize the average case, and of all the factoring algorithms tested, our implementation of SuperSPAR is the fastest on average for semiprime integers of size between 49 bits and 62 bits. A possible application of this new factoring algorithm is for factoring left overs after sieving in algorithms like the number field sieve using multiple large primes (see [23, §6.1.4]). In this case, SuperSPAR should be optimized for the median case (or some other percentile) with the idea that SuperSPAR will only perform a bounded number of operations in an attempt to factor an integer. This is sufficient in terms of a factoring sub-algorithm since left-overs from the sieve stage of other factoring algorithms are generated relatively quickly, and not every left-over needs to be factored.

1.1 Organization and Contributions of this Thesis

Chapters 2, 3, and 4 cover background material, while Chapters 5, 6, 7, and 8 discuss our implementations in detail.

To begin, one contribution of this thesis is improved arithmetic in the ideal class group of imaginary quadratic number fields. The background material on ideal class groups useful to understanding the improvements discussed in this thesis are covered in Chapter 2. Roughly half of the computational effort of ideal class multiplication is in computing solutions of the extended greatest common divisor. Chapter 5, discusses several solutions to this problem, as well as an adaptation to a left-to-right binary extended GCD that we propose in Section 5.3. The results in Section 5.7 show that this method is the fastest on average for a large range of integer sizes, when compared against Pari 2.5.3 [11] and GMP 5.1.1 [3] as reference implementations. The background material on the ideal class group in Chapter 2 is also the basis of our implementation of ideal class arithmetic. The experiments that lead to our improved implementation are discussed in Chapter 6. The results in Section 6.2 show that our implementation, on average, is roughly 3 to 4 times faster than the implementation of

Pari 2.5.3 [11], which we use as a reference implementation, at least for arithmetic in class groups with negative discriminants of size less than 128 bits. These results directly improve the performance of our implementation of SuperSPAR.

Another contribution of this thesis is improved exponentiation by large power primorials in the ideal class group. Again, these results are useful to our implementation of SuperSPAR. Chapter 3 discusses some methods from the literature for exponentiating. In particular, we discuss double base number systems and some approaches for computing double base representations useful for exponentiation. In Chapter 7, we propose several adaptations to these methods. In particular, we propose a method that we call *left-to-right best approximations* in Section 7.6. In Section 7.10 we provide results that show that for our implementation of ideal class arithmetic and when exponentiating by large power primorials, a left-to-right best approximations technique performs the best on average. This is the technique used to compute 2,3 representations of power primorials used in our implementation of SuperSPAR.

A final contribution of this thesis is an improved integer factoring algorithm, named SuperSPAR. Chapter 4 discusses the original SPAR factoring algorithm, published by Schnorr and Lenstra [53], but named after Shanks, Pollard, Atkin, and Rickert [45, p.484]. Our implementation employs the results of Chapters 5, 6, and 7, as well as the bounded primorial steps algorithm of Sutherland [58], and several modifications to the original algorithm. The result, demonstrated in Section 8.8, is an implementation of an integer factoring algorithm that is the fastest on average for semiprime integers of the size 49 bits to 62 bits of the implementations tested. Again, this is useful for factoring left overs after sieving in factoring algorithms like the number field sieve using multiple large primes [23, §6.1.4].

The source code for the libraries used throughout this thesis is available online. An optimized implementation of 128 bit arithmetic and functions to compute the extended GCD are available as `liboptarith` [4]. Our implementations of ideal class arithmetic is available as `libqform` [5]. This library exposes several programming interfaces. One can

specifically target the 64 bit, 128 bit, or GMP implementation of ideal class arithmetic, or we also expose a generic interface that will automatically select the implementation best suited for a given discriminant size. Finally, our implementation of the SuperSPAR factoring algorithm is available as `libsspar` [6]. We point out that this library is currently optimized to minimize average case performance. A function is provided where one gives the integer to be factored and the library automatically selects suitable parameters for the average case. An additional function is exposed where one can manually provide configuration parameters as well. Future work will likely include an entry point optimized for the median case. By making each of these projects publicly available, the hope is for greater collaboration and eventual inclusion into common mathematical packages such as Pari/GP [11] and Sage [13].

Chapter 2

Ideal Arithmetic

A focus of this thesis is arithmetic and exponentiation in the ideal class group of imaginary quadratic number fields. We begin with the relevant theory of quadratic number fields, then discuss quadratic orders and ideals of quadratic orders. Finally, we discuss arithmetic in the ideal class group. The theory presented here is available in detail in reference texts on algebraic number theory such as [21], [34], or [37].

2.1 Quadratic Number Fields

A quadratic number field is an algebraic number field of degree 2 over the rational numbers $\mathbb{Q}$, and is defined as

$$\mathbb{Q}(\alpha) = \{u + v\alpha : u, v \in \mathbb{Q}\}$$

for some quadratic irrational $\alpha \in \mathbb{C}$. The element α must be a root of a quadratic polynomial $ax^2 + bx + c$ with integer coefficients. By the quadratic formula

$$\alpha = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

and therefore,

$$\mathbb{Q}(\sqrt{b^2 - 4ac}) = \mathbb{Q}(\alpha).$$

Let $f^2D = b^2 - 4ac$ for $f, D \in \mathbb{Z}$ and D square-free, then $\mathbb{Q}(\sqrt{D}) = \mathbb{Q}(\alpha)$ also. When $D < 0$, we call $\mathbb{Q}(\sqrt{D})$ an *imaginary* quadratic number field, and when $D > 0$, we call $\mathbb{Q}(\sqrt{D})$ a *real* quadratic number field. The assumption throughout this thesis is that D is negative.

When $\xi \in \mathbb{Q}(\sqrt{D})$ is a root of a *monic* quadratic polynomial with integer coefficients, then ξ is a *quadratic integer*, or more simply an integer of $\mathbb{Q}(\sqrt{D})$.

Proposition 2.1.1. [37, Proposition 13.1.1] The integers of a quadratic field $\mathbb{Q}(\sqrt{D})$ can be written as

$$\xi = \begin{cases} x + y\sqrt{D} & \text{when } D \equiv 2, 3 \pmod{4} \\ x + y\frac{1+\sqrt{D}}{2} & \text{when } D \equiv 1 \pmod{4} \end{cases}$$

for $x, y \in \mathbb{Z}$. See [37, p.189] for a proof.

Let $[\omega_1, \omega_2]$ denote the $\mathbb{Z}$ -module generated by the quadratic integers ω_1 and ω_2 , as

$$\omega_1\mathbb{Z} + \omega_2\mathbb{Z} = \{\omega_1x + \omega_2y : x, y \in \mathbb{Z}\}.$$

Then by Proposition 2.1.1, ξ is a quadratic integer of $\mathbb{Q}(\sqrt{D})$ if and only if $\xi \in [1, \omega]$ where

$$\omega = \begin{cases} \sqrt{D} & \text{when } D \equiv 2, 3 \pmod{4} \\ \frac{1+\sqrt{D}}{2} & \text{when } D \equiv 1 \pmod{4}. \end{cases}$$

The set of all the quadratic integers in $\mathbb{Q}(\sqrt{D})$ forms a ring [21, p.47], which leads us to the concept of a *quadratic order*.

Definition 2.1.2. A *quadratic order* $\mathcal{O}$ of $\mathbb{Q}(\sqrt{D})$ is a sub-ring of the quadratic integers of $\mathbb{Q}(\sqrt{D})$ containing 1.

When a quadratic order $\mathcal{O}$ consists of all the quadratic integers in $\mathbb{Q}(\sqrt{D})$ then it is called the *maximal order*, since it is not contained within any larger order. From [40, p.80], the $\mathbb{Z}$ -module $[1, \omega]$ is the maximal order of $\mathcal{K} = \mathbb{Q}(\sqrt{D})$ and we denote this $\mathcal{O}_{\mathcal{K}}$.

Example 2.1.3. $[1, 2\sqrt{-5}]$ is an order of $\mathbb{Q}(\sqrt{-5})$ that is *not* maximal since it is contained in the order $[1, \sqrt{-5}]$.

Definition 2.1.4. [40, Definition 4.16] Let $\mathcal{O} = [\xi_1, \xi_2]$ be any order of $\mathbb{Q}(\sqrt{D})$. Then the *discriminant* Δ of $\mathcal{O}$ is defined as

$$\Delta = \begin{vmatrix} \xi_1 & \xi_2 \\ \bar{\xi}_1 & \bar{\xi}_2 \end{vmatrix}^2 = (\xi_1\bar{\xi}_2 - \bar{\xi}_1\xi_2)^2$$

where $\bar{\xi}$ denotes the complex conjugate of ξ .

Example 2.1.5. The discriminant Δ' of the non-maximal order $[1, 2\sqrt{-5}]$ is

$$\Delta' = \begin{vmatrix} 1 & 2\sqrt{-5} \\ 1 & -2\sqrt{-5} \end{vmatrix}^2 = (-4\sqrt{-5})^2 = -80,$$

while the discriminant Δ of the maximal order $[1, \sqrt{-5}]$ is

$$\Delta = \begin{vmatrix} 1 & \sqrt{-5} \\ 1 & -\sqrt{-5} \end{vmatrix}^2 = (-2\sqrt{-5})^2 = -20.$$

By [50, p.13], the discriminant Δ of the maximal order $\mathcal{O}_K = [1, \omega]$ can be written as

$$\Delta = \begin{cases} 4D & \text{when } D \equiv 2, 3 \pmod{4} \\ D & \text{when } D \equiv 1 \pmod{4}, \end{cases} \quad (2.1)$$

and is an invariant of the quadratic field $\mathbb{Q}(\sqrt{D})$. A discriminant of a maximal order is called a *fundamental discriminant*. When $\mathcal{O}'$ is a non-maximal order of $\mathbb{Q}(\sqrt{D})$, we have the following proposition (see [21, p.216] for a proof).

Proposition 2.1.6. Let $\mathcal{O}'$ be a non-maximal order of $\mathbb{Q}(\sqrt{D})$ with discriminant Δ' . Then $\Delta' = f^2\Delta$ for some integer f , and Δ a fundamental discriminant.

We call f the *conductor* of the non-maximal order $\mathcal{O}'$. Notice that in Example 2.1.5, we had $\Delta' = -80 = 4(-20) = f^2\Delta$ where $f = 2$ and $\Delta = -20$.

2.2 Ideals and the Class Group

Definition 2.2.1. [40, Definition 4.20] An (integral) *ideal* $\mathfrak{a}$ is an additive subgroup of an order $\mathcal{O}$ with the property that for any $a \in \mathfrak{a}$ and $\xi \in \mathcal{O}$, it holds that ξa is an element of the ideal $\mathfrak{a}$.

Example 2.2.2. Let $\mathcal{O} = [1, \sqrt{-5}]$ be the maximal order of $\mathbb{Q}(\sqrt{-5})$. Then let $\mathfrak{a} = \{2x + \sqrt{-5}y : x, y \in \mathcal{O}\}$ be an additive subgroup of $\mathcal{O}$. Then by Proposition 2.1.1, for some $\xi \in \mathcal{O}$

we have $\xi = u + \sqrt{-5}v$ for $u, v \in \mathbb{Z}$ and for some $\alpha \in \mathfrak{a}$ we have $\alpha = 2\omega_1 + \sqrt{-5}\omega_2$ for $\omega_1, \omega_2 \in \mathcal{O}$. Since $\omega_1, \omega_2 \in \mathcal{O}$, there exists $x_1, y_1, x_2, y_2 \in \mathbb{Z}$ with $\omega_1 = x_1 + \sqrt{-5}y_1$ and $\omega_2 = x_2 + \sqrt{-5}y_2$. Now it follows that

$$\begin{aligned}
\xi\alpha &= (u + \sqrt{-5}v)(2\omega_1 + \sqrt{-5}\omega_2) \\
&= (u + \sqrt{-5}v)(2(x_1 + \sqrt{-5}y_1) + \sqrt{-5}(x_2 + \sqrt{-5}y_2)) \\
&= (u + \sqrt{-5}v)(2x_1 + 2\sqrt{-5}y_1 + \sqrt{-5}x_2 - 5y_2) \\
&= 2x_1u + 2\sqrt{-5}y_1u + \sqrt{-5}x_2u - 5y_2u + 2\sqrt{-5}x_1v - 10y_1v - 5x_2v - 5\sqrt{-5}y_2v \\
&= 2((x_1u - 5y_1v) + \sqrt{-5}(y_1u + x_1v)) + \sqrt{-5}((x_2u - 5y_2v) + \sqrt{-5}(y_2u + x_2v)).
\end{aligned}$$

Let $\zeta_1 = (x_1u - 5y_1v) + \sqrt{-5}(y_1u + x_1v)$ and $\zeta_2 = (x_2u - 5y_2v) + \sqrt{-5}(y_2u + x_2v)$. Since $\zeta_1, \zeta_2 \in \mathcal{O}$, it follows that $\xi\alpha = 2\zeta_1 + \sqrt{-5}\zeta_2$ and $\xi\alpha \in \mathfrak{a}$. Therefore, $\mathfrak{a}$ is an ideal of $\mathcal{O}$.

Let $\mathcal{O}_\Delta$ be a maximal order. Then by [50, p.16], for $\alpha, \beta \in \mathcal{O}_\Delta$, the set $\mathfrak{a} = \{x\alpha + y\beta : x, y \in \mathcal{O}_\Delta\}$ is an ideal of the order $\mathcal{O}_\Delta$ and is denoted (α, β) . We call α and β the *generators* of $\mathfrak{a}$. Every ideal of a quadratic order $\mathcal{O}_\Delta$ can be represented by at most two generators [21, p.125–126], while some can be represented by a single generator. An ideal represented by a single generator is a *principal* ideal and is denoted $(\alpha) = \{x\alpha : x \in \mathcal{O}_\Delta\}$ [40, p.87].

The product of two ideals, $\mathfrak{a} = (\alpha_1, \beta_1)$ and $\mathfrak{b} = (\alpha_2, \beta_2)$ of $\mathcal{O}_\Delta$, is

$$\mathfrak{a}\mathfrak{b} = (\alpha_1\alpha_2, \alpha_1\beta_2, \beta_1\alpha_2, \beta_1\beta_2) = (\alpha_3, \beta_3) \quad (2.2)$$

for some $\alpha_3, \beta_3 \in \mathcal{O}_\Delta$ [40, Definition 4.27], and $\mathfrak{a}\mathfrak{b}$ is also an ideal of $\mathcal{O}_\Delta$. The principal ideal $(1) = \mathcal{O}_\Delta$ is the *identity* element with respect to ideal multiplication, since $\mathfrak{a} = \mathfrak{a}\mathcal{O}_\Delta = \mathcal{O}_\Delta\mathfrak{a}$. If there exists an ideal $\mathfrak{c}$ such that $\mathfrak{a}\mathfrak{c} = \mathfrak{b}$, then $\mathfrak{a}$ *divides* $\mathfrak{b}$ and we write $\mathfrak{a} \mid \mathfrak{b}$. Finally, an ideal $\mathfrak{p} \neq (1)$ is *prime* when $\mathfrak{p} \mid \mathfrak{a}\mathfrak{b}$ implies that either $\mathfrak{p} \mid \mathfrak{a}$ or $\mathfrak{p} \mid \mathfrak{b}$. For this reason, a prime ideal is divisible only by the identity ideal and itself.

Equation (2.2) shows that ideals are closed under multiplication, and by [21, p.117], they are associative, commutative, and have identity. For ideals to form a group, we need inverses, and for this we define *fractional ideals*.

Definition 2.2.3. [52, Definition 2.23] A *fractional ideal* of $\mathcal{O}$ is a subset $\mathfrak{a}$ of $\mathbb{Q}(\sqrt{\Delta})$ such that $d\mathfrak{a}$ is an integral ideal of $\mathcal{O}$ for some non-zero $d \in \mathcal{O}$.

Example 2.2.4. Let $\mathcal{O} = [1, \sqrt{-5}]$ be the maximal order in the field $\mathbb{Q}(\sqrt{-5})$. Then the set

$$\mathfrak{f} = \left\{ \frac{2x + \sqrt{-5}y}{7} : x, y \in \mathcal{O} \right\}$$

is a fractional ideal because

$$7\mathfrak{f} = \{2x + \sqrt{-5}y : x, y \in \mathcal{O}\}$$

is an integral ideal in the order $\mathcal{O}$.

Using fractional ideals, we are able to define inverses.

Proposition 2.2.5. [48, pp.153–154] Let $\mathfrak{a}$ be a fractional ideal of $\mathcal{O}_\Delta$, where $\mathcal{O}_\Delta$ is a maximal order in the quadratic field $\mathbb{Q}(\sqrt{\Delta})$. The inverse of $\mathfrak{a}$ is defined as

$$\mathfrak{a}^{-1} = \{d \in \mathbb{Q}(\sqrt{\Delta}) : d\mathfrak{a} \subseteq \mathcal{O}_\Delta\}$$

and satisfies $\mathfrak{a}\mathfrak{a}^{-1} = \mathfrak{a}^{-1}\mathfrak{a} = \mathcal{O}_\Delta$.

Example 2.2.6. Let $\mathcal{O} = [1, \sqrt{-5}]$ be the maximal order in the field $\mathbb{Q}(\sqrt{-5})$. Then let $\Delta = -20$ and

$$\mathfrak{a} = \left\{ 7x + \frac{(8 + \sqrt{\Delta})y}{2} : x, y \in \mathcal{O} \right\} = \left(7, \frac{8 + \sqrt{\Delta}}{2} \right)$$

is an ideal of $\mathcal{O}$. The inverse of $\mathfrak{a}$ is given as

$$\mathfrak{a}^{-1} = \left\{ x + \frac{-8 + \sqrt{\Delta}}{14}y : x, y \in \mathcal{O} \right\} = \frac{1}{7} \left(7, \frac{-8 + \sqrt{\Delta}}{2} \right),$$

which is verified by

$$\begin{aligned} \frac{1}{7} \left(7, \frac{8 + \sqrt{\Delta}}{2} \right) \left(7, \frac{-8 + \sqrt{\Delta}}{2} \right) &= \frac{1}{7} \left(49, \frac{-56 + 7\sqrt{\Delta}}{2}, \frac{56 + 7\sqrt{\Delta}}{2}, 21 \right) \\ &= \left(7, \frac{-8 + \sqrt{\Delta}}{2}, \frac{8 + \sqrt{\Delta}}{2}, 3 \right) \\ &= (1). \end{aligned}$$

All fractional ideals of $\mathcal{O}_\Delta$ are invertible, and the set of fractional ideals forms a group under ideal multiplication [52, Theorem 2.26]. We now define an equivalence relation on the invertible ideals of $\mathcal{O}_\Delta$.

Definition 2.2.7. [40, p.88] Two ideals $\mathfrak{a}$ and $\mathfrak{b}$ are *equivalent* if there exist principal ideals $\alpha, \beta \in \mathcal{O}_\Delta$ such that $\alpha\beta \neq 0$ and $(\alpha)\mathfrak{a} = (\beta)\mathfrak{b}$.

Example 2.2.8. Let $\mathcal{O} = [1, \sqrt{-5}]$ be the maximal order in $\mathbb{Q}(\sqrt{-5})$. Then let $\Delta = -20$ and let $\mathfrak{a} = (3, (4 + \sqrt{\Delta})/2)$ and $\mathfrak{b} = (3\sqrt{\Delta}, -10 + 2\sqrt{\Delta})$ be two ideals of $\mathcal{O}$. Ideals $\mathfrak{a}$ and $\mathfrak{b}$ are equivalent since

$$(\sqrt{\Delta})\mathfrak{a} = (1)\mathfrak{b}$$

implies

$$(\sqrt{\Delta}) \left(3, \frac{4 + \sqrt{\Delta}}{2} \right) = (1)(3\sqrt{\Delta}, -10 + 2\sqrt{\Delta})$$

for principle ideals $(\sqrt{\Delta})$ and (1) of $\mathcal{O}$.

We write $[\mathfrak{a}]$ to denote the set of all ideals equivalent to $\mathfrak{a}$, and call this an *ideal class*. The *ideal class group*, denoted Cl_Δ , is the set of all equivalence classes of invertible ideals, with the group operation defined as the product of class representatives. By [21, p.136], the group operation is well defined and the ideal class group is a finite Abelian group.

Definition 2.2.9. The number of ideal classes in the class group Cl_Δ is the *class number* and is denoted h_Δ [40, p.153].

Example 2.2.10. Let $\Delta = -20$ and the number of ideal classes in Cl_Δ is $h_\Delta = 2$.

C. Siegel [23, p.247] showed that $h_\Delta = |\Delta|^{1/2+o(1)}$ as $\Delta \rightarrow -\infty$, which will be useful in Chapters 4 and 8.

2.3 Ideal Arithmetic

When performing ideal arithmetic, we represent invertible fractional ideals following Jacobson [38, p.13, Equation 2.1] using the $\mathbb{Z}$ -module

$$\mathfrak{a} = q \left[a, \frac{b + \sqrt{\Delta}}{2} \right]$$

for $a, b \in \mathbb{Z}, q \in \mathbb{Q}_{>0}, a > 0, \gcd(a, b, (b^2 - \Delta)/4a) = 1, b^2 \equiv \Delta \pmod{4a}$, and b is unique mod $2a$. We also use $q(a, b)$, for short, to represent the fractional ideal $\mathfrak{a}$. When $q = 1$, $\mathfrak{a}$ is a *primitive* ideal, and we use the pair (a, b) to represent $\mathfrak{a}$. Our implementation of ideal arithmetic also maintains the value $c = (b^2 - \Delta)/4a$, which will be useful throughout the remainder of this chapter.

Using this representation, the inverse $\mathfrak{a}^{-1}$ of an ideal is given by [38, pp.14–15] as

$$\mathfrak{a}^{-1} = \frac{q}{\mathcal{N}(\mathfrak{a})} \left[a, \frac{-b + \sqrt{\Delta}}{2} \right] \quad (2.3)$$

where $\mathcal{N}(\mathfrak{a}) = q^2 a$ is the norm of $\mathfrak{a}$ and is multiplicative. For a primitive invertible ideal (a, b) , the inverse is given by $\frac{1}{a}(a, -b)$. Let $[\mathfrak{a}]$ be an ideal class represented by the primitive ideal $\mathfrak{a}$. Then the inverse $[\mathfrak{a}]^{-1}$ is given simply as the class represented by $(a, -b)$ [50, p.20]. Therefore, computing the inverse of an ideal class is virtually free.

Since an ideal class contains an infinitude of ideals, we work with reduced representatives. This also makes arithmetic faster, since the size of generators are typically smaller. Subsection 2.3.1 defines the reduced form of a representative and gives an algorithm for finding the reduced form. For two reduced class representatives, Subsection 2.3.2 shows how to compute their product. Subsection 2.3.3 discusses how to perform multiplication such that the result is a reduced or almost reduced representative. This is then extended to the case of computing the square (Subsection 2.3.4) and cube (Subsection 2.3.5) of an ideal class.

2.3.1 Reduced Representatives

Definition 2.3.1. A primitive ideal $\mathfrak{a} = [a, (b + \sqrt{\Delta})/2]$ with $\Delta < 0$ is a *reduced* representative of the ideal class $[\mathfrak{a}]$ when $-a < b \leq a < c$ or when $0 \leq b \leq a = c$ for $c = (b^2 - \Delta)/4a$ [23, p.241]. Often, we refer to the ideal $\mathfrak{a}$ simply as being *reduced*.

Algorithm 2.1 Ideal Reduction ([39, p.90]).

Input: An ideal class representative $\mathfrak{a}_1 = [a_1, (b_1 + \sqrt{\Delta})/2]$ and $c_1 = (b_1^2 - \Delta)/4a_1$.

Output: A reduced representative $\mathfrak{a} = [a, (b + \sqrt{\Delta})/2]$ and $c = (b^2 - \Delta)/4a$.

```

1:  $(a, b, c) \leftarrow (a_1, b_1, c_1)$ 
2: while  $a > c$  or  $b > a$  or  $b \leq -a$  do
3:   if  $a > c$  then
4:     swap  $a$  with  $c$  and set  $b \leftarrow -b$ 
5:   if  $b > a$  or  $b \leq -a$  then
6:      $b \leftarrow b'$  such that  $-a < b' \leq a$  and  $b' \equiv b \pmod{2a}$ 
7:      $c \leftarrow (b^2 - \Delta)/4a$ 
8:   if  $a = c$  and  $b < 0$  then
9:      $b \leftarrow -b$ 
10: return  $[a, (b + \sqrt{\Delta})/2]$  and  $c = (b^2 - \Delta)/4a$ 

```

In an imaginary quadratic field, every ideal class contains exactly one reduced ideal [50, p.20]. There are several algorithms to compute a reduced ideal, many of which are listed in [39]. Here we present the algorithm we use, which is given by Algorithm 2.1. We adapt the work presented in [39, p.90] and [40, p.99]. If $\mathfrak{a} = [a, (b + \sqrt{\Delta})/2]$ is a representative of the class $[\mathfrak{a}]$ then

$$\mathfrak{b} = \left[\frac{b^2 - \Delta}{4a}, \frac{-b + \sqrt{\Delta}}{2} \right] \quad (2.4)$$

is equivalent, which can be verified by

$$\left(-(b - \sqrt{\Delta})/2 \right) \mathfrak{a} = (a)\mathfrak{b}.$$

Since $c = (b^2 - \Delta)/4a$ we have

$$\mathfrak{b} = \left[c, \frac{-b + \sqrt{\Delta}}{2} \right]. \quad (2.5)$$

The first step is to decrease a by setting $\mathfrak{a} = [c, (-b + \sqrt{\Delta})/2]$, if $a > c$. Then b is decreased since b is unique modulo $2a$. These steps are repeated while $\mathfrak{a}$ is not reduced. In the case that $a = c$, we use the absolute value of b , since by Equation (2.5) the ideals $[a, (b + \sqrt{\Delta})/2]$ and $[c, (-b + \sqrt{\Delta})/2]$ are equivalent.

Example 2.3.2. Let $\mathfrak{a} = [7, (8 + \sqrt{\Delta})/2]$ with $\Delta = -20$ be an ideal of the order $\mathcal{O} = [1, \sqrt{-5}]$. Initially, $a = 7, b = 8, c = 3$. Using Algorithm 2.1, the steps to compute a reduced ideal equivalent to $\mathfrak{a}$ are as follows.

- | | | | | |
|----|---------|----------|---------|--|
| 1. | $a = 7$ | $b = 8$ | $c = 3$ | {initial values} |
| 2. | $a = 3$ | $b = -8$ | $c = 7$ | {swap a and c and negate b } |
| 3. | $a = 3$ | $b = -2$ | $c = 2$ | {decrease $b \bmod 2a$, recalculate c } |
| 4. | $a = 2$ | $b = 2$ | $c = 3$ | {swap a and c and negate b } |

The final ideal $\mathfrak{a}' = [2, (2 + \sqrt{\Delta})/2]$ is reduced as it satisfies Definition 2.3.1.

We point out here that by Definition 2.3.1, $|b| \leq a$. It follows that $-\Delta = 4ac - b^2 \geq 4ac - a^2$, and using $a \leq c$, we have $|b| \leq a \leq \sqrt{|\Delta|/3}$. These bounds will be useful to our implementations of ideal class arithmetic in Chapter 6.

2.3.2 Ideal Class Multiplication

The ideal class group operation is multiplication of ideal class representatives. Given two representative ideals $\mathfrak{a} = [a_1, (b_1 + \sqrt{\Delta})/2]$ and $\mathfrak{b} = [a_2, (b_2 + \sqrt{\Delta})/2]$ in reduced form, the

(non-reduced) product $\mathfrak{a}\mathfrak{b}$ is computed using (adapted from [40, §5.4])

$$c_2 = (b_2^2 - \Delta)/4a_2,$$

$$s = \gcd(a_1, a_2, (b_1 + b_2)/2) = Ya_1 + Va_2 + W(b_1 + b_2)/2, \quad (2.6)$$

$$U = (V(b_1 - b_2)/2 - Wc_2) \bmod (a_1/s), \quad (2.7)$$

$$a = (a_1a_2)/s^2, \quad (2.8)$$

$$b = (b_2 + 2Ua_2/s) \bmod 2a, \quad (2.9)$$

$$\mathfrak{a}\mathfrak{b} = s \left[a, \frac{b + \sqrt{\Delta}}{2} \right].$$

Example 2.3.3. Let $\Delta = -20$ and let $\mathfrak{a} = [3, (4 + \sqrt{\Delta})/2]$ and $\mathfrak{b} = [6, (10 + \sqrt{\Delta})/2]$ be reduced ideals in the order $\mathcal{O} = [1, \sqrt{-5}]$. Multiplication using the above gives

$$\mathfrak{a}\mathfrak{b} = \left[18, \frac{22 + \sqrt{\Delta}}{2} \right].$$

The remainder of this subsection is used to derive the above equations. We adapt much of the presentation given in [40, pp.117–118]. Using Equation (2.2) for the product of two ideals, $\mathfrak{a}$ and $\mathfrak{b}$, gives

$$\mathfrak{a}\mathfrak{b} = a_1a_2\mathbb{Z} + \frac{a_1b_2 + a_1\sqrt{\Delta}}{2}\mathbb{Z} + \frac{a_2b_1 + a_2\sqrt{\Delta}}{2}\mathbb{Z} + \frac{b_1b_2 + (b_1 + b_2)\sqrt{\Delta} + \Delta}{4}\mathbb{Z}. \quad (2.10)$$

By the multiplicative property of the norm we have

$$\begin{aligned} N(\mathfrak{a}\mathfrak{b}) &= s^2a = N(\mathfrak{a})N(\mathfrak{b}) = a_1a_2 \\ \Rightarrow a &= \frac{a_1a_2}{s^2}, \end{aligned}$$

which gives Equation (2.8). Now, by the second term of equation (2.10) we know that $(a_1b_2 + a_1\sqrt{\Delta})/2 \in \mathfrak{a}\mathfrak{b}$. It follows that there is some $x, y \in \mathbb{Z}$ such that

$$\frac{a_1b_2 + a_1\sqrt{\Delta}}{2} = xsa + ys \left(\frac{b + \sqrt{\Delta}}{2} \right).$$

Equating irrational parts gives

$$\frac{a_1\sqrt{\Delta}}{2} = \frac{ys\sqrt{\Delta}}{2}.$$

Hence, $s \mid a_1$. Similarly, by the third and fourth terms of equation (2.10) we have $(a_2b_1 + a_2\sqrt{\Delta})/2 \in \mathfrak{ab}$, which implies that $s \mid a_2$, and $(b_1b_2 + (b_1 + b_2)\sqrt{\Delta} + \Delta)/4 \in \mathfrak{ab}$, which implies that $s \mid (b_1 + b_2)/2$.

By the second generator, $s(b + \sqrt{\Delta})/2$, of $\mathfrak{ab}$ and the entire right hand side of equation (2.10) there exists $X, Y, V, W \in \mathbb{Z}$ such that

$$\frac{sb + s\sqrt{\Delta}}{2} = Xa_1a_2 + Y\frac{a_1b_2 + a_1\sqrt{\Delta}}{2} + V\frac{a_2b_1 + a_2\sqrt{\Delta}}{2} + W\frac{b_1b_2 + (b_1 + b_2)\sqrt{\Delta} + \Delta}{4}.$$

Grouping rational and irrational parts gives

$$\frac{sb + s\sqrt{\Delta}}{2} = \left(Xa_1a_2 + Y\frac{a_1b_2}{2} + V\frac{a_2b_1}{2} + W\frac{b_1b_2 + \Delta}{4} \right) + \left(Y\frac{a_1}{2} + V\frac{a_2}{2} + W\frac{b_1 + b_2}{4} \right) \sqrt{\Delta}. \quad (2.11)$$

Again, by equating irrational parts we have

$$\begin{aligned} \frac{s\sqrt{\Delta}}{2} &= \left(Y\frac{a_1}{2} + V\frac{a_2}{2} + W\frac{b_1 + b_2}{4} \right) \sqrt{\Delta} \\ s &= Ya_1 + Va_2 + W\frac{b_1 + b_2}{2}, \end{aligned} \quad (2.12)$$

which is the same as Equation (2.6). Since s divides each of a_1, a_2 , and $(b_1 + b_2)/2$, take $s = \gcd(a_1, a_2, (b_1 + b_2)/2)$ to be the largest such common divisor.

It remains to compute $b \pmod{2a}$. Recall that $a = a_1a_2/s^2$. This time, by equating the rational parts of (2.11) we have

$$\begin{aligned} \frac{sb}{2} &= Xa_1a_2 + Y\frac{a_1b_2}{2} + V\frac{a_2b_1}{2} + W\frac{b_1b_2 + \Delta}{4} \\ b &= 2X\frac{a_1a_2}{s} + Y\frac{a_1b_2}{s} + V\frac{a_2b_1}{s} + W\frac{b_1b_2 + \Delta}{2s} \\ b &\equiv Y\frac{a_1b_2}{s} + V\frac{a_2b_1}{s} + W\frac{b_1b_2 + \Delta}{2s} \pmod{2a}. \end{aligned} \quad (2.13)$$

This gives b . However, Equation (2.13) can be rewritten with fewer divisions and multipli-

cations. Equation (2.12) gives

$$\begin{aligned} s &= Y a_1 + V a_2 + W \frac{b_1 + b_2}{2} \\ 1 &= Y \frac{a_1}{s} + V \frac{a_2}{s} + W \frac{b_1 + b_2}{2s} \\ Y \frac{a_1}{s} &= 1 - V \frac{a_2}{s} - W \frac{b_1 + b_2}{2s}. \end{aligned}$$

Then substituting into Equation (2.13) gives

$$\begin{aligned} b &\equiv b_2 \left(1 - V \frac{a_2}{s} - W \frac{b_1 + b_2}{2s}\right) + V \frac{a_2 b_1}{s} + W \frac{b_1 b_2 + \Delta}{2s} \pmod{2a} \\ &\equiv b_2 - V \frac{a_2 b_2}{s} - W \frac{b_1 b_2 + b_2^2}{2s} + V \frac{a_2 b_1}{s} + W \frac{b_1 b_2 + \Delta}{2s} \pmod{2a} \\ &\equiv b_2 + V \frac{a_2(b_1 - b_2)}{s} + W \frac{\Delta - b_2^2}{2s} \pmod{2a} \\ &\equiv b_2 + V \frac{2a_2(b_1 - b_2)}{2s} + W \frac{2a_2(\Delta - b_2^2)}{2a_2 \cdot 2s} \pmod{2a} \\ &\equiv b_2 + \frac{2a_2}{s} \left(V \frac{b_1 - b_2}{2} + W \frac{\Delta - b_2^2}{4a_2} \right) \pmod{2a}. \end{aligned}$$

Let $c_2 = (b_2^2 - \Delta)/4a_2$ and $U = (V(b_1 - b_2)/2 - Wc_2) \pmod{(a_1/s)}$ and we have

$$b \equiv b_2 + \frac{2a_2}{s} U \pmod{2a},$$

which completes the derivation of Equation (2.9). Note that the product ideal $\mathfrak{a}\mathfrak{b}$ is not reduced and that the size of its coefficients can be as much as twice that of the ideal factors $\mathfrak{a}$ and $\mathfrak{b}$ [40, p.118].

2.3.3 Fast Ideal Multiplication (NUCOMP)

Shanks [56] gave an algorithm for multiplying two ideal class representatives such that their product is reduced or almost reduced. The algorithm is known as NUCOMP and stands for “New COMPosition”. This algorithm is often faster in practice as the intermediate numbers are smaller and the final product requires at most two applications of the reduction operation to be converted to reduced form [40, pp.439–441]. The description of NUCOMP

provided here is a high level description of the algorithm based on Jacobson and Williams [40, §5.4, pp. 119-123].

Equations (2.6), (2.8), and (2.9) from the previous subsection give a solution to the ideal product $\mathfrak{a}\mathfrak{b} = s[a, (b + \sqrt{\Delta})/2]$. In this case, the product ideal is not necessarily reduced. Computing the reduced ideal class representative corresponds to computing the simple continued fraction expansion of $(b/2)/a$ [40, p.119], but in this case, the coefficients a and b may be as large as Δ [40, p.118]. Instead of computing the simple continued fraction expansion of $(b/2)/a$, Jacobson and Williams [40, p.119] use sU/a_1 where U is given by Equation (2.7). To see why this is sufficient, recall from Subsection 2.3.1 that a_2 is approximately $\sqrt{|\Delta|}$ in size and s is small, so $s^2/2a_2 \approx 1/\sqrt{|\Delta|}$ and

$$\frac{b}{2a} = \frac{b_2 + 2Ua_2/s}{2a_1a_2/s^2} = \frac{s^2b_2 + s2Ua_2}{2a_1a_2} = \frac{s^2b_2}{2a_1a_2} + \frac{sU}{a_1} \approx \frac{sU}{a_1}.$$

Following [40, pp.120-121], we develop the simple continued fraction expansion of $sU/a_1 = \langle q_0, q_1, \dots, q_i, \phi_{i+1} \rangle$ using the recurrences

$$q_i = \lfloor R_{i-2} / R_{i-1} \rfloor \tag{2.14}$$

$$R_i = R_{i-2} - q_i R_{i-1} \tag{2.15}$$

$$C_i = C_{i-2} - q_i C_{i-1} \tag{2.16}$$

until we have R_i and R_{i-1} such that

$$R_i < \sqrt{a_1/a_2} |\Delta/4|^{1/4} < R_{i-1}. \tag{2.17}$$

Initial values for the recurrence are given by

$$\begin{bmatrix} R_{-2} & C_{-2} \\ R_{-1} & C_{-1} \end{bmatrix} = \begin{bmatrix} sU & -1 \\ a_1 & 0 \end{bmatrix}.$$

We then compute

$$\begin{aligned}
M_1 &= \frac{R_i a_2 + s C_i (b_1 - b_2)/2}{a_1}, \\
M_2 &= \frac{R_i (b_1 + b_2)/2 - s C_i c_2}{a_1}, \\
a &= (-1)^{i+1} (R_i M_1 - C_i M_2), \\
b &= \left(\frac{2(R_i a_2/s - C_{i-1} a)}{C_i} - b_2 \right) \bmod 2a
\end{aligned} \tag{2.18}$$

for the product $\mathbf{ab} = [a, (b + \sqrt{\Delta})/2]$ where $\mathbf{ab}$ is at most two steps from being reduced [40, p.122]. Note that this procedure assumes that $\mathcal{N}(\mathbf{a}) \leq \mathcal{N}(\mathbf{b})$ and that if $a_1 < \sqrt{a_1/a_2} |\Delta/4|^{1/4}$ then R_{-1} and R_{-2} satisfy Equation (2.17) and we compute the product $\mathbf{ab}$ as in the previous subsection without expanding the simple continued fraction sU/a_1 . When $a_1 \geq \sqrt{a_1/a_2} |\Delta/4|^{1/4}$, at least one iteration of the recurrence (2.16) is performed and so $C_i \neq 0$ and there will not be a division by zero in Equation (2.18).

Our implementation of fast ideal multiplication includes many practical optimizations developed by Imbert, Jacobson, and Schmidt [35, Algorithm 6]. For example, by Equation (2.6), $s \mid a_1$ and $s \mid a_2$, and so after computing s , we use $a_1' = a_1/s$ and $a_2' = a_2/s$ throughout. Furthermore, they separate Equation (2.6) into the following computations. Let

$$s' = \gcd(a_1, a_2) = Y' a_1 + V' a_2 \tag{2.19}$$

$$s = \gcd(s', (b_1 + b_2)/2) = V'' s' + W(b_1 + b_2)/2. \tag{2.20}$$

Only when $s' \neq 1$ do we compute Equation (2.20), in which case $V = V' V''$. When $s' = 1$, we do not compute Equation (2.20) and instead set $s = 1$, $V = V'$, and $W = 0$ (which simplifies Equation (2.7)). Also notice that Y from Equation (2.6) (and correspondingly Y' from Equation (2.19)) is not used throughout the ideal product calculation. For this reason, we do not compute this coefficient when computing Equation (2.19).

Chapter 5 discusses practical optimizations for computing the extended GCD used in Equations (2.19) and (2.20), as well as the simple continued fraction expansion of sU/a_1 ,

which is essentially the extended GCD computation [40, §3.2]. Chapter 6 gives a more thorough treatment of our implementation of ideal class arithmetic. Pseudo-code for the approach described here is given in Algorithm 2.2.

Algorithm 2.2 NUCOMP – Fast Ideal Multiplication ([40, pp.441-443]).

Input: Reduced representatives $\mathbf{a} = [a_1, (b_1 + \sqrt{\Delta})/2]$, $\mathbf{b} = [a_2, (b_2 + \sqrt{\Delta})/2]$ with $c_1 = (b_1^2 - \Delta)/4a_1$, $c_2 = (b_2^2 - \Delta)/4a_2$, and discriminant Δ .

Output: A reduced or almost reduced representative $\mathbf{ab}$.

```

1: ensure  $\mathcal{N}(\mathbf{a}) \leq \mathcal{N}(\mathbf{b})$  by swapping  $\mathbf{a}$  with  $\mathbf{b}$  if  $a_1 > a_2$ 
2: compute  $s', V' \in \mathbb{Z}$  such that  $s' = \gcd(a_1, a_2) = Y'a_1 + V'a_2$  for  $Y' \in \mathbb{Z}$ 
3:  $s \leftarrow 1$ 
4:  $U \leftarrow V'(b_1 - b_2)/2 \bmod a_1$ 
5: if  $s' \neq 1$  then
6:   compute  $s, V, W \in \mathbb{Z}$  such that  $s = \gcd(s', (b_1 + b_2)/2) = Vs' + W(b_1 + b_2)/2$ 
7:    $(a_1, a_2) \leftarrow (a_1/s, a_2/s)$ 
8:    $U \leftarrow (VU - Wc_2) \bmod a_1$ 
9: if  $a_1 < \sqrt{a_1/a_2} |\Delta/4|^{1/4}$  then
10:   $a \leftarrow a_1 a_2$ 
11:   $b \leftarrow (2a_2 U + b_2) \bmod 2a$ 
12:  return  $[a, (b + \sqrt{\Delta})/2]$ 
13:  $\begin{bmatrix} R_{-2} & C_{-2} \\ R_{-1} & C_{-1} \end{bmatrix} \leftarrow \begin{bmatrix} U & -1 \\ a_1 & 0 \end{bmatrix}$ 
14:  $i \leftarrow -1$ 
15: while  $R_i > \sqrt{a_1/a_2} |\Delta/4|^{1/4}$  do
16:   $i \leftarrow i + 1$ 
17:   $q_i = \lfloor R_{i-2} / R_{i-1} \rfloor$ 
18:   $R_i = R_{i-2} - q_i R_{i-1}$ 
19:   $C_i = C_{i-2} - q_i C_{i-1}$ 
20:  $M_1 \leftarrow (R_i a_2 + C_i(b_1 - b_2)/2)/a_1$   $\{(b_1 - b_2)/2 \text{ is reused from above}\}$ 
21:  $M_2 \leftarrow (R_i(b_1 + b_2)/2 - sC_i c_2)/a_1$ 
22:  $a \leftarrow (-1)^{i+1}(R_i M_1 - C_i M_2)$ 
23:  $b \leftarrow ((2(R_i a_2 - C_{i-1} a)/C_i) - b_2) \bmod 2a$   $\{R_i a_2 \text{ is reused from above}\}$ 
24: return  $[a, (b + \sqrt{\Delta})/2]$   $\{\text{optionally return } c = (b^2 - \Delta)/4a\}$ 

```

2.3.4 Fast Ideal Squaring (NUDUPL)

When the two input ideals for multiplication are the same, as is the case when squaring, much of the arithmetic simplifies. In this case, $a_1 = a_2$, $b_1 = b_2$, and Equations (2.6) and

(2.7) simplify to

$$s = \gcd(a_1, b_1) = Xa_1 + Yb_1$$

$$U = -Yc_1 \bmod (a_1/s).$$

One then computes the continued fraction expansion of sU/a_1 , but using the bound

$$R_i < |\Delta/4|^{1/4} < R_{i-1}.$$

Computing the ideal class representative simplifies as well – we have

$$\begin{aligned} M_1 &= R_i, \\ M_2 &= \frac{R_i b_1 - s C_i c_1}{a_1}, \\ a &= (-1)^{i+1} (R_i^2 - C_i M_2), \\ b &= \left(\frac{2(R_i a_1/s - C_{i-1} a)}{C_i} - b_1 \right) \bmod 2a. \end{aligned}$$

Pseudo-code for our implementation is given in Algorithm 2.3.

2.3.5 Fast Ideal Cubing (NUCUBE)

When we consider binary-ternary representations of exponents, cubing is required. In general, if we want to compute $\mathfrak{a}^3$ for an ideal class representative $\mathfrak{a} = [a_1, (b_1 + \sqrt{\Delta})/2]$, we can take advantage of the simplification that happens when expanding the computation of $\mathfrak{a}^2 \mathfrak{a}$. Here we provide a high level description of a technique for cubing based on similar ideas to NUCOMP and NUDUPL, namely that of computing the quotients of a continued fraction expansion. A detailed description and analysis of this technique can be found in [35].

Similar to ideal squaring, compute integers s' and Y' such that

$$s' = \gcd(a_1, b_1) = X'a_1 + Y'b_1.$$

Note that X' is unused and need not be computed. If $s' \neq 1$, compute

$$s = \gcd(s'a_1, b_1^2 - a_1 c_1) = Xs'a_1 + Y(b_1^2 - a_1 c_1)$$

Algorithm 2.3 NUDUPL – Fast Ideal Squaring.

Input: Reduced representative $\mathfrak{a} = [a_1, (b_1 + \sqrt{\Delta})/2]$ with $c_1 = (b_1^2 - \Delta)/4a_1$ and discriminant Δ .

Output: A reduced or almost reduced representative $\mathfrak{a}^2$.

```

1: compute  $s, Y \in \mathbb{Z}$  such that  $s = \gcd(a_1, b_1) = Xa_1 + Yb_1$  for  $X \in \mathbb{Z}$ 
2:  $a_1 \leftarrow a_1/s$ 
3:  $U \leftarrow -Yc_1 \bmod a_1$ 
4: if  $a_1 < |\Delta/4|^{1/4}$  then
5:    $a \leftarrow a_1^2$ 
6:    $b \leftarrow (2Ua_1 + b_1) \bmod 2a$ 
7:   return  $[a, (b + \sqrt{\Delta})/2]$ 
8:  $\begin{bmatrix} R_{-2} & C_{-2} \\ R_{-1} & C_{-1} \end{bmatrix} \leftarrow \begin{bmatrix} U & -1 \\ a_1 & 0 \end{bmatrix}$ 
9:  $i \leftarrow -1$ 
10: while  $R_i > |\Delta/4|^{1/4}$  do
11:    $i \leftarrow i + 1$ 
12:    $q_i = \lfloor R_{i-2} / R_{i-1} \rfloor$ 
13:    $R_i = R_{i-2} - q_i R_{i-1}$ 
14:    $C_i = C_{i-2} - q_i C_{i-1}$ 
15:  $M_2 \leftarrow (R_i b_1 - s C_i c_1)/a_1$ 
16:  $a \leftarrow (-1)^{i+1} (R_i^2 - C_i M_2)$ 
17:  $b \leftarrow ((2(R_i a_1 + C_{i-1} a)/C_i) - b_1) \bmod 2a$ 
18: return  $[a, (b + \sqrt{\Delta})/2]$  {optionally return  $c = (b^2 - \Delta)/4a$ }

```

for $s, X, Y \in \mathbb{Z}$. If $s' = 1$ then let $s = 1$, too. Then compute U using

$$U = \begin{cases} Y'c_1(Y'(b_1 - Y'c_1a_1) - 2) \bmod a_1^2 & \text{if } s' = 1 \\ -c_1(XY'a_1 + Yb_1) \bmod a_1^2/s & \text{otherwise.} \end{cases}$$

Next, develop the simple continued fraction expansion of sU/a_1^2 until

$$R_i < \sqrt{a_1}|\Delta/4|^{1/4} < R_{i-1}.$$

Finally, compute the representative $\mathfrak{a}^3 = [a, (b + \sqrt{\Delta})/2]$ using the equations

$$\begin{aligned}
M_1 &= \frac{R_i + C_i U}{a_1}, \\
M_2 &= \frac{R_i(b_1 + Ua_1) - sC_i c_1}{a_1^2}, \\
a &= (-1)^{i+1} (R_i M_1 - C_i M_2), \\
b &= \left(\frac{2(R_i a_1/s - C_{i-1} a)}{C_i} - b_1 \right) \bmod 2a.
\end{aligned}$$

By [35, p.15 Theorem 5.1], the ideal $[a, (b + \sqrt{\Delta})/2]$ is at most two reduction steps from being reduced. Pseudo-code for our implementation of fast ideal cubing is given in Algorithm 2.4.

Algorithm 2.4 NUCUBE – Fast Ideal Cubing ([35, p.26]).

Input: A reduced representative $\mathfrak{a} = [a_1, (b_1 + \sqrt{\Delta})/2]$.

Output: A reduced or almost reduced representative $\mathfrak{a}^3$.

```

1: compute  $s', Y' \in \mathbb{Z}$  such that  $s' = \gcd(a_1, b_1) = X'a_1 + Y'b_1$  and  $X' \in \mathbb{Z}$ 
2: if  $s' = 1$  then
3:    $s \leftarrow 1$ 
4:    $U \leftarrow Y'c_1(Y'(b_1 - Y'c_1a_1) - 2) \bmod a_1^2$ 
5: else
6:   compute  $s, X, Y \in \mathbb{Z}$  such that  $s = \gcd(s'a_1, b_1^2 - a_1c_1) = Xs'a_1 + Y(b_1^2 - a_1c_1)$ 
7:    $U \leftarrow -c_1(XY'a_1 + Yb_1) \bmod a_1^2/s$ 
8: if  $a_1^2/s < \sqrt{a_1} |\Delta/4|^{1/4}$  then
9:    $a \leftarrow a_1^3/s^2$ 
10:   $b \leftarrow (b_1 + 2Ua_1/s) \bmod 2a$ 
11:  return  $[a, (b + \sqrt{\Delta})/2]$  {optionally return  $c = (b^2 - \Delta)/4a$ }
12:   $\begin{bmatrix} R_{-2} & C_{-2} \\ R_{-1} & C_{-1} \end{bmatrix} \leftarrow \begin{bmatrix} U & -1 \\ a_1^2/s & 0 \end{bmatrix}$ 
13:   $i \leftarrow -1$ 
14:  while  $R_i > \sqrt{a_1} |\Delta/4|^{1/4}$  do
15:     $i \leftarrow i + 1$ 
16:     $q_i = \lfloor R_{i-2} / R_{i-1} \rfloor$ 
17:     $R_i = R_{i-2} - q_i R_{i-1}$ 
18:     $C_i = C_{i-2} - q_i C_{i-1}$ 
19:     $M_1 \leftarrow (R_i + C_i U) / a_1$ 
20:     $M_2 \leftarrow (R_i(b_1 + Ua_1) - sC_i c_1) / a_1^2$ 
21:     $a \leftarrow (-1)^{i+1} (R_i M_1 - C_i M_2)$ 
22:     $b \leftarrow (2(R_i a_1 / s - C_{i-1} a) / C_i - b_1) \bmod 2a$ 
23:  return  $[a, (b + \sqrt{\Delta})/2]$  {optionally return  $c = (b^2 - \Delta)/4a$ }

```

The next chapter discusses some exponentiation techniques that use the ideal arithmetic presented in this chapter, namely fast multiplication, squaring, and cubing.

Chapter 3

Exponentiation

Exponentiation has many applications. Diffie-Hellman key exchange uses exponentiation so that two parties may jointly establish a shared secret key over an insecure channel. An application discussed in detail in this thesis is that of computing the order of an ideal class. The approach used in Chapter 4 is to exponentiate an ideal class by the product, E , of several small primes. The result is an ideal class whose order is likely to not be divisible by any of these primes. The order is then computed using a variant of Shanks' baby-step giant-step algorithm where only powers relatively prime to E are computed. Computing the order of several ideal classes is one method to factor an integer associated with the ideal class group. Faster exponentiation means the approach used there is faster.

In Sections 3.1 and 3.2 we discuss standard exponentiation techniques that rely on a base 2 representation of the exponent. In Section 3.3 we describe double-base number systems, which, as the name implies, are number systems that make use of two bases in the representation of a number. We are particularly interested in representations that use bases 2 and 3, since our implementation of ideal class group arithmetic provides multiplication, squaring, and cubing. In Section 3.4 we discuss some methods to compute double-base representations found in the literature. Throughout this chapter, the running time and space complexities are given in terms of the number of group operations and elements required, respectively. They do not take into consideration the binary costs associated with each. Throughout this chapter, we use big O and big Ω notation, which are defined as

$$O(g(n)) = \{f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0} \mid \exists c_2 > 0, \exists n_0 > 0, \forall n \geq n_0 : 0 \leq f(n) \leq c_2 g(n)\}$$
$$\Omega(g(n)) = \{f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0} \mid \exists c_1 > 0, \exists n_0 > 0, \forall n \geq n_0 : 0 \leq c_1 g(n) \leq f(n)\}.$$

3.1 Binary Exponentiation

The simplest method of exponentiation is binary exponentiation (see [20, §9.1.1]). Let g be an element in a group G and n a positive integer. To compute g^n , we first represent n in binary as

$$n = \sum_{i=0}^{\lfloor \log_2 n \rfloor} b_i 2^i$$

where $b_i \in \{0, 1\}$ such that b_i represents the i^{th} bit of n . We then represent g^n as

$$g^n = \prod_{i=0}^{\lfloor \log_2 n \rfloor} g^{b_i 2^i}.$$

and compute g^{2^i} by repeated squaring of g . The result, g^n , is the product of each g^{2^i} where $b_i = 1$. This description is known as right-to-left binary exponentiation because the result is computed by generating the terms of the product from right to left in the written binary representation of n . The left-to-right variant evaluates the bits of the exponent n from high order to lower order by repeatedly squaring an accumulator and multiplying this with the base element g when $b_i = 1$. The left-to-right variant has the advantage that one of the values in each multiplication, namely g , remains fixed throughout the evaluation. There also exist windowed variants where g^w is precomputed for each w in some window $0 \leq w < 2^k$ for some k (typically chosen to be cache efficient). The exponent n is then expressed in base 2^k . For further discussion of windowed techniques, see [20, Subsection 9.1.3. p.149].

For a randomly chosen exponent, binary exponentiation requires $\lfloor \log_2 n \rfloor$ squarings and $(\lfloor \log_2 n \rfloor + 1)/2$ multiplications in expectation. This is the case since a multiplication is only necessary when $b_i = 1$ and the probability of $b_i = 1$, for a randomly chosen exponent, is $1/2$.

3.2 Non-Adjacent Form Exponentiation

The Non-Adjacent Form (NAF) of an integer is a *signed* base two representation such that no two non-zero terms in the representation are adjacent (see [20, §9.1.4]). Each integer, n ,

has a unique representation in non-adjacent form. Formally, an integer n is represented by

$$n = \sum_{i=0}^{\lfloor \log_2 n \rfloor + 1} s_i 2^i$$

where $s_i \in \{0, 1, -1\}$ and $s_i \cdot s_{i+1} = 0$, for $0 \leq i \leq \lfloor \log_2(n) \rfloor + 1$. For example, suppose $n = 23814216$. In binary we have

$$23814216 = 2^3 + 2^6 + 2^{13} + 2^{14} + 2^{16} + 2^{17} + 2^{19} + 2^{21} + 2^{22} + 2^{24} \quad (3.1)$$

and in non-adjacent form we have

$$23814216 = 2^3 + 2^6 - 2^{13} - 2^{15} - 2^{18} - 2^{20} - 2^{23} + 2^{25}. \quad (3.2)$$

Similar to the binary case, we compute

$$g^n = \prod_{i=0}^{\lfloor \log_2 n \rfloor + 1} g^{s_i 2^i}. \quad (3.3)$$

Since $s_i \in \{0, 1, -1\}$, exponentiation using the non-adjacent form requires inversion. However, when computing in the ideal class group, the cost of inversion is negligible (see Section 2.3). When inversion is expensive, we can instead compute

$$g^n = \left(\prod_{i: s_i = 1} g^{2^i} \right) \cdot \left(\prod_{i: s_i = -1} g^{2^i} \right)^{-1}$$

which requires at most one inversion (but this is not necessary for our purposes).

To compute the non-adjacent form of an integer n , inspect n two bits at a time from least significant to most significant. Let $n = \sum b_i 2^i$ be the binary representation of n and let $j = 0$. If the bit pattern $\langle b_{j+1}, b_j \rangle = 01_2$ then let $s_j = 1$ and subtract 2^j from n . If $\langle b_{j+1}, b_j \rangle = 11_2$ then let $s_j = -1$ and add 2^j to n . When the bit pattern $\langle b_{j+1}, b_j \rangle$ is 00_2 or 10_2 , let $s_j = 0$. Next, increment j and repeat while $n \neq 0$.

In our experiments, we use a variation of the above, originally due to Reitwiesner [51], that maintains a carry flag c (see Algorithm 3.1). Instead of adding 2^i to n , set $c = 1$, and instead of subtracting 2^i from n , set $c = 0$. When inspecting n two bits at a time, we

Algorithm 3.1 Computes g^n using right-to-left non-adjacent form (Reitwiesner [51]).

Input: $g \in G, n \in \mathbb{Z}_{\geq 0}$

Output: g^n

```

1:  $c \leftarrow 0$ 
2:  $T \leftarrow g$ 
3:  $R \leftarrow 1_G$ 
4:  $i \leftarrow 0$ 
5: while  $n \geq 2^i$  do
6:   if  $\lfloor n/2^i \rfloor + c \equiv 1 \pmod{4}$  then
7:      $R \leftarrow R \cdot T$ 
8:      $c \leftarrow 0$ 
9:   else if  $\lfloor n/2^i \rfloor + c \equiv 3 \pmod{4}$  then
10:     $R \leftarrow R \cdot T^{-1}$ 
11:     $c \leftarrow 1$ 
12:     $T \leftarrow T^2$ 
13:     $i \leftarrow i + 1$ 
14: if  $c = 1$  then
15:    $R \leftarrow R \cdot T$ 
16: return  $R$ 

```

consider the bit pattern $(m + c) \bmod 4$ where $m = 2b_{i+1} + b_i$. This technique is faster since addition and subtraction is performed with constant sized integers.

Computing $g^{23814216}$ using the binary expansion from Equation (3.1) requires 24 squarings and 9 multiplications, while exponentiation using the non-adjacent form representation (3.2) requires 25 squarings, 7 multiplications, and 5 inversions. When inversion is inexpensive, an advantage of the non-adjacent form is that it requires at most $\lfloor \log_2 n \rfloor + 1$ squares and on average $(\lfloor \log_2 n \rfloor + 2)/3$ multiplications, as opposed to $(\lfloor \log_2 n \rfloor + 1)/2$ for binary exponentiation. To see this, recall that non-adjacent form requires that no two non-zero terms be adjacent. Consider any two adjacent terms. The possible outcomes are $(0, 0)$, $(s, 0)$, or $(0, s)$ where $s \in \{-1, 1\}$. This means that 2/3 of the time, 1/2 of the terms will be non-zero, and so the probability of any given term being non-zero is 1/3.

As with binary exponentiation, there exist windowed variants of non-adjacent form (see [20, Algorithm 9.20. p.153]). In this case, g^w is computed for each w in some window $-2^{k-1} < w \leq 2^{k-1}$, and the exponent n is repeatedly evaluated modulo 2^k using the smallest

residue in absolute value, i.e. the residue w such that $-2^{k-1} < w \leq 2^{k-1}$.

3.3 Double-Base Number Systems

Binary representation and non-adjacent form use only a single base, namely base 2. Double-base number systems (DBNS), on the other hand, use two bases. Our presentation here is derived from that of Dimitrov and Cooklev [24, 25]. Given two coprime integers p and q and an integer n , we represent n as the sum and difference of the product of powers of p and q ,

$$n = \sum_{i=1}^k s_i p^{a_i} q^{b_i} \quad (3.4)$$

where $s_i \in \{-1, 1\}$ and $a_i, b_i, k \in \mathbb{Z}_{\geq 0}$. This thesis pays particular attention to representations using bases $p = 2$ and $q = 3$ such that $n = \sum s_i 2^{a_i} 3^{b_i}$. Such representations are referred to as *2,3 representations*.

As an example of a 2,3 representation, consider the number $n = 23814216$ again. Given the bases $p = 2$ and $q = 3$, *one* possible representation of n is

$$23814216 = 2^3 3^3 - 2^4 3^5 + 2^5 3^6 + 2^7 3^7 + 2^9 3^8 + 2^{10} 3^9. \quad (3.5)$$

Another possible representation is

$$23814216 = 2^3 3^2 - 2^{13} 3^2 + 2^{15} 3^6. \quad (3.6)$$

There may be many possible 2,3 representations for a given number, and different representations will trade off between cubings, squarings, and the number of terms. Exponentiation using Equation (3.5) requires 10 squarings, 9 cubings, 1 inversion, and 5 multiplications, while Equation (3.6) requires 15 squarings, 6 cubings, 1 inversion, and 3 multiplications. Contrast this with the binary representation (Equation (3.1)), which requires 24 squarings, 0 inversions, and 9 multiplications, and the non-adjacent form (Equation (3.2)), which requires 25 squarings, 5 inversions, and 7 multiplications. The best representation will depend on the needs of the application and the cost of each operation. Later, we shall see some

algorithms that take this into account (see Chapter 7), but many are designed to either find representations quickly, of a special form, or with few terms.

3.3.1 Chained 2,3 Representations

One way to classify algorithms that compute 2,3 representations is by the constraints placed on the partition of an integer n .

Definition 3.3.1. [35, §1] A *partition* of n is written $n = \pm x_1 \pm x_2 \pm \cdots \pm x_k$ where the terms x_i are positive integers that increase monotonically.

One such constraint is on the divisibility of consecutive terms.

Definition 3.3.2. [35, §1] A partition of an integer $n = x_1 \pm x_2 \pm \cdots \pm x_k$ is *chained* if every term x_i divides every term x_j for $i < j$. A partition is said to be *strictly chained* if it is chained and x_i is *strictly* less than x_j for each $i < j$.

Binary and non-adjacent form are special types of strictly chained partitions, since for any two non-zero terms where $i < j$, we have $x_i = 2^i$, $x_j = 2^j$, $x_i \mid x_j$, and $x_i < x_j$. The 2,3 representation of $23814216 = 2^3 3^2 - 2^{13} 3^2 + 2^{15} 3^6$ is another example of a strictly chained partition, since $2^3 3^2 \mid 2^{13} 3^2 \mid 2^{15} 3^6$.

The benefit of restricting 2,3 representations to chained representations is the ease with which one can compute g^n when n is given as a chain. For example $g^{23814216} = g^{2^3 3^2 - 2^{13} 3^2 + 2^{15} 3^6}$ can be computed by first computing $x_0 = g$, $x_1 = g^{2^3 3^2}$, $x_2 = x_1^{2^{10}}$, $x_3 = x_2^{2^2 3^4}$, each term being computed by repeated squaring and cubing from the previous term. Finally $g^{23814216} = x_1 \cdot x_2^{-1} \cdot x_3$. When n is given as a chained 2,3 representation, Algorithm 3.2 will compute g^n using exactly a_k squares, b_k cubes, $k - 1$ multiplications, and at most k inverses. Since $x_i \mid x_{i+1}$, an implementation need only retain x_i in order to compute x_{i+1} , and so only requires storage of a constant number of group elements.

Recall that when inversion is expensive, the product of terms with negative exponents can be computed separately from terms with positive exponents – the inverse is then computed

Algorithm 3.2 Computes g^n given n as a chained 2,3 partition (Dimitrov et al [26]).

Input: $g \in G$, $n = \sum_{i=1}^k s_i 2^{a_i} 3^{b_i}$, $s_1, \dots, s_k \in \{-1, 1\}$, $0 \leq a_1 \leq \dots \leq a_k \in \mathbb{Z}$, $0 \leq b_1 \leq \dots \leq b_k \in \mathbb{Z}$.

Output: g^n

```

1:  $i \leftarrow 1$ 
2:  $a \leftarrow 0$                                      {current power of 2}
3:  $b \leftarrow 0$                                      {current power of 3}
4:  $T \leftarrow g$                                      {loop invariant:  $T = g^{2^a 3^b}$ }
5:  $R \leftarrow 1_G$ 
6: while  $i \leq k$  do
7:   while  $a < a_i$  do
8:      $T \leftarrow T^2, a \leftarrow a + 1$ 
9:   while  $b < b_i$  do
10:     $T \leftarrow T^3, b \leftarrow b + 1$ 
11:    $R \leftarrow R \cdot T^{s_i}$                          {multiply with  $T$  or  $T^{-1}$ }
12:    $i \leftarrow i + 1$ 
13: return  $R$ 

```

only once for this product. Since computing inverses is negligible in the ideal class group, we instead compute the product of all terms directly.

3.3.2 Unchained 2,3 Representations

There is evidence [35] that the shortest possible chained representations require a linear number of terms in relation to the size of the input. This is in contrast to unchained representations for which there are algorithms where the number of terms in the representation is provably sublinear in the length of the input [27, 19].

Given an unchained 2,3 representation for an integer $n = \sum s_i 2^{a_i} 3^{b_i}$, Méloni and Hasan [46, Section 3.2] gave an algorithm that achieves the same bound on the number of group operations, i.e. $O(\log n)$, as Algorithm 3.2, but with a bound of $O(\min\{\max a_i, \max b_i\})$ on the number of group elements stored. Suppose $\max b_i < \max a_i$ and that the terms are labelled and sorted such that $a_1 \geq \dots \geq a_k$. The algorithm works by precomputing a table of $T_b = g^{3^b}$ for $0 \leq b \leq \max b_i$. Let $i = 1$ and begin with the first term $s_i 2^{a_i} 3^{b_i}$. Look up $T_{b_i} = g^{3^{b_i}}$ and, after applying the sign s_i , multiply $T_{b_i}^{s_i}$ with the running result. Let

$a' = a_i - a_{i+1}$ when $i < k$ and $a' = a_k$ when $i = k$. Then square the running result a' times. The algorithm then removes the term $s_i 2^{a_i} 3^{b_i}$ from the list of terms, and continues in this way with the next largest a_i . The algorithm terminates when there are no more terms in the list. Algorithm 3.3 gives pseudo-code for this approach and requires the storage of $O(\max b_i)$ group elements. When $\max a_i < \max b_i$, we use a related algorithm that requires the terms to be sorted such that $b_1 \geq \dots \geq b_k$; it precomputes $T_a = g^{2^a}$ for $0 \leq a \leq \max a_i$ and works similar to Algorithm 3.3 but with cubing and squaring appropriately swapped.

Algorithm 3.3 Compute g^n for a 2,3 representation of n ([46, Section 3.2]).

Input: $g \in G$, $n = \sum_{i=1}^k s_i 2^{a_i} 3^{b_i}$, $s_1, \dots, s_k \in \{-1, 1\}$, $a_1 \geq \dots \geq a_k \in \mathbb{Z}_{\geq 0}$, $b_1, \dots, b_k \in \mathbb{Z}_{\geq 0}$.
Output: g^n

- 1: $T_b \leftarrow g^{3^b}$ for $0 \leq b \leq \max\{b_1, \dots, b_k\}$ {by repeated cubing}
- 2: $R \leftarrow 1_G$
- 3: $i \leftarrow 1$
- 4: **while** $i < k$ **do**
- 5: $R \leftarrow R \cdot T_{b_i}^{s_i}$ {multiply with T_{b_i} or $T_{b_i}^{-1}$ }
- 6: $a' \leftarrow a_i - a_{i+1}$
- 7: $R \leftarrow R^{2^{a'}}$ {by squaring a' number of times}
- 8: $i \leftarrow i + 1$
- 9: $R \leftarrow R \cdot T_{b_k}^{s_k}$
- 10: $R \leftarrow R^{2^{a_k}}$ {by squaring a_k number of times}
- 11: **return** R

For example, the 2,3 representation $23814216 = 2^{15}3^6 - 2^83^5 - 2^43^6 + 2^33^3$ is sorted by decreasing a_i . The algorithm first computes $T_b = g^{3^b}$ for $0 \leq b \leq 6$. Note that it is sufficient to store only the values of $g^{3^{b_i}}$ that actually occur in terms (in this case T_3 , T_5 , and T_6). Figure 3.1 depicts the following steps to compute $g^{2^{15}3^6 - 2^83^5 - 2^43^6 + 2^33^3}$. Let $R_0 = 1_G$ and then

compute

$$\begin{aligned}
R_1 &= R_0 T_6 && \Rightarrow g^{3^6}, \\
R_2 &= R_1^{2^7} && \Rightarrow g^{2^7 3^6}, \\
R_3 &= R_2 T_5^{-1} && \Rightarrow g^{2^7 3^6 - 3^5}, \\
R_4 &= R_3^{2^4} && \Rightarrow g^{2^{11} 3^6 - 2^4 3^5}, \\
R_5 &= R_4 T_6^{-1} && \Rightarrow g^{2^{11} 3^6 - 2^4 3^5 - 3^6}, \\
R_6 &= R_5^{2^1} && \Rightarrow g^{2^{12} 3^6 - 2^5 3^5 - 2^1 3^6}, \\
R_7 &= R_6 T_3 && \Rightarrow g^{2^{12} 3^6 - 2^5 3^5 - 2^1 3^6 + 3^3}, \\
R_8 &= R_7^{2^3} && \Rightarrow g^{2^{15} 3^6 - 2^8 3^5 - 2^4 3^6 + 2^3 3^3},
\end{aligned}$$

using 15 squares, 6 cubes, 2 inverses, and 3 multiplications. The result of the computation is $R_8 = g^{2^{38} 142^{16}}$.

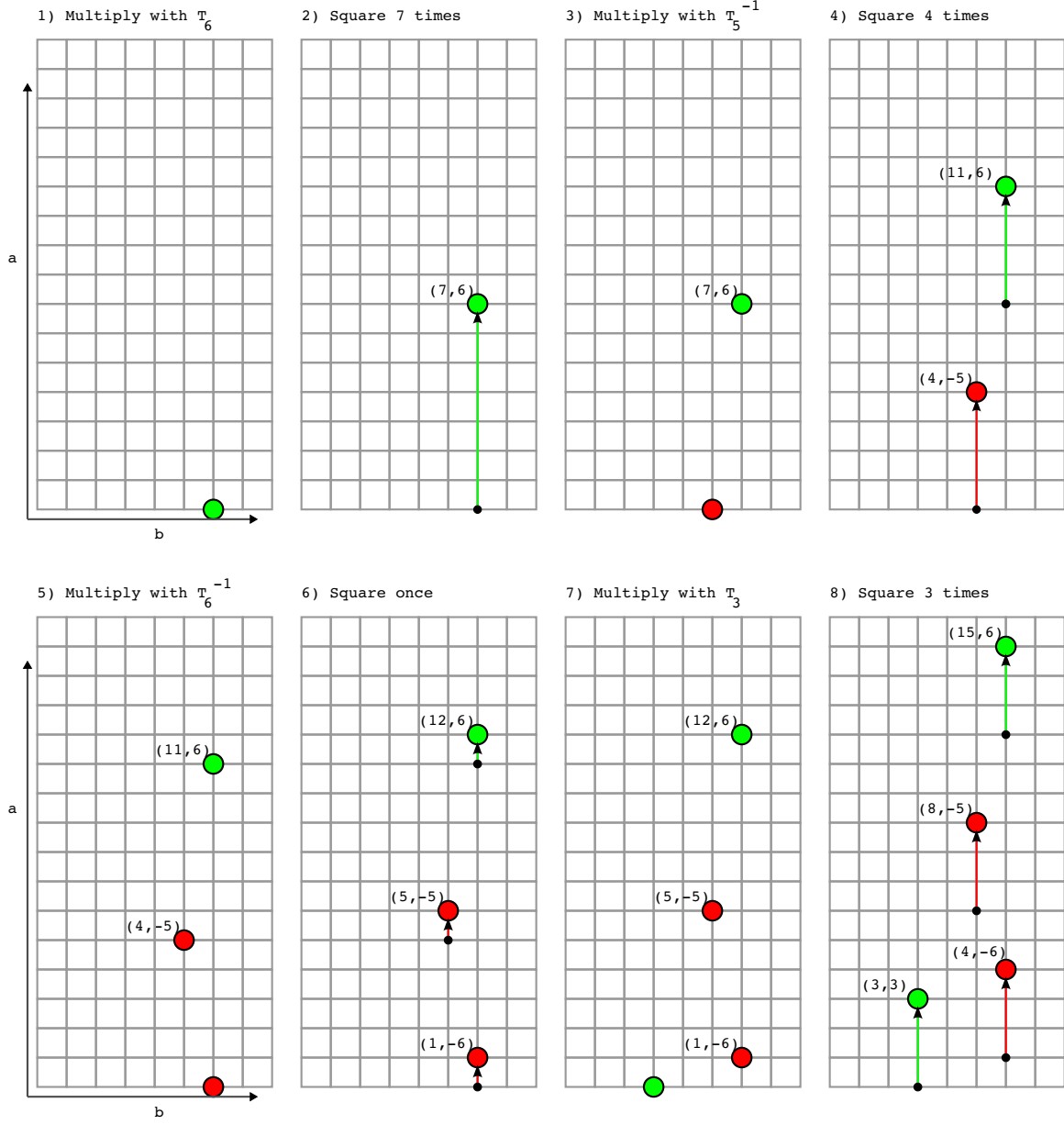


Figure 3.1: The construction of $2^{15}3^6 - 2^83^5 - 2^43^6 + 2^33^3$ using Algorithm 3.3. Steps are executed from left-to-right, top-to-bottom. Positive terms in the exponent are indicated with green, while negative terms use red.

3.4 Methods for Computing 2,3 Chains/Representations

The section discusses some of the methods from the literature for computing 2,3 representations. The first method generates strict chains from low order to high order (right-to-left), while the second method generates representations (both chained and unchained) from high order to low order (left-to-right). The third technique generates strict chains using a tree-based approach, while the final method computes additive only strict chains of shortest length in a manner similar to chains generated from low order to high order.

These methods trade off the time to compute a representation against the time to exponentiate using that representation. When the exponent is known in advance, the corresponding chain or representation best suited to the application can also be computed in advance. Chapter 4 discusses two factoring algorithms that use representations of exponents, computed in advance, in order to speed their computations. While none of the methods presented in this chapter take into account the relative cost of multiplying, squaring, or cubing ideals, Chapter 7 looks at additional techniques based on these methods that attempt to minimize the cost of exponentiation given the average costs of group operations.

3.4.1 Right-to-Left Chains (from low-order to high-order)

The first method we present computes a strictly chained 2,3 partition that is generated from low order to high order and is from Ciet et al [18, Figure 8]. We begin by recalling the technique for binary exponentiation that computes from low order to high order. Given an element $g \in G$ and an integer n , the function

$$\text{bin}(g, n) = \begin{cases} 1 & \text{if } n = 0 \\ \text{bin}(g, n/2)^2 & \text{if } n > 0 \text{ and } n \equiv 0 \pmod{2} \\ \text{bin}(g, n-1) \cdot g & \text{if } n > 0 \text{ and } n \equiv 1 \pmod{2} \end{cases}$$

will compute the binary exponentiation of g^n from low order to high order. This algorithm repeatedly removes factors of 2 from n . When n is not divisible by 2, it subtracts 1 such

that the input to the recursive call will be divisible by 2. The recursion terminates with the base case of $n = 0$.

This concept is extended to a 2,3 number system by repeatedly removing factors of 2 from n , and then factors of 3 from n . At this point, either $n \equiv 1 \pmod{6}$ or $n \equiv 5 \pmod{6}$. When $n \equiv 1 \pmod{6}$, we recurse on $n - 1$ and the input will be divisible by both 2 and 3. When $n \equiv 5 \pmod{6}$, we recurse on $n + 1$. Again, the input to the recursive call will be divisible by both 2 and by 3. Using this idea, we perform a 2,3 exponentiation recursively as

$$\text{rtl}(g, n) = \begin{cases} 1 & \text{if } n = 0 \\ \text{rtl}(g, n/2)^2 & \text{if } n > 0 \text{ and } n \equiv 0 \pmod{2} \\ \text{rtl}(g, n/3)^3 & \text{if } n > 0 \text{ and } n \equiv 0 \pmod{3} \\ \text{rtl}(g, n - 1) \cdot g & \text{if } n > 0 \text{ and } n \equiv 1 \pmod{3} \\ \text{rtl}(g, n + 1) \cdot g^{-1} & \text{if } n > 0 \text{ and } n \equiv 2 \pmod{3}. \end{cases}$$

Algorithm 3.4 describes a non-recursive function with group operations that correspond to those generated by the function rtl . The idea is as follows: let $a = 0$, $b = 0$, and $i = 1$. While $n > 0$, repeatedly remove factors of 2 from n and increment a for each factor of 2 removed. Then repeatedly remove factors of 3 from n and increment b for each factor of 3 removed. At this point, either $n \equiv 1 \pmod{6}$ or $n \equiv 5 \pmod{6}$ and so continue on $n - 1$ or $n + 1$ respectively. When we continue on $n - 1$, this corresponds to adding the current term, so we set $s_i = 1$, and when we continue on $n + 1$, this corresponds to subtracting the current term, so we set $s_i = -1$. Let $a_i = a$ and $b_i = b$ and then increment i and repeat this process while $n > 0$. We then use Algorithm 3.2 to compute the exponentiation given the strictly chained 2,3 partition. When we are not able to compute the chain in advance, it is relatively straightforward to interleave the computation of the partition with the computation of the exponentiation, since the terms $s_i 2^{a_i} 3^{b_i}$ are computed in increasing order for $i = 1..k$.

To see the correctness of the above procedure, consider a modification to the recursive

Algorithm 3.4 2,3 strict chains from low order to high order (Ciet et al [18]).

Input: $n \in \mathbb{Z}_{\geq 0}$

Output: $(a_1, b_1, s_1), \dots, (a_k, b_k, s_k)$ satisfying $n = \sum_{i=1}^k s_i 2^{a_i} 3^{b_i}$ with $s_i \in \{1, -1\}$, $a_i < a_{i+1}$, and $b_i < b_{i+1}$ for $1 \leq i < k$

```

1:  $(a, b) \leftarrow (0, 0)$ 
2:  $i \leftarrow 1$ 
3: while  $n > 0$  do
4:   while  $n \equiv 0 \pmod{2}$  do
5:      $n \leftarrow n/2, a \leftarrow a + 1$ 
6:   while  $n \equiv 0 \pmod{3}$  do
7:      $n \leftarrow n/3, b \leftarrow b + 1$ 
8:   if  $n \equiv 1 \pmod{3}$  then
9:      $n \leftarrow n - 1, s \leftarrow 1$ 
10:  else if  $n \equiv 2 \pmod{3}$  then
11:     $n \leftarrow n + 1, s \leftarrow -1$ 
12:     $(s_i, a_i, b_i) \leftarrow (s, a, b)$ 
13:     $i \leftarrow i + 1$ 
14:   $k \leftarrow i$ 
15: return  $(a_1, b_1, s_1), \dots, (a_k, b_k, s_k)$ 

```

function rtl such that it returns a partition of the input n as a list of terms $s_i 2^{a_i} 3^{b_i}$. When the result of the recursive call is squared, this corresponds to incrementing a_i in each term of the list. Similarly, when the result is cubed, this corresponds to incrementing b_i in each term of the list. When the result is multiplied with g , we prepend a term of $+1$ to the partition, and when the result is multiplied with g^{-1} , we prepend a term of -1 to the partition. On each iteration of the loop, either $n \equiv 0 \pmod{2}$ or $n \equiv 0 \pmod{3}$, so either a increases or b increases. Since every term $|s_i 2^{a_i} 3^{b_i}|$ is strictly less than $|s_j 2^{a_j} 3^{b_j}|$ when $i < j$, the partition is strictly chained.

3.4.2 Left-to-Right Representations (from high-order to low-order)

The previous section gives a procedure for generating a strictly chained 2,3 partition for an integer n such that the terms are ordered from smallest absolute value to largest. Here we present a greedy approach, suggested by Berthé and Imbert [16], which generates the terms in order of the largest absolute value to the smallest. The idea is to find a term, $s 2^a 3^b$, that

is closest to the remaining target integer n and then repeat on $n - s2^a3^b$. Let

$$\text{closest}(n) = s2^a3^b$$

such that $a, b \in \mathbb{Z}_{\geq 0}$ minimize $||n| - 2^a3^b|$. When $n < 0$, let $s = -1$, otherwise let $s = 1$. A recursive function to compute a 2,3 representation greedily is

$$\text{greedy}(n) = \begin{cases} 0 & \text{if } n = 0 \\ \text{closest}(n) + \text{greedy}(n - \text{closest}(n)) & \text{otherwise.} \end{cases}$$

Note that the representation generated may not be a chained partition. To generate a chained partition we restrict the maximum powers of 2 and 3 generated by the function closest. We bound the function closest, such that it returns the triple

$$\text{closest}'(n, A, B) = (s2^a3^b, a, b)$$

where $0 \leq a \leq A$, $0 \leq b \leq B$, a and b minimize $||n| - 2^a3^b|$, and $s = -1$ when $n < 0$ and $s = 1$ when $n > 0$. Our recursive function is then

$$\text{greedy}'(n, A, B) = \begin{cases} 0 & \text{if } n = 0 \\ v + \text{greedy}'(n - v, a, b) & \text{where } (v, a, b) = \text{closest}'(n, A, B). \end{cases}$$

We present pseudo-code in Algorithm 3.5. Note that on successive invocations of greedy' , the absolute value of $v = |s2^a3^b|$ returned by $\text{closest}'$ is monotonically decreasing. Reversing the terms of the partition gives a chained 2,3 partition of n that we can use to perform exponentiation using Algorithm 3.2.

To compute the 2,3 term closest to n , a straightforward approach is to compute the set

$$V = \{2^a3^b, 2^{a+1}3^b : 0 \leq b \leq B \leq \lceil \log_3 |n| \rceil, a = \lfloor \log_2 |n|/(3^b) \rfloor \text{ when } a \leq A\}.$$

Then take the element $v \in V$ that is closest to $|n|$, and take $s \in \{-1, 1\}$ based on the sign of n . Since the set V contains $O(\log |n|)$ elements, computing the term closest to n by this method takes $\Omega(\log |n|)$ steps. When a and b are not constrained (i.e. $A \geq \lceil \log_2 |n| \rceil$ and

Algorithm 3.5 Greedy left to right representation (Berthé and Imbert [16]).

Input: $n \in \mathbb{Z}$, $A, B \in \{\mathbb{Z}_{\geq 0}, +\infty\}$ $\{+\infty \text{ for unbounded } a \text{ or } b\}$
Output: $(s_1, a_1, b_1), \dots, (s_k, a_k, b_k)$ satisfying $n = \sum_{i=1}^k s_i 2^{a_i} 3^{b_i}$, $s_i \in \{1, -1\}$, $a_i, b_i \in \mathbb{Z}_{\geq 0}$.
1: $L \leftarrow$ empty list
2: **while** $n \neq 0$ **do**
3: compute integers a and b that minimize $||n| - 2^a 3^b|$
 such that $0 \leq a \leq A$ and $0 \leq b \leq B$
4: $s \leftarrow -1$ when $n < 0$ and 1 otherwise
5: push (s, a, b) on to the front of L
6: optionally set $(A, B) \leftarrow (a, b)$ when a chain is desired
7: $n \leftarrow n - s 2^a 3^b$
8: **return** L

$B \geq \lceil \log_3 |n| \rceil$, and we simply want to compute the 2,3 term closest to n , Berthé and Imbert [16] present a method that requires at most $O(\log \log |n|)$ steps.

They also found that applying a global bound A^* and B^* such that $0 \leq a \leq A^*$ and $0 \leq b \leq B^*$ often lead to representations with a lower density. In the unchained case, recursive calls to `greedy'` use the global values of A^* and B^* rather than the values a and b generated by `closest'`. Finding the best greedy representation is then a matter of iterating over the global bounds A^* and B^* to compute 2,3 representations constrained appropriately. We discuss some of the results of this in Chapter 7.

3.4.3 Pruned Tree of ± 1 Nodes

The next technique that we discuss for finding strictly chained 2,3 partitions was suggested by Doche and Habsieger [28]. The idea is similar to the method for generating chains from right to left as described in Subsection 3.4.1 above, but differs by generating multiple values that may be further reduced by powers of 2 and 3. The procedure is given in Algorithm 3.6. The idea is to maintain a tree, T , with at most L leaf nodes. At each iteration, each leaf node $v \in T$ generates two new leaves, $v - 1$ and $v + 1$, which are then reduced as much as possible by removing factors of 2 and 3. We then discard any duplicate nodes and all but the smallest L elements generated. The path from the root to the first leaf with a value of

1 represents a chained 2,3 partition of the number n .

Algorithm 3.6 Chain from ± 1 Pruned Tree (Doche and Habsieger [28]).

Input: $n \in \mathbb{Z}_{>0}$ and a bound $L \in \mathbb{Z}_{>0}$.

Output: $(a_1, b_1, s_1), \dots, (a_k, b_k, s_k)$ satisfying $n = \sum_{i=1}^k s_i 2^{a_i} 3^{b_i}$ with $s_i \in \{1, -1\}$, $a_i, b_i \in \mathbb{Z}_{\geq 0}$.

- 1: $T \leftarrow$ a binary tree on the node n
 - 2: **while** no leaf is 1 **do**
 - 3: **for all** leaf nodes $v \in T$ **do**
 - 4: insert as a left child $(v - 1)$ with all factors of 2 and 3 removed
 - 5: insert as a right child $(v + 1)$ with all factors of 2 and 3 removed
 - 6: discard any duplicate leaves
 - 7: discard all but the smallest L leaves
 - 8: **return** the chained 2,3 partition generated by the path from the root to the first leaf node containing 1
-

Larger values of L sometimes produce chains with fewer terms, but take longer to compute. When the input integer n is known in advance, this might not be a problem, however, large values of L can still be prohibitively expensive. Empirically, the authors found that $L = 4$ was a good compromise between the length of the chain generated and the time to compute the chain.

3.4.4 Shortest Additive 2,3 Chains

In the previous subsection, the search for a 2,3 chain iterates on ± 1 the value of the L smallest candidates. When we further restrict a chain to contain only positive terms, the number of possible 2,3 chains is reduced. Imbert and Philippe [36, §3] considered searching for additive 2,3 strictly chained partitions that contain as few terms as possible. They gave the following recursive function to compute the minimum number of terms in such a chain.

Let $s(n)$ denote the smallest k such that n can be represented as $n = \sum_{i=1}^k 2^{a_i} 3^{b_i}$. They

define $s(n)$ as

$$s(n) = \begin{cases} \min\{s(n/3), s(n/2)\} & \text{when } n \equiv 0 \pmod{6} \\ 1 + s(n-1) & \text{when } n \equiv 1 \pmod{6} \\ s(n/2) & \text{when } n \equiv 2 \pmod{6} \\ \min\{s(n/3), 1 + s((n-1)/2)\} & \text{when } n \equiv 3 \pmod{6} \\ \min\{s(n/2), 1 + s((n-1)/3)\} & \text{when } n \equiv 4 \pmod{6} \\ 1 + s((n-1)/2) & \text{when } n \equiv 5 \pmod{6} \end{cases}$$

where the base cases are handled by $s(n) = 1$ when $n \leq 2$.

The corresponding 2,3 chain is computed by memoizing a shortest chain for each solution to $s(n)$ encountered. When a recursive call uses $n/2$, the chain for n is the chain for $n/2$ with each term multiplied by 2. Similarly, if the recursion uses $n/3$, each term is multiplied by 3. When the recursion uses $n-1$, we simply add the term 1 to the chain representing $n-1$.

3.5 Summary

This chapter outlined some exponentiation techniques from the literature. We started with binary exponentiation based on a binary representation of the exponent. Next we described non-adjacent form using a signed base 2 encoding of the exponent. Since cubing is often faster than combined multiplication with squaring, we discussed 2,3 number systems where an integer can have many representations as the sum of the products of 2 and 3. Exponentiation based on 2,3 representations fall under two classes: chained and unchained. Chained representations can typically be computed while interleaved with the exponentiation operation. They also require storage of only a constant number of group elements in addition to the input arguments. Unchained representations often have fewer terms or operations in their representation. Exponentiation of a group element using an unchained representation

of the exponent can be performed using a linear number of group elements in the size of the exponent.

Chapter 7 discusses several variations of chained and unchained 2,3 representations, many of which take into account the average time to multiply, square, and cube elements from the ideal class group. The actual performance of these variations guide our implementation of a factoring algorithm called “SuperSPAR”. In the next chapter, we provide the background for SPAR and the SuperSPAR factoring algorithm.

Chapter 4

SuperSPAR

One contribution of this thesis is to improve the speed of arithmetic in the ideal class group of imaginary quadratic number fields with an application to integer factoring. Chapter 2 describes the ideal class group of imaginary quadratic number fields, and Chapter 3 gives methods for exponentiation in generic groups. This chapter makes a connection between the two and integer factorization. Section 4.1 discusses an algorithm due to Schnorr and Lenstra [53], called SPAR, that uses the ideal class group to factor an integer associated with the discriminant. Section 4.2 expands on SPAR by incorporating a primorial steps algorithm, due to Sutherland [58, §4.1], and discussed in Subsection 4.2.1. When the order of any element from a set is sufficient, Sutherland [58, §5.4] gives an algorithm with subexponential complexity, discussed in Subsection 4.2.2. Finally, Subsection 4.2.3 reconsiders the factoring algorithm SPAR in the context of primorial steps for order finding. We call this new algorithm “SuperSPAR”.

This chapter uses big and little O notation, which are defined as

$$o(g(n)) = \{f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0} \mid \forall c_2 > 0, \exists n_0 > 0, \forall n \geq n_0 : 0 \leq f(n) \leq c_2 g(n)\}$$

$$O(g(n)) = \{f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0} \mid \exists c_2 > 0, \exists n_0 > 0, \forall n \geq n_0 : 0 \leq f(n) \leq c_2 g(n)\}.$$

4.1 SPAR

SPAR is an integer factoring algorithm that works by finding a reduced ambiguous class with a discriminant associated with the integer to be factored. The algorithm was published by Schnorr and Lenstra [53], but is named SPAR after Shanks, Pollard, Atkin, and Rickert [45, p.484].

4.1.1 Ambiguous Classes and the Factorization of the Discriminant

The description of SPAR (see [53]) uses the group of equivalence classes of binary quadratic forms – a group isomorphic to the ideal class group of imaginary quadratic fields (see Frölich and Taylor [30]). Since this thesis discusses the ideal class group, SPAR is described in that setting here. This thesis uses reduced representatives for elements of the ideal class group, and the equivalence class $[\mathfrak{a}]$ for a reduced representative $\mathfrak{a}$ is denoted using the $\mathbb{Z}$ -module $\mathfrak{a} = [a, (b + \sqrt{\Delta})/2]$. Our implementation also maintains a third term $c = (b^2 - \Delta)/4a$, and we represent an ideal class using the triple (a, b, c) .

Definition 4.1.1. The *ambiguous classes* are the classes $[\mathfrak{a}]$ such that $[\mathfrak{a}]^2$ is the identity class [53, p.302]. Notice that the identity ideal class $[\mathcal{O}_\Delta] \in Cl_\Delta$ is an ambiguous class.

According to [53, p.303], every reduced representative of an ambiguous class with negative discriminant has either $b = 0$, $a = b$, or $a = c$. Since our implementations use the triple (a, b, c) to represent an ideal class, where the coefficients satisfy $\Delta = b^2 - 4ac$, these reduced representatives correspond to a factorization of the discriminant. For a reduced ambiguous ideal class either

$$\begin{aligned} \Delta &= -4ac && \text{when } b = 0, \\ \Delta &= b(b - 4c) && \text{when } a = b, \text{ or} \\ \Delta &= (b - 2a)(b + 2a) && \text{when } a = c. \end{aligned}$$

Suppose we wish to find a factor of an odd integer N . Since $\Delta = b^2 - 4ac$ we must have $\Delta \equiv 0, 1 \pmod{4}$. Therefore, for some square-free integer k , let $\Delta = -kN$ when $-kN \equiv 1 \pmod{4}$ and $\Delta = -4kN$ otherwise. Now to find a factor of N , we find a reduced ambiguous class representative with discriminant Δ . If $b = 0$ or $a = b$, we compute $d = \gcd(a, N)$, and otherwise we compute $d = \gcd(b - 2a, N)$ (in Chapter 5 we discuss how to compute $\gcd(N, m)$). Unfortunately, this is not guaranteed to find a non-trivial factor of N . The ambiguous class may lead to the trivial factorization, $1 \cdot \Delta$, but also, since the discriminant

$\Delta = -kN$ or $\Delta = -4kN$, the ambiguous class may lead to a factorization of Δ but not N . Since the multiplier k is typically small, future work could use divisibility tests to determine if the ambiguous class leads to a factor of N .

4.1.2 SPAR Algorithm

The SPAR factoring algorithm uses two stages, an exponentiation stage and a search stage, in order to find an ambiguous class with a discriminant associated with the integer to factor. Recall from Section 2.2 that the ideal class group Cl_Δ has a finite number of elements. This means that for a random ideal class $[\mathfrak{a}]$, there exists an integer h such that $[\mathfrak{a}]^h = [\mathcal{O}_\Delta]$. We say that h is the *order* of the element $[\mathfrak{a}]$ and denote this by $h = \text{ord}([\mathfrak{a}])$. When the order is even, then $[\mathfrak{b}] = [\mathfrak{a}]^{h/2}$ is an ambiguous ideal class, because $[\mathfrak{b}]^2 = [\mathcal{O}_\Delta]$. Let $h = 2^j h_0$ with h_0 odd. The exponentiation stage of SPAR chooses an exponent E such that, with a certain probability, E is a multiple of h_0 , and by repeated squaring of $[\mathfrak{a}]^E$, the algorithm will find an ambiguous class.

If the algorithm fails to find an ambiguous class as a result of the exponentiation stage, the algorithm continues with the search stage. The search stage uses at most the same number of group operations as in the exponentiation stage, and attempts to find a multiple of the order of $[\mathfrak{a}]$ by performing a random walk on the ideal class generated by the exponentiation stage. Knowing a multiple of the order of $[\mathfrak{a}]$, the algorithm then attempts to find an ambiguous class and a factor of N . If the search stage fails, the algorithm starts over using a different square-free multiplier k .

Following Schnorr and Lenstra [53], the exponentiation stage takes the first t primes $p_1 = 2, p_2 = 3, \dots, p_t \leq N^{1/2r}$ for $r = \sqrt{\ln N / \ln \ln N}$. Let $E = \prod_{i=2}^t p_i^{e_i}$ for $e_i = \max\{v : p_i^v \leq p_t^2\}$ and compute $[\mathfrak{b}] = [\mathfrak{a}]^E$. Notice that we exponentiate $[\mathfrak{a}]$ to the product of only *odd* prime powers. The reason for this is that if $\text{ord}([\mathfrak{a}])$ divides $2^j E$ for some j , then we can compute $[\mathfrak{b}]^{2^j}$ for the smallest j such that $[\mathfrak{b}]^{2^j} = [\mathcal{O}_\Delta]$. It follows that $[\mathfrak{b}]^{2^{j-1}}$ is an ambiguous ideal class and we attempt to factor N . Since $\text{ord}([\mathfrak{a}])$ may not divide $2^j E$,

Schnorr and Lenstra [53, p.291] bound j to be no larger than $\ell = \lfloor \log_2 \sqrt{N} \rfloor$. If this stage fails to find $[\mathbf{b}]^{2^j} = [\mathcal{O}_\Delta]$ for some j , the algorithm continues with the search stage.

The exponentiation stage computes $[\mathbf{b}] = [\mathbf{a}]^E$ and $[\mathbf{c}] = [\mathbf{b}]^{2^\ell}$. The search stage performs a random walk through the cyclic group generated by the ideal class $[\mathbf{c}]$ in an attempt to find the order $h = \text{ord}([\mathbf{c}])$. Let $\langle [\mathbf{c}] \rangle$ denote the group generated by $[\mathbf{c}]$ and let $f : \langle [\mathbf{c}] \rangle \rightarrow \langle [\mathbf{c}] \rangle$ be a function from one element in the group to another. The function f should have the property that if x is known for some $[\mathbf{c}]^x$, then y can be determined for $[\mathbf{c}]^y = f([\mathbf{c}]^x)$. Let $[\mathbf{c}_1] = [\mathbf{c}]$ and repeatedly compute

$$[\mathbf{c}_{i+1}] = f([\mathbf{c}_i])$$

until there is some $j < k$ such that $[\mathbf{c}_j] = [\mathbf{c}_k]$. By the function f , we compute u and v such that $[\mathbf{c}_j] = [\mathbf{c}]^u$ and $[\mathbf{c}_k] = [\mathbf{c}]^v$. Then $h = v - u$ is a multiple of the order of $[\mathbf{c}]$, and we attempt to find an ambiguous class by computing $[\mathbf{d}]^{2^j}$ for $[\mathbf{d}] = [\mathbf{b}]^h$ and the smallest such $j \leq \lfloor \log_2 \sqrt{N} \rfloor$.

4.1.3 SPAR Complexity

The original publication of SPAR by Schnorr and Lenstra [53] claimed that every composite integer N could be factored in $o\left(\exp \sqrt{\ln N \ln \ln N}\right)$ bit operations in expectation. This was the first factoring algorithm for which this runtime had been conjectured, and it was also the first for which this conjecture had to be withdrawn [45].

The first stage of the algorithm exponentiates a random ideal class $[\mathbf{a}] \in Cl_\Delta$ to the product $\prod_{i=2}^t p_i^{e_i}$ of the first t primes where $e_i = \max\{v : p_i^v \leq p_t^2\}$. Using binary exponentiation, this takes $O(p_t)$ group operations since there are about $p_t / \log p_t$ prime powers in the product, each of which is at most $\lceil 2 \log_2 p_t \rceil$ in size. According to [53, p.290], for a random composite integer $m \in [0, N]$, this stage will factor m with probability $\geq r^{-r}$. The search stage performs a random walk of at most $O(p_t)$ group operations and with probability $\geq (r-2)^{-(r-2)}$ will factor m [53, p.290]. Schnorr and Lenstra [53, p.290] claim that if the

exponentiation stage is run on each integer kN for $k \leq r^r$, then every composite integer N will be factored within $o\left(\exp\sqrt{\ln N \ln \ln N}\right)$ bit operations.

This claim was based on a false assumption – for a complete discussion, see Lenstra and Pomerance [45, § 11]. In short, the original assumption was that for fixed N and variable k , the class number (h_Δ for $\Delta = -kN$) was as likely to have no prime divisors larger than p_t as the class number associated with a random discriminant of approximately the same size. This assumption meant that one could take both k and p_t to be no larger than $N^{1/2r} = \exp\left(\frac{1}{2}\sqrt{\ln N \ln \ln N}\right)$, leading to an upper bound of $\exp\left(\sqrt{\ln N \ln \ln N}\right)$ for the expected running time. Lenstra and Pomerance showed [45, §11] that this assumption is incorrect for a sufficiently dense sequence of integers, however, future work (Section 9.3) could use a median case analysis of SPAR, which might show a subexponential median case running time.

4.2 SuperSPAR

This section discusses SuperSPAR – an integer factoring algorithm and our motivation for improving the performance of exponentiation in the ideal class group. Similar to SPAR, SuperSPAR attempts to find an ambiguous class with a discriminant associated with N , the odd integer we wish to factor. Also similar is that SuperSPAR operates in two stages: an exponentiation stage and a search stage. The first stage of SuperSPAR is the same as in SPAR, although we empirically optimize the choice of the exponent in this stage (see Sections 8.3 and 8.6). The second stage of SuperSPAR differs from SPAR, however. In both algorithms, this stage attempts to find the order of an ideal class. Schnorr and Lenstra [53, p.294] suggest a Pollard-Brent recursion [17] based on Pollard’s Rho method [49], but also remark [53, p.298] that Shanks’ baby-step giant-step method [55] could be used to deterministically find the order. SuperSPAR instead uses a bounded primorial steps algorithm, due to Sutherland [58, §4.1], and discussed in Subsection 4.2.1 (later in Section 8.6

we discuss the empirical optimisation of parameters for the search stage of SuperSPAR). In the case of SuperSPAR, any ideal class with a discriminant associated with N is a candidate for factoring N . For this reason, the algorithm does not need to find the order of a particular ideal class, but the order of any of several ideal classes will suffice. Sutherland suggests a bounded primorial steps algorithm in this context [58, §5.4], which we describe in Subsection 4.2.1. Finally, Subsection 4.2.3 gives a high level description of the operation of SuperSPAR.

4.2.1 Bounded Primorial Steps

The bounded primorial steps algorithm [58] is an order finding algorithm for generic groups requiring $o(\sqrt{M})$ group operations in the worst case where M is a bound on the order of the group element. This means that for sufficiently large values of M , a bounded primorial steps algorithm will require fewer group operations in the worst case than previously known order finding algorithms for generic groups, such as the Pollard-Brent method [17] and Shanks' baby-step giant-step method [55], both of which require at most $O(\sqrt{M})$ group operations in the worst case. The algorithm improves upon Shanks' baby-step giant-step method by using a group element whose order is known to not have any small divisors.

Shanks' baby-step giant-step method computes the order of an element α in a group G . Given a bound M on the order of the element α , let $s = \lceil \sqrt{M} \rceil$ and compute $\alpha^1, \alpha^2, \dots, \alpha^s$ storing the each key/value pair (α^i, i) in a table – these are the baby steps. If any $\alpha^i = 1_G$, then $\text{ord}(\alpha) = i$ and we are finished. Otherwise compute $\alpha^{2s}, \alpha^{3s}, \dots$. For each α^{js} , if α^{js} is in the table, then $\alpha^{js} = \alpha^i$ for some i and $js - i$ is the order of α . These are the giant steps. Notice that s is chosen such that after an equal number of baby steps and giant steps, the last giant step has an exponent $s^2 \geq M$. Following Sutherland [58, p.50], we point out that if computing the inverse of an element is cheaper than multiplying two elements, the number of multiplications is reduced by letting $s = \lceil \sqrt{M/2} \rceil$ and computing the giant steps $\alpha^{2s}, \alpha^{-2s}, \alpha^{4s}, \alpha^{-4s}, \dots$ instead.

Sutherland observed [58, p.56] that if we know that $h = \text{ord}(\alpha)$ is odd, then computing

only odd powers $\alpha^1, \alpha^3, \dots, \alpha^{s-1}$ for the baby steps is sufficient. We still need to compute giant steps $\alpha^{2s}, \alpha^{3s}, \dots$, for some s that is even since we want to find some $\alpha^{js} = \alpha^i$ where $js - i$ is odd. In this case, $s = \lceil \sqrt{2M} \rceil$ so that after roughly $\sqrt{M/2}$ baby steps and $\sqrt{M/2}$ giant steps, the last giant step has exponent $\geq M$.

The problem is that $\text{ord}(\alpha)$ may not be odd. However, by repeated squaring of α it is easy to find an element whose order is guaranteed to be odd [58, p.56]. Given a bound M on the group order, compute $\beta = \alpha^{2^\ell}$ where $\ell = \lfloor \log_2 M \rfloor$ and now run the modified algorithm on β to find $h' = \text{ord}(\beta)$. The order of α can be found by computing $\zeta = \alpha^{h'}$ and then repeatedly squaring ζ until $\zeta^{2^k} = 1_G$ for some k . The order of α is then $2^k h'$.

Algorithm 4.1 Primorial Steps (Sutherland [58, p.57]).

Input: $\alpha \in G$, a bound $M \geq \text{ord}(\alpha)$, and a fast order algorithm $\mathcal{A}(\alpha, E)$.

Output: h satisfying $\alpha^h = 1_G$.

```

1: maximize  $w$  such that  $P_w \leq \sqrt{M}$ 
2: maximize  $m$  such that  $m^2 P_w \phi(P_w) \leq M$ 
3:  $s = mP_w$ 
4:  $E = \prod_{i=1}^n p_i^{\lfloor \log_{p_i} M \rfloor}$ 
5:  $\beta \leftarrow \alpha^E$ 
6: for  $i$  from 1 to  $s$  where  $i$  is coprime to  $P_w$  do
7:   compute  $\beta^i$  and store  $(\beta^i, i)$  in the table                                {baby steps}
8:   if  $\beta^i = 1_G$  then
9:     return  $i \cdot \mathcal{A}(\alpha^i, E)$ 
10: for  $j = 2s, 3s, \dots$  do
11:   if  $(\beta^j, i)$  is in the table for some  $i$  then                                {giant steps}
12:      $h' = j - i$ 
13:     return  $h' \cdot \mathcal{A}(\alpha^{h'}, E)$ 

```

This approach is extended to computing $\beta = \alpha^E$ where $E = 2^{\lfloor \log_2 M \rfloor} 3^{\lfloor \log_3 M \rfloor}$ and the order of β is coprime to both 2 and 3. In this case, compute baby steps with exponents coprime to 6 and giant steps that are a multiple of 6. More generally, this works for any primorial P_w such that

$$P_w = 2 \times 3 \times \dots \times p_w = \prod_{i=1}^w p_i$$

where p_i is the i^{th} prime. Following Sutherland [58, p.57], select the largest $P_w \leq \sqrt{M}$

and then maximize m such that $m^2 P_w \phi(P_w) \leq M$, where $\phi(P_w)$ is the number of integers coprime to P_w given as

$$\phi(P_w) = (2-1) \times (3-1) \times \cdots \times (p_w-1) = \prod_{i=1}^w (p_i-1). \quad (4.1)$$

Let $e_i = \lfloor \log_{p_i} M \rfloor$ for $1 \leq i \leq w$ and $E = 2^{e_2} \times 3^{e_3} \times \cdots \times p_w^{e_w}$. Then compute $\beta = \alpha^E$. The bound on the largest baby step is $s = mP_w$, and we compute $h' = \text{ord}(\beta)$ by taking at most $m\phi(P_w)$ baby steps coprime to P_w and at most $m\phi(P_w)$ giant steps of size mP_w . Pseudo-code for the bounded Primorial Steps technique is given in Algorithm 4.1. By [58, p.59 Proposition 4.2], the number of group operations in the worst case is bound by $O(\sqrt{M/\log \log M})$.

To compute $h = \text{ord}(\alpha)$ given $h' = \text{ord}(\beta)$, one uses a fast order finding algorithm. One fast order finding algorithm, $\mathcal{A}(\alpha^{h'}, E)$, uses the known factorization of $E = \prod p_i^{e_i}$. Notice that $\alpha^{Eh'} = \beta^{h'} = 1_G$. The idea is to iterate on the factors of E , removing each factor p before computing $\zeta = \alpha^{E'h'}$ for the quotient $E' = E/p$. If $\zeta \neq 1_G$, then $\text{ord}(\alpha)$ does not divide $E'h'$ and p must be a factor of the order of α . The algorithm then continues with the next prime factor of E' . Additional fast order finding algorithms are given in [58, Chapter 7].

4.2.2 Primorial Steps for a Set

Both SPAR and SuperSPAR use the order of an ideal class to find an ambiguous ideal class. For this reason, their success in factoring an integer is not limited to computing the order of a particular ideal class, but the order of any ideal class from a set of ideal classes may work. More generally, let $\{\alpha_i \in G_i\}$ be a set of elements from different groups such that the order of α_i is distributed uniformly at random on the interval $[1, M]$ where M is a bound on the largest order of the elements α_i . When the order of any element α_i from the set will suffice, Sutherland [58, §5.4] gives an algorithm with subexponential complexity.

To begin, an integer x is y -smooth if all of its prime factors are no larger than y . By [58, p.81], the probability that a random integer x is $x^{1/u}$ smooth is $u^{-u+o(1)}$. Assuming

that there is some α_i such that $\text{ord}(\alpha_i)$ is $M^{1/u}$ smooth, let $M' = M^{2/u}$ and then attempt to compute $\text{ord}(\alpha_i)$ using M' as a bound for the bounded primorial steps algorithm. This will use $o(M^{1/u})$ group operations since the bounded primorial steps algorithm uses $o(\sqrt{M'})$ operations for a bound M' . If the algorithm fails to find the order of α_i , we try again for the next α_{i+1} in the set. Using this approach, according to [58, pp.81–82] the expected running time to find the order of some α_i is approximately

$$u^{u+o(1)} M^{1/u} = \exp \left(\frac{1}{u} \log M + u \log u + o(1) \right).$$

The cost is minimized for $u \approx \sqrt{2 \log M / \log \log M}$, which gives an expected running time of

$$\exp \left(\left(\sqrt{2} + o(1) \right) \sqrt{\log M \log \log M} \right).$$

Notice that the idea behind the SPAR factoring algorithm is to find an ambiguous ideal for one of the class groups with valid discriminant $\Delta = -kN$ for $1 \leq k \leq r^r$ where $r = \sqrt{\ln N / \ln \ln N}$. In this case, the success of factoring a composite integer N is not limited to finding an ambiguous ideal within a single group, but instead to finding an ambiguous ideal from any of several groups. For this reason, the above approach is directly applied to SuperSPAR factoring algorithm. Unfortunately, as shown by Lenstra and Pomerance [45, § 11], there exist integers N for which there is no multiplier k such that the class group Cl_Δ for $\Delta = -kN$ is sufficiently smooth. Such integers have large square factors, and in the next subsection (and throughout this thesis) we assume that the integers we wish to factor are square-free. Such square-free integers are also likely to occur as left-overs from the sieve stage of the number field sieve, which is one possible application of this work.

4.2.3 SuperSPAR Algorithm

As mentioned previously, SuperSPAR works in two stages: an exponentiation stage and a search stage. First, let N be the odd composite integer that we wish to factor. Then, for some square-free integer k , choose a discriminant $\Delta = -kN$ or $\Delta = -4kN$ such that

$\Delta \equiv 0, 1 \pmod{4}$. The exponentiation stage of SuperSPAR is the same as in SPAR, but we quickly recap this here.

In the exponentiation stage of SuperSPAR, we take the first t primes $p_1 = 2, p_2 = 3, \dots, p_t \leq N^{1/2r}$ for $r = \sqrt{\ln N / \ln \ln N}$, let $E = \prod_{i=2}^t p_i^{e_i}$ be the product of the odd prime powers where $e_i = \max\{v : p_i^v \leq p_t^2\}$, and $[\mathbf{b}] = [\mathbf{a}]^E$ for a random ideal class $[\mathbf{a}] \in Cl_\Delta$. If $[\mathbf{b}] = [\mathcal{O}_\Delta]$, then $\text{ord}([\mathbf{a}])$ divides E and is odd, and we cannot find an ambiguous ideal class from $[\mathbf{a}]$. Then, we try again with a different random ideal class in Cl_Δ . Assuming $[\mathbf{b}] \neq [\mathcal{O}_\Delta]$, we then compute $[\mathbf{b}]^{2^j}$, by repeated squaring, for $1 \leq j \leq \lfloor \log_2 \sqrt{|\Delta|} \rfloor$ or until $[\mathbf{b}]^{2^j}$ is an ambiguous ideal. If there is some j such that $[\mathbf{b}]^{2^j}$ is an ambiguous ideal, we attempt to factor the integer N , and if this fails, we try again for another ideal class in Cl_Δ . If there is no j such that $[\mathbf{b}]^{2^j}$ is an ambiguous ideal class, then we have computed $[\mathbf{c}] = [\mathbf{b}]^{2^\ell}$ for $\ell = \lfloor \log_2 \sqrt{|\Delta|} \rfloor$, and SuperSPAR moves on to the search stage.

The search stage of SuperSPAR differs from that of SPAR. Using SPAR, one attempts to find the order of $[\mathbf{c}]$ by performing a random walk using at most $O(p_t)$ group operations. With SuperSPAR, on the other hand, one attempts to find the order of $[\mathbf{c}]$ using a variation of the bounded primorial steps algorithm such that the exponent of the final giant step, $m^2 \phi(P_w) P_w$, is as large as possible, while the total number of steps taken is still at most $O(p_t)$. Notice that E and 2^ℓ , from the exponentiation stage, were chosen such that the order of $[\mathbf{c}]$ is likely to be free of prime divisors $\leq p_t$. Let η be the number of group operations performed during the exponentiation stage, and we have $\eta \in O(p_t)$. Then choose $w \leq t$ to be as large as possible such that $c_1 \eta / 2 \leq m \phi(P_w) < c_2 \eta / 2$ for some integer m and constants $c_1, c_2 \in \mathbb{R}$. The choice of c_1 and c_2 are somewhat arbitrary but are needed to ensure that the total number of group operations in the search stage is $O(p_t)$. Letting $c_1 = 1$ and $c_2 = 3/2$ works well in practice. Bounding $w \leq t$ guarantees that we do not take steps coprime to a primorial larger than the one used in the exponentiation stage.

To continue, let $s = m P_w$ and $d = m \phi(P_w)$. Then take baby steps $[\mathbf{c}]^i$ for $1 \leq i \leq s$

with i coprime to P_w . If any $[\mathbf{c}]^i = [\mathcal{O}_\Delta]$, then we have computed the order $h' = i$ of $[\mathbf{c}]$. Otherwise, we compute giant steps $[\mathbf{c}]^j$ for $j = 2s, -2s, 4s, -4s, \dots, 2sd, -2sd$. Notice that this sequence of giant steps is used since computing the inverse in the ideal class group is essentially free. If there exists a giant step $[\mathbf{c}]^j$ such that $[\mathbf{c}]^j = [\mathbf{c}]^i$ for some corresponding baby step $[\mathbf{c}]^i$, then we have computed a multiple of the order $h' = j - i$ of $[\mathbf{c}]$.

If we successfully compute $h' = \text{ord}([\mathbf{c}])$ during the search stage, then assuming that $\text{ord}([\mathbf{b}])$ is even, we compute an ambiguous ideal class by repeated squaring of $[\mathbf{b}]^{h'}$, and then attempt to factor the integer associated with the discriminant. Since the order of $[\mathbf{b}]^{h'}$ might not be even, we do not square more than $\ell = \left\lfloor \log_2 \sqrt{N} \right\rfloor$ times, as in the exponentiation stage. If the order is odd, we choose a different random ideal class and start over with the exponentiation stage. On the other hand, if we fail to compute the order of $[\mathbf{c}]$ during the search stage, then we start from the beginning with a different square-free multiplier k of the integer N .

The work by Lenstra and Pomerance [45, §11] precludes a subexponential running time complexity for both SPAR and SuperSPAR, however, their results dictate a worst case running time. Future work would include a median case complexity analysis of SuperSPAR (Section 9.3), since by [58, p.15] this type of analysis is more relevant when there is a large standard deviation in the running time across inputs. In Chapter 8, we empirically search for parameters for SuperSPAR, such as the exponent E and the multiple of a primorial mP_w , in order to minimize the average running time to factor integers in the range where SuperSPAR is competitive with other factoring algorithms. We also discuss several ways in which our implementation of SuperSPAR differs from the description given here in order to improve the performance of SuperSPAR in practice. Details of our implementation are given in Algorithm 8.4.

4.3 Summary

This chapter discusses SPAR, an integer factoring algorithm based on the ideal class group and our motivation for practical improvements to the performance of arithmetic and exponentiation in the class group. This chapter also discusses a primordial steps algorithm, due to Sutherland [58], which we apply to the search stage of SPAR. The resulting algorithm is referred to as “SuperSPAR”. In the upcoming chapters, we detail our experiments and results to lead to practical improvements in ideal class arithmetic, exponentiation, and finally our implementation of SuperSPAR.

Chapter 5

Extended Greatest Common Divisor Experiments

One contribution of this thesis is an efficient implementation of arithmetic in the ideal class group of imaginary quadratic number fields. Experiments show that roughly half of the computational effort of ideal class multiplication (Algorithm 2.2) is in computing integral solutions to equations of the form

$$s = Ua + Vb$$

where a and b are fixed integers given as input, and s is the greatest common divisor of both a and b . Solutions to this equation are referred to as the *extended greatest common divisor* (or extended GCD for short). A first step to improving the performance of arithmetic in the ideal class group is to improve the performance of extended GCD computations. This chapter discusses several algorithms for computing solutions to the extended GCD.

Section 5.1 discusses the standard extended Euclidean Algorithm, which uses multiplication and division. Binary extended GCD computations, in contrast, emphasize bit shifting instead of multiplication and division. Section 5.2 discusses an extended GCD algorithm that works from low-order to high-order, referred to as *right-to-left* as this is the order in which bits are operated on in the binary representation. Subsection 5.2.1 discusses a windowed variant of this algorithm that operates on several bits of the input for each iteration. An extended GCD can also be computed from high-order to low-order, and such a method is referred to as *left-to-right*. One such technique, suggested by Shallit and Sorenson [54], is discussed in Section 5.3. There we propose a simplification of this algorithm that performs more than twice as fast on average for 128 bit inputs. When the inputs are larger than a single machine word, Lehmer [44] observed that much of the computation of the extended GCD can still be performed using just the most significant machine word of the interme-

diates. Section 5.4 discusses Lehmer’s standard extended GCD as well as some variations that we tried, namely precomputing intermediate solutions for 8 bit machine words and the use of a binary extended GCD for 64 bit machine words. Section 5.5 briefly describes each of these extended GCD computations in the context of the partial extended GCD, which is useful for computing the simple continued fraction expansion used by NUCOMP, NUDUPL, and NUCUBE from Subsections 2.3.3, 2.3.4, and 2.3.5 respectively.

To improve the performance of these algorithms in practice, we specialized much of our implementation for the x64 architecture. The details of this specialization are discussed in Section 5.6. Many of our routines benefit when the input is bounded by a single machine word, i.e. 64 bits, but we were also able to take advantage of integers that fit within two machine words by implementing a custom library for 128 bit arithmetic. When integers are larger than 128 bits, we use the GNU Multiple Precision (GMP) arithmetic library [3]. All the software in this chapter was developed using the GNU C compiler version 4.7.2 on Ubuntu 12.10. Assembly language was used for 128 bit arithmetic and processing features not available in the C language. The hardware platform was a 2.7GHz Intel Core i7-2620M CPU and 8Gb of memory. The CPU has four cores, only one of which was used during timing experiments. Section 5.7 shows the average time to compute the extended GCD for pairs of random positive integers (a, b) where a and b are the same number of bits in size. We use GMP [3], Pari [11], and MPIR [9] as reference implementations.

5.1 The Euclidean Algorithm

The Euclidean Algorithm (see [20, §9.3.2]) is an algorithm for computing the greatest common divisor of two integers. The input is the two positive integers a and b . At each iteration of the algorithm, we subtract the smaller of the two numbers from the larger, until one of them is 0. At this point, the non-zero number is the largest divisor of a and b . Since the smaller number may still be smaller after a single iteration, we use fewer steps by subtracting

an integer multiple of the smaller number from the larger one.

The Euclidean Algorithm is extended by using a system of equations of the form

$$s = Ua + Vb \quad (5.1)$$

$$t = Xa + Yb. \quad (5.2)$$

Initially, let

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} = \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}$$

and Equations (5.1) and (5.2) hold. We maintain the invariant that $s \geq t$. When $s < t$, simply swap the rows in the matrix representation above. At each iteration, subtract $q = \lfloor s/t \rfloor$ times the second row from the first, and then swap rows to maintain the invariant. When $t = 0$, the first row of the matrix is a solution such that s is the largest positive divisor of a and b . The algorithm is given in Algorithm 5.1. Negative inputs a and b are handled by using $a' = |a|$ and $b' = |b|$ as inputs instead and modifying the output such that $U' = U \cdot \text{sign}(a)$ and $V' = V \cdot \text{sign}(b)$ where

$$\text{sign}(x) = \begin{cases} -1 & \text{when } x < 0 \\ 0 & \text{when } x = 0 \\ 1 & \text{when } x > 0. \end{cases}$$

In practice, these operations are performed by manipulating each variable directly, rather than by using matrix arithmetic. Furthermore, we typically implement division with remainder, which solves $s = qt + r$ for $q, r \in \mathbb{Z}$ and $|r| < |t|$. Notice that $r = s - qt$ is the target value of t for each iteration of the Euclidean Algorithm.

5.2 Right-to-Left Binary Extended GCD

The extended Euclidean algorithm uses divide with remainder, which is often expensive. Binary extended GCD algorithms emphasize bit shifting over multiplication and division.

Algorithm 5.1 Extended Euclidean Algorithm.

Input: $a, b \in \mathbb{Z}_{\geq 0}$ **Output:** (s, U, V) satisfying $s = Ua + Vb$ with $s = \gcd(a, b)$.

```
1:  $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}$ 
2: if  $t > s$  then
3:    $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$            {Swap rows. Maintain  $s \geq t$ .}
4: while  $t \neq 0$  do
5:    $q \leftarrow \lfloor s/t \rfloor$ 
6:    $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & -q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$    {Subtract  $q$  times 2nd row and swap.}
7: return  $(s, U, V)$            {Such that  $s = Ua + Vb$ .}
```

Such algorithms may perform better than other extended GCD algorithms in practice (see Section 5.7 for results). Here we describe a binary extended GCD algorithm, originally published by Stein [57], that works from the least significant bit to the most significant bit. We refer to this as *right-to-left* since this is the direction in which we process the written binary representation.

To compute the greatest common divisor of two positive numbers, we repeatedly apply the following identities,

$$\gcd(a, b) = \begin{cases} 2 \cdot \gcd(a/2, b/2) & \text{when both } a \text{ and } b \text{ are even} \\ \gcd(a/2, b) & \text{when only } a \text{ is even} \\ \gcd(a, b/2) & \text{when only } b \text{ is even} \\ \gcd((a-b)/2, b) & \text{when } a \geq b \text{ and both are odd} \\ \gcd((b-a)/2, a) & \text{when } a < b \text{ and both are odd.} \end{cases}$$

In the case that only a is even, we divide a by 2 since 2 is not a common divisor of both. The same is true when only b is even. When both a and b are odd, their difference is even and so is further reduced by 2. Notice that each relation reduces at least one of the arguments and so the recursion terminates with either $\gcd(a, 0) = a$ or $\gcd(0, b) = b$.

When both a and b are even, $\gcd(a, b) = 2 \cdot \gcd(a/2, b/2)$. So the first step of the algorithm

presented here is to remove all common powers of two from a and b . Let r be the number of times 2 is removed from both. Now either a or b or both are odd. If a is even, then swap a and b so that a is guaranteed to be odd. We will compute the extended GCD using this a and b rather than the input values. Using this, the final solution to the original input is $s2^r = Ua2^r + Vb2^r$.

As before, begin with the matrix representation

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} = \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}.$$

An invariant of the algorithm is that s is odd at the beginning of each iteration. Since a was chosen to be odd, s is initially odd. First remove any powers of 2 from t . While t is even, we would like to apply the operation

$$(t, X, Y) \leftarrow \left(\frac{t}{2}, \frac{X}{2}, \frac{Y}{2} \right)$$

but this may result in rational values for X and Y if either are odd. We first point out that

$$\begin{aligned} t &= Xa + Yb \\ &= Xa + Yb + (ab - ab) \\ &= (X + b)a + (Y - a)b. \end{aligned}$$

Using this, we can simultaneously add b to X and subtract a from Y when it suits us. To continue, we use the following theorem and proof, which we were unable to find in the literature.

Theorem 5.2.1. Assume that a is odd. When t is even, either both X and Y are even, or Y is odd and both $X + b$ and $Y - a$ are even.

Proof. Assume t is even and that Y is odd. We have

$$\begin{aligned} t &\equiv Xa + Yb && (\text{mod } 2) \\ \Rightarrow 0 &\equiv X + b && (\text{mod } 2) \quad \{\text{Since } t \text{ is even and } Y \text{ and } a \text{ are odd.}\} \\ \Rightarrow 0 &\equiv X + b \equiv Y - a && (\text{mod } 2) \quad \{\text{Since } Y - a \text{ is even.}\}. \end{aligned}$$

Now assume t is even and that X is odd. We have

$$\begin{aligned}
& t \equiv Xa + Yb \pmod{2} \\
\Rightarrow & 0 \equiv 1 + Yb \pmod{2} \quad \{\text{Since } t \text{ is even and } X \text{ and } a \text{ are odd.}\} \\
\Rightarrow & 1 \equiv Yb \pmod{2} \quad \{\text{Both } Y \text{ and } b \text{ are odd.}\} \\
\Rightarrow & 0 \equiv X + b \equiv Y - a \pmod{2}.
\end{aligned}$$

Therefore, if t is even, either both X and Y are even, or Y is odd and both $X + b$ and $Y - a$ are even. $\square$

By Theorem 5.2.1, we have a way to reduce t by 2 and maintain integer coefficients.

While t is even,

$$(t, X, Y) \leftarrow \begin{cases} \left(\frac{t}{2}, \frac{X}{2}, \frac{Y}{2}\right) & \text{if } Y \text{ is even} \\ (t, X, Y) \leftarrow \left(\frac{t}{2}, \frac{X+b}{2}, \frac{Y-a}{2}\right) & \text{otherwise.} \end{cases}$$

At this point, both s and t are odd. If $s \geq t$ then let

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$$

otherwise let

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ -1 & 1 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}.$$

This ensures that s is odd, t is even, and that both s and t are positive. Repeat the steps of reducing t by powers of 2 and then subtracting one row from the other until $t = 0$. The complete algorithm is given in Algorithm 5.2. Note that in practice, integer division by 2 is performed using a bit shift right.

5.2.1 Windowed Right-to-Left Binary Extended GCD

Windowing is a common technique used to extend the base of an algorithm. We saw this earlier in our discussion of binary exponentiation in Section 3.1. The idea there was to

Algorithm 5.2 Right-to-left Binary Extended GCD (Stein [57]).

Input: $a, b, \in \mathbb{Z}_{>0}$.

Output: (s, U, V) satisfying $s = Ua + Vb$ with $s = \gcd(a, b)$.

```
1: let  $r$  be the largest integer such that  $2^r$  divides both  $a$  and  $b$ 
2:  $a \leftarrow a/2^r, b \leftarrow b/2^r$ 
3: swap  $a$  and  $b$  if  $a$  is not odd
4:  $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}$ 
5: while  $t \neq 0$  do
6:   while  $t$  is even do
7:     if  $Y$  is odd then
8:        $(X, Y) \leftarrow (X + b, Y - a)$ 
9:        $(t, X, Y) \leftarrow (\frac{t}{2}, \frac{X}{2}, \frac{Y}{2})$ 
10:    if  $s \geq t$  then
11:       $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$ 
12:    else
13:       $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ -1 & 1 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$ 
14: return  $(s2^r, U, V)$  if  $a$  and  $b$  were not swapped and  $(s2^r, V, U)$  otherwise
```

precompute g^w for each w in some window $0 \leq w < 2^k$ for some k and then to iterate over the exponent k bits at a time. Here we apply this technique to the right-to-left binary extended GCD. The application of windowing to a right-to-left binary extended GCD did not appear in the literature and we do this here as our own extension.

To recap, the non-windowed right-to-left binary extended GCD of Subsection 5.2 works as follows: it repeatedly reduces either the equation $t = Xa + Yb$ or the equation $t = (X + b)a + (Y - a)b$ by 2. When Y is odd, we simultaneously add b to X and subtract a from Y in order to make both X and Y even. Suppose that t was a multiple of 4. We could simultaneously add b to X and subtract a from Y repeatedly until both X and Y were divisible by 4. Choose m such that $ma \equiv Y \pmod{4}$, and then $t = (X + mb)a + (Y - ma)b$ is evenly divisible by 4 when t is divisible by 4.

This is easily extended for any 2^k where k is a positive integer. The algorithm first computes $x_j = mb$ and $y_j = ma$ for $0 \leq m < 2^k$ where $j = ma \bmod 2^k$. While t is divisible

by 2^h for some $h \leq k$, we look up x_j and y_j for $j = Y \bmod 2^h$ and compute $(X + x_j)/2^h$ and $(Y - y_j)/2^h$. The complete algorithm is given in Algorithm 5.3. Future work should consider an analysis of the theoretically optimal window size.

Algorithm 5.3 Windowed Right-to-left Binary Extended GCD.

Input: $a, b, \in \mathbb{Z}_{>0}$ and let $k \in \mathbb{Z}_{>0}$ be the window size in bits.

Output: (s, U, V) satisfying $s = Ua + Vb$ with $s = \gcd(a, b)$.

```

1: let  $r$  be the largest integer such that  $2^r$  divides both  $a$  and  $b$ 
2:  $a \leftarrow a/2^r, b \leftarrow b/2^r$ 
3: swap  $a$  and  $b$  if  $a$  is not odd
4: for  $m$  from  $2^k - 1$  downto 0 do
5:    $j \leftarrow ma \bmod 2^k$ 
6:    $x_j \leftarrow mb$ 
7:    $y_j \leftarrow ma$ 
8:    $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}$ 
9: while  $t \neq 0$  do
10:   while  $t$  is even do
11:     let  $h$  be the largest integer such that  $h \leq k$  and  $2^h$  divides  $t$ 
12:      $j \leftarrow Y \bmod 2^h$ 
13:      $(t, X, Y) \leftarrow \left( \frac{t}{2^h}, \frac{X+x_j}{2^h}, \frac{Y-y_j}{2^h} \right)$  {Reduce by  $2^h$ }
14:   if  $s \geq t$  then
15:      $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & -1 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$ 
16:   else
17:      $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ -1 & 1 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$ 
18: return  $(s2^r, U, V)$  if  $a$  and  $b$  were not swapped and  $(s2^r, V, U)$  otherwise

```

5.3 Left-to-Right Binary Extended GCD

Just as exponentiation can be performed from high-order to low-order, so too can an extended binary GCD computation. This is termed a left-to-right binary GCD, since it works from the left most bit to the right most bit of the written binary representation of the inputs.

Recall that at each iteration of the extended Euclidean Algorithm, we subtract $q = \lfloor s/t \rfloor$ times the equation $t = Xa + Yb$ from the equation $s = Ua + Vb$ and then swap (s, U, V) with (t, X, Y) . Computing $q = \lfloor s/t \rfloor$ uses integer division, and then subtracting q times one

equation from the other uses multiplication. Since it is not necessary to subtract exactly q times the equation, the idea is instead to use a value $q' = 2^k$ such that q' is *close* in some sense to q . Subtracting q' times the second equation from the first can then be done using a binary shift left by k bits.

Shallit and Sorenson [54] propose to select $q' = 2^k$ such that $q't \leq s < 2q't$. If $s - q't < 2q't - s$, compute

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -q' \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix},$$

otherwise, compute

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 2q' \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}.$$

Notice that after this step s has the previous value of t and that the binary representation of t is one digit shorter than the binary representation of the previous value of s , i.e. the left most set bit of the previous value of s is now cleared. As with the extended Euclidean Algorithm, maintain the invariant that $s \geq t$; if after the above operation $s < t$, then swap the rows of the matrix to restore the invariant. The complete algorithm is given in Algorithm 5.4.

In practice, to find $q = 2^k$ we compute the number of bits in both s and t and then use the difference as a candidate for k . Let $k' = (\lfloor \log_2 s \rfloor + 1) - (\lfloor \log_2 t \rfloor + 1) = \lfloor \log_2 s \rfloor - \lfloor \log_2 t \rfloor$ be our candidate. If $t2^{k'} \leq s$ then $k = k'$, otherwise $k = k' - 1$. Notice that either $k = k'$ in which case $t2^k = t2^{k'}$, or $k = k' - 1$ and then $t2^{k+1} = t2^{k'}$. Either way $t2^{k'}$ can be reused for one half of the comparison of $s - qt < 2qt - s$. Also, if $k' = 0$ then $t \leq s$ (by our invariant) and so there is no possibility of using $t2^{-1}$, since we will use $t2^{k'}$ and $t2^{k'+1}$ in the comparison.

This approach requires us to first compare $t2^{k'}$ to s and then compare one of $t2^{k'-1}$ or $t2^{k'+1}$ to s in order to find which is closer. Because of this, we also experimented with a simplified version of the algorithm. We only compute $k = \lfloor \log_2 s \rfloor - \lfloor \log_2 t \rfloor$. Let $q = 2^k$ and

Algorithm 5.4 Shallit and Sorenson Left-to-Right Binary Extended GCD ([54]).

Input: $a, b \in \mathbb{Z}$

Output: (s, U, V) satisfying $s = Ua + Vb$ with $s = \gcd(a, b)$.

```

1:  $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}$ 
2: if  $t > s$  then
3:    $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$            {Swap rows. Maintain  $s \geq t$ .}
4: while  $t \neq 0$  do
5:   find  $q = 2^k$  such that  $qt \leq s < 2qt$ 
6:   if  $s - qt < 2qt - s$  then
7:      $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$ 
8:   else
9:      $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -1 & 2q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$ 
10:  if  $t > s$  then
11:     $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$            {Swap rows. Maintain  $s \geq t$ .}
12: return  $(s, U, V)$ 

```

compute

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 1 & -q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}.$$

If $qt > s$ then the resulting value for $s - qt$ is negative, and so we negate the first row of the product matrix to ensure that the new value for s is positive. The result is fewer comparisons for the inner loop of the GCD overall. The results in Section 5.7 show that this simplified left-to-right binary extended GCD takes between only 45% and 77% as long as our implementation of Shallit and Sorenson's left-to-right binary extended GCD.

5.4 Lehmer's Extended GCD

In the extended Euclidean algorithm (Section 5.1), each iteration subtracts a multiple, $q = \lfloor s/t \rfloor$, of the smaller number from the larger number. Lehmer [44] noticed that most of the quotients, q , were small and that those small quotients could be computed from the

leading digits (or machine word) of the numerator and denominator.

The idea is similar to the extended Euclidean algorithm, only that there is an inner loop that performs an extended GCD computation using values that fit within a single machine word. As in the extended Euclidean algorithm, start by setting

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}$$

for positive integers a and b . Assume that $s \geq t$ (if this is not the case, then swap the rows of the matrix). Compute $s' = \lfloor s/2^k \rfloor$ for some k such that s' fits within a single machine word and is as large as possible (if s already fits within a single machine word, then let $k = 0$), and compute $t' = \lfloor t/2^k \rfloor$ for the same value k . The idea is to then perform an extended GCD computation on the values s' and t' but only so long as the quotient $q_i' = \lfloor s'/t' \rfloor$, generated by the i^{th} step of a single precision extended Euclidean GCD computation, is equal to the quotient $q_i = \lfloor s/t \rfloor$ that would be generated by the i^{th} step of a corresponding full precision extended Euclidean GCD computation.

To determine when the single precision quotient q_i' would differ from the full precision quotient q_i , we use a method originally proposed by G. E. Collins but described by Jebelean [41, Theorem 1]. Let

$$\begin{bmatrix} s' & A & B \\ t' & C & D \end{bmatrix} \leftarrow \begin{bmatrix} s' & 1 & 0 \\ t' & 0 & 1 \end{bmatrix}$$

be the initial matrix consisting of single precision integers. We then perform a single precision extended GCD on the above matrix until

$$t' < -D \quad \text{or} \quad s' - t' < C - A \quad \text{when } i \text{ is even,}$$

$$t' < -C \quad \text{or} \quad s' - t' < D - B \quad \text{when } i \text{ is odd.}$$

The resulting matrix

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}$$

represents the concatenation of the operations performed during the single precision extended GCD. If $B \neq 0$, these operations are combined with the outer loop of the larger extended GCD by computing

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} A & B \\ C & D \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}.$$

We then continue with the outer loop of the computation until $t = 0$. In the event that $B = 0$, then s and t differ in length by more than a single machine word, and we use a step of the full precision extended Euclidean GCD computation to adjust their lengths.

Lehmer's original description [44] uses the extended Euclidean algorithm to compute each step of the single precision extended GCD, however, other single precision extended GCD computations will work. In Section 5.7, we give timing results for extended GCD computations using an implementation of Lehmer's extended GCD for both a single precision extended Euclidean GCD and a single precision left-to-right binary extended GCD. We also experimented with precomputing the result of the single precision extended GCD for 8 bit machine words. Pseudo-code for Lehmer's extended GCD is given in Algorithm 5.5.

5.5 Partial Extended GCD

Subsections 2.3.3, 2.3.4, and 2.3.5 describe algorithms to perform ideal class multiplication (NUCOMP), squaring (NUDUPL), and cubing (NUCUBE) respectively, where the result requires at most two reduction operations to be reduced [40, p.122]. Each of these algorithms involves computing a simple continued fraction expansion, which is done using an extended GCD computation (see [40, §3.2]). Unlike the extended GCD algorithms previously described in this chapter, the extended GCD used to compute the simple continued fraction for ideal multiplication terminates once the next remainder computed is within a given bound. For this reason, this computation is referred to as the *partial extended GCD*.

The descriptions of NUCOMP, NUDUPL, and NUCUBE use the variables R_i and C_i .

Algorithm 5.5 Lehmer's extended GCD ([44]).

Input: $a, b, \in \mathbb{Z}$ and $a \geq b > 0$. Let m be the number of bits in a machine word.

Output: (s, U, V) satisfying $s = Ua + Vb$ with $s = \gcd(a, b)$.

```

1:  $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} a & 1 & 0 \\ b & 0 & 1 \end{bmatrix}$ 
2: while  $t \neq 0$  do
3:    $k \leftarrow \lfloor \log_2 s \rfloor + 1 - m$ 
4:    $s' \leftarrow \lfloor s/2^k \rfloor$  {Shift right for most significant word.}
5:    $t' \leftarrow \lfloor t/2^k \rfloor$ 
6:    $\begin{bmatrix} A & B \\ C & D \end{bmatrix} \leftarrow \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ 
7:    $i \leftarrow 0$ 
8:   while  $t' \neq 0$  do
9:     perform one step of a single precision extended GCD on  $\begin{bmatrix} s' & A & B \\ t' & C & D \end{bmatrix}$ 
10:    if  $i$  is even then
11:      if  $(t' < -D$  or  $s' - t' < C - A)$  then use previous  $A, B, C, D$  and break
12:    else
13:      if  $(t' < -C$  or  $s' - t' < D - B)$  then use previous  $A, B, C, D$  and break
14:       $i \leftarrow i + 1$ 
15:    if  $B = 0$  then
16:       $q \leftarrow \lfloor s/t \rfloor$  {Full precision step.}
17:       $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & -q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$ 
18:    else
19:       $\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} A & B \\ C & D \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$  {Combine step.}

```

These variables can be related to the description of the extended Euclidean algorithm (see Section 5.1) using

$$\begin{bmatrix} s & U \\ t & X \end{bmatrix} = \begin{bmatrix} R_{i-1} & C_{i-1} \\ R_i & C_i \end{bmatrix}.$$

Since the coefficients V and Y of the extended GCD are not used, our implementations of the partial extended GCD do not compute them. To expand the simple continued fraction a/b , let

$$\begin{bmatrix} R_{-2} & C_{-2} \\ R_{-1} & C_{-1} \end{bmatrix} = \begin{bmatrix} a & -1 \\ b & 0 \end{bmatrix}$$

and then apply the recurrences

$$q_i = \lfloor R_{i-2} / R_{i-1} \rfloor$$

$$R_i = R_{i-2} - q_i R_{i-1}$$

$$C_i = C_{i-2} - q_i C_{i-1}$$

until we have R_i less than or equal to the given termination bound. This is essentially the same as computing the extended GCD except for the starting matrix, the termination bound, and one set of coefficients. An invariant of this method is that $R_{i-1} \geq R_i$.

By [23, §5.6.1], each of the operations performed on the matrix

$$\begin{bmatrix} R_{i-1} & C_{i-1} \\ R_i & C_i \end{bmatrix}$$

during the partial extended GCD must be representable by an invertible 2×2 matrix with determinant ± 1 (known as a *unimodular* matrix). This is necessary in order to ensure that the reduced ideal (after applying the coefficients from the partial extended GCD) is in the same equivalence class as the product ideal. This is important since only extended GCD computations that are restricted to operations representable using invertible 2×2 matrices can be used for the partial extended GCD.

Of the extended GCD algorithms in this chapter, only the right-to-left binary extended GCD of Section 5.2 (and its windowed variants of Subsection 5.2.1) cannot be adapted to the partial extended GCD. The reason is due to the step where t is reduced by 2. The algorithm computes

$$(t, X, Y) \leftarrow \begin{cases} \left(\frac{t}{2}, \frac{X}{2}, \frac{Y}{2}\right) & \text{if } Y \text{ is even} \\ (t, X, Y) \leftarrow \left(\frac{t}{2}, \frac{X+b}{2}, \frac{Y-a}{2}\right) & \text{otherwise.} \end{cases}$$

The step $(t, X, Y) \leftarrow (t/2, (X+b)/2, (Y-a)/2)$ cannot be expressed as a unimodular matrix.

5.6 Specialized Implementations of the Extended GCD

To further improve performance, we specialized implementations of each of the GCD algorithms discussed in this section for 32 bit, 64 bit, and 128 bit arithmetic. All algorithms are implemented for the GNU C compiler version 4.7.2 and assembly language is used for 128 bit arithmetic and processor specific features not available in the C programming language.

Many of the techniques used here are described in the book “Hacker’s Delight” [60]. We use `and2` to denote bitwise ‘and’, `or2` for bitwise ‘or’, `⊕` for bitwise ‘exclusive or’, and `¬` for bitwise negation. Computing $x2^k$ corresponds to shifting x left by k bits, while computing the integer $\lfloor x/2^k \rfloor$ corresponds to shifting x right by k bits. To compute $x \bmod 2^k$ use $x \text{ and}_2 (2^k - 1)$.

Assuming a two’s complement representation of machine words, the most significant bit of a machine word x is set if $x < 0$ and clear otherwise. Let m denote the number of bits in a machine word; then the result of an arithmetic shift right on x by $m - 1$ bits is either a word with m set bits when $x < 0$ or a word with m clear bits otherwise. Let `sign_mask( $x$ )` denote this operation. Notice that a word with m set bits corresponds to the integer -1 under a signed two’s complement representation.

Using these operations, the absolute value of a signed machine word x can be computed without using any conditional statements. Let $y = \text{sign_mask}(x)$, then the absolute value of x is $(x \oplus y) - y$. To see this, suppose $x \geq 0$. Then y is 0 and so $(x \oplus y) - y = x$. When $x < 0$, y has all of its bits set (and is also -1). Therefore, $(x \oplus y) - y = \neg x + 1 = -x$.

Similarly, a word x can be conditionally negated depending on a bit-mask y . This is useful since the extended GCD algorithms described previously expect their input, a and b , to be positive integers. First compute the absolute value of a and b by computing a signed mask for each and then conditionally negate each. Since the extended GCD algorithm computes a solution to $s = Ua + Vb$, where a and b have been made positive, the solution for the original inputs is to conditionally negate U based on the signed mask of a , and V based on

the signed mask of b .

Furthermore, many of the algorithms maintain the invariant that $s > t$ and that both are positive. Suppose the algorithm computes

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & -q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$$

when $s \geq t$ and

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ -1 & q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$$

otherwise. The conditional instruction is removed by first computing $m = \mathbf{sign_mask}(s - t)$ and then

$$\begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix} \leftarrow \begin{bmatrix} 0 & 1 \\ 1 & -q \end{bmatrix} \cdot \begin{bmatrix} s & U & V \\ t & X & Y \end{bmatrix}$$

and conditionally negating the triple (t, X, Y) based on the signed mask m .

To swap two machine words, x_0 and y_0 , without using additional words, compute $x_1 = x_0 \oplus y_0$, $y_1 = x_1 \oplus y_0$, and then $x_2 = x_1 \oplus y_1$. Notice that x_2 expands to

$$x_2 = (x_0 \oplus y_0) \oplus ((x_0 \oplus y_0) \oplus y_0)$$

and this reduces to $x_2 = y_0$. Also, y_1 expands to $(x_0 \oplus y_0) \oplus y_0$, which is just x_0 . In practice, each assignment to x_i and y_i overwrites the previous x_{i-1} and y_{i-1} and so is performed in place.

Two words, x and y , are conditionally swapped when $x < y$ by using two additional words. Let $d = x - y$. Notice that simply computing $x \leftarrow x - d$ and $y \leftarrow y + d$ swaps x and y . Instead, let $m = \mathbf{sign_mask}(d)$ and then $x \leftarrow x - (d \mathbf{and}_2 m)$ and $y \leftarrow y + (d \mathbf{and}_2 m)$ swaps x and y only when $x < y$. In the left-to-right binary GCD, we conditionally swap the triple (s, U, V) with (t, X, Y) when $s < t$. By fixing $m = \mathbf{sign_mask}(s - t)$, but letting d take on $s - t$, and then $U - X$, and finally $V - Y$, we conditionally swap the triples when $s < t$. The algorithm is given in Algorithm 5.6. On the x64 architecture we optimize the

Algorithm 5.6 Conditionally swap (s, U, V) with (t, X, Y) when $s < t$.

Input: (s, U, V) and (t, X, Y) .

Output: the triples are swapped in place satisfying $s \geq t$.

```

1:  $d \leftarrow s - t$ 
2:  $m \leftarrow \text{sign\_mask}(d)$ 
3:  $d \leftarrow d \text{ and}_2 m$ 
4:  $s \leftarrow s - d$ 
5:  $t \leftarrow t + d$ 
6:  $d \leftarrow (U - X) \text{ and}_2 m$ 
7:  $U \leftarrow U - d$ 
8:  $X \leftarrow X + d$ 
9:  $d \leftarrow (V - Y) \text{ and}_2 m$ 
10:  $V \leftarrow V - d$ 
11:  $Y \leftarrow Y + d$ 

```

computation of $d \leftarrow s - t$ and $m \leftarrow \text{sign_mask}(d)$ since the operation $s - t$ sets the carry flag when the result is negative. Using a subtract with borrow, we subtract m from itself. This sets m to 0 when the carry is clear, and -1 when the carry is set.

Often our implementation uses the number of bits in a positive integer x . In the algorithm description we use $\lfloor \log_2 x \rfloor + 1$. On the x64 architecture the instruction `bsr` returns the index of the most significant set bit, while the instruction `bsf` return the index of the least significant set bit. This allows us to quickly compute either value. On non-x64 platforms, our code uses a logarithmic search to find the most (or least) significant set bit. To find the most significant bit, let $k = \lfloor m/2 \rfloor$ where m is the number of bits in a machine word and let i be the computed index (initially $i \leftarrow 0$). We first check if $x \geq 2^k$. If it is, then $i \leftarrow i + k$ and x is shifted right by k . We repeat on $k \leftarrow \lfloor k/2 \rfloor$ until $k = 0$. At this point, i is the index of the most significant set bit (see Algorithm 5.7). Notice that we can use the signed mask of $2^k - x$ instead of the conditional ‘if’ statement, and we can unroll the while loop for fixed values of m .

Finally, Lehmer’s extended GCD algorithm is especially useful when the inputs are several machine words long. For 128 bit inputs, we implemented Lehmer’s extended GCD using the extended Euclidean algorithm for 32 and 64 bit machine words, and our simplified left-

Algorithm 5.7 Return the index of the most significant set bit of x .

Input: $x \in \mathbb{Z}_{\geq 0}$, $m \in \mathbb{Z}_{>0}$, with $x < 2^m$.**Output:** i such that i is the index of the most significant set bit of x .

```
1:  $i \leftarrow 0$ 
2:  $k \leftarrow \lfloor m/2 \rfloor$  { $m$  is the number of bits in a machine word.}
3: while  $k \neq 0$  do
4:   if  $x \geq 2^k$  then
5:      $x \leftarrow \lfloor x/2^k \rfloor$ 
6:      $i \leftarrow i + k$ 
7: return  $i$ 
```

to-right binary extended GCD for 64 bit machine words. For 32 and 64 bit inputs, we implemented Lehmer's extended GCD using 8 bit machine words. In this case, it is possible to compute the resulting 2×2 matrix for all possible 8 bit values for the two inputs in advance, i.e. all 65536 pairs of values. The coefficients of the resulting 2×2 matrix can be bound by 8 bits each, and so the resulting table requires 256Kb of memory. This simplifies the extended GCD computation since the inner loop of Lehmer's extended GCD becomes a lookup from this table.

5.7 Experimental Results

We specialized the implementation of each of the extended GCD algorithms for 32, 64, and 128 bit arithmetic. For each k from 1 to 127, we generated 1,000,000 pairs of pseudorandom positive integers of no more than k bits each. For the 32-bit implementations, we measured the time to compute the extended GCD of each pair for $1 \leq k \leq 31$, then for the 64-bit implementations, we measure the time for each pair for $1 \leq k \leq 63$, and finally for the 128-bit implementations, we measure the time for each pair for $1 \leq k \leq 127$. Only positive integers were considered since the prologue and epilogue code to handle negative inputs is the same in each implementation, namely the sign of the inputs is recorded, the inputs are made positive, the extended GCD is computed, and then the sign of the output coefficients is corrected.

To verify that this work has lead to practical performance improvements, we use Pari 2.5.3, MPIR 2.6.0, and GMP 5.1.1 as reference implementations of the extended GCD. In the case of both GMP and MPIR, we use instances of `mpz_t` modified to use at most two 64-bit words of locally allocated stack space for limbs. The only operations timed are calls to `mpz_xgcd`. Since MPIR is a fork of GMP, we simply change the `#include` directive, recompile, and then link against the appropriate library. For Pari, we link directly with the Pari library from C. For 64 bit integers, Pari `GEN` objects are created using `stoi`, while for 128 bit integers we use `mkintn(4, ...)`. All pairs of input integers are first converted to Pari `GEN` objects. As such, Pari timings only include calls to `bezout`, the symbol for Pari’s implementation of the extended GCD. Garbage collection in Pari is performed by recording Pari’s stack pointer (`avma`) and then restoring it at the end of each batch timing. For each of Pari, MPIR, and GMP, we statically link with the corresponding library.

The source for these experiments was developed using the GNU C compiler version 4.7.2 on Ubuntu 12.10. The experiments were performed on a 2.7GHz Intel Core i7-2620M CPU with 8Gb of memory. The CPU has four cores, only one of which was used during testing. Since we interfaced with Pari, GMP, and MPIR through the C library, and we ran all our experiments sequentially, we did not make library calls on multiple threads. We also observed the CPU load during experiments to verify that at most one core was being used. The timing data occasionally displays an interval that is not smooth. Since repeating the experiment either produces a smooth result, or the non-smooth area occurs at a different time during the experiment, we believe that this is an artefact of the timing measurements and not of the function being timed.

We present our data in three sections, relating to our implementations specialized for 32, 64, and 128 bit arithmetic. In each case, we show that our 32-bit implementation is no slower than our 64-bit implementation, which is no slower than our 128-bit implementation. For input integers smaller than 32 bits, our implementation of the standard extended Euclidean

algorithm performs best. For input integers larger than 31 bits but smaller than 64 bits, our implementation of a simplified left-to-right binary extended GCD is the fastest. Finally for input integers larger than 63 bits but smaller than 128 bits, our implementation of a simplified left-to-right binary extended GCD is fastest up to 118 bits and then our implementation of Lehmer’s extended GCD using our simplified left-to-right binary extended GCD for 64 bit machine words becomes faster. For input integers larger than 127 bits, we defer to one of the reference implementations.

Ideal class arithmetic uses a variant of the extended GCD where only s and U are computed for the solution $s = Ua + Vb$ for input integers a and b . We refer to this as a *left-only* extended GCD. Ideal class arithmetic also uses the partial extended GCD, which is similar to the left-only extended GCD but the algorithm is terminated when the next quotient is less than a specified bound, and the initial value for U is -1 . Future work (see Section 9.1) would test each implementation of the left-only and partial extended GCD specifically, however, we simply use the associated version of the left-only and partial extended GCD for whichever version of the full extended GCD performs best for inputs of a specific size. Since none of the reference implementations tested provide a partial extended GCD, we implemented one for `mpz_t` types using Lehmer’s partial extended GCD and a 64 bit extended Euclidean GCD for the inner extended GCD.

5.7.1 32-bit Extended GCDs

Figures 5.1 through 5.12 show performance timings for each of our implementations of the extended GCD, as well as the reference implementations, for inputs less than 32 bits. For each bit size k from 1 through 31, a set of 1,000,000 pseudorandom positive integers of at most k bits was created. The times shown are the average time for a single extended GCD computation. In each case, our 32-bit implementations perform faster than our corresponding 64-bit and 128-bit implementations. Figure 5.1 shows that for inputs of this size, Pari is the fastest of the reference implementations tested. Figure 5.8 shows the performance of our 32

bit implementations of Stein’s right-to-left binary extended GCD for window sizes from 1 bit to 5 bits. Of these implementations, the 1 bit window is fastest for inputs up to 15 bits, and then the 3 bit window is faster up to 29 bits.

Each of the implementations tested here have been verified to produce correct results for inputs up to 31 bits, with the exception of our 32-bit implementations of Stein’s right-to-left binary extended GCD, which fails verification for inputs larger than 29 bits – this appears to be due to arithmetic overflow. The output for each extended GCD algorithm is a solution to the equation $s = Ua + Vb$. We verify that the equation is satisfied by the output values given the inputs, and that s is the greatest common divisor of both a and b .

Figure 5.12 shows the best performing implementations from among the reference implementations and each of our 32-bit implementations. In the case of our implementations of Stein’s right-to-left binary extended GCD, this figure only includes the 1 bit and 3 bit windowed implementations, since these were the best performing of the windowed implementations. This figure shows that for inputs less than 32 bits, our 32-bit implementation of the extended Euclidean performs best, and takes roughly only 63% as long as the fastest reference implementation, for inputs of this size.

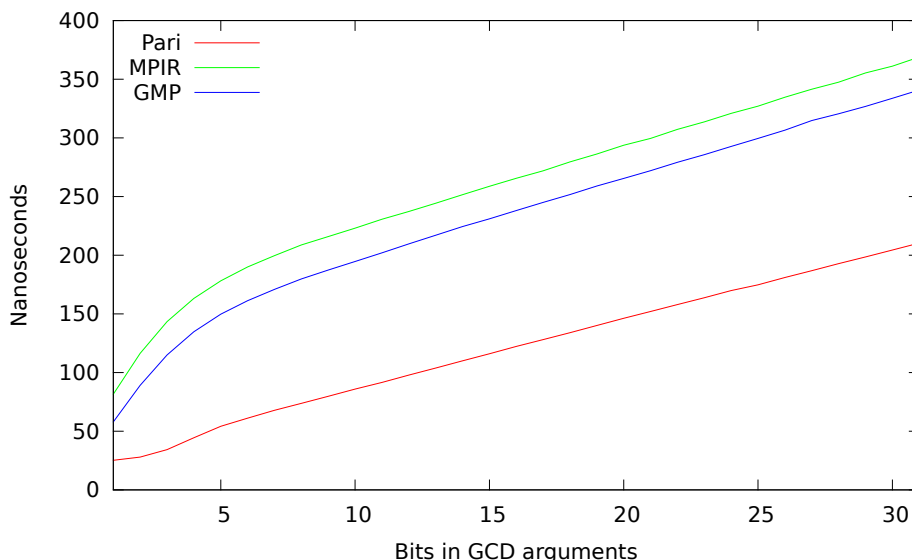


Figure 5.1: Reference Implementations for 32-bit Inputs.

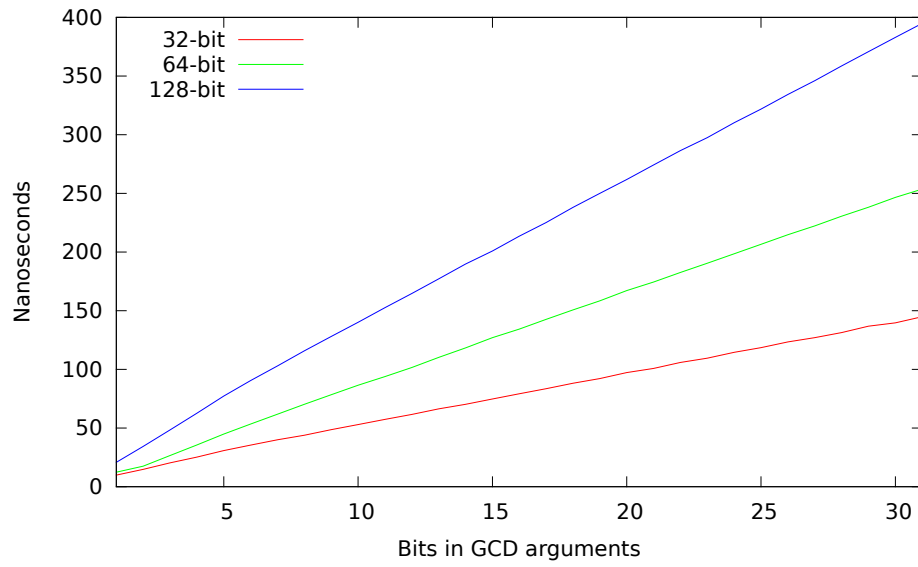


Figure 5.2: Extended Euclidean GCD for 32-bit Inputs.

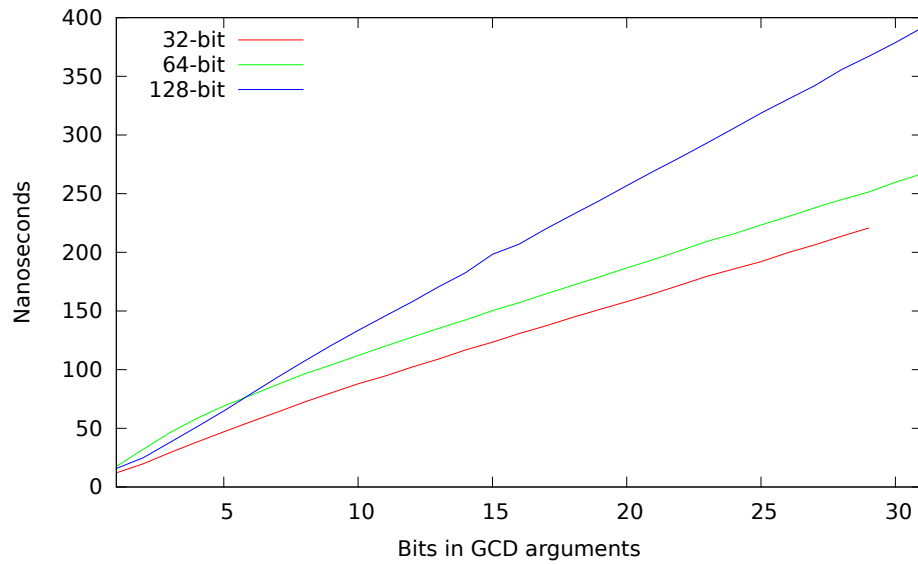


Figure 5.3: Stein's Extended GCD for 32-bit Inputs.

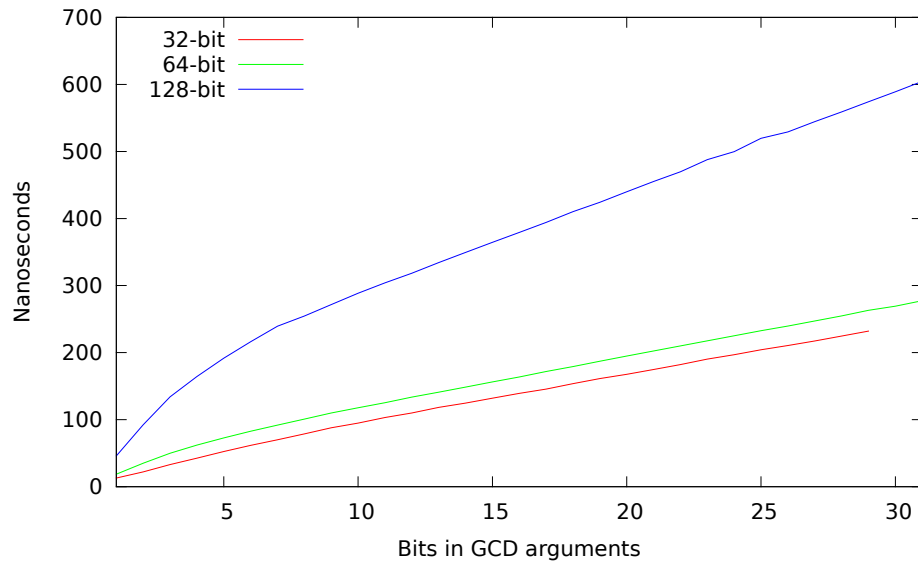


Figure 5.4: 2 bit Windowed Stein's Extended GCD for 32-bit Inputs.

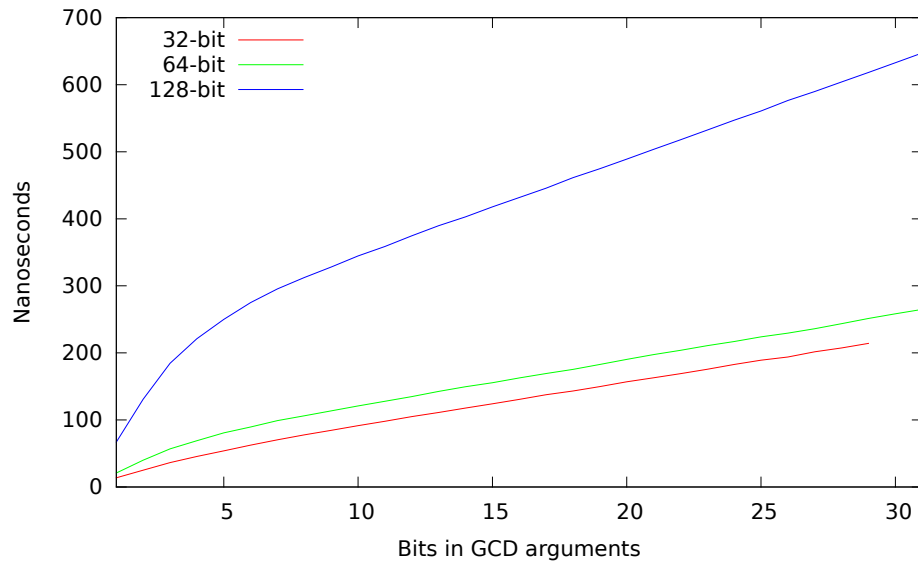


Figure 5.5: 3 bit Windowed Stein's Extended GCD for 32-bit Inputs.

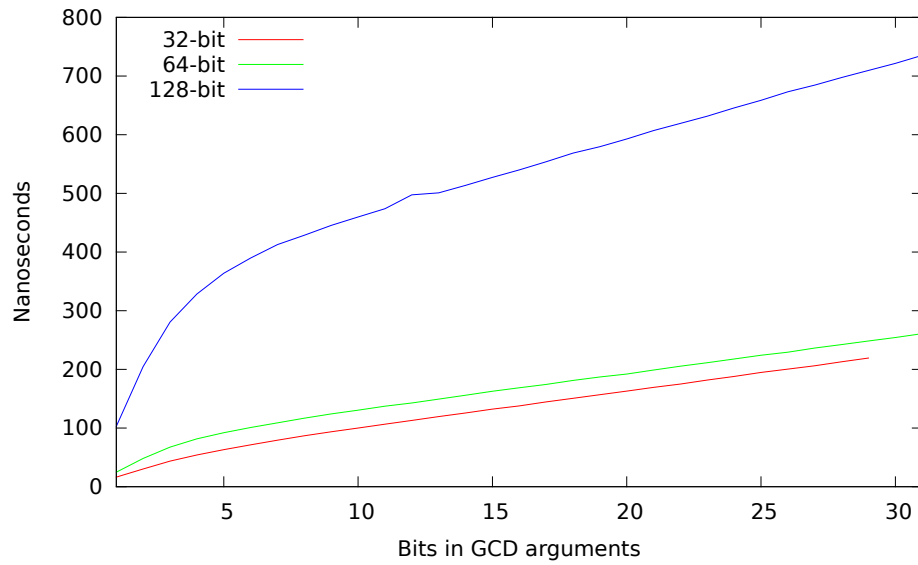


Figure 5.6: 4 bit Windowed Stein's Extended GCD for 32-bit Inputs.

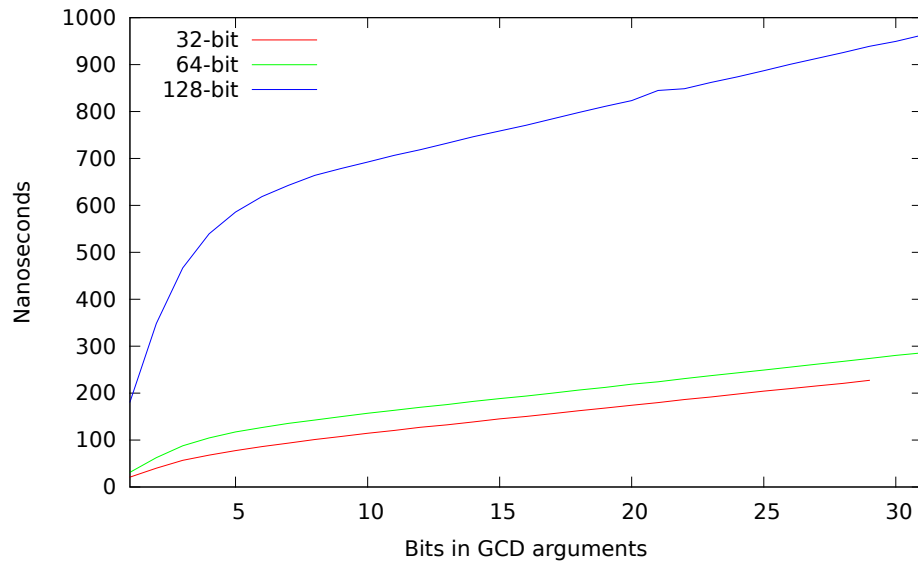


Figure 5.7: 5 bit Windowed Stein's Extended GCD for 32-bit Inputs.

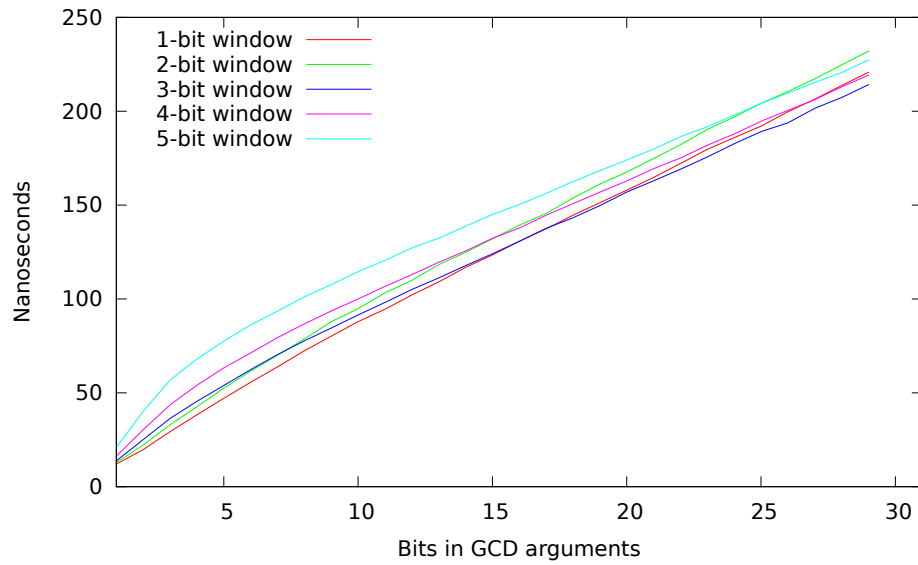


Figure 5.8: All 32-bit Implementations of Stein's Extended GCD.

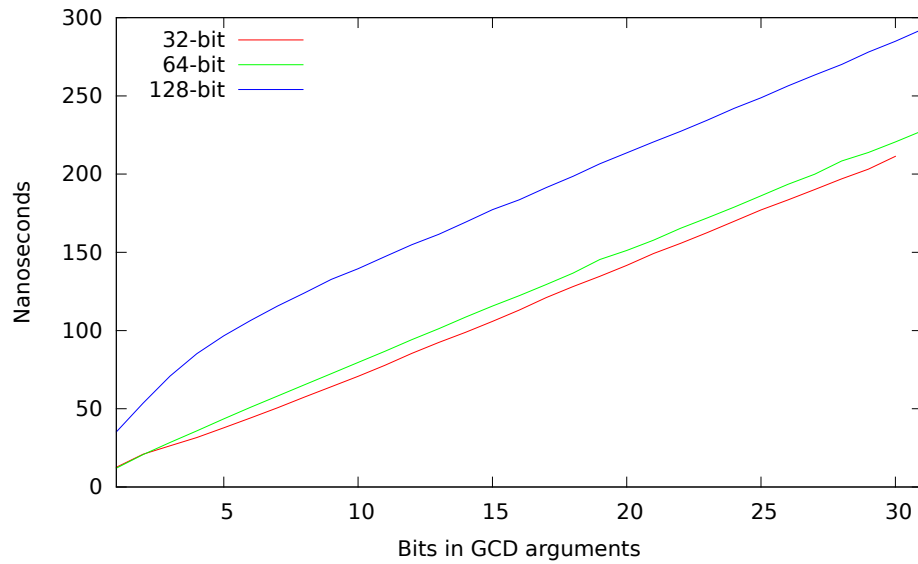


Figure 5.9: Shallit's Left-to-Right Binary Extended GCD for 32-bit Inputs.

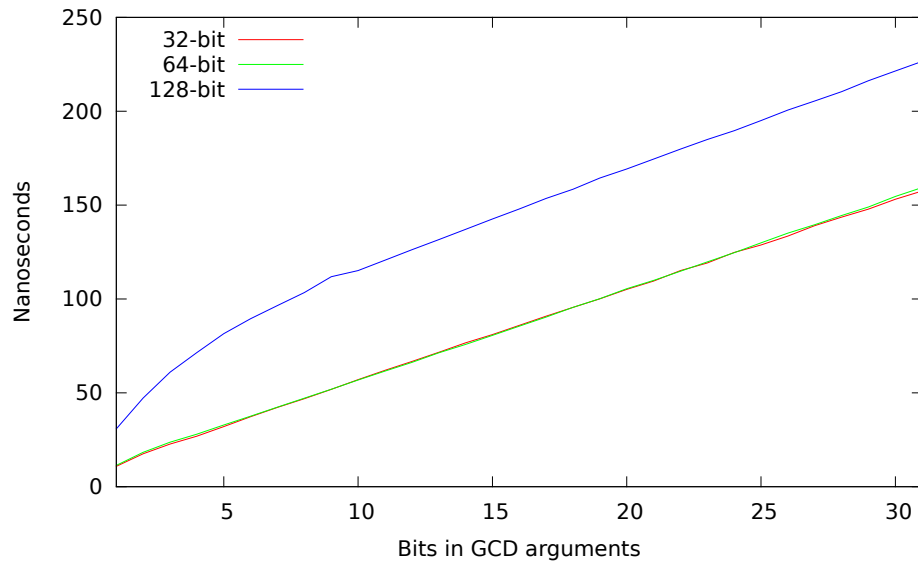


Figure 5.10: Simplified Left-to-Right Binary Extended GCD for 32-bit Inputs.

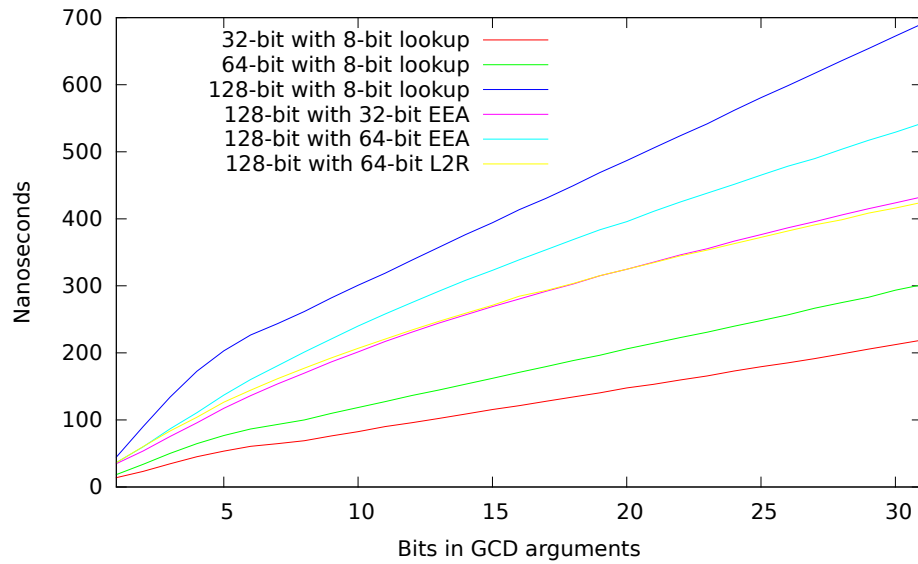


Figure 5.11: Lehmer's Extended GCD for 32-bit Inputs.

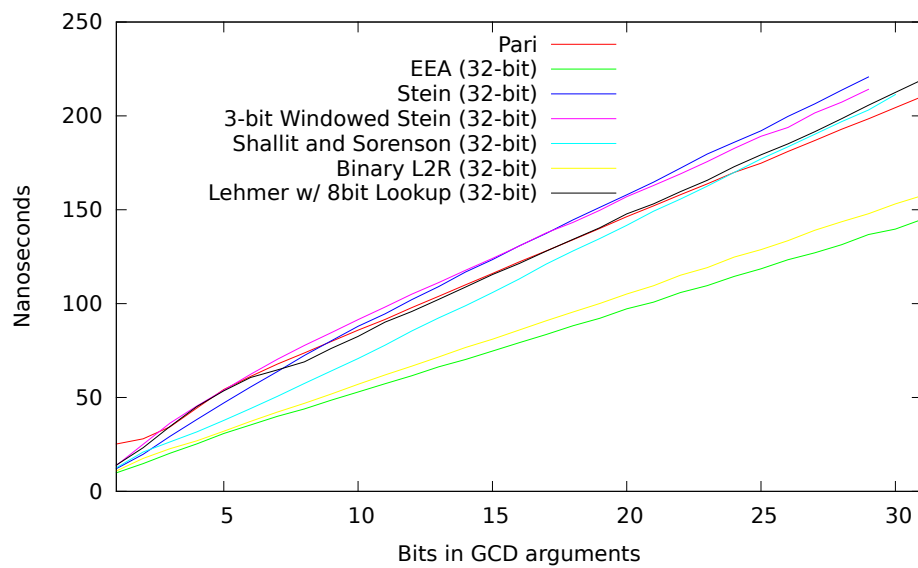


Figure 5.12: Best Extended GCDs for 32-bit Inputs.

5.7.2 64-bit Extended GCDs

Figures 5.13 through 5.24 show the average time to compute the extended GCD for a pair of pseudorandom positive integers of k bits for k from 1 through 63 using each of our 64-bit and 128-bit implementations of the extended GCD, as well as for the reference implementations. For each bit size, k , 1,000,000 pairs of pseudorandom positive integers were generated and used as inputs for each implementation of the extended GCD. In each case, our 64-bit implementation is faster than the corresponding 128-bit implementation. Figure 5.13 shows that for inputs of this size, Pari performs the fastest of the reference implementations.

Figure 5.20 shows each of our 64-bit implementations of Stein’s right-to-left binary extended GCD for window sizes from 1 bit to 5 bits. Of these implementations, the implementation using a 4 bit window is consistently faster than our other windowed implementations. Future work should include a theoretical analysis of optimal window sizes.

Finally, Figure 5.24 shows each of our 64-bit implementations, as well as the fastest reference implementation, Pari. This figure only includes timings for our 4 bit windowed implementation of Stein’s right-to-left binary extended GCD. This figure shows that for inputs larger than 31 bits but less than 64 bits, our simplified left-to-right binary extended GCD is the fastest and takes roughly 77% of the time of Pari. Each of the implementations tested in this subsection produce correct results up to 63 bits, with the exception of our implementations of Stein’s right-to-left binary extended GCD, which only produce correct results for inputs up to 60 bits.

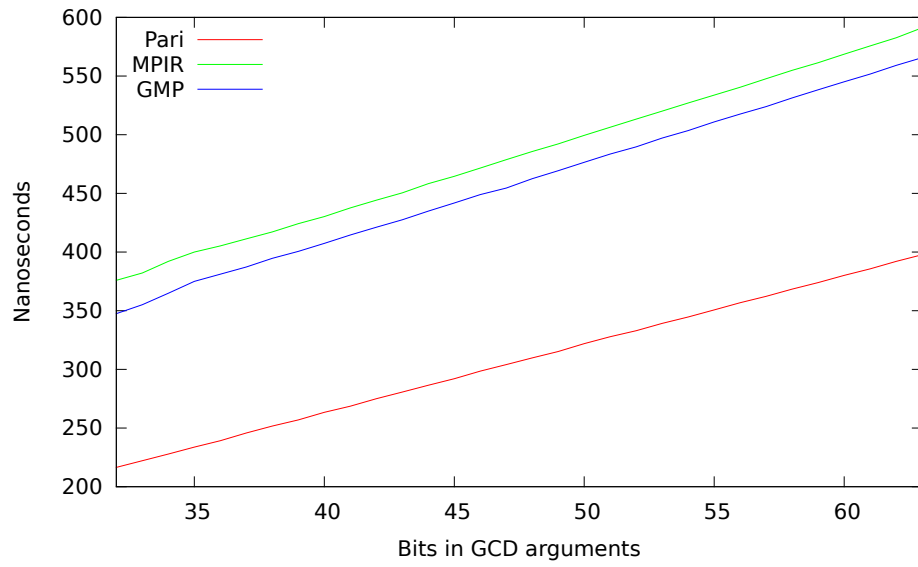


Figure 5.13: Reference Implementations for 64-bit Inputs.

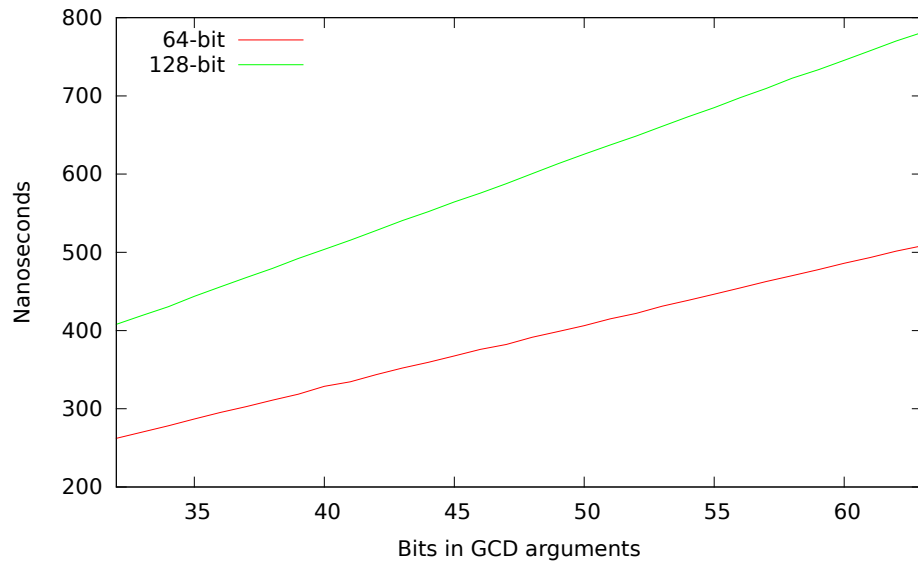


Figure 5.14: Extended Euclidean GCD for 64-bit Inputs.

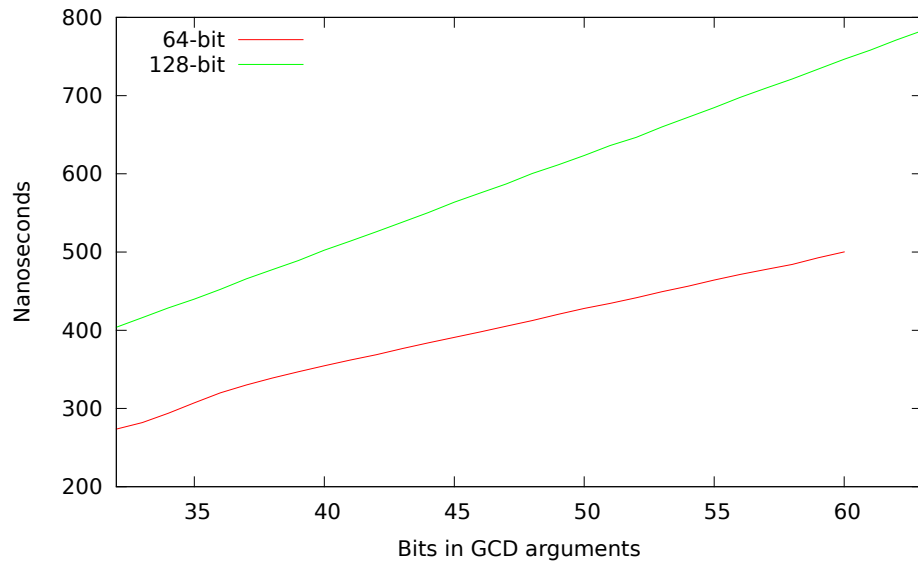


Figure 5.15: Stein's Extended GCD for 64-bit Inputs.

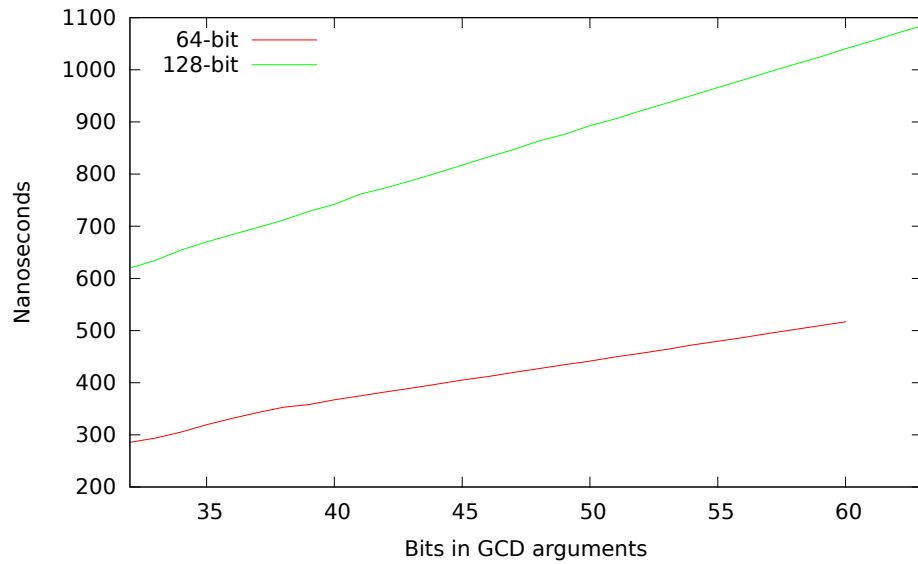


Figure 5.16: 2 bit Windowed Stein's Extended GCD for 64-bit Inputs.

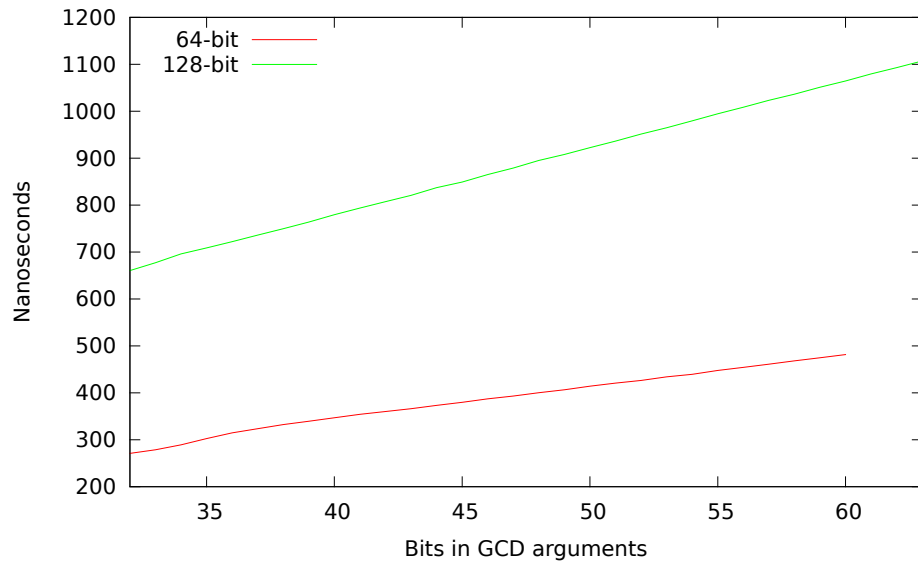


Figure 5.17: 3 bit Windowed Stein's Extended GCD for 64-bit Inputs.

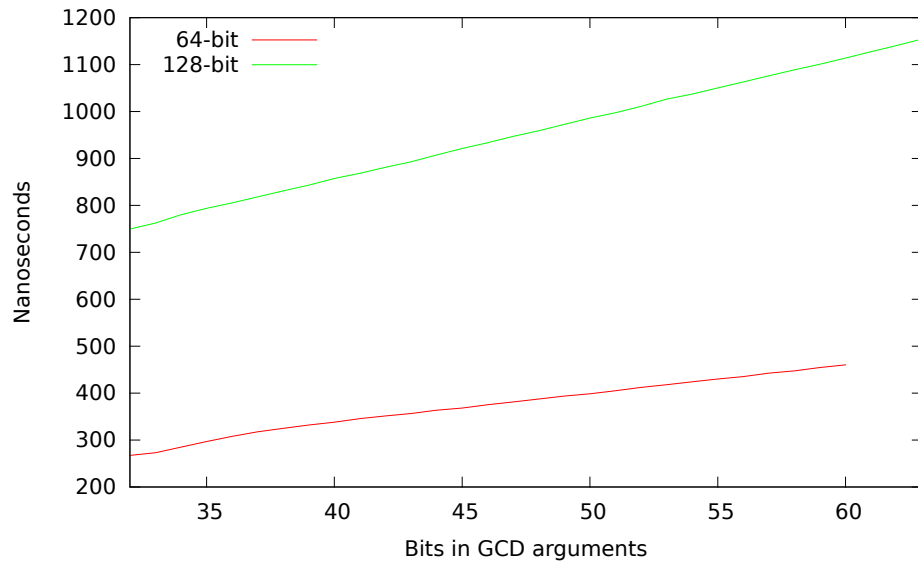


Figure 5.18: 4 bit Windowed Stein's Extended GCD for 64-bit Inputs.

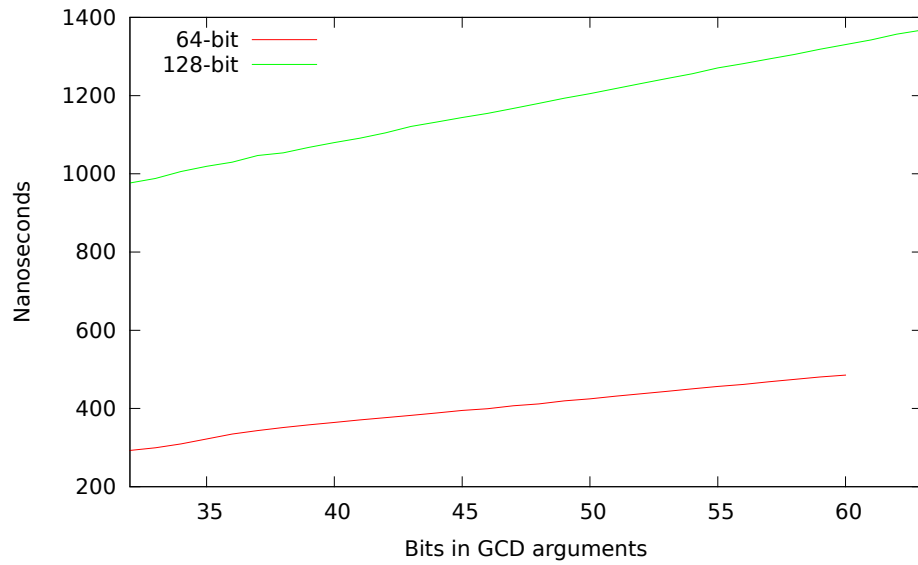


Figure 5.19: 5 bit Windowed Stein's Extended GCD for 64-bit Inputs.

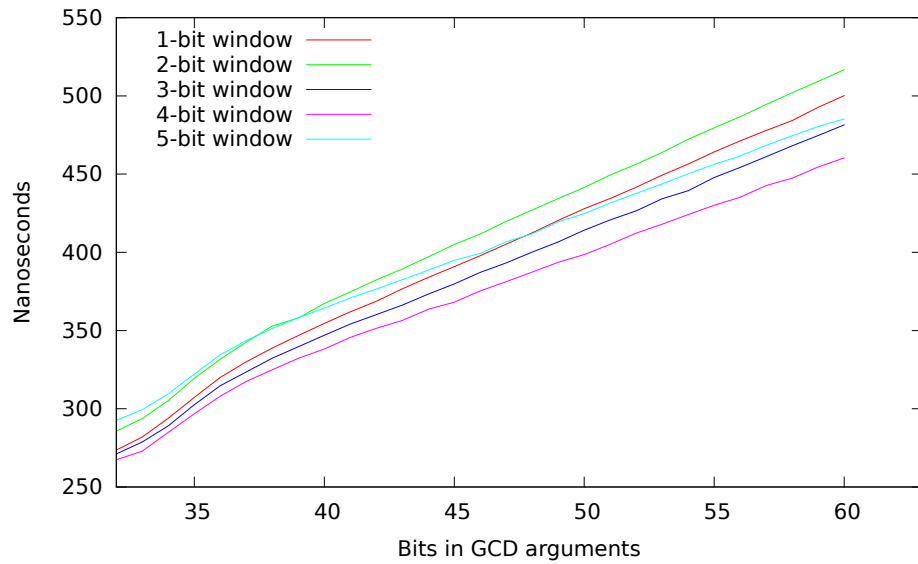


Figure 5.20: All 64-bit Implementations of Stein's Extended GCD.

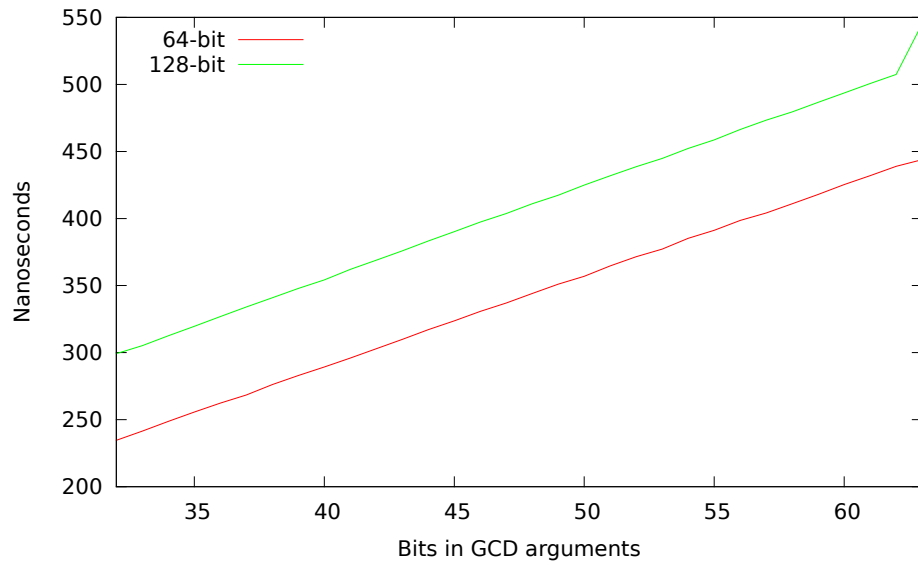


Figure 5.21: Shallit's Left-to-Right Binary Extended GCD for 64-bit Inputs.

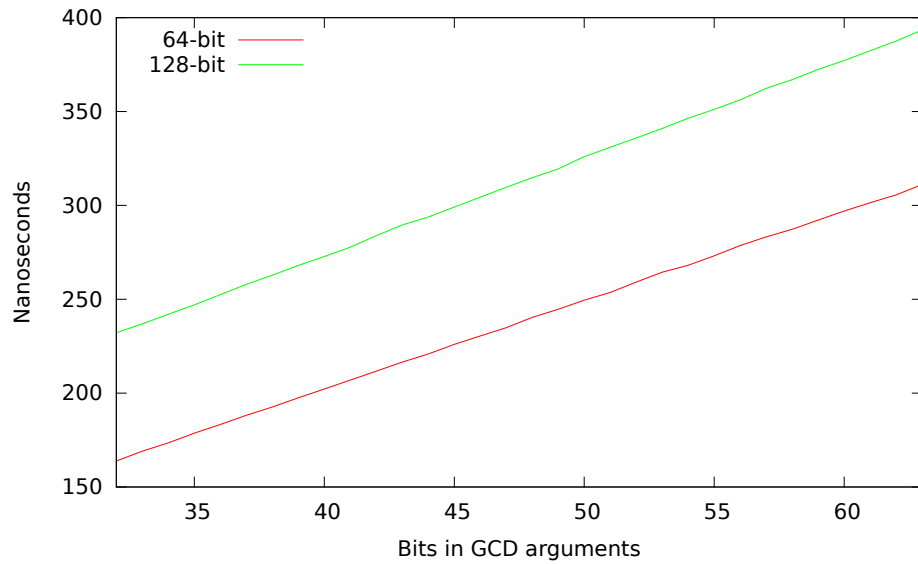


Figure 5.22: Simplified Left-to-Right Binary Extended GCD for 64-bit Inputs.

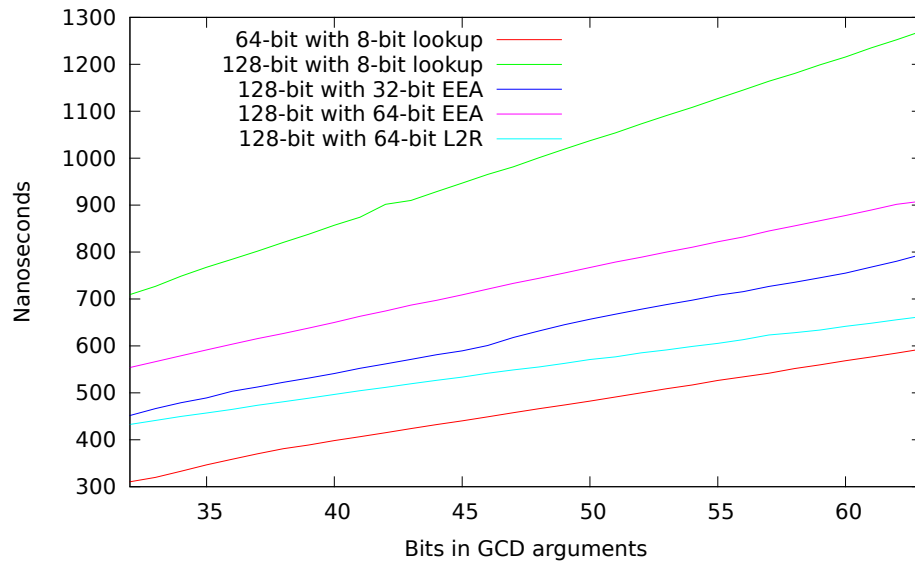


Figure 5.23: Lehmer's Extended GCD for 64-bit Inputs.

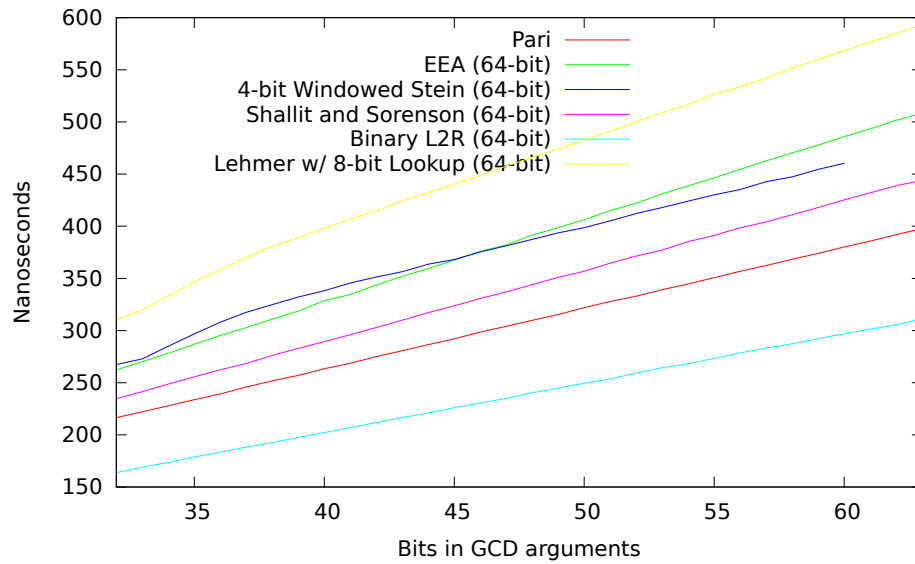


Figure 5.24: Best Extended GCDs for 64-bit Inputs.

5.7.3 128-bit Extended GCDs

Figures 5.25 through 5.28 show the performance timings of each of our 128-bit implementations of the extended GCD, including the reference implementations. Figure 5.25 shows that, of the reference implementations, Pari and GMP are fastest on average. Figure 5.26 show each of our windowed implementations of Stein’s right-to-left binary extended GCD, and that our non-windowed implementation is fastest on average. Figure 5.27 shows each of our implementations of a 128-bit Lehmer’s extended GCD. Our implementation of a 128 bit Lehmer’s extended GCD using a 64-bit implementation of our simplified left-to-right binary extended GCD is the fastest on average among these. Finally, Figure 5.28 shows the best of each of our 128-bit implementations as well as the best of the reference implementations. In this case, our 128-bit implementation of our simplified left-to-right binary extended GCD is the best performing on average for inputs no larger than 118 bits, otherwise our implementation our 128-bit implementation of Lehmer’s extended GCD using our 64 bit simplified left-to-right binary extended GCD is fastest on average.

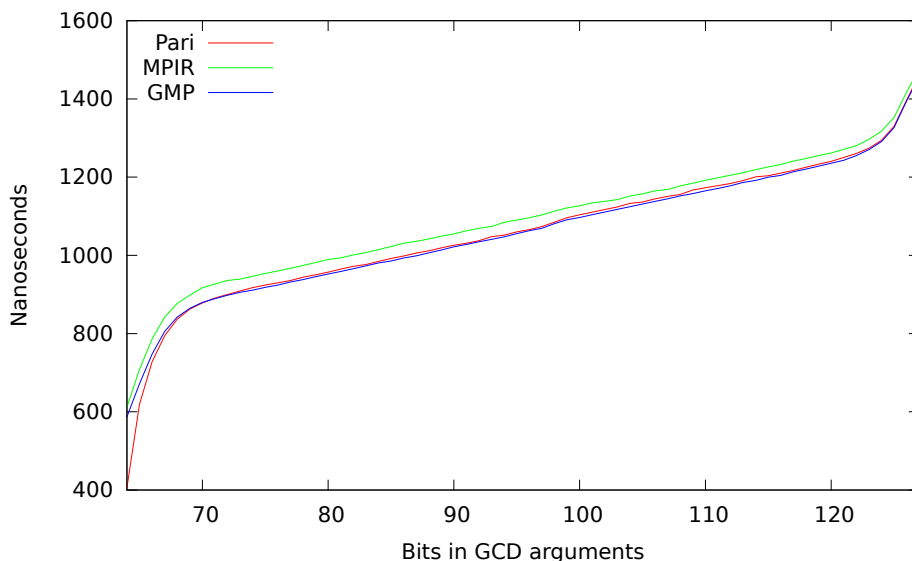


Figure 5.25: Reference Implementations for 128-bit Inputs.

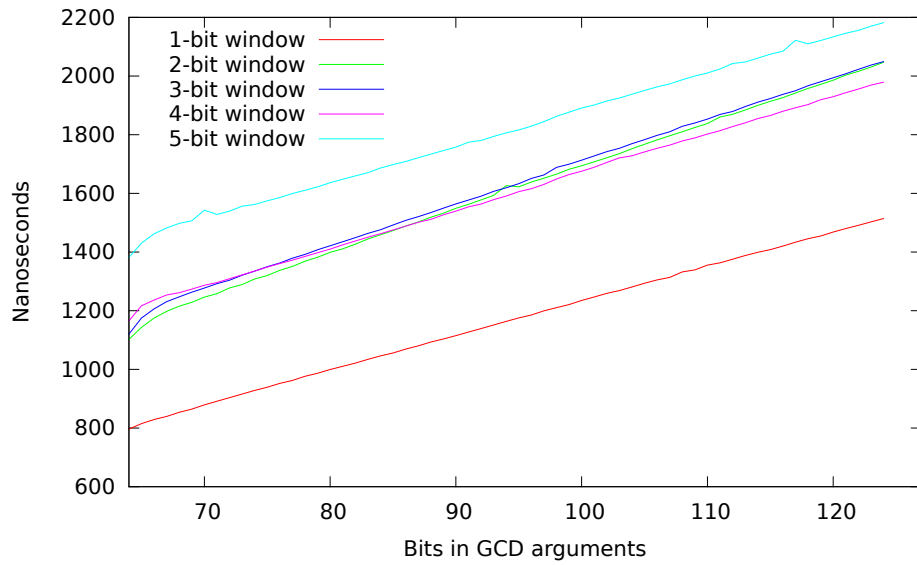


Figure 5.26: All 128-bit Implementations of Stein's Extended GCD.

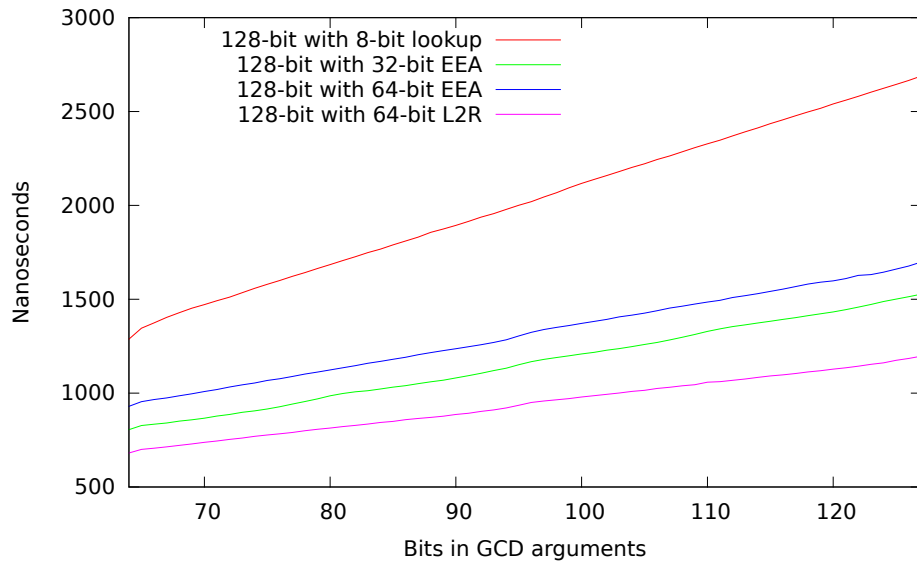


Figure 5.27: Lehmer's Extended GCD for 128-bit Inputs.

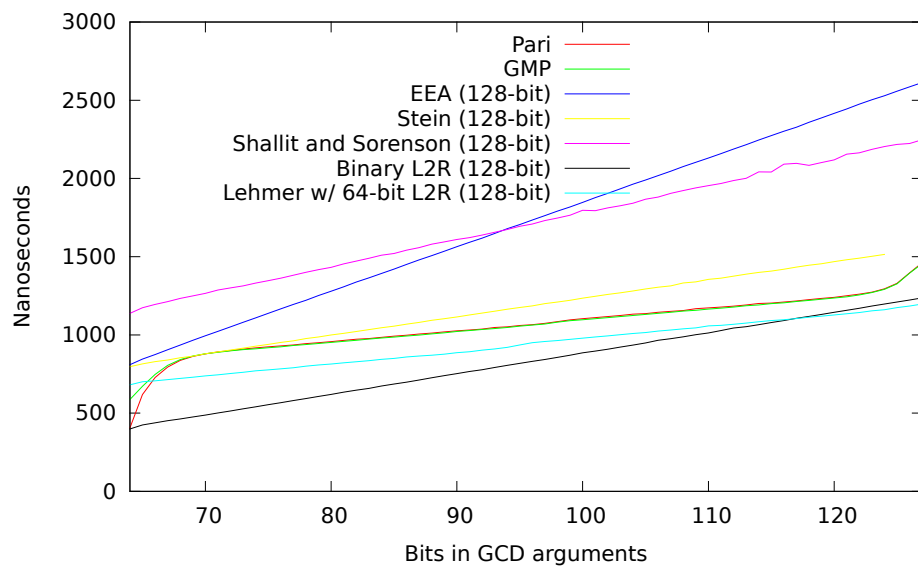


Figure 5.28: Best Extended GCDs for 128-bit Inputs.

5.8 Summary

We briefly restate the results in this section. For inputs less than 32 bits, our 32-bit implementation of the extended Euclidean algorithm is the fastest on average and on average takes roughly 63% of the time of the best reference implementation, which is Pari. For inputs less than 64 bits, our 64-bit implementation of our simplified left-to-right binary extended GCD is the fastest on average and takes on average roughly 77% of the time of the best reference implementation, which, again, is Pari. For inputs less than 119 bits, our 128-bit implementation of our simplified left-to-right binary extended GCD is the fastest on average and typically only takes roughly 95% of the time of either Pari or GMP. For inputs larger than 118 bits, our 128-bit implementation of Lehmer's extended GCD using our simplified left-to-right binary extended GCD for 64 bit words is the fastest on average, taking roughly 88% of the time of Pari or GMP on average.

All of the 32-bit implementations tested in this Chapter were verified to produce correct results for inputs up to 31 bits, with the exception of each of the variants of Stein's right-to-left binary extended GCD, which were only correct for inputs up to 29 bits. Similarly, our 64-bit implementations were verified for inputs up to 63 bits, but our implementations of Stein's right-to-left binary extended GCD were only verified to be correct for inputs no larger than 60 bits. Finally, our 128-bit implementations were verified to be correct for inputs up to 127 bits, with the exception of our 128-bit implementations of Stein's right-to-left binary extended GCD, which were only verified to be correct for inputs no larger than 124 bits. In the case of Stein's right-to-left binary extended GCD, for each implementation, when the algorithm did not verify correctly, it is believe that the variables U and V overflowed.

For each of our implementations (with the exception of our implementations of Stein's right-to-left binary extended GCD), overflows of intermediates do not occur and we did not have to use arithmetic of a larger word size. However, this is not the case for our implementations of Stein's right-to-left binary extended GCD. In this case, we could have

used arithmetic of a larger word size, but this would have made the implementation slower than it currently is and we wanted each of our implementations to be as fast as possible for comparison. Our implementation of Stein’s right-to-left binary extended GCD is not the fastest on average for any bit range, and so altering it to handle intermediate overflow did not seem useful.

This chapter discussed our implementations of the extended GCD and demonstrated a practical improvement on average for inputs less than 128 bits. We introduced a simplified left-to-right binary extended GCD that is based on Shallit and Sorrenson [54] method. Our 32-bit implementation of our simplified left-to-right binary extended GCD takes on average 78% of our 32-bit implementation of Shallit and Sorrenson’s left-to-right binary extended GCD. Our 64-bit implementation takes on average 71% of the time, and our 128-bit implementation takes on average 45% of the time. For this reason, our implementations of our simplified left-to-right binary extended GCD demonstrate a practical improvement on average over our implementations of Shallit and Sorenson’s left-to-right binary extended GCD.

The next chapter studies the performance of our implementations of ideal class arithmetic. Since ideal class arithmetic makes heavy use of the extended GCD, the left-only extended GCD, and the partial extended GCD, we adapted each of our implementations of the full extended GCD to a left-only and partial extended GCD. We then use the results of this chapter to determine which implementation of the full, left-only, and partial extended GCDs to use within our implementations of ideal class arithmetic.

Chapter 6

Ideal Arithmetic Experiments

The previous chapter demonstrated results that lead to practical improvements in computing the extended GCD – a computation that occupies roughly half the time to compute reduced ideal multiplication, squaring, and cubing. This chapter discusses our implementations of ideal arithmetic, which make extensive use of the results of the previous chapter. In particular, when computing either the full, left-only, or partial extended GCD, if arguments are less than 32 bits, we use our 32-bit implementation of the extended Euclidean algorithm. If arguments are larger than 31 bits, but smaller than 64 bits, we use our 64-bit implementation of the simplified left-to-right binary extended GCD. When arguments are larger than 63 bits and smaller than 119 bits, we use our 128-bit implementation of the simplified left-to-right binary extended GCD, and when they are larger than 118 bits, we use our 128-bit implementation of Lehmer’s extended GCD using our 64-bit implementation of our simplified left-to-right binary extended GCD for the inner extended GCD. Finally, when arguments are larger than 127 bits, we use the implementation of the extended GCD and left-only extended GCD provided by GMP, and for the partial extended GCD we use an implementation of Lehmer’s extended GCD using a 64 bit extended Euclidean algorithm for GMP types.

To further improve the performance of ideal arithmetic, specialized implementations using at most a single machine word, i.e. 64 bits, or at most two machine words, i.e. 128 bits were developed. A reference implementation using GMP (or MPIR), that works for unbounded discriminant sizes, was developed for comparison. To verify the correctness of our implementations, we used a randomized test suite. We continuously multiplied, squared, and cubed random prime ideals using each implementation and verified that our 64-bit and

128-bit implementations gave the same result as our reference implementations. In addition, we verified the results of ideal class exponentiation using each of the techniques discussed in Chapter 7 for random exponents up to 16384 bits in size. Our 64-bit implementation of ideal arithmetic has been verified to be correct for negative discriminants less than 60 bits in size, while our 128-bit implementation of ideal arithmetic has also been verified to be correct for discriminants less than 119 bits. This holds for all class group operations such as ideal class multiplication, squaring, and cubing. When verification fails, this is due to arithmetic overflow in our implementation of ideal arithmetic.

As a general rule, in our 64-bit implementation, if intermediate terms fit within a 32 bit word, we use 32 bit arithmetic. If terms do not fit within a 32 bit word, but within a 64 bit word, we use 64 bit arithmetic. Only when terms do not fit within a 64 bit word do we use two machine words and 128 bit arithmetic. Similarly, for our 128-bit implementation, if terms fit within a single 64 bit word, then we use 64 bit arithmetic. Only when terms do not fit within a single 64 bit word do we use two 64 bit words and perform 128 bit arithmetic. In the extreme case that an intermediate term does not fit within two 64 bit words, i.e. the term is larger than 128 bits, only then do we fall back on GMP for multiple precision arithmetic. The size of arithmetic operations used are determined at points in the algorithm where overflow is likely to occur.

Our implementation, available as `libqform` [5], exposes several different interfaces. A user can specifically target our 64 bit, 128 bit, or GMP implementation. In addition, we expose a generic interface that will automatically select the best suited implementation when given a discriminant of a specific size. The best suited implementation is determined by the results of this chapter given in Section 6.2.

The software developed for this chapter uses the GNU C compiler version 4.7.2, and assembly language for 128 bit arithmetic, and was developed on Ubuntu 12.10 using a 2.7GHz Intel Core i7-2620M CPU with 8Gb of memory. Experiments are restricted to using only a

single core of the CPU's four cores. Since we link directly to `libpari`, and we run all the experiments sequentially and on a single thread, we do not call into `libpari` on more than one thread. We were also able to observe the CPU load during the experiments to verify that at most a single CPU core was being used. The timing data occasionally displays an interval that is not smooth. Since repeating the experiment either produces a smooth result, or the non-smooth area occurs at a different time during the experiment, we believe that this is an artefact of the timing measurements and not of the function being timed.

6.1 Specialized Implementations of Ideal Arithmetic

Throughout this thesis, an ideal class $[\mathfrak{a}]$ is represented using the $\mathbb{Z}$ -module basis for the reduced ideal representative $\mathfrak{a} = [a, (b + \sqrt{\Delta})/2]$. Ideal class multiplication and ideal reduction use the value $c = (b^2 - \Delta)/4a$, which is also useful in determining if an ideal is an ambiguous ideal (see Subsection 4.1.1 for additional details). For this reason, our implementation represents an ideal class using the triple (a, b, c) . We represent the ideal class group Cl_{Δ} using the discriminant Δ , and maintain this separately from the representation of each ideal class. We do this to reduce the memory footprint of the representation of each ideal class.

Our implementations of ideal class arithmetic use all the improvements discussed in Subsection 2.3.3 (see [35, Algorithm 6]). This includes separating the computation of

$$s = \gcd(a_1, a_2, (b_1 + b_2)/2) = Ya_1 + Va_2 + W(b_1 + b_2)/2$$

into two GCD computations, the second of which is only computed if the first GCD is not equal to 1. Additional optimizations discussed in this section are an approximation of the termination bound, minimizing the size of intermediates, branch free reduction operations, and the computation of prime ideal classes.

In Subsections 2.3.3, 2.3.4, and 2.3.5 we gave algorithms for fast ideal class multiplication, squaring, and cubing. These compute a termination bound for the partial extended

GCD useful for computing partially reduced coefficients of the product ideal class. The termination bound for ideal multiplication is $\sqrt{a_1/a_2}|\Delta/4|^{1/4}$, for ideal squaring it is $|\Delta/4|^{1/4}$, and for ideal cubing it is $\sqrt{a_1}|\Delta/4|^{1/4}$. In our implementations of ideal class arithmetic, we approximate each of these termination bounds. The results in Section 6.2, particularly in Figures 6.1, 6.2, 6.3, and 6.4, show that using an approximate termination bound is faster on average.

Each of the termination bounds is approximated as follows. In each case, the termination bound contains the coefficient $|\Delta/4|^{1/4}$. Since the coefficients of the partial extended GCD are integers, once the discriminant Δ of the class group Cl_Δ is known, we store both $\lceil |\Delta/4|^{1/4} \rceil$ and $\lceil |\Delta/4|^{1/2} \rceil$ along side Δ in our representation of the class group. These values are computed as accurately as possible, since these are only computed once per class group. The termination bound for ideal class multiplication is approximated as

$$\sqrt{a_1/a_2}|\Delta/4|^{1/4} = \sqrt{(a_1/a_2)|\Delta/4|^{1/2}} \approx \sqrt{(a_1/a_2) \lceil |\Delta/4|^{1/2} \rceil}.$$

For ideal squaring we simply use $\lceil |\Delta/4|^{1/4} \rceil$, and for ideal cubing we use $\sqrt{a_1} \lceil |\Delta/4|^{1/2} \rceil \approx \sqrt{a_1}|\Delta/4|^{1/4}$.

We further approximate the computation of integer square roots when computing the termination bound for fast multiplication. We note that

$$\begin{aligned} x^{1/2} &= 2^{(\log_2 x)/2} \approx 2^{\lfloor (\log_2 x)+1 \rfloor / 2} \\ \Rightarrow x/x^{1/2} &= x/2^{(\log_2 x)/2} \approx x/2^{\lfloor (\log_2 x)+1 \rfloor / 2}, \end{aligned}$$

where $\lfloor (\log_2 x) + 1 \rfloor$ is the number of bits in the binary representation of x . Using this approximation, we compute the integer square root of x as roughly $\lfloor x/2^{\lfloor (\log_2 x)+1 \rfloor / 2} \rfloor$. This is simply a bit shift right by half the number of bits in x .

Other practical considerations are to minimize the size of variables and intermediate values, as smaller operands often lead to faster arithmetic. Recall from Subsection 2.3.1 that $|b| \leq a \leq \sqrt{\Delta/3}$. In our 64-bit implementation, both a and b require only 32 bits at most, so we take advantage of 32 bit operations when possible (such as our 32-bit implementation

of the extended Euclidean GCD from Section 5.1). Similarly in our 128-bit implementation, both a and b fit within one 64 bit machine word. Again, we use 64 bit arithmetic when possible (such as our simplified 64 bit left-to-right binary extended GCD from Section 5.3), and even 32 bit when operands are small. However, since intermediates may be larger, we sometimes require the full 64 bits or 128 bits of the implementation. Cubing sometimes requires even larger intermediates, for example, when computing modulo a_1^2 . Since a_1^2 can be as large as Δ , intermediates before reduction might be larger than Δ in size. Therefore, our 64-bit implementation of cubing requires some 128 bit arithmetic, while our 128-bit implementation uses GMP when necessary.

To keep intermediates small, when possible, we compute using the smallest absolute representative of a residue class. This has the added benefit that it reduces the likelihood of overflow and the necessity of multiple precision arithmetic. For example, Equation (2.7) states that we can compute $U \pmod{a_1/s}$ and Equation (2.9) allows us to compute $b \pmod{2a}$. When an intermediate is known to be close to zero, we do not perform a complete division with remainder, but only add or subtract the divisor as appropriate (often using the signed mask discussed in Section 5.6).

In the specific case of ideal reduction (see Algorithm 2.1), one step is to compute b' such that $-a < b' \leq a$ and $b' \equiv b \pmod{2a}$. To compute $b \pmod{2a}$ we effectively compute $b = q(2a) + r$ for $q, r \in \mathbb{Z}$ where $|r| < 2a$. We note that $q = \lfloor b/(2a) \rfloor = \lfloor (b/a)/2 \rfloor$, and instead compute $b = q'a + r'$ for $q', r' \in \mathbb{Z}$ where $|r'| < a$. Now q' is at least twice q . If q' is even, then $b = q'a + r' = (q'/2)2a + r'$ and we let $b' = r'$. Suppose q' is odd and r' is positive. Then $b = q'a + r' = (q' + 1)a + (r' - a) = ((q' + 1)/2)2a + (r' - a)$. Since $0 < r' < a$, we have $b' = r' - a$ and $-a < b' \leq a$. The proof is similar when q is odd and r' is negative.

This leads to the following implementation, which we optimize to be free of conditionals.

Using two's complement for fixed sized words, we compute

$$\begin{aligned}
b &= q'a + r' && \{\text{division with remainder}\} \\
q_m &= -(q \text{ and}_2 1) && \{q_m \text{ has all bits set when } q \text{ is odd}\} \\
r_m &= \neg \text{sign_mask}(r) && \{r_m \text{ has all bits set when } r \geq 0\} \\
a' &= (a \oplus r_m) - r_m && \{\text{negate } a \text{ when } r_m \text{ is all set}\} \\
r &= r' + (a' \text{ and}_2 q_m) && \{\text{move } r \text{ towards zero when } q \text{ is odd}\} \\
d &= r_m \text{ or}_2 1 && 1 \text{ when } r < 0 \text{ and } -1 \text{ otherwise} \\
q &= (q' - (d \text{ and}_2 q_m))/2 && \{\text{adjust } q \text{ with respect to } r\}
\end{aligned}$$

and finally $b' = r$.

Since we represent an ideal class using the triple (a, b, c) , it is necessary to compute a new value $c' = (b^2 - \Delta)/4a$ after a reduction step. This can be simplified using Tenner's algorithm (see [40, §3.4]) as

$$\begin{aligned}
c &= (b^2 - \Delta)/4a \\
&= ((2aq + r)^2 - \Delta)/4a \\
&= (4a^2q^2 + 4aqr + r^2 - \Delta)/4a \\
&= ((r^2 - \Delta)/4a) + aq^2 + qr \\
&= c' + q(aq + r) \\
&= c' + q(2aq + r - aq) \\
&= c' + q(b - aq) \\
&= c' + q(b + r)/2.
\end{aligned}$$

Using this, we have

$$c' = c - q(b + r)/2.$$

Note, the last step above is obtained by rewriting $b = 2aq + r$ as

$$b - 2aq = r$$

$$2b - 2aq = b + r$$

$$b - aq = (b + r)/2.$$

Since $b - aq \in \mathbb{Z}$, the divide by 2 can be performed with an arithmetic bit shift right.

The final performance improvement is for generating prime ideals. For a given prime p , we would like to find an ideal of the form $[p, (b + \sqrt{\Delta})/2]$. Since $b^2 - 4pc = \Delta$, and p is known, this gives $b^2 \equiv \Delta \pmod{p}$ and $b \equiv \pm\sqrt{\Delta} \pmod{p}$. Computing b is a matter of computing a square root modulo a prime, if one exists. There are efficient algorithms for this (see Bach and Shallit [15]), but since we are interested in finding some prime ideal, rather than a specific prime ideal, we instead use a table of all square roots modulo each prime p for sufficiently many small p . Note, the size of the table is $O(n^2/\log n)$ for n primes. For our purposes, all primes $p < 1000$ is enough. Finding $b \equiv \pm\sqrt{\Delta} \pmod{p}$ is now a matter of looking up $\Delta \bmod p$ from a table.

Having found a value for b , we have not necessarily found a prime ideal $[p, (b + \sqrt{\Delta})/2]$. Since $\Delta = b^2 - 4pc$, this implies that $c = (b^2 - \Delta)/4p$ must have an integral solution. Our implementation maintains the value c for an ideal class, so we compute c and if c is an integer, then we have found a prime ideal. Otherwise, we try again for some other prime (usually the smallest prime greater than p). Note that the table of square roots modulo a prime is part of our library and available as global data. Also, an optimization overlooked in our implementation is to compute the square roots modulo $4p$ for p prime. This way, if a square root exists, c is guaranteed to be integral.

6.2 Average Time for Operations

This section compares the performance of our implementations of ideal class arithmetic with our reference implementation, Pari 2.5.3 [11]. Let k be the target number of bits in the discriminant Δ . We iterate k from 16 to 140, and for each k we repeat the following 10,000 times: We compute two prime numbers p and q such that p is $\lfloor k/2 \rfloor$ bits and q is $k - \lfloor k/2 \rfloor$ bits. We use the negative of their product $\Delta = -pq$ as the discriminant for a class group, unless $\Delta \not\equiv 1 \pmod{4}$. In this case, we try again with a different p and q . We then pick a prime ideal class, $[\mathfrak{a}]$, randomly from among the primes less than 1000. One motivation for limiting this to primes less than 1000 is the size of the lookup table for $\sqrt{\Delta \bmod p} \pmod{p}$.

For ideal multiplication, squaring, and cubing, we use Algorithms 2.2, 2.3, and 2.4 respectively. For ideal reduction we use Algorithm 2.1. Each of the optimizations mentioned in this chapter is applied to each implementation, as well as the results from Chapter 5 for computing the extended GCD. The general strategy for our 64-bit implementation is to use 32 bit arithmetic when operands are less than 32 bits, otherwise to use 64 bit arithmetic, and only when this fails do we use 128 bit arithmetic. For our 128-bit implementation, we use 64 bit arithmetic when operands are less than 64 bits, otherwise we use 128 bit arithmetic, and only when intermediates are larger than 128 bits do we use GMP arithmetic.

To time the performance of squaring and cubing, we iterate 1000 times either $[\mathfrak{a}] \leftarrow [\mathfrak{a}]^2$ or $[\mathfrak{a}] \leftarrow [\mathfrak{a}]^3$ respectively. Dividing the total cost by 10,000,000 gives the average time to square or cube an ideal. For multiplication, we initially set $[\mathfrak{b}] \leftarrow [\mathfrak{a}]$. We then iterate 1000 times the sequence of operations $[\mathfrak{c}] \leftarrow [\mathfrak{a}] \cdot [\mathfrak{b}]$, $[\mathfrak{a}] \leftarrow [\mathfrak{b}]$, and $[\mathfrak{b}] \leftarrow [\mathfrak{c}]$.

To time the performance of Pari, we statically link with the Pari library. We determine the discriminant and prime ideal class in exactly the same way as with the other implementations. We then convert the ideal class into a Pari GEN object using `qfi`, Pari's function to create a binary quadratic form. The only operations timed are calls to `gmul`, in the case of multiplication, `gsqr`, in the case of squaring, and `gpowsq` (or a combination of `gmul` and

`gsqr`), for cubing. Pari does not appear to implement ideal class cubing and so we timed both the general powering function, `gpows`, as well as multiplication with the square. We perform garbage collection in Pari by recording Pari's stack pointer, `avma`, and then restoring it after all the timings are completed.

Figures 6.1, 6.2, 6.3, and 6.4 compare the average time to multiply two ideals or to cube an ideal using either the approximate termination bound for the partial extended GCD or the full termination bound. We show this for both our 64-bit and 128-bit implementations. In each case, it is faster on average to use the approximate termination bound.

Figures 6.5, 6.6, and 6.7 show the average time for ideal multiplication, squaring, and cubing respectively, alongside the timings for each operation using the reference implementation from Pari. For all discriminant sizes, our 64-bit implementation performs faster than our 128-bit implementation, which performs faster than our GMP implementation. Table 6.1 shows the relative costs on average for ideal multiplication, squaring, and cubing for each implementation compared with Pari. For our 64-bit implementation, we only average the times for discriminants less than 60 bits in size, for our 128-bit implementation, we use the times for discriminants less than 119 bits in size, and for our GMP implementation, we use the times for discriminants less than 140 bits in size.

In Figures 6.8, 6.9, 6.10, and 6.11 we compare the average time to cube against the average time to multiply a representative with its square. For both our 64-bit implementation and our GMP implementation, cubing is faster than multiplying with the square. For Pari, multiplying an ideal with its square (using `gmul` and `gsqr`) is faster than using the general powering function (`gpows`).

One thing to notice about Figure 6.9 is that our 128-bit implementation of cubing performs more poorly than that of multiplying an ideal with its square when the discriminant is larger than 69 bits. In order to improve the performance of our 128-bit implementation of cubing, we implemented several code paths depending on the predicted size of intermediate

operands. When operands are likely to be larger than 128 bits, we use GMP for multiple precision arithmetic. Profiling revealed that for discriminants smaller than 65 bits, none of the GMP code paths were being executed, but that as the discriminant size grew above 65 bits, the GMP code paths were being executed more frequently.

The GMP code paths are only necessary in a few places for our 128-bit implementation of ideal cubing. First, recall that the input to ideal cubing as described by Algorithm 2.4 is an ideal $\mathfrak{a} = [a_1, (b_1 + \sqrt{\Delta})/2]$ where a_1 may be as large as $\sqrt{|\Delta|/3}$ (see Subsection 2.3.1). This means that a_1^2 may be as large as Δ in size. There are three equations in Algorithm 2.4 that compute using a_1^2 ,

$$\begin{aligned} U &= Y'c_1(Y'(b_1 - Y'c_1a_1) - 2) \bmod a_1^2 && \{\text{line 4}\} \\ U &= -c_1(XY'a_1 + Yb_1) \bmod a_1^2/s && \{\text{line 7}\} \\ M_2 &= \frac{R_i(b_1 + Ua_1) - sC_i c_1}{a_1^2} && \{\text{line 20}\}. \end{aligned}$$

We tried several methods to compute each of the above. The method that worked the best in practice is as follows. When computing U , we look at the size of a_1^2 or a_1^2/s . If this value is larger than 64 bits, then division with remainder uses a 128 bit denominator, namely a_1^2 or a_1^2/s . Since the denominator is a 128 bit argument, we use GMP for multiple precision arithmetic. In this case, we compute U without computing the remainder until the very last step. If the denominator is smaller than 64 bits, then we use 64 bit arithmetic and compute the remainder after each multiplication. When computing M_2 , on the other hand, the method that works best in practice is to estimate the size of the numerator. If the numerator is predicted to be larger than 128 bits, then we use GMP for multiple precision arithmetic. Otherwise, we use 128 bit arithmetic.

For computing U , we tried two additional methods. For each multiplication performed, we estimate the size of the product. If the predicted size is larger than 128 bits, then we use GMP to compute the product, and otherwise we use 128 bit arithmetic. The two methods here differ as to when we compute the remainder modulo a_1^2 or a_1^2/s . In the first case,

we compute the remainder after each product, and in the second case we only compute the remainder if the product is larger than a_1^2 or a_1^2/s as appropriate. The results are shown in Figure 6.12. These two methods perform essentially the same, and both are slower than the method described above.

	64-bit Implementation / Pari
Multiplication	0.23405
Squaring	0.26703
Cubing	0.17120
	128-bit Implementation / Pari
Multiplication	0.28567
Squaring	0.26703
Cubing	0.23718
	GMP Implementation / Pari
Multiplication	0.96476
Squaring	0.84217
Cubing	0.55412

Table 6.1: Relative cost of each implementation on average compared with Pari.

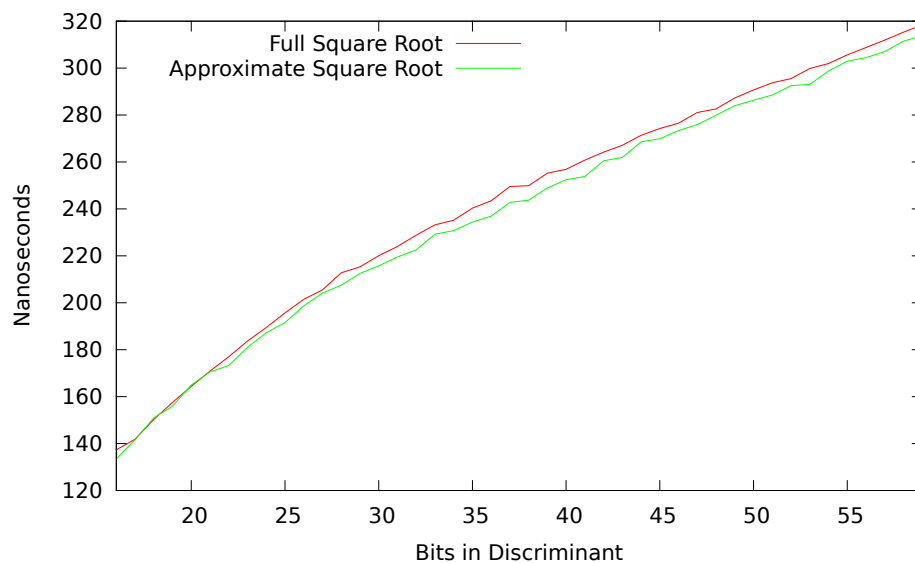


Figure 6.1: Full or Approximate Square Root for 64 bit Ideal Multiplication

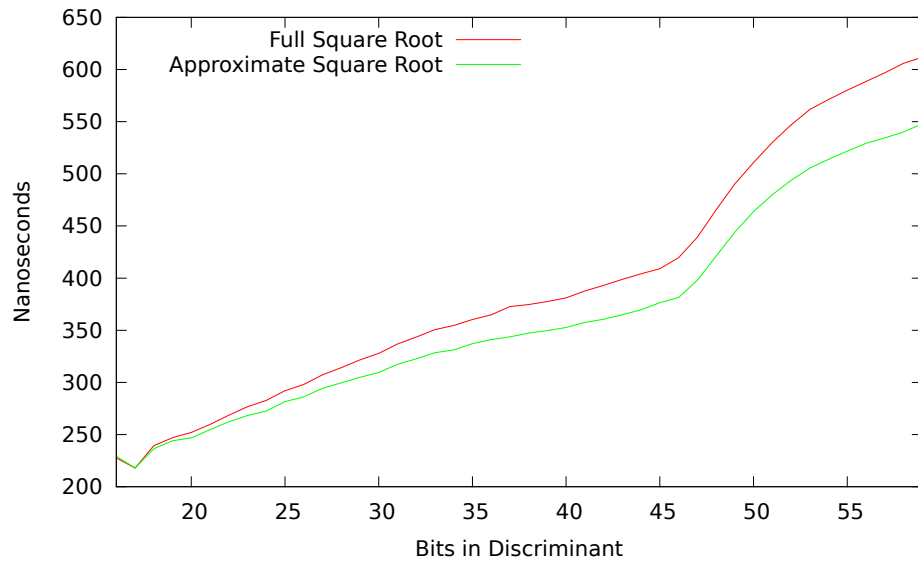


Figure 6.2: Full or Approximate Square Root for 64 bit Ideal Multiplication

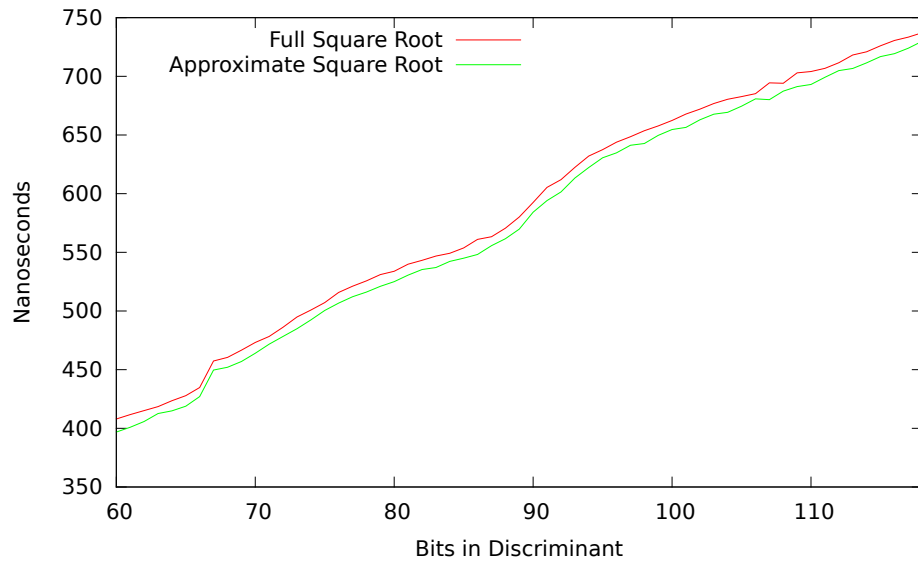


Figure 6.3: Full or Approximate Square Root for 128 bit Ideal Multiplication

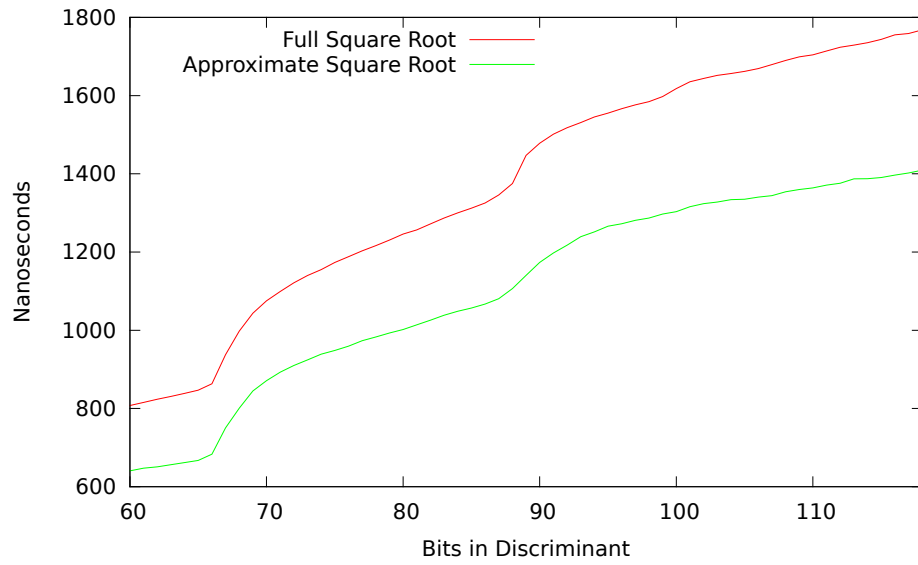


Figure 6.4: Full or Approximate Square Root for 128 bit Ideal Cubing

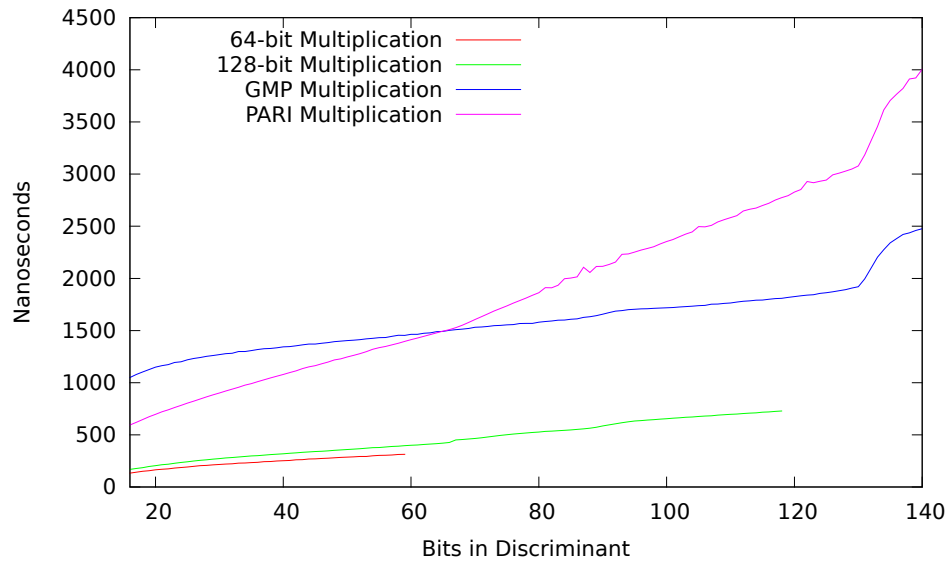


Figure 6.5: Average Time to Multiply Reduced Ideal Class Representatives.

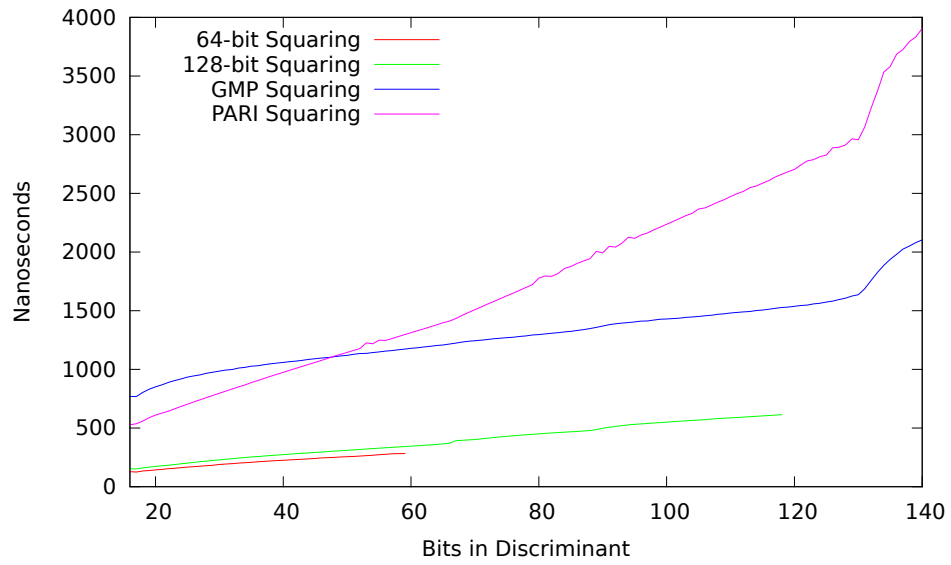


Figure 6.6: Average Time to Square Reduced Ideal Class Representatives.

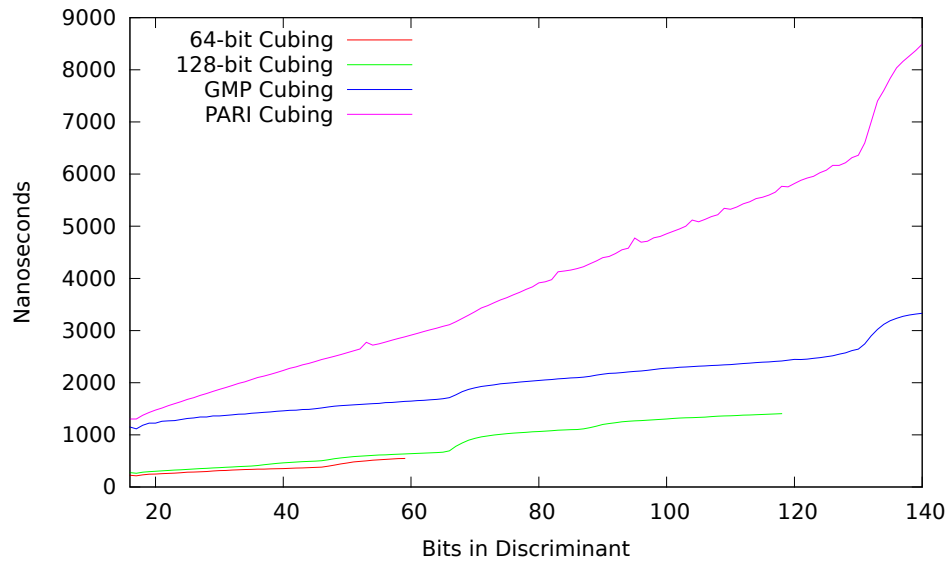


Figure 6.7: Average Time to Cube Reduced Ideal Class Representatives.

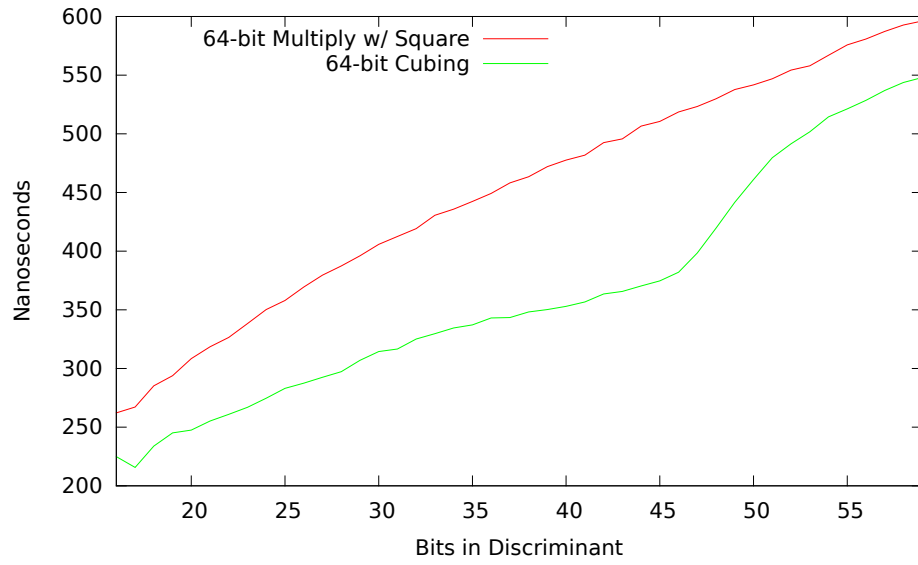


Figure 6.8: Time to compute $[\mathfrak{a}^3]$ in our 64-bit implementation.

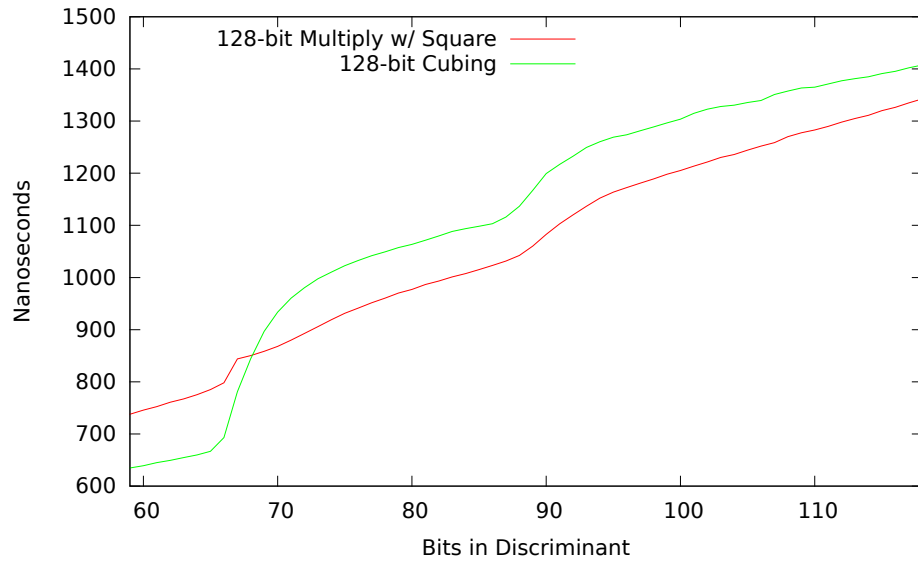


Figure 6.9: Time to compute $[\mathfrak{a}^3]$ in our 128-bit implementation.

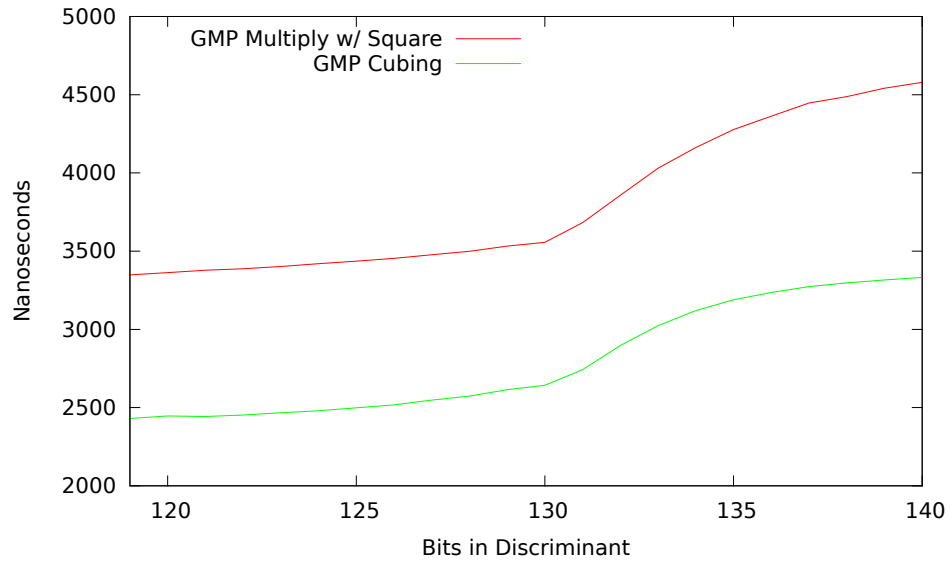


Figure 6.10: Time to compute $[\mathfrak{a}^3]$ in our GMP implementation.

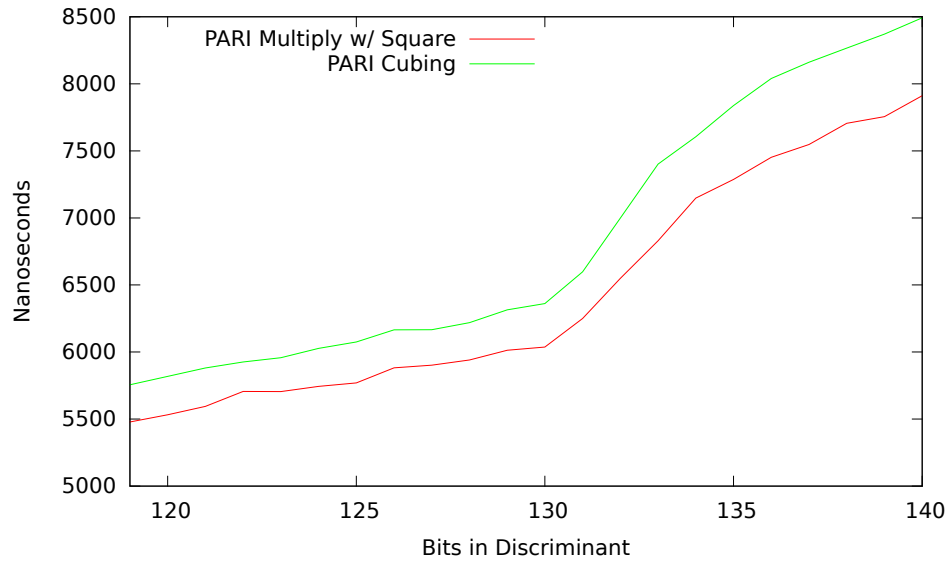


Figure 6.11: Time to compute $[\mathfrak{a}^3]$ using Pari.

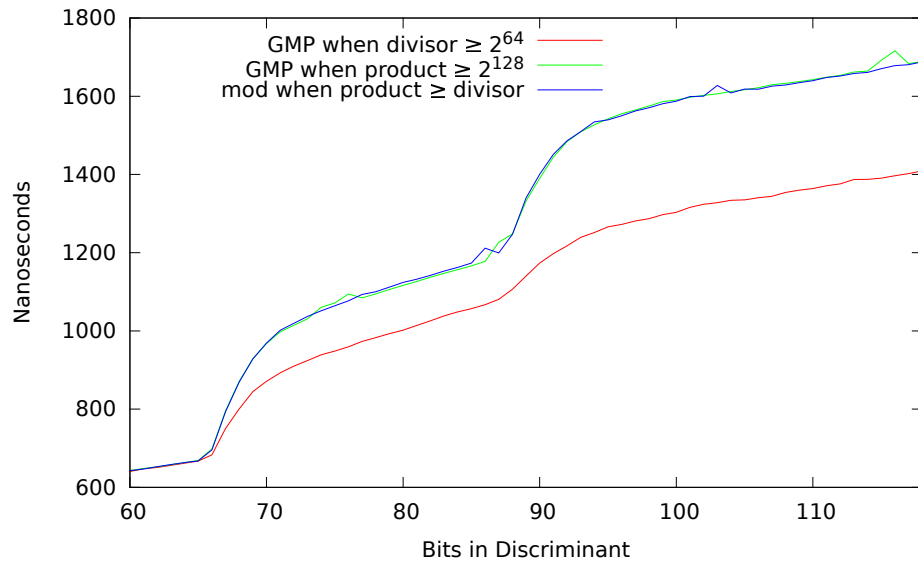


Figure 6.12: Methods of Handling a_1^2 in 128 bit Cubing.

6.3 Summary

This chapter discussed our implementations of ideal class arithmetic. We then demonstrated the average time for multiplication, squaring, and cubing. We showed that for discriminants smaller than 60 bits, our 64-bit implementation of ideal class arithmetic is significantly faster than the reference implementation provided by Pari. For discriminants larger than 59 bits, but smaller than 119 bits, our 128-bit implementation of ideal class arithmetic is significantly faster than the reference implementation provided by Pari. Finally, for discriminants larger than 118 bits, our GMP implementation is still faster on average than the reference implementation provided by Pari.

Chapter 7

Exponentiation Experiments

Subsection 4.2.1 discusses the bounded primorial steps algorithm of Sutherland [58, §4.1] – an order finding algorithm for generic groups. Part of this algorithm is to exponentiate a group element to a power primorial, i.e. the product of many small prime powers. Since this exponent is determined by a given bound, when the range of bounds is known in advance, an efficient representation of each exponent can also be computed in advance. Section 4.2 discussed the SuperSPAR integer factoring algorithm. This algorithm uses a variant of the bounded primorial steps algorithm to compute the order of an ideal class. C. Siegel [23, p.247] gave a bound on the class number (see Section 2.2), and for this reason, efficient representations of the exponent used by the bounded primorial steps portion of the SuperSPAR algorithm can be computed in advance. This also holds for any algorithm using the bounded primorial steps algorithm where bounds on the order of a group element are known in advance.

This chapter discusses several approaches to exponentiating an ideal class to large exponents that are the product of all prime powers less than some bound (known as *power primorials*), particularly where the set of exponents is known in advance. We begin by recalling exponentiation techniques from Chapter 3. These include techniques that use signed and unsigned base 2 representations, right-to-left and left-to-right double base representations (for bases 2 and 3), and a tree-based approach.

In Section 7.6, we propose a method that combines the left-to-right 2,3 representations of Berthé and Imbert [16] (see Subsection 3.4.2 and Section 7.4) with a method of iterating on the best partial representations of Doche and Habsieger [28] (see Subsection 3.4.3 and Section 7.5). The results of Section 7.10 show that this new method is faster on average for

our implementations of ideal class arithmetic and power primorial exponents.

In Section 7.7, we propose two extensions to a method by Imbert and Philippe [36, §3] (see Subsection 3.4.4) that, under certain constraints, are able to exhaustively compute all such 2,3 representations of a given exponent. Then in Section 7.8, we propose an alternative approach to computing 2,3 representations. Rather than searching for a representation for a particular exponent, we instead generate a set of representations for many exponents. For each exponent, there may be several representations in the set and only the “best” representation for each exponent is returned. Unfortunately, the techniques of Section 7.7 and Section 7.8 appear to require an exponential amount of time with respect to the size of the largest exponent used. For this reason, we limit the use of these techniques to computing representations for all exponents of 16 bits or less. This still has practical uses, since we can represent a larger exponent by partitioning it into 16 bit blocks or by using its factorization.

Finally, we compare each method on a sequence of primorials to find the best method in practice. Binary exponentiation and non-adjacent form exponentiation are used as reference implementations since they are widely used in practice, and they are the only two implementations discussed here that do not make use of ideal class cubing.

The results in Section 7.10 use the results from the previous chapter, i.e. Section 6.2. For our 64-bit implementation, the largest discriminants supported are those that fit within 59 bits, and for our 128-bit implementation, the largest discriminants fit within 118 bits. The average cost of arithmetic in class groups with discriminants of these sizes represents an upper bound on the average cost of arithmetic for each implementation. For this reason, our exponentiation experiments focus on improving the time to exponentiate assuming the average cost of operations for 59 bit and 118 bit discriminants. Fixing the size of the discriminants also means that the average time to exponentiate varies only with the representation computed by a particular technique. For each technique discussed in this chapter, we assume that a 2,3 representation is computed in advance and then use the results from Section 6.2

to compute the average cost of exponentiation using Algorithm 3.3 from Subsection 3.3.2. Future work could time the actual exponentiation of a large set of ideal classes in order to compute an average. However, a concern about this is that if the set of ideal classes is too small, the average time may not be representative.

7.1 Binary and Non-Adjacent Form

Sections 3.1 and 3.2 discussed binary exponentiation and non-adjacent form exponentiation in detail. Briefly again, binary exponentiation uses the binary representation of an exponent n . The representation can be parsed from high-order to low-order or from low-order to high-order – we typically refer to these methods as left-to-right or right-to-left respectively. Using either approach, we use $\lfloor \log_2 n \rfloor$ squares and $\lfloor \log_2 n \rfloor / 2$ multiplications on average.

A non-adjacent form exponentiation uses a signed base 2 representation of the exponent such that no two non-zero terms are adjacent. More formally, non-adjacent form is written as $n = \sum s_i 2^i$ for $s_i \in \{0, -1, 1\}$, and by “non-adjacent” we mean that $s_i \cdot s_{i+1} = 0$ for all i . Similar to binary exponentiation, when computing a non-adjacent form we can parse the exponent from left-to-right or from right-to-left. Either direction, we use $\lfloor \log_2 n \rfloor + 1$ squares and $(\lfloor \log_2 n \rfloor + 1)/3$ multiplications on average.

The results in Section 7.10 compute the average time to exponentiate using Algorithm 3.3 from Subsection 3.3.2 given a 2,3 representation of an exponent. Since the representation is unique for both binary and non-adjacent form, we compute the representation using right-to-left variants of each.

7.2 Right-to-Left 2,3 Chains

In Subsection 3.4.1, we describe a method from Ciet et al [18, Figure 8] for computing 2,3 chains from low-order to high-order that is a natural extension of the binary representation or non-adjacent form of an exponent. In this method, we reduce the exponent n by 2

while it is even, and by 3 while it is divisible by 3, at which point either $n \equiv 1 \pmod{6}$ or $n \equiv 5 \pmod{6}$ and we add or subtract 1 so that n is a multiple of 6. The resulting partition of n is then reversed such that the number of squares or cubes in successive terms is monotonically increasing and we can use Algorithm 3.2 from Subsection 3.3.1 to compute the exponentiation.

Since this approach evaluates the exponent modulo 3 and may reduce the exponent by 3, efficient methods to compute $n \bmod 3$ and $n/3$ will speed the computation of the chain. This does not, however, improve the time to exponentiate if the representation is computed in advance, as in Section 7.10.

To begin, notice that $4 \equiv 1 \pmod{3}$. As such, we can write n as

$$n = 4 \lfloor n/4 \rfloor + (n \bmod 4) \equiv \lfloor n/4 \rfloor + (n \bmod 4) \pmod{3}$$

and then recursively write $\lfloor n/4 \rfloor \pmod{3}$ the same way. This provides us with a method to quickly compute $n \bmod 3$ – we partition the binary representation of n into 2 bit blocks and sum each block modulo 4. The resulting sum s is $s \equiv n \pmod{3}$. We further improve the performance of this algorithm by noting that $4^m \equiv 1 \pmod{3}$. Since our target architecture (and language) has 64 bit words, we partition the binary representation into 64 bit blocks and sum each block modulo 2^{64} . We then add each 32 bit word of the resulting sum modulo 2^{32} , then each 16 bit word of the sum modulo 2^{16} , and finally each 8 bit word modulo 2^8 . We look up the final answer modulo 3 from a table of 256 entries (see Algorithm 7.1).

Division by 3 is relatively expensive when compared to computing the remainder. A common approach is to precompute a single word approximation of the inverse of 3, and then to use multiplication by an approximation of the inverse and then to adjust the result (see [33, 60, 47] for additional information). As the GNU Multiple Precision (GMP) library implements exact division by 3, we use this.

Algorithm 7.1 Fast $n \bmod 3$ (adapted from Hacker’s Delight [60]).

Input: $n \in \mathbb{Z}$.

Output: $n \bmod 3$.

```

1:  $s \leftarrow 0$ 
2: for  $i$  from 0 to  $\lfloor \log_2 n \rfloor$  by 64 do
3:    $t \leftarrow \lfloor n/2^i \rfloor \bmod 2^{64}$ 
4:    $s \leftarrow (s + t) \bmod 2^{64}$ 
5:  $s \leftarrow (s + \lfloor s/2^{32} \rfloor) \bmod 2^{32}$ 
6:  $s \leftarrow (s + \lfloor s/2^{16} \rfloor) \bmod 2^{16}$ 
7:  $s \leftarrow (s + \lfloor s/2^8 \rfloor) \bmod 2^8$ 
8: return  $s \bmod 3$ 

```

{Lookup from a table with 256 entries.}

7.3 Windowed Right-to-Left Chains

Windowing improves the performance of right-to-left binary exponentiation and right-to-left binary GCD computations. Similarly, windowing is used to improve the performance of a right-to-left 2,3 chain (see [29]). We experimented with a window size of $2^2 3^2$. Again, while the exponent is even, we reduce it by 2, and while it is divisible by 3, we reduce it by 3. In a non-windowed variant, we add or subtract 1 to make the remainder a multiple of 6. In the windowed variant, we evaluate the exponent n modulo the window $2^2 3^2 = 36$, which gives us $n \equiv 1, 5, 7, 11, 13, 17, 19, 23, 25, 29, 31, 35 \pmod{36}$. As there are 12 different residue classes, and for each residue class we could either add or subtract 1 from n , we have 2^{12} different strategies to evaluate. For each strategy, we measured the cost to exponentiate the primorials P_{100k} for $1 \leq k \leq 50$. For 48 out of 50 of the primorials tested, the same strategy lead to the cheapest cost of exponentiation, which was to subtract 1 from n when $n \equiv 1, 5, 13, 17, 25, 29 \pmod{36}$ and to add 1 otherwise. For the primorial P_{200} , this particular strategy was the third most efficient, and for P_{500} it was the second most efficient.

Since the windowed variant of a 2,3 chain computes $n \bmod 36$, we give a fast method for this. First write $n = 4x + r_4$ for integers x and r_4 such that $0 \leq r_4 < 4$. Then write $x = 9y + r_9$ for integers y and r_9 such that $0 \leq r_9 < 9$. Substituting the second equation

into the first we have

$$\begin{aligned} n &= 4(9y + r_9) + r_4 \\ &= 36y + 4r_9 + r_4. \end{aligned}$$

Notice that $(4r_9 + r_4)$ is $n \bmod 36$ where $r_4 = n \bmod 4$ and $r_9 = \lfloor n/4 \rfloor \bmod 9$. To compute $n \bmod 9$, we point out that $64 \equiv 1 \pmod{9}$. Similar to the case of computing $n \bmod 3$, we write

$$n = 64 \lfloor n/64 \rfloor + (n \bmod 64) \equiv \lfloor n/64 \rfloor + (n \bmod 64) \pmod{9}$$

and recursively write $\lfloor n/64 \rfloor \pmod{9}$. To compute $n \bmod 9$, we partition n into blocks of 6 bits (since $2^6 = 64$ and $64 \equiv 1 \pmod{9}$) and sum each 6 bit block modulo 64. We then compute the sum $s \bmod 9$. Again, since our target architecture has 64 bit machine words, we improve upon this by noting that 64 divides 192 and that $2^{192} \equiv 1 \pmod{9}$. Using this, we first partitioning n into 192 bit blocks, and then compute an intermediate sum of each 192 bit block. We then partition the intermediate sum into 6 bit blocks and compute the final solution.

7.4 Left-to-Right 2,3 Representations

In Subsection 3.4.2 we describe a greedy algorithm for computing 2,3 representations from high-order to low-order by Berthé and Imbert [16]. Briefly, for a given exponent n and for $a_i, b_i \in \mathbb{Z}_{\geq 0}$ and $s_i \in \{-1, 1\}$, we compute the term $s_i 2^{a_i} 3^{b_i}$ such that $|n - s_i 2^{a_i} 3^{b_i}|$ is as small as possible. We then recurse on the result $n - s_i 2^{a_i} 3^{b_i}$. As is, this produces unchained representations, but a chain can be generated by restricting each a_i and b_i such that it is no greater than the a_{i-1} and b_{i-1} from the previous term. Placing a bound on the number of squares and cubes can also be done globally, i.e. for every a_i, b_i we have $0 \leq a_i \leq A$ and $0 \leq b_i \leq B$. When a bound is applied globally, the cost of a left-to-right chain or representation varies as the global bound varies.

Figure 7.1 shows the average time to exponentiate using a left-to-right representation, for an exponent that is the product of the first 1000 primes (a number with 11271 bits), while the global bound A varies. The time to compute the representation is not included; only the average time to exponentiate using the representation is given here. The horizontal axis indicates a global bound A such that all $a_i \leq A$. A global bound B is chosen for the exponent n such that $B = \lceil \log_3(n/2^A) \rceil$. The end points of the graph correspond to a representation that uses only cubing and multiplication (on the left side) and a representation that uses only squaring and multiplication (on the right side).

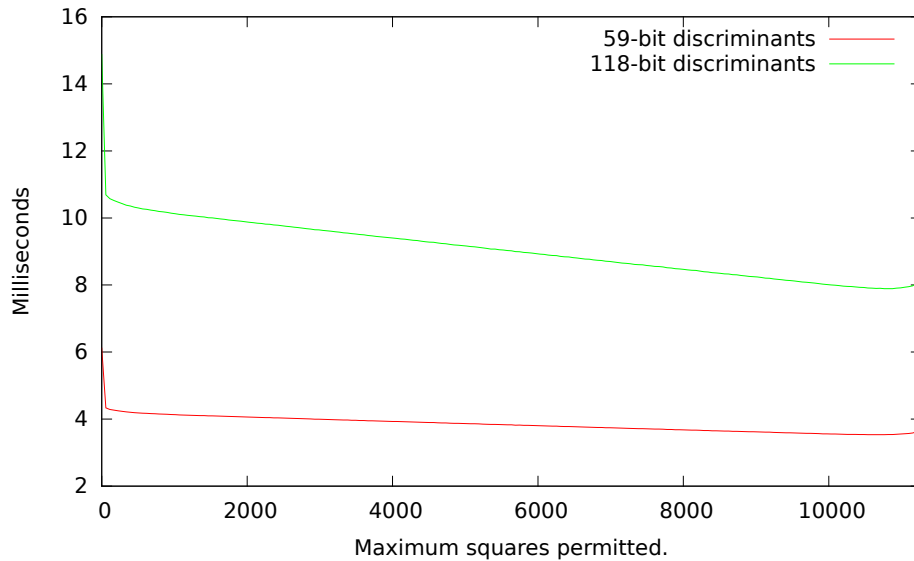


Figure 7.1: The time to exponentiate by the 1000th primorial, n , while varying the maximum number of squares permitted, A . The maximum number of cubes is $B = \lceil \log_3(n/2^A) \rceil$.

In general, bounding the maximum number of squares and cubes gives representations that lead to faster exponentiation in the ideal class group than representations where the number of squares or cubes is left unbounded. For this particular exponent, a representation generated without a global bound takes approximately 7.3 milliseconds for a 59 bit discriminant and 17.2 milliseconds for a 118 bit discriminant. This is slower than all bounded representations for both implementations.

Section 7.10 compares the average time to exponentiate an ideal class to a power primorial

given a 2,3 representation of that power primorial. The 2,3 representation is assumed to be known in advance and for this reason, the results do not include the time to generate the 2,3 representation. When computing representations using the technique described here, we compute representations using global bounds A and B by iterating B such that $0 \leq B \leq \lceil \log_3 n \rceil$ and $A = \lceil \log_2(n/3^B) \rceil$. We iterate B rather than A as $\lceil \log_3 n \rceil \leq \lceil \log_2 n \rceil$. For each representation, we use the timing results of Section 6.2 in order to estimate the average time to exponentiate an ideal class using that representation, and then the representation with the best average time is used for comparisons with other techniques.

7.5 Pruned Trees

A right-to-left 2,3 chain can be generated by repeatedly reducing the exponent by 2 and 3 and then either adding or subtracting 1 and repeating this process until the exponent is 1 [18, Figure 8]. In the windowed variant [29], after reducing the exponent by 2 and 3, we had 12 residues modulo $2^2 3^2$ and this lead to 2^{12} different strategies for adding or subtracting 1 from the residue.

In Subsection 3.4.3, we discussed a tree based method, from Doche and Habsieger [28], where a set of at most L *partial representations* is maintained (a partial representation can be written as $n = x + \sum s_i 2^{a_i} 3^{b_i}$). At each iteration, the x term from each partial representation generates two new elements $x - 1$ and $x + 1$. After reducing each new element by powers of 2 and 3, at most L representations with the *smallest* x are kept (or for two partial representations with the same x , when the cost of the 2,3 summation is less). More formally, an element x generates new integral elements $(x \pm 1)/(2^c 3^d)$.

Doche and Habsieger suggest a generalization of this approach where each element x generates a set of integral elements $(x \pm y)/(2^c 3^d)$ for values of y chosen from some fixed set [28, §5]. Here we consider two possibilities. The first generates integer values of the form $(x \pm 2^a 3^b)/(2^c 3^d)$, while the second uses the form $(x \pm 2^a \pm 3^b)/(2^c 3^d)$. The assumption is

that by adjusting x by more than just 1, we increase the likelihood of finding a multiple of a large $2^c 3^d$ window. Notice that the first variation includes the forms $(x \pm 1)/(2^c 3^d)$ and so we expect it to typically give representations no worse than the method suggested by Doche and Habsieger [28] (covered in Subsection 3.4.3). The results in Section 7.10, particularly Figures 7.10 and 7.11 also indicate this.

For both variations, we bound a and b such that $0 \leq a \leq U$ and $0 \leq b \leq U$. Figure 7.2 shows the average time to exponentiate by the 1000th primorial as the bound U increases (using the $k = 4$ best partial representations). As U increases, computing representations using this approach quickly becomes intractable. We found that for our purposes, $U = 16$ is an acceptable trade off between the average time to exponentiate and the time to generate the 2,3 representation.

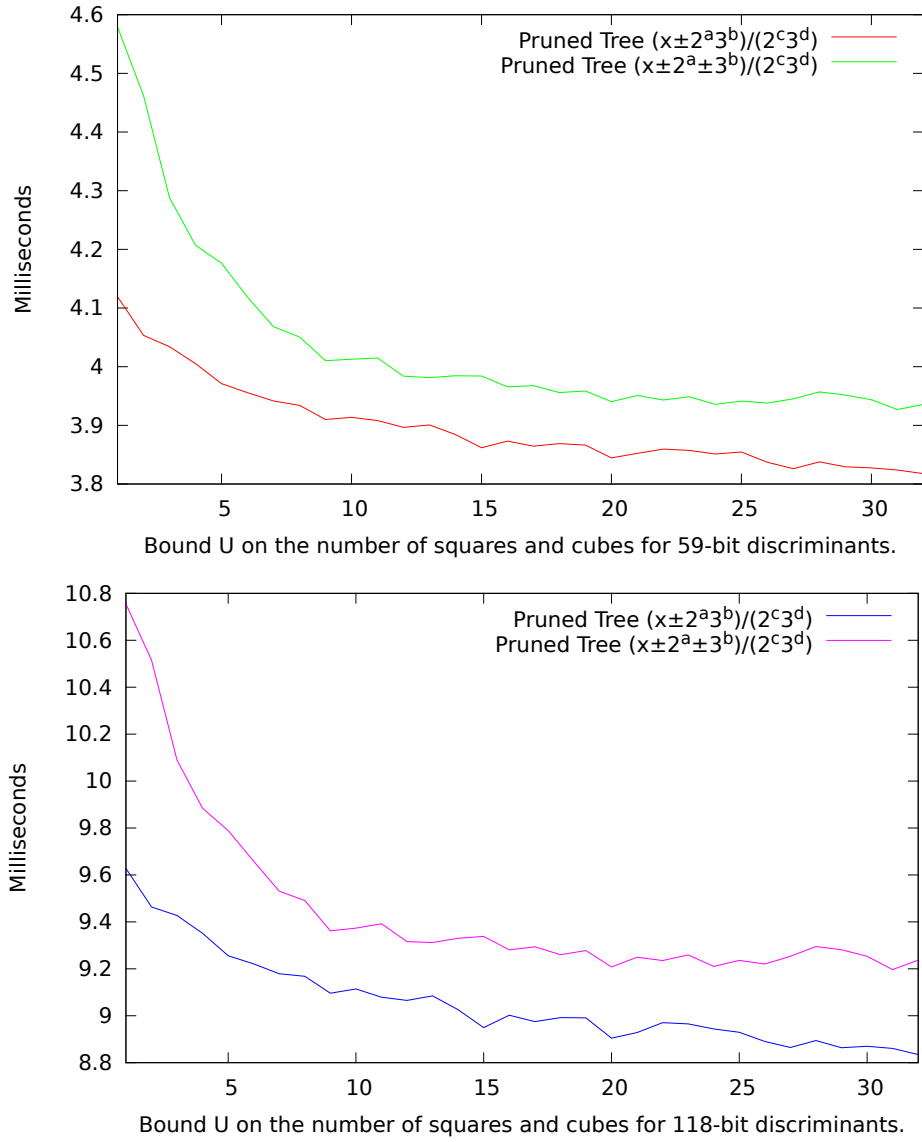


Figure 7.2: Performance of the bounds $a, b \leq U$ when exponentiating by the primorial P_{1000} .

7.6 Left-to-Right Best Approximations

In this section, we propose a method that combines the approach of maintaining a set of the L best partial representations, suggested by Doche and Habsieger [28] (see Section 7.5), with that of computing left-to-right 2,3 representations, suggested by Berthé and Imbert [16] (see Section 7.4). The results of Section 7.10 show that this method gives the best performance on average of the methods studied when exponentiating by power primorials.

For an integer x we say that $2^a 3^b$ is a *best approximation* of $|x|$ when $||x| - 2^a 3^b|$ is minimal. The algorithm for the left-to-right representation (Algorithm 3.5) finds a best approximation for x and then repeats on the positive difference. What we propose here is that instead of only iterating on the best approximation, we iterate on the L best approximations. In other words, each value from the set of partial representations generates new values of the form $||x| - 2^a 3^b|$ (being careful to record the sign of x), and then at most L of the best partial representations are retained for some bound L .

Let A and B represent global bounds on the exponents 2^a and 3^b such that $0 \leq a \leq A$ and $0 \leq b \leq B$. To compute the set of best approximations for x , we iterate b from $0 \leq b \leq B$ and let $a_1 = \lfloor \log_2(x/3^b) \rfloor$ and $a_2 = \lceil \log_2(x/3^b) \rceil$ bounding both $a_1, a_2 \leq A$. Note that using $a_2 = a_1 + 1$ is sufficient since either $\lceil \log_2(x/3^b) \rceil = \lfloor \log_2(x/3^b) \rfloor$ or $\lceil \log_2(x/3^b) \rceil = \lfloor \log_2(x/3^b) \rfloor + 1$. We then use $||x| - 2^{a_1} 3^b|$ and $||x| - 2^{a_2} 3^b|$ as candidates for the new set. Since there are at most L partial representations, each element x from a partial representation generates many new partial representations, and we maintain at most L of the best representations from the union of each set.

As in Section 7.4 on left-to-right 2,3 representations, iterating the bound on the number of squares and cubes varies the cost of the representations generated. When the exponent n is known in advance, as is the assumption in Section 7.10, we iterate the global bounds A and B for $0 \leq B \leq \lceil \log_3 n \rceil$ and $A = \lceil \log_2(n/3^B) \rceil$. The average time to exponentiate using each representation is then estimated using the results from Section 6.2, and the representation

with the best average time is returned.

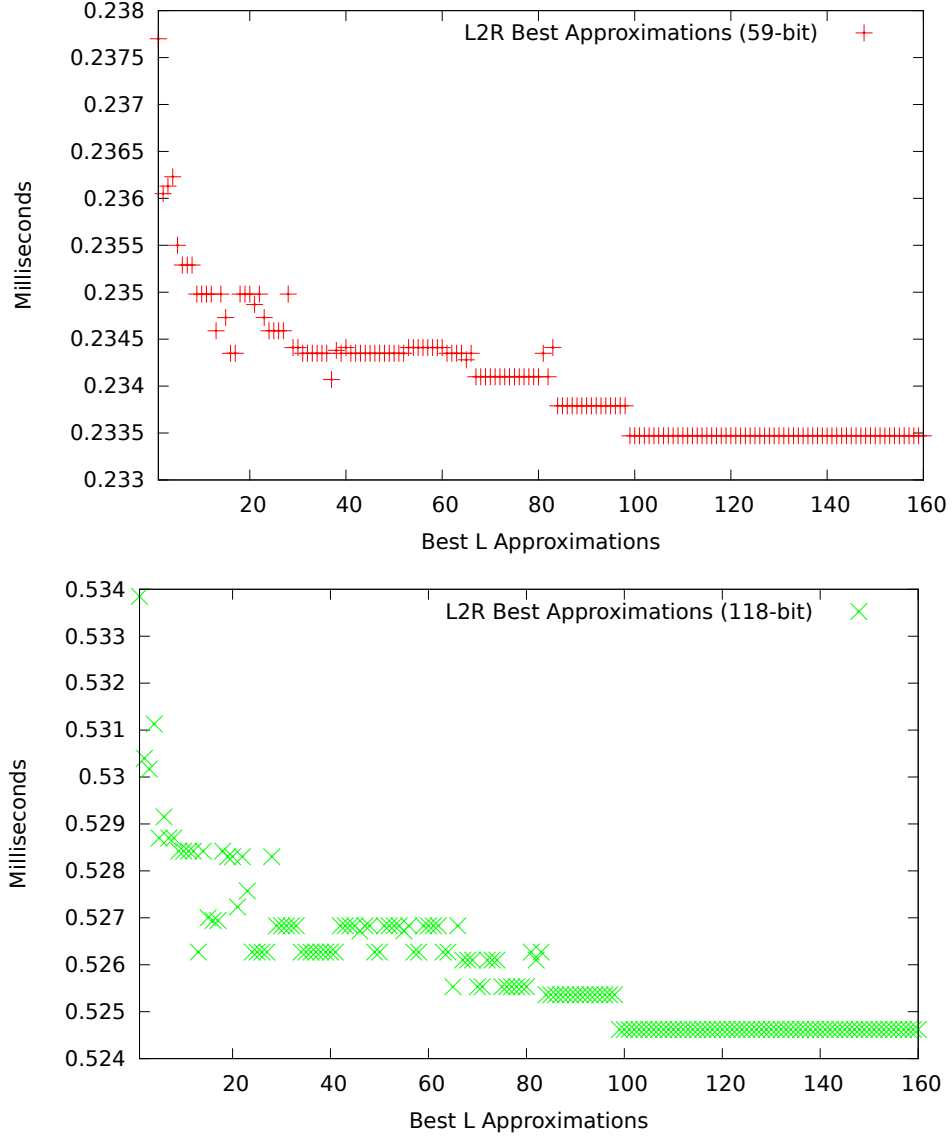


Figure 7.3: Time to exponentiate using P_{100} when number of the best approximations maintained is varied.

Assuming that global bounds A and B are chosen to give a representation with the best average performance, Figure 7.3 shows the times to exponentiate the 100th primorial when the number of best approximations L is varied. Note that when $L = 1$, this technique produces the same representations as the left-to-right representations discussed in Section 7.4. For 59 bit discriminants, a value of $L = 100$ takes 0.23347 milliseconds on average.

This is an improvement over $L = 1$, which takes 0.2377 milliseconds on average. For 118 bit discriminants, $L = 100$ requires 0.52462 milliseconds on average, while $L = 1$ uses 0.53385 milliseconds on average.

7.7 Additive 2,3 Chains

Imbert and Philippe [36, §3] provide a method (see Subsection 3.4.4) to compute the shortest additive strictly chained 2,3 partition, suitable for smaller integers. Such partitions do not permit subtraction and every term is strictly less than and divides all subsequent terms. These representations take the form

$$n = \sum_{i=0}^k 2^{a_i} 3^{b_i}$$

for $a_i, b_i \in \mathbb{Z}_{\geq 0}$ where $2^{a_i} 3^{b_i}$ divides $2^{a_j} 3^{b_j}$ and $a_i < a_j$ and $b_i < b_j$ for all $i < j$.

For our purposes, we propose two modifications of this algorithm to compute 2,3 chains that minimize the average cost of arithmetic in the ideal class group – the resulting additive 2,3 chains should give a better average time for exponentiation, while they are not necessarily the shortest possible. Future work should compare the cost of exponentiation using representations generated by the functions from this section, with representations generated by the non-modified function $s(n)$ in Subsection 3.4.4, which is the same as that proposed by Imbert and Philippe.

Let M , S , and C be the average cost to multiply, square, and cube respectively. Our

modified function is as follows

$$s'(n) = \begin{cases} \min\{S + s'(n/2), C + s'(n/3)\} & \text{when } n \equiv 0 \pmod{6} \\ M + s'(n-1) & \text{when } n \equiv 1 \pmod{6} \\ S + s'(n/2) & \text{when } n \equiv 2 \pmod{6} \\ \min\{C + s'(n/3), M + S + s'((n-1)/2)\} & \text{when } n \equiv 3 \pmod{6} \\ \min\{S + s'(n/2), M + C + s'((n-1)/3)\} & \text{when } n \equiv 4 \pmod{6} \\ M + S + s'((n-1)/2) & \text{when } n \equiv 5 \pmod{6} \end{cases}$$

where $s'(1) = 0$, $s'(2) = S$, and $s'(3) = C$ are the base cases.

We also experimented with computing 2,3 strictly chained partitions that allow for both positive and negative terms. The corresponding function is

$$f(n) = \begin{cases} \min\{S + f(n/2), C + f(n/3)\} & \text{when } n \equiv 0 \pmod{6} \\ \min\{M + f(n-1), M + S + f((n+1)/2)\} & \text{when } n \equiv 1 \pmod{6} \\ \min\{S + f(n/2), M + C + f((n+1)/3)\} & \text{when } n \equiv 2 \pmod{6} \\ \min\{C + f(n/3), M + S + f((n-1)/2), \\ \quad M + S + f((n+1)/2)\} & \text{when } n \equiv 3 \pmod{6} \\ \min\{S + f(n/2), M + C + f((n-1)/3)\} & \text{when } n \equiv 4 \pmod{6} \\ \min\{M + f(n+1), M + S + f((n-1)/2)\} & \text{when } n \equiv 5 \pmod{6} \end{cases}$$

again, where $f(1) = 0$, $f(2) = S$, and $f(3) = C$ are the base cases. One thing to notice is that the function s' computes a subset of the representations computed by the function f . For this reason, the average cost to exponentiate using a representation computed by f should be no worse than the average cost to exponentiate using a representation computed by s' . We were able to confirm this for each exponent n such that $1 \leq n < 65536$.

We point out that the functions $s'(n)$ and $f(n)$ are highly redundant in their computation. For this reason, our implementations memoize the result for each input in order to improve

the running time. Furthermore, since we compute the result for each exponent n for $1 \leq n < 65536$, computing $s'(n)$ benefits from the computation of $s'(m)$ for each $m < n$, and the same holds for $f(n)$ respectively. This reduces the running time to compute representations for all exponents n for $1 \leq n < 65536$, but even so, the running time of each algorithm appears to be exponential in the size of the largest input and so we only use these functions up to 16 bit inputs. We recommend a thorough analysis of the running time as future work.

7.8 Incremental Searching

Section 7.7 imposes additional constraints on 2,3 chains so that we are able to compute all such chains for a given input. Here we propose an alternative approach to computing 2,3 representations. Rather than searching for a representation for a particular exponent, we instead generate a set of representations for many exponents. For each exponent, there may be several representations in the set and only the *cheapest* representation for each exponent is recorded.

Consider first the single term representations $2^{a_1}3^{b_1}$ for $0 \leq a_1 \leq A$ and $0 \leq b_1 \leq B$, and then the two term representations $2^{a_1}3^{b_1} \pm 2^{a_2}3^{b_2}$ for $0 \leq a_1 < a_2 \leq A$, $0 \leq b_1 \leq B$, and $0 \leq b_2 \leq B$. In general, we compute the set of representations

$$R = R_1 \cup R_2 \cup \dots \cup R_m$$

for some maximum number of terms m where R_k is a set of k term representations defined as

$$R_k = \left\{ \sum_{i=1}^k \pm 2^{a_i} 3^{b_i} : 0 \leq a_1 < \dots < a_k \leq A \text{ and } 0 \leq b_j \leq B \text{ for } 1 \leq j \leq k \right\}.$$

We iterate over the set R and for each integer represented, we record the *lowest cost* representation for that integer in R . Notice that the powers of 2 are constrained such that $0 \leq a_i < a_j \leq A$ for each $i < j$, while the powers of 3 are only constrained such that $0 \leq b_j \leq B$. This removes a dimension of redundancy, while still allowing for representations to be unchained.

In our experiments, we searched for representations for 16 bit integers. We initially chose $A = \lceil \log_2(2^{16}) \rceil = 16$, $B = \lceil \log_3(2^{16}) \rceil = 11$, and iterated the number of terms k from 1 through 5. For representations of $k = 6$ terms, our implementation had not finished after a week of execution. We then ran our search again using $A = 18$ and $B = 12$ and found that none of our minimal representations were improved.

Since the method proposed here is computationally expensive, we only computed chains for 16 bit exponents. This is still useful when we consider methods that partition large exponents into smaller blocks using their binary representation, or when we consider repeated exponentiation by a list of prime powers, as we do in the next section.

7.9 Big Exponents from Small Exponents

The techniques of the previous two sections appear to require an exponential amount of time in the size of the input and for this reason, we limited our search to representations of 16 bit integers. One way to use such representations is with a left-to-right 2^k -ary algorithm [20, §9.1.2]. Write the exponent n in 16 bit blocks as

$$n = \sum_{i=0}^k 2^{16i} b_i$$

where $0 \leq b_i < 2^{16}$. Assuming that we can exponentiate a group element g to a 16 bit exponent b_i , Algorithm 7.2 computes the exponentiation g^n using an approach similar to a left-to-right windowed binary exponentiation.

Algorithm 7.2 16 bit Blocked Exponentiation [20, Algorithm 9.7].

Input: $n \in \mathbb{Z}$, $g \in G$ and a method to compute g^{b_i} for $0 \leq b_i < 2^{16}$.

Output: g^n

```

1:  $R \leftarrow 1_G$ 
2: for  $i$  from  $\lceil \log_{2^{16}} n \rceil$  downto 0 do
3:    $R \leftarrow R^{2^{16}}$                                      {By repeated squaring.}
4:    $b_i \leftarrow \lfloor n/2^{16i} \rfloor \bmod 2^{16}$ 
5:    $R \leftarrow R \cdot g^{b_i}$                                {Externally compute  $g^{b_i}$ .}
6: return  $R$ 
```

Another way we can use 16 bit representations is when we know the prime factorization of the exponent n and n is the product of prime powers smaller than 2^{16} . Since we are interested in exponentiation by power primorials (i.e. the product of the first w prime powers), the prime factorization is trivial. Given a power primorial $E = \prod_{i=1}^w p_i^{e_i}$ where p_i is the i^{th} prime and each $p_i^{e_i} < 2^{16}$, we can compute g^E as

$$\left(\left(\left(g^{p_1^{e_1}} \right)^{p_2^{e_2}} \right) \cdots \right)^{p_w^{e_w}}$$

using a series of w small exponentiations.

Figures 7.12 and 7.13 in Section 7.10 show that when exponentiating by a power primorial, it is faster on average to exponentiate by a series of small prime powers than by using 16 bit blocks.

7.10 Experimental Results

The goal of this section is to determine a method of exponentiating to a power primorial that performs well on average using our implementation of ideal class arithmetic. The SPAR integer factoring algorithm (see Section 4.1) uses the exponent

$$E_t = \prod_{i=2}^t p_i^{e_i} \text{ where } e_i = \max\{v : p_i^v \leq p_t^2\}. \quad (7.1)$$

With this in mind, we compare the techniques discussed in this chapter on exponents E_t for values of $t \leq 1000$. We point out that the theoretically optimal value of t for a 100 bit number is about 5273, which is less than 7919, the 1000th prime.

This section builds on the results from Section 6.2. For our 64-bit implementation, the largest discriminants supported are those that fit within 59 bits, and for our 128-bit implementation, the largest discriminants fit within 118 bits. We let the average cost of arithmetic in class groups with discriminants of these sizes represent an upper bound on the average cost of arithmetic operations for each implementation. Using this, the results here estimate the average time to exponentiate assuming the average cost of operations for

59 bit and 118 bit discriminants. Fixing the size of the discriminants also means that the average time computed varies only with the representation of the exponent computed by a particular technique. For each technique discussed in this chapter, we assume that a 2,3 representation is computed in advance. We then compute the average cost of exponentiation using Algorithm 3.3 from Subsection 3.3.2 for a given representation.

Section 7.9 discusses two techniques to compute g^n using a series of smaller exponentiations g^b such that $0 \leq b < 2^{16}$. We begin by determining the best method to exponentiate using 16 bit exponents, so that we may then compare these methods using power primorial exponents. To do so, we compute the average time to exponentiate for exponents 1 through 65535 using 59 bit and 118 bit discriminants. Table 7.1 shows the number of exponents for which each method was the fastest (methods that were not the fastest for any of the exponents tested are not shown). Table 7.2 shows the average time, in microseconds, for each method to exponentiate a 16 bit exponent. This is based on the average of the average time to exponentiate for each exponent 1 through 65535. For each of the pruned tree methods and for the left-to-right best approximations method, we maintain the best 4 leaf nodes. We also bound $a \leq 4$ and $b \leq 4$ for each of the pruned tree methods.

Method	59 bit Discriminants	118 bit Discriminants
Left-to-Right 2,3 Representation	2536	7097
Left-to-Right Best Approximations	9091	7230
Pruned Tree $(x \pm 2^a 3^b)/(2^c 3^d)$	47969	45468
Pruned Tree $(x \pm 2^a \pm 3^b)/(2^c 3^c)$	3333	2756
Recursive $\sum \pm 2^a 3^b$ Chains	2406	2970
Incremental Search	199	13

Table 7.1: The number of 16 bit exponents where an exponentiation technique was fastest.

The method that maintains the 4 best partial representations of the form $(x \pm 2^a 3^b)/(2^c 3^d)$ was the best performer in general. However, since we were interested in precomputing 16 bit representations for use with a block exponentiation and exponentiation by a list of prime factors, we used the best representation available for each exponent.

To determine the best method to exponentiate a random ideal class by a power primorial,

Method	59 bit Discriminants	118 bit Discriminants
Binary	6.46404	14.61132
Non-Adjacent Form	6.04918	13.59860
Right-to-Left 2,3 Chain	5.90276	13.93929
$2^2 3^2$ Windowed Right-to-Left 2,3 Chain	6.01505	13.89539
Left-to-Right 2,3 Representation	5.54176	12.52248
Left-to-Right Best Approximations	5.18680	11.84543
Pruned $(x \pm 1)/(2^c 3^d)$	5.34933	12.43606
Pruned Tree $(x \pm 2^a 3^b)/(2^c 3^d)$	4.59421	10.75899
Pruned Tree $(x \pm 2^a \pm 3^b)/(2^c 3^c)$	5.98289	14.05320
Recursive $\sum 2^a 3^b$ Chains	5.58740	12.84866
Recursive $\sum \pm 2^a 3^b$ Chains	5.26825	12.02264
Incremental Search	5.18618	11.84825

Table 7.2: Average time (microseconds) to exponentiate 16 bit exponents.

we compute the average time to exponentiate the exponent E_t from Equation (7.1) for the values $1 \leq t \leq 1000$. We categorized the different exponentiation techniques as those that use only base 2 (our reference implementations), those that generate 2,3 chains/representations from right-to-left, from left-to-right, that add or subtract to a partial representation and then reduce by $2^c 3^d$, and those that make use of the best 16 bit representations. We then compared the average time to exponentiate each method from a category, and finally compared the best performers of each category to determine the best performer overall.

We found that for both our 64-bit and 128-bit implementations and for all power primorials tested, a left-to-right best approximations method, proposed in Section 7.6, leads to representations with the fastest time to exponentiate. This is in contrast to a pruned tree using $(x \pm 2^a 3^b)/(2^c 3^d)$ that lead to the best timings for 16 bit integers. In each case, this was an improvement over the average time to exponentiate using either the binary representation or the non-adjacent form. Naturally, the results of both can be improved by iterating on the L best partial representations for $L > 4$ at the expense of longer precomputation, and the results of a pruned tree using $(x \pm 2^a 3^b)/(2^c 3^d)$ can also be improved by allowing larger bounds for a and b . Since a pruned tree using $(x \pm 2^a 3^b)/(2^c 3^d)$ is faster for 16 bit exponents, while a left-to-right best approximations is faster for the power primorials tested, future

work should investigate the cross-over point for these two techniques for various parameters.

Figures 7.4 and 7.5 compare exponentiation using the binary representation against the non-adjacent form. The non-adjacent form is faster on average in all cases. Figures 7.6 and 7.7 compare the non-windowed right-to-left 2,3 chain method to the 2^23^2 windowed method. The 2^23^2 windowed method performs faster on average than the non-windowed method for all exponents tested. Figures 7.8 and 7.9 compare left-to-right 2,3 representations with that of a left-to-right best approximations technique. In this case, a left-to-right best approximations technique performs best on average. Figures 7.10 and 7.11 compare the three different techniques of adding or subtracting a value to a partial representation and then reducing by a power of 2 and 3. Here, computing candidates $(x - 2^a3^b)/(2^c3^d)$ leads to representations that exponentiate the fastest on average. Figures 7.12 and 7.13 compare the two methods that rely on the best found 16 bit representations. In this case, when the factorization of the exponent is easily known, it is faster on average to exponentiate by the list of prime factors than it is to represent the exponent using 16 bit blocks. Finally, Figures 7.14 and 7.15 compare the best performer from each category.

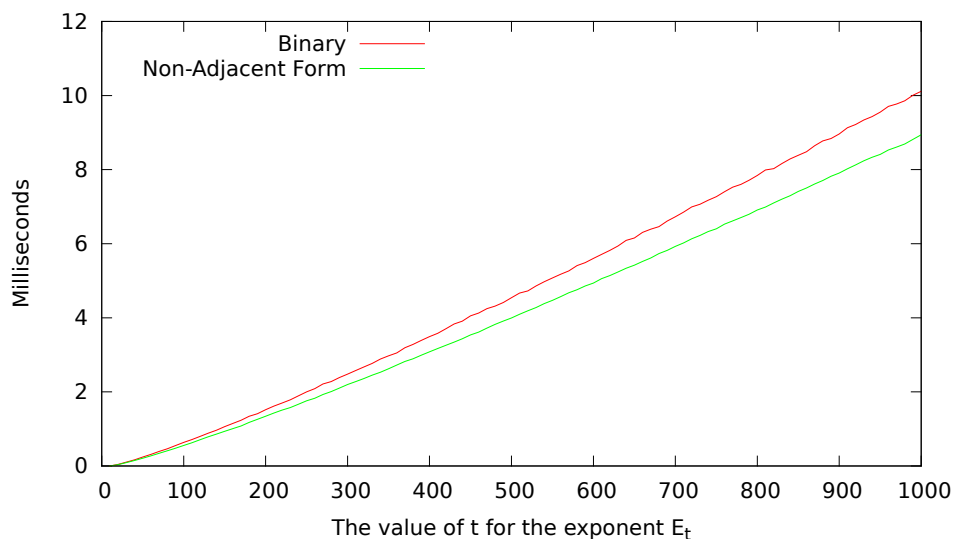


Figure 7.4: Base 2 Exponentiation (59 bit Discriminants).

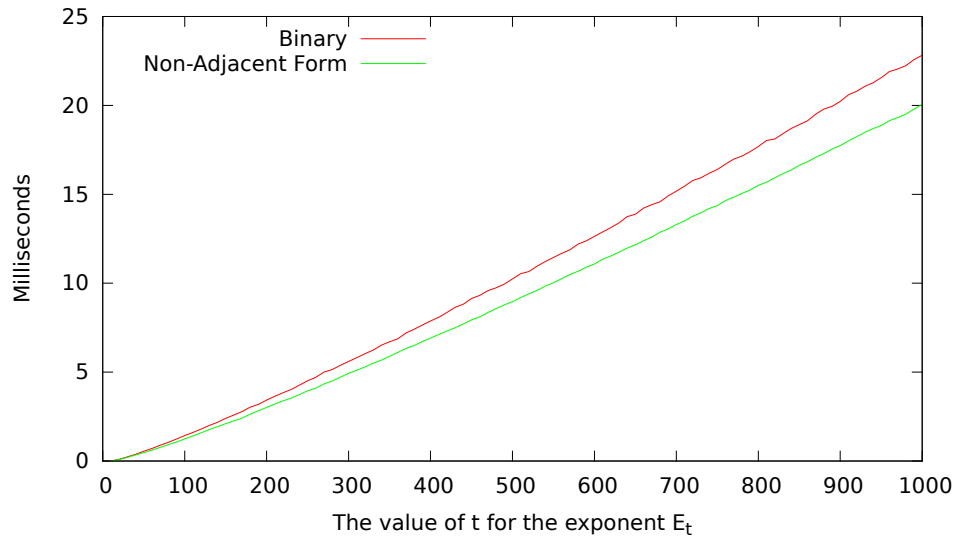


Figure 7.5: Base 2 Exponentiation (118 bit Discriminants).

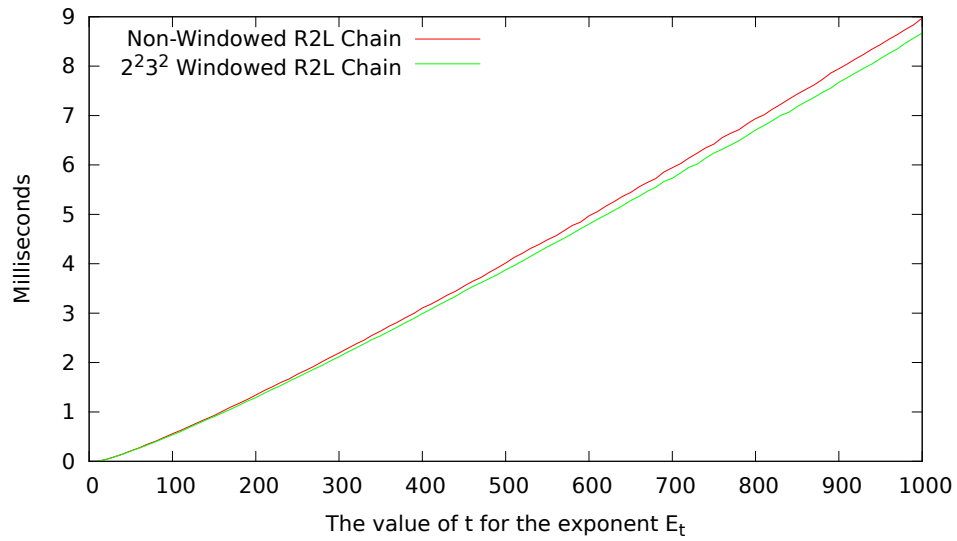


Figure 7.6: Right-to-Left 2,3 Chains (59 bit Discriminants).

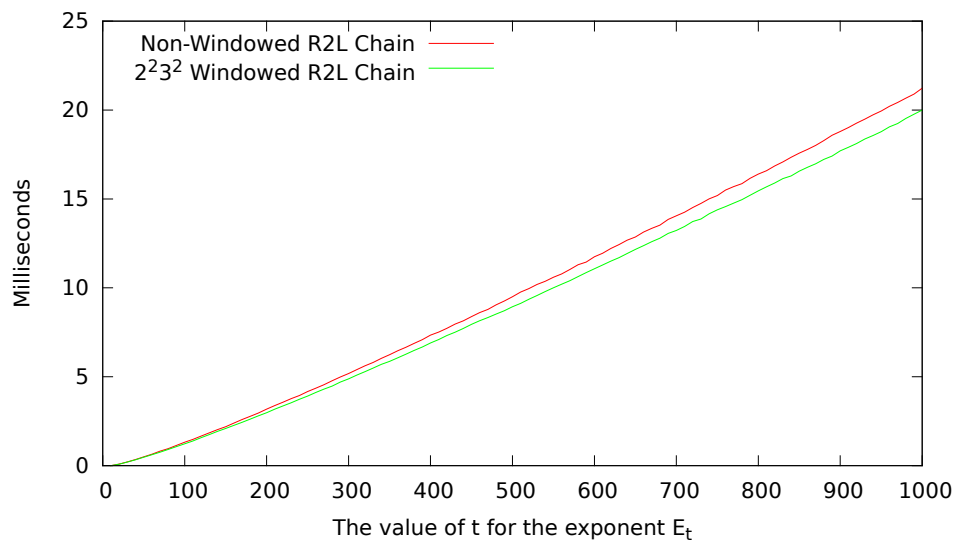


Figure 7.7: Right-to-Left 2,3 Chains (118 bit Discriminants).

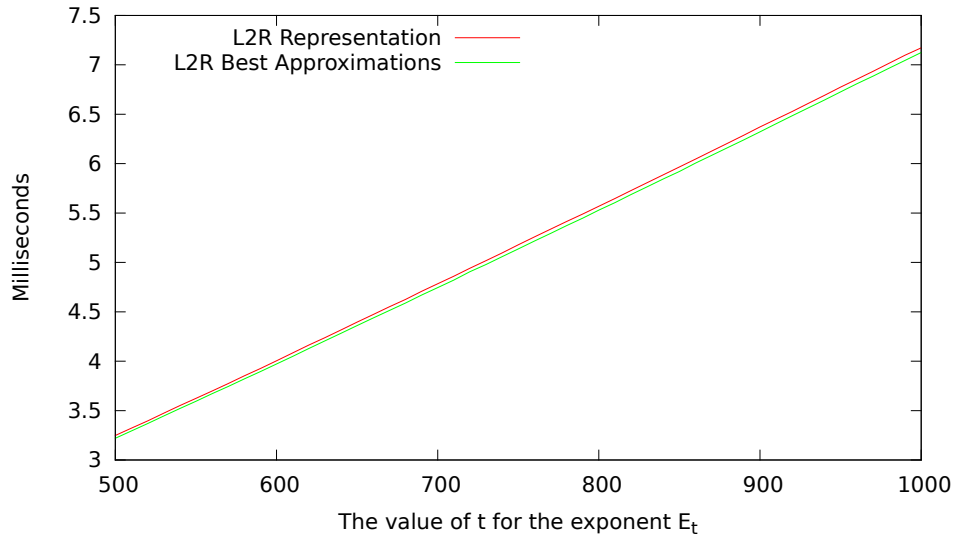
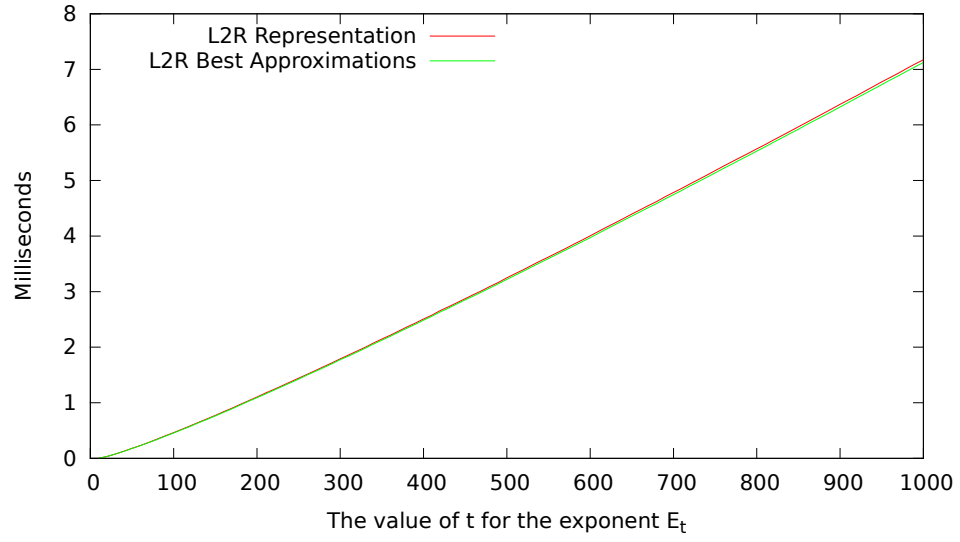


Figure 7.8: Left-to-Right 2,3 Representations (59 bit Discriminants).

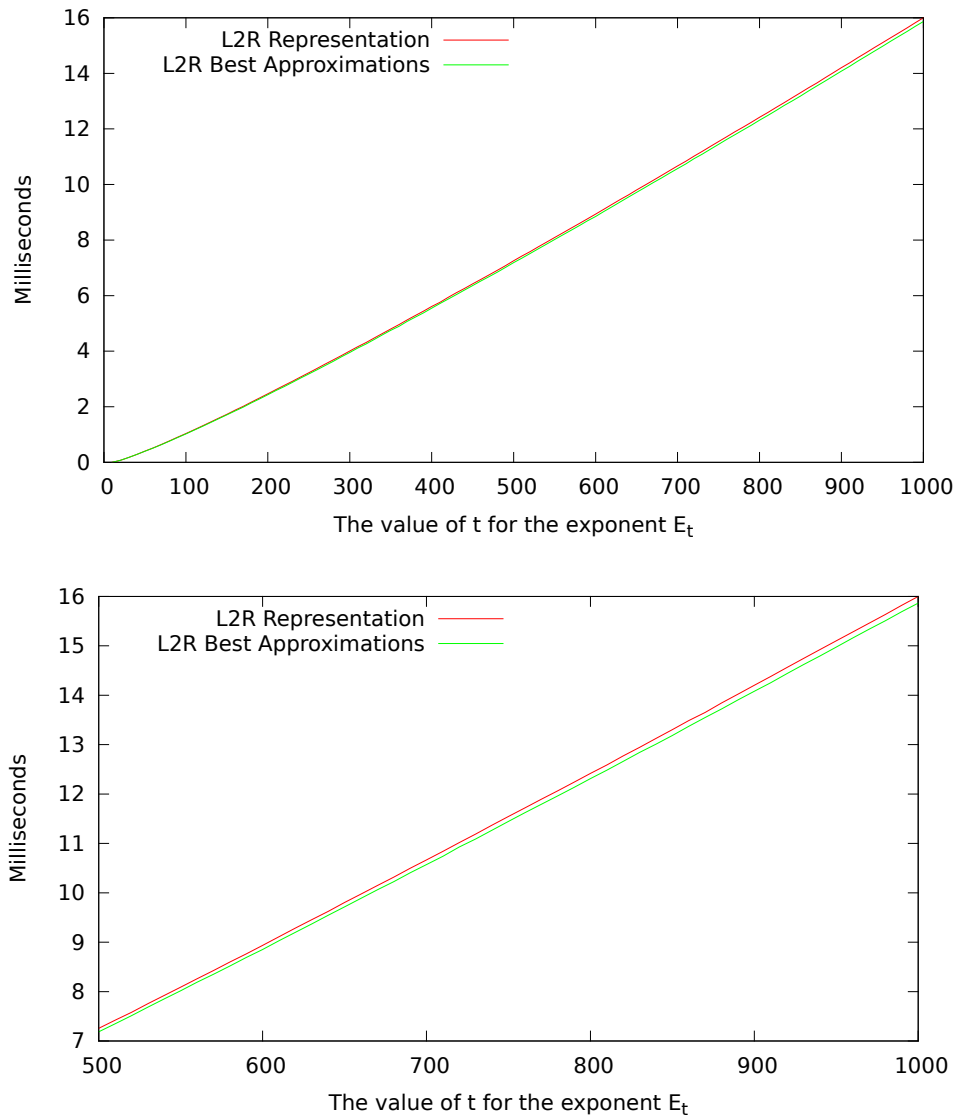


Figure 7.9: Left-to-Right 2,3 Representations (118 bit Discriminants).

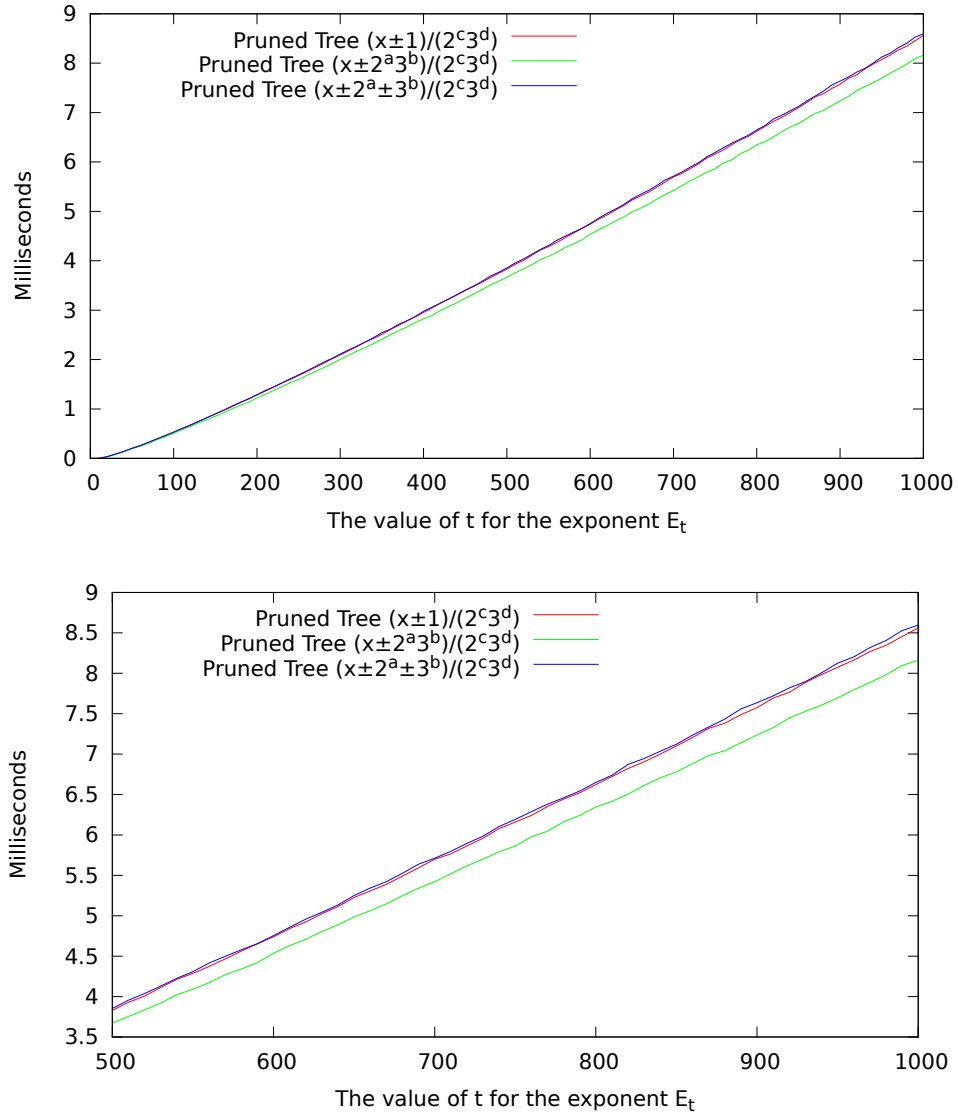


Figure 7.10: Pruned Trees using the 4 Best $(x \pm \dots)/(2^c 3^d)$ (59 bit Discriminants).

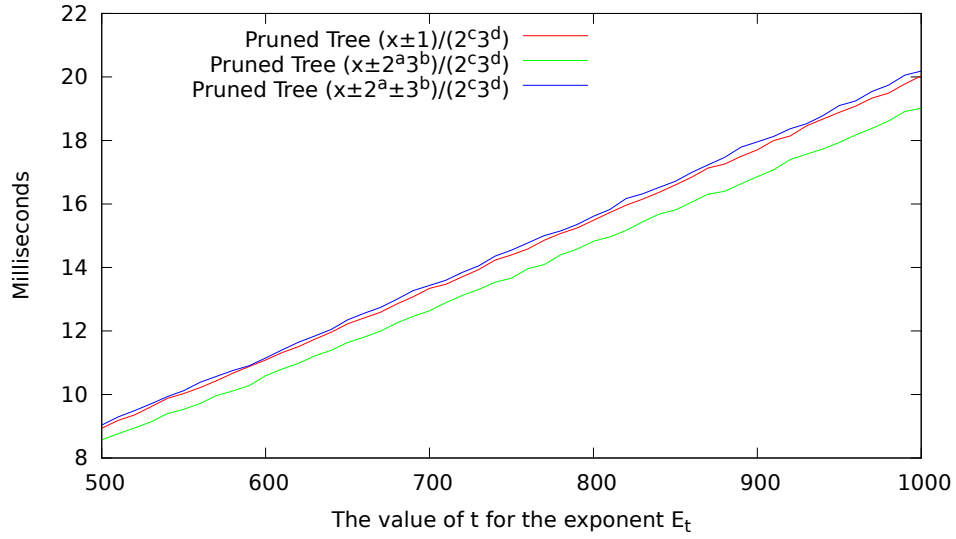
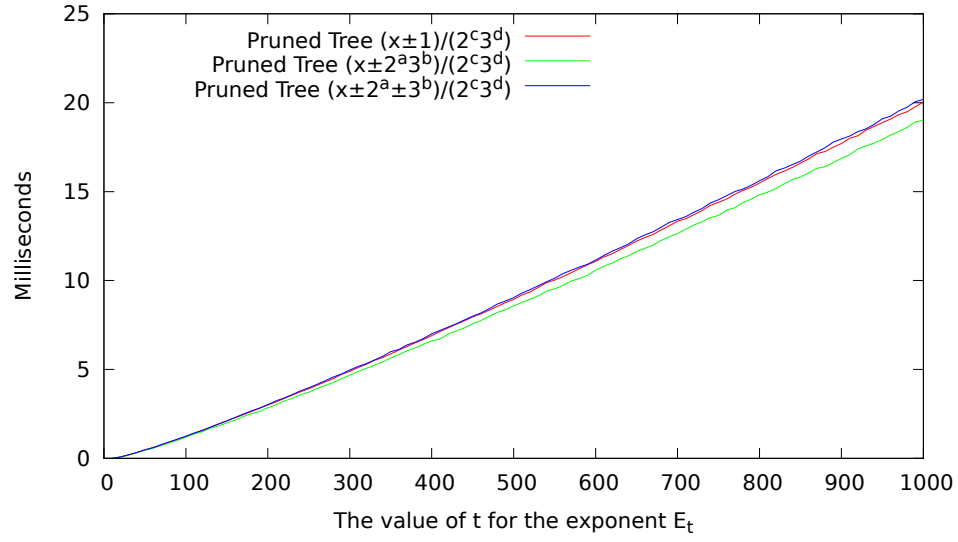


Figure 7.11: Pruned Trees using the 4 Best $(x \pm \dots)/(2^c 3^d)$ (118 bit Discriminants).

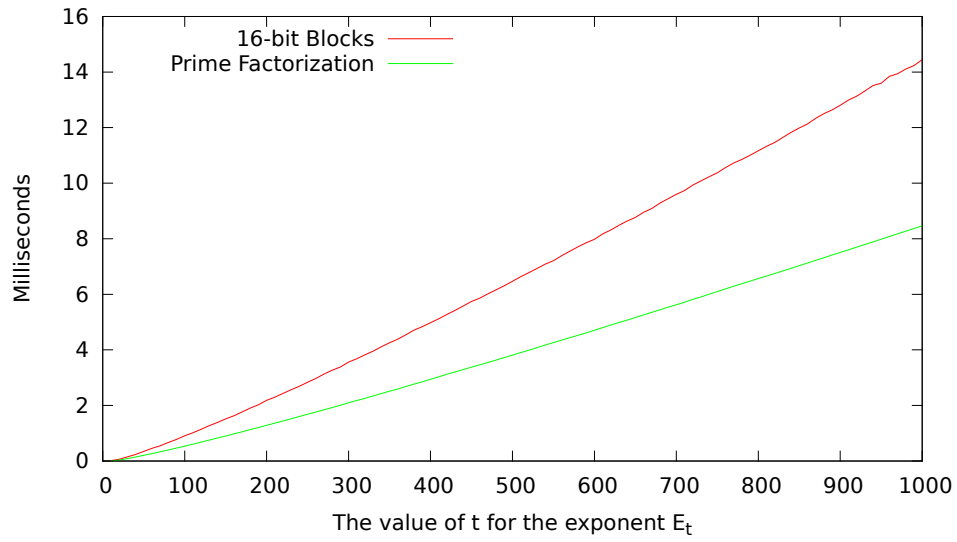


Figure 7.12: Use g^b for 16 bit b (59 bit Discriminants).

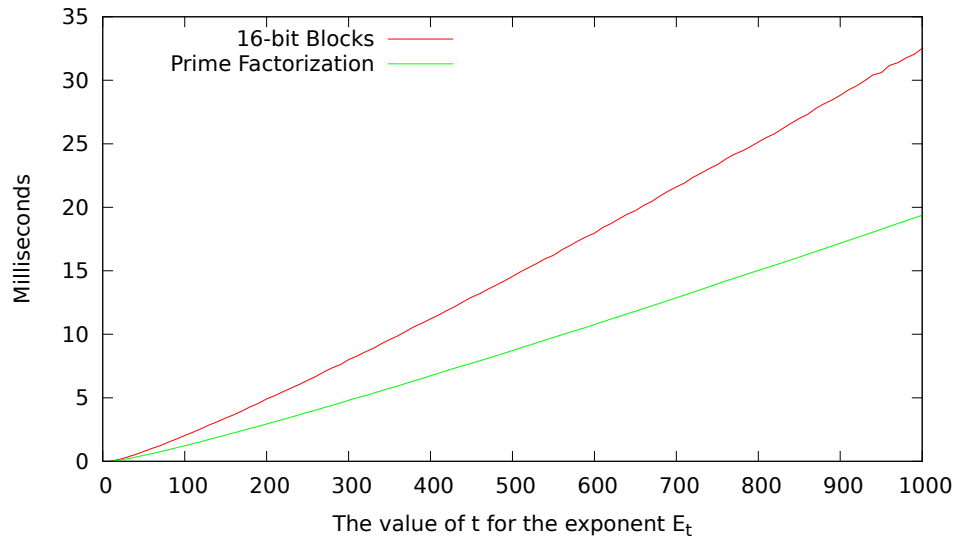


Figure 7.13: Use g^b for 16 bit b (118 bit Discriminants).

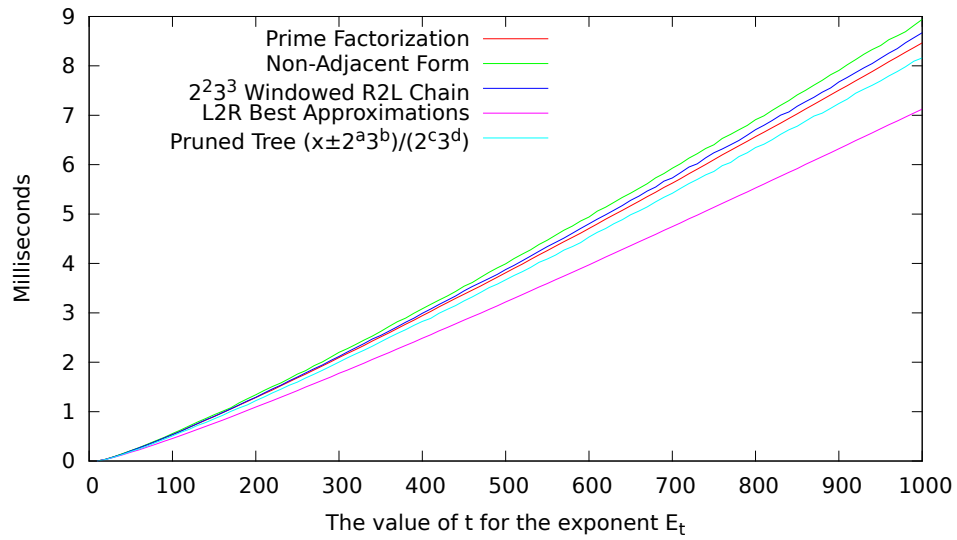


Figure 7.14: The best performers from each category (59 bit Discriminants).

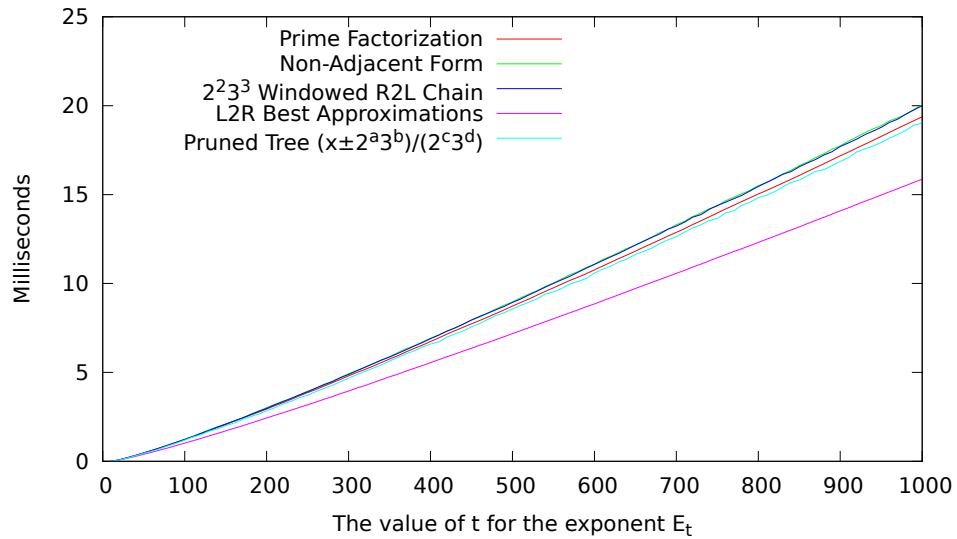


Figure 7.15: The best performers from each category (118 bit Discriminants).

7.11 Summary

The goal of this chapter was to determine efficient methods for exponentiating an ideal class to a power primorial, assuming that the exponent is known in advance. In Section 7.6, we proposed a method that combines a technique for computing left-to-right 2,3 representations suggested by Berthé and Imbert [16] with a method of maintaining the L best partial representations suggested by Doche and Habsieger [28]. The results of Section 7.10 show that when exponentiating to power primorials using our implementations of ideal class arithmetic for class groups with either 59 bit or 118 bit negative discriminants, representations computed using a left-to-right best approximations technique are the fastest on average. This is useful when applied to a bounded primorial steps algorithm (see Subsection 4.2.1) and a bound on the class group is known in advance. We will see an example of this in the next chapter where we bring together practical improvements for computing the extended greatest common divisor (Chapter 5), improvements to ideal class arithmetic (Chapter 6), and the results of this chapter in our implementation of SuperSPAR.

Chapter 8

SuperSPAR Experiments

The goal of this chapter is to demonstrate that the SPAR integer factoring algorithm (Section 4.1) can be modified and implemented so that, on average, it is the fastest available factoring implementation for integers of the size 49 bits to 62 bits – this is an implementation of SuperSPAR (Section 4.2). Our improvements to computing the extended GCD and to ideal arithmetic play a key role in our implementation of SuperSPAR. Much of this chapter describes how we found suitable parameters to improve the average performance of SuperSPAR in practice. Our final implementation of SuperSPAR automatically selects appropriate parameters based on the size of the input integer and the results of this chapter.

One way in which factoring integers of these sizes is useful is after the sieve step of other integer factoring algorithms, such as the number field sieve (see [23, §6.2]). In this case, the sieve removes small primes from a set of integers and what remains are often semiprime, not too large, and need to be factored by other means. For this use, SuperSPAR could be optimized for the median case (or some other percentile) with the idea that SuperSPAR will only perform a bounded number of operations in an attempt to factor an integer. This is sufficient in terms of a factoring sub-algorithm since left-overs from the sieve stage are generated relatively quickly, and not every left-over needs to be factored. This is left as future work. Here we optimize SuperSPAR for the average case.

Although Lenstra and Pomerance [45, p.511, §11] give a proof that, in the worst case, SPAR (and hence SuperSPAR) requires an exponential number of group operations, this is for input integers that have a large square factor. This chapter optimizes the average performance of SuperSPAR for semiprime integers, i.e. integers $N = p \cdot q$ such that $p \neq q$ and p and q are roughly the same size. Such numbers are often left-over from the sieve stage

of other factoring algorithms such as the number field sieve.

SPAR factors an integer N by attempting to compute the order of a random ideal class in an order with discriminant $\Delta = -kN$ for some square-free integer k . An implementation first exponentiates a random ideal class to the product of many small primes (the exponentiation stage) and then performs a random walk in an attempt to find an ambiguous ideal class (the search stage). Our implementation of SuperSPAR improves upon the search stage of SPAR by using a bounded primorial steps search, due to Sutherland [58] and discussed in Subsection 4.2.1. The results of this improvement are demonstrated in Section 8.7.

Our implementation of SuperSPAR also improves upon the exponentiation stage of SPAR. In this stage, SPAR exponentiates a random ideal class to the product of many small primes. Since our implementations of ideal class arithmetic include cubing, we take advantage of 2,3 representations for exponentiation, which is an improvement over binary exponentiation (see Section 7.10). For SuperSPAR, we compute 2,3 representations in advance using a left-to-right best approximations technique of Section 7.6, which Chapter 7 shows to perform the best on average for large exponents that are the product of many primes and for ideal classes for orders with discriminants no larger than 118 bits.

Another way in which our implementation of SuperSPAR improves upon SPAR is by bounding the number of group operations used by each stage independently from each other and the theoretically optimal values. In SPAR, each stage uses $O(p_t)$ group operations where $p_t \approx N^{1/2r}$ and $r = \sqrt{\ln N / \ln \ln N}$ (see Subsection 4.1.2 for additional details). The value for r is chosen to theoretically minimize the expected running time of the algorithm, since the assumption is that the order of a random ideal class $[\mathfrak{a}] \in Cl_\Delta$ with $\Delta = -kN$ is $N^{1/2r}$ smooth with probability r^{-r} . In practice, other values of p_t are more efficient.

Furthermore, balancing both stages to use $O(p_t)$ group operations is not ideal. This is partly because the actual costs of multiplication, squaring, and cubing differ, but also because the success of each stage depends on different properties of the class number. The

exponentiation stage is successful when the order of a random ideal class is smooth with respect to the exponent E used in the exponentiation stage, while the search stage is successful when the non-smooth part of the order is sufficiently small. Selecting bounds for each stage independently of r varies the time spent during each stage inversely to the probability of its success. For this reason, our implementation of SuperSPAR selects the largest prime p_t , used in the exponent $E = \prod p_i^{e_i}$ in the exponentiation stage, separately from the multiple of a primorial, mP_w , used in the search stage. Note that the largest prime p_t used during the exponentiation stage is larger than the largest prime p_w used by the search stage, since the search stage assumes that the order of $[\mathbf{b}] = [\mathbf{a}]^E$ is coprime to E .

In addition to allowing the largest prime p_t used in the exponent $E = \prod p_i^{e_i}$ to differ from the theoretically optimal value, we also bound the prime exponents e_i differently from the theoretical values suggested by Schnorr and Lenstra, who incidentally advise the use of smaller exponents in practice (see [53, p.293] and Subsection 8.3.1 of this thesis).

Since there are several parameters affecting the performance of our implementation of SuperSPAR, we first discuss the methodology used in this chapter. This chapter, like previous chapters, emphasizes empirical results, however, factoring a single integer is considerably slower than computing a single extended GCD or performing a single arithmetic operation in the ideal class group. In the case of computing the extended GCD, a single computation takes on average between a few nanoseconds to a few hundred nanoseconds. This means that we can empirically test 1,000,000 computations for random integers of each size from 1 bit to 127 bits in a modest amount of time. Even for ideal class arithmetic, multiplication, squaring, and cubing take between a few hundred nanoseconds to a few microseconds. Again, because of this we are able to empirically time each operation on 10,000,000 random ideal classes for discriminants of size 16 bits to 140 bits in a few hours. On the other hand, factoring a single integer with our implementation of SuperSPAR takes less than 100 microseconds on average for properly chosen parameters for 32 bit integers, but over 50 milliseconds on

average for poorly chosen parameters for 100 bit integers – this is roughly between 10,000 times to 500,000 times slower than an extended GCD computation. So even for small sets of integers (a few 1000), this is quite slow and may not lead to parameters that work well on average.

To aid in the selection of efficient parameters for our implementation of SuperSPAR, Section 8.3 discusses the statistical analysis of a set of roughly 6,000,000 ideal classes in order to find bounds on several parameters. Subsection 8.3.1 shows that the prime factorization of the order of ideal classes consists of prime powers with exponents that are typically 1 for all but the smallest primes. This informs the value of each e_i in the exponent E for the exponentiation stage. Additionally, we found that the order of an ideal class $[\mathfrak{a}_1]$ was, with high probability (about 97.8% in our experiments), either the same as or a multiple of the order of some other ideal class $[\mathfrak{a}_2]$ in the same class group (see Subsection 8.3.2). For this reason, if the algorithm finds some h' such that $[\mathfrak{a}_1]^{h'(2^j)}$ is an ambiguous class for some j , but is unsuccessful at factoring N , the algorithm simply attempts to find an ambiguous class by repeated squaring of $[\mathfrak{a}_i]^{h'}$ for several $[\mathfrak{a}_i]$. Since this may fail for several ideal classes, we let c be the maximum number of ideal class classes tried before the algorithm starts over with a different multiplier k for the discriminant. Although the statistical data shows that in expectation a value of $c \approx 2$ is all that is needed (see Subsection 8.3.4), we rely on empirical testing to show that much larger values of c perform better in practice. Similarly, the largest prime p_t used in the exponent E for the exponentiation stage, and the multiple of a primorial mP_w used for the search stage are determined empirically in Section 8.6.

The software used to generate timing information was developed using GNU C compiler version 4.7.2 on 64 bit Ubuntu 12.10. Assembly language was used for 128 bit arithmetic on x64 processors and for processor specific features. The C programming language was used for all other implementation details. The CPU is a 2.7GHz Intel Core i7-2620M CPU with 8Gb of RAM and has four cores, only one of which was used for timing experiments. Unless

otherwise specified, the timings associated with factoring integers are the average time to factor one integer from a set of 10,000 semiprime integers, such that each integer is the product of two primes of equal size. All empirical timings for integers of a given size use the same set of 10,000 semiprime integers.

Exponents used for the exponentiation stage were computed in advance using a left-to-right best approximations technique from Section 7.6, which Chapter 7 shows performs the fastest for exponents that are the products of many small primes and for our implementations of ideal class group arithmetic. The implementation of ideal class arithmetic used is that discussed in Chapter 6, specifically, we use the 64-bit implementation for all class groups with discriminant no larger than 59 bits, and we use the 128-bit implementation for class group with discriminant larger than 59 bits, but no larger than 118 bits. Our implementation of SuperSPAR will work for integers larger than 118 bits, however, empirical tests show that SuperSPAR was not competitive for larger integers.

Extended GCD computations used for ideal arithmetic are based on the results of Chapter 5. For an ideal $\mathfrak{a} = [a, (b + \sqrt{\Delta})/2]$, the coefficients a and b are roughly half the size of the discriminant Δ . For this reason, our 64-bit implementation of ideal arithmetic favours our 32-bit implementation of the extended Euclidean algorithm, however, when arguments are larger than 32 bits, our implementation uses the 64-bit implementation of our simplified left-to-right binary extended GCDs. Our 128-bit implementation of ideal arithmetic favours the 64-bit implementation of our simplified left-to-right binary extended GCDs, but similarly, when arguments are larger than 64 bits, our implementation uses the 128-bit implementation of Lehmer's extended GCD using our 64-bit simplified left-to-right binary extended GCD for 64 bit machine words. In each case, when a left-only or partial extended GCD is needed, our implementation uses the left-only or partial implementation of the corresponding full extended GCD. Our GMP implementation of ideal arithmetic is not utilized during these tests since our implementation of SuperSPAR was not competitive for integers larger than

118 bits, which is the largest size supported by our 128-bit implementation of ideal class arithmetic.

The sections in this chapter are as follows. Section 8.1 discusses two techniques to generate a list of integers (or deltas between integers) coprime to a primorial up to a given bound. This is useful during the search stage of SuperSPAR when we take baby steps coprime to the multiple of a primorial mP_w . The search stage requires the use of a lookup table. Section 8.2 discusses how our implementation hashes an ideal class and then compares two methods of resolving hash collisions, namely chaining with list heads and open addressing. Open addressing performs faster on average. As discussed in Subsection 4.2.1, for the search stage to work, the order of the element should have no factors in common with the primorial used for the search stage. Section 8.3 discusses the data and experiments used to bound the exponents of the prime factors of the exponent used in the exponentiation stage so that (with high probability) the order of the resulting ideal class will have no factors in common with the primorial used during the search stage. This section also discusses the number of ideal classes to try in expectation for a given class group. To enable the discovery of bounds useful for the search stage, Section 8.5 discusses a method of generating suitable candidates. With all this in place, Section 8.6 discusses the method we use to determine the exponent for the exponentiation stage and the bound for the search stage that work well in practice for integers of specific sizes. Finally, a comparison of our implementation of SuperSPAR to several public implementations of factoring algorithms is provided. We show that for pseudorandom semiprime integers between 49 bits and 62 bits inclusive, SuperSPAR performs the best on average of the implementations tested.

8.1 Coprime Finding

The bounded primorial steps algorithm from Subsection 4.2.1 and adapted to the SuperSPAR factoring algorithm (Section 4.2) computes baby-steps $[\mathbf{a}]^i$ for $1 \leq i \leq s$ where i is coprime

to some primorial P_w and s is a multiple of P_w . By considering only values coprime to P_w , the running time complexity of the bounded primorial steps algorithm achieves an upper bound of $O(\sqrt{M/\log \log M})$ group operations in the worst case. If an implementation has to test whether each i from 1 to P_w is coprime to P_w , the algorithm cannot achieve this. For this reason, the algorithm assumes a set of values coprime to P_w is globally available. Of course, one can generate such a set by testing whether $\gcd(i, P_w) = 1$ for i from 1 to P_w , but there are more efficient methods. This section discusses two such methods, namely sieving and wheeling. Additional information on each technique can be found in [23, pp.117–127] and [59, p.494].

8.1.1 Coprime Sieve

Using a coprime sieve to determine the values coprime to some primorial P_w is straightforward since the factorization $P_w = 2 \times 3 \times \cdots \times p_w$ is known. We describe the approach we used here. Let $[1, b]$ denote the range to be sieved. First, create a table mapping the values 1 through b to *true*. Then for each prime factor p_j of P_w , and for every multiple mp_j such that $1 \leq mp_j \leq b$, set the table entry at index mp_j to *false*. At this point, each index with a *true* value is coprime to P_w . To see this, note that an index x is *false* when there was some multiple m of a prime p_j dividing P_w such that $mp_j = x$. Therefore, the value of x is false if and only if x and P_w share a common factor, namely p_j .

8.1.2 Coprime Wheel

When the value b from the previous section is equal to the primorial P_w , a coprime wheel may be faster in practice. First, suppose that for some primorial P_w , the set of all integers $1 \leq x < P_w$ coprime to P_w is already known. More formally, let

$$\mathcal{C}_w = \{x : \gcd(x, P_w) = 1, 1 \leq x < P_w\}.$$

As such, there is no $x \in \mathcal{C}_w$ that is divisible by any prime p_j that divides P_w . The values $x + mP_w$ for $x \in \mathcal{C}_w$ and $m \in \mathbb{Z}$ are also coprime to P_w , and so act as a set of representatives of the integers modulo P_w that are coprime to P_w . Let $\mathcal{C}_w + mP_w$ denote the set $\{x + mP_w : x \in \mathcal{C}_w\}$. Computing

$$\bigcup_{0 \leq m < p_{w+1}} \mathcal{C}_w + mP_w$$

generates the set of all positive integers less than P_{w+1} that are coprime to P_w . Removing all multiples of p_{w+1} from this set implies that no element is divisible by a prime $p_j \leq p_{w+1}$. With this idea, the set

$$\mathcal{C}_{w+1} = \left(\bigcup_{0 \leq m < p_{w+1}} \mathcal{C}_w + mP_w \right) \setminus \{m'p_{w+1} : 1 \leq m' < P_w\}$$

is the set of all values $1 \leq x < P_{w+1}$ that are coprime to P_{w+1} .

The primorial $P_1 = 2$ has the set $\mathcal{C}_1 = \{1\}$ of integers coprime to 2. This acts as a base case, from which the representative set of integers coprime to primorials $P_2, P_3, \dots, P_w$ are computed by recursive application of the above steps.

Our implementation of SuperSPAR iterates baby steps for consecutive integers coprime to some primorial P_w . For this reason, the difference between coprime integers is used instead, and the above technique is adapted to work with lists of deltas between consecutive coprime integers. Each list begins with the difference from the first coprime integer 1 (which is coprime to all integers) to the next integer coprime to some primorial P_w . Let $\phi(P_w) = \prod_{p \leq P_w} (p-1)$ be the number of positive integers coprime to P_w and no larger than P_w . Then let

$$\mathcal{D}_w = d_1, \dots, d_{\phi(P_w)}$$

such that $d_i = c_{i+1} - c_i$ for consecutive integers c_i and c_{i+1} coprime to P_w where $c_1 = 1$. The list $\mathcal{D}_1$ consists of the single value 2.

Algorithm 8.1 computes $\mathcal{D}_{w+1}$ given $\mathcal{D}_w$, P_w , and the next prime p_{w+1} . The algorithm starts with the current candidate integer, c , coprime to P_{w+1} and a delta counter $d = 0$. The algorithm cycles through the delta list $\mathcal{D}_w$ adding each element encountered to c . On each

Algorithm 8.1 Compute deltas for P_{w+1} given deltas for P_w .

Input: A primorial P_w , the delta list $\mathcal{D}_w$, and the next prime p_{w+1} . Let $P_{w+1} = p_{w+1}P_w$ and $\phi(P_{w+1}) = (p_{w+1} - 1)\phi(P_w)$.

Output: The delta list $\mathcal{D}_{w+1}$.

```

1:  $i \leftarrow 1, j \leftarrow 1, c \leftarrow 1, d \leftarrow 0$ 
2: while  $j \leq \phi(P_{w+1})$  do
3:    $c \leftarrow c + d_i$  {each  $c$  is coprime to  $P_w$ }
4:    $d \leftarrow d + d_i$ 
5:   if  $c \not\equiv 0 \pmod{p_{w+1}}$  then
6:      $d'_j \leftarrow d, d \leftarrow 0, j \leftarrow j + 1$  {output  $d$  when  $c$  is coprime to  $P_{w+1}$ }
7:    $i \leftarrow i + 1$ 
8:   if  $i > \phi(P_w)$  then
9:      $i \leftarrow 1$ 
10: return  $d'_1, \dots, d'_{\phi(P_{w+1})}$ 

```

iteration, if c is not a multiple of p_{w+1} , then the delta counter d is appended to the output list $\mathcal{D}_{w+1}$ and then set to 0. Otherwise, the algorithm continues with the next delta from the input list (cycling to the beginning of the list when the end is reached). Once $\phi(P_{w+1})$ deltas have been appended to the output list, the algorithm terminates.

8.1.3 Comparison

The coprime sieve requires b bits of memory, while the coprime wheel requires $\phi(P_w)$ integers for the list of deltas, where $\phi(P_w) = \prod_{p \leq p_w} (p - 1)$ for p prime is the number of positive integers coprime to P_w and no larger than P_w . Table 8.1 shows the values of $\phi(P_w)$ for the first 12 primorials. The delta list for the tenth primorial, $\phi(P_{10}) = 1021870080$, requires over a billion integers. For this reason, the coprime sieve is preferred when the primorial is large, while the coprime wheel may be more efficient for large values of b . This is because the coprime wheel generates deltas directly, whereas each entry in the sieve must be inspected to find integers coprime to the primorial.

Our implementation of SuperSPAR computes baby steps coprime to a multiple of a primorial, and does so by iterating a list of deltas from one coprime value to the next. Since this list consists of $\phi(P_w)$ integers, in practice our implementation is limited to coprime delta

w	p_w	$P_w = \prod_{p \leq p_w} p$	$\phi(P_w) = \prod_{p \leq p_w} (p - 1)$
1	2	2	1
2	3	6	2
3	5	30	8
4	7	210	48
5	11	2310	480
6	13	30030	5760
7	17	510510	92160
8	19	9699690	1658880
9	23	223092870	36495360
10	29	6469693230	1021870080
11	31	200560490130	30656102400
12	37	7420738134810	1103619686400

Table 8.1: The first 12 primorials.

lists that fit within available memory, and as such to primorials P_w that are not too large. On the other hand, our implementation is still able to use much larger primorials for the exponentiation stage, since the space needed to represent a primorial is $O(\log(P_w))$ while the space needed to represent the list of coprime deltas is $O(\phi(P_w))$. One reason we might want to use a larger primorial in the exponentiation stage is that by doing so, the order of the resulting ideal class $[\mathbf{b}] = [\mathbf{a}]^E$ is likely to be coprime to E and so any factors of the order of $[\mathbf{a}]$ that were in common with the factors of E are likely to be removed by the exponentiation stage. The order of the resulting ideal class $[\mathbf{b}]$ will be smaller and as such, there is a greater probability that the search stage will discover the order of the ideal class $[\mathbf{b}]$. We note that the largest prime used in the exponent of the exponentiation stage can be no smaller than the largest prime in the primorial of the search stage. If this were the case, the search stage would step coprime to a possible factor of the order of the ideal class $[\mathbf{a}]$ and then could not possibly discover the order of the ideal class $[\mathbf{b}]$.

8.2 Lookup Tables

The SuperSPAR Algorithm 8.4 (and more generally, the bounded primorial steps algorithm 4.1) requires a lookup table mapping group elements $[\mathbf{b}]^i$ to exponents i . The baby steps

generate group elements $[\mathbf{b}]^i$ with i coprime to some primorial P_w and store the mapping $[\mathbf{b}]^i \mapsto i$ in the lookup table. The giant steps lookup group elements $[\mathbf{b}]^{2js}$ and $[\mathbf{b}]^{-2js}$ from the lookup table. Since we require these operations, the performance of the algorithm directly corresponds to the performance of the lookup table. This section describes the experimental results for SuperSPAR using two different implementations of lookup tables, as well as the method used to map group elements to exponents in our implementation.

Our implementation of ideal class arithmetic represents an ideal class $[\mathbf{a}] \in Cl_\Delta$ using the triple (a, b, c) where $\mathbf{a} = [a, (b + \sqrt{\Delta})/2]$ is a reduced representative for the ideal class and $c = (b^2 - \Delta)/4a$ (see Section 6.1). Since inversion of ideal classes is virtually free, SuperSPAR computes giant steps for ideal classes $[\mathbf{b}]^{2js}$ and $[\mathbf{b}]^{-2js}$, but rather than perform two table lookups, we take advantage of the way in which the inverse of an ideal class is computed. Recall from Section 2.3 that the inverse of an ideal class for a representative $[a, (b + \sqrt{\Delta})/2]$ is given by the representative $[a, (-b + \sqrt{\Delta})/2]$. Since $|b| = \pm\sqrt{\Delta + 4ac}$, we use the pair (a, c) for the key and our implementation is able to look up both $[\mathbf{b}]^{2sj}$ and $[\mathbf{b}]^{-2js}$ using a single table lookup. However, once a giant step finds a corresponding $[\mathbf{b}]^i$ in the table, both the exponents $2js - i$ and $2js + i$ must be tested to determine which one is a multiple of the order of $[\mathbf{b}]$. Figure 8.1 contrasts this with an implementation using two lookups per giant step. Both implementations use identical parameters on the same sets of semiprime integers, i.e. the executions of the two implementations only differ in the number of table lookups performed during the giant step portion of the search stage. In the implementation using two table lookups per giant step, the pair (a, b) is used for the hash key in order to distinguish between $[\mathbf{b}]^{2js}$ and $[\mathbf{b}]^{-2js}$, however, when some $[\mathbf{b}]^i$ is found, a multiple of the order is immediately known.

We implement the lookup table as a hash table with 32 bit keys and 32 bit values (both as unsigned integers). 32 bit values are sufficient since the theory predicts that for integers as large as 128 bits, the largest exponent generated by the search stage still fits within

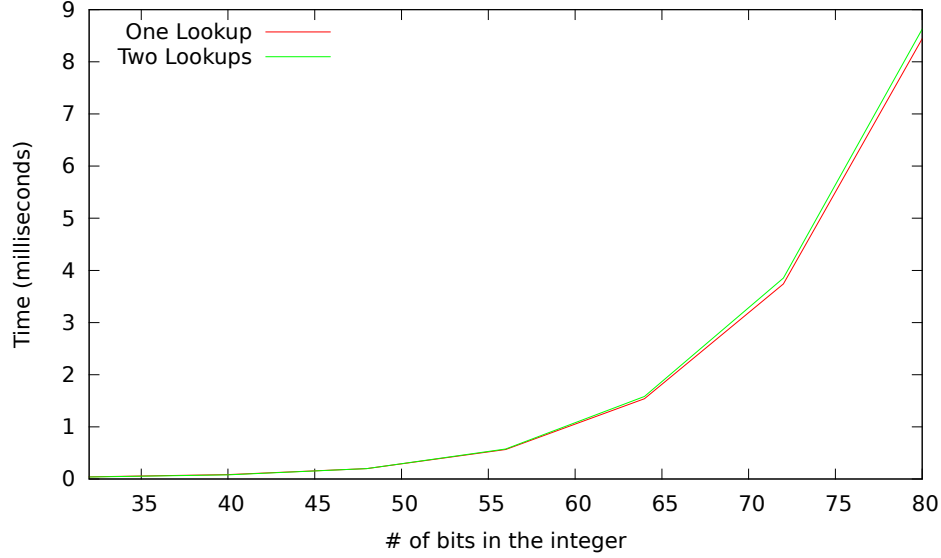


Figure 8.1: Average time to factor an integer by either using a single lookup for each giant step, with exponentiation to verify a multiple of the order, or two lookups for each giant step, but without the need to verify a multiple of the order.

31 bits (see Table 8.7 and Section 8.5 for more details). To generate the 32 bit key, our implementation maps the pair (a, c) associated with an ideal class using

$$\text{hash}_{32}(a, c) = (2654435761 \times (2654435761a + c)) \bmod 2^{32}.$$

The multiplier 2654435761 was chosen because it is the largest prime less than $2^{32} \times (\sqrt{5} - 1)/2$, and $(\sqrt{5} - 1)/2$ has been shown by Knuth [42, Section 6.4] to have good scattering properties. Other multipliers were also tried but had little impact on performance. Since the number of baby steps is a function of the size of the integer to be factored (see Section 8.6), the maximum number of entries in the hash table is known at initialization. We found that a table of 2^k slots for the smallest k such that $2^k \geq 3m\phi(P_2)$, where $m\phi(P_w)$ is the number of baby steps, worked well in practice. Different multiples of the number of baby steps were tried, as well as setting the number of slots to a prime near some multiple of the number of baby steps. There was little difference in performance, with the exception of a table having too few slots (resulting in an insertion failing), or a table having an excessive number of slots (in which case, table initialization dominated). Using a power of 2 for the

number of slots has the added benefit that once a hash key is computed, the corresponding slot is found by using a binary `and2` operation. However, regardless of the hash function used, multiple elements may hash to the same slot. Two methods of collision resolution are considered here, each being selected for cache coherency properties (see [22, Subsection 11.2] and [43]).

8.2.1 Chaining with List Heads

The first method is chaining with list heads (see [22, Subsection 11.2]). The idea is for the hash table to consist of 2^k slots, such that each slot operates as a linked list, but that the head of the linked lists are separate from the tails. Two areas of memory are used: one for the heads, and the other for the tails (also known as the overflow). In the head, each slot consists of the 32 bit value for a specified key, as well as the 32 bit hash key itself (since multiple keys may resolve to the same slot). A special pair of constants are used to indicate that a slot is unoccupied, e.g. (0xAAAAAAAA, 0x55555555). The overflow area consists of an array of no less than 2^k entries, each entry consisting of the 32 bit value for a specified hash key, the 32 bit hash key itself, and an index into the overflow area of the next node in the list, if any. A special constant, e.g. 0xFFFFFFFF, is used to indicate the end of list. The first 2^k entries in the overflow area are reserved and correspond to the second node in the linked list of each slot. To perform a lookup, the hash key resolves to a slot (using $\text{hash}_{32}(a, c) \bmod 2^k$) and the list is traversed until either the specified hash key or the end of the list is found. To insert or update, again, the list is traversed until either the specified hash key is found, at which point an update is performed, or the end of list is found. If the end of list is found, the next available entry in the overflow area is set appropriately and appended to the end of the list.

8.2.2 Open Addressing

The second method of collision resolution considered is open addressing, and in particular, we implement the same probing function as in CPython [43]. In this method, each slot consists of a 32 bit hash key, as well as the 32 bit value associated with that hash key. Any hash key can occupy any slot, and a special pair of constants are used to indicate that a slot is unoccupied, e.g. (0xAAAAAAAA, 0x55555555). When an element is hashed, the table is probed at slot indices $s_0, s_1, \dots$ generated by a probing function until a slot is found where either the hash key occupying the slot is the same as the hash key for the element in question, or the slot is unoccupied. Our implementation defines probe indices using the function defined in [43, p.299],

$$\begin{aligned}s_0 &= \text{hash}_{32}(a, b) \bmod 2^k \\ s_i &= (5s_{i-1} + 1 + \lfloor \text{hash}_{32}(a, b) / 2^{5i} \rfloor) \bmod 2^k.\end{aligned}$$

8.2.3 Chaining vs. Open Addressing

Figure 8.2 shows the average time to factor semiprime integers of the specified size using the collision resolution methods described above. In this context and for each integer range tested, open addressing for collision resolution performs better.

8.3 Factorization of the Order of Ideal Classes

In the introduction to this chapter, we identified several configuration parameters for our implementation of SuperSPAR based on the size of the input integer. One method for finding configuration parameters that perform well on average is to iterate the range of reasonable values for each parameter for each integer size, but this would be slow even for small sets of input integers. Furthermore, training the configuration parameters on a small set of integers is unlikely to lead to parameters that perform well on average.

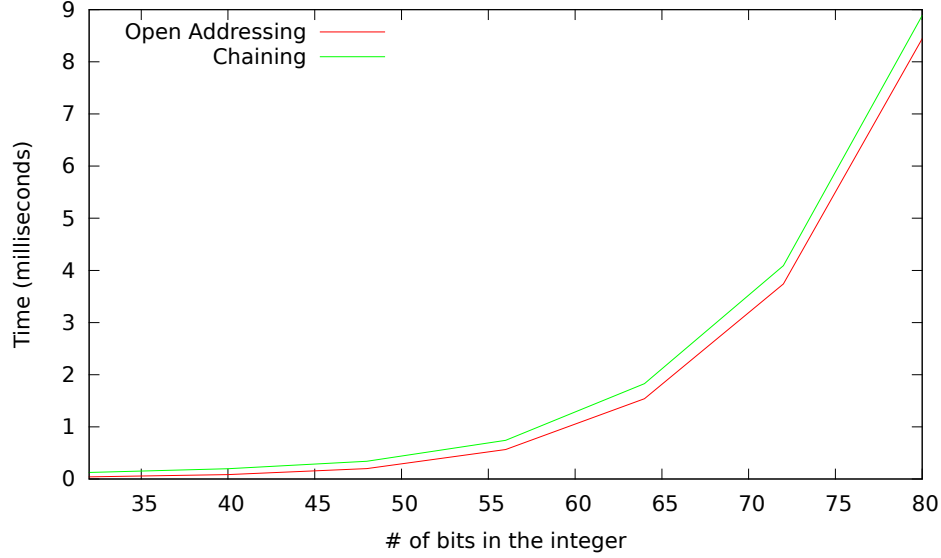


Figure 8.2: The average time to factor an integer using either open addressing or chaining with list heads for collision resolution in a hash table.

Since this chapter is concerned with the performance of SuperSPAR in practice, the following integer sizes were selected to highlight the range in which SuperSPAR is competitive with other factoring algorithms, but also in order to minimize the size of operands. Pari/GP was used to generate as large a data set as possible, but that still fit within the available memory (8Gb) and could be processed reasonably quickly. For bit sizes $n \in \{32, 40, 48, 56, 64, 72, 80\}$, roughly 250,000 unique semiprime integers $N = p \cdot q$ were generated such that p and q are prime and half the size of N , while N is n bits in size. Then for each square-free multiplier $k \in \{1, 2, 3, 5, 6, 7, 10\}$, discriminants $\Delta = -kN$ or $\Delta = -4kN$ such that $\Delta \equiv 0, 1 \pmod{4}$ were generated. For each corresponding ideal class group Cl_Δ , ideal class representatives $[\mathfrak{p}] \in Cl_\Delta$ with $\mathfrak{p} = [p, (b + \sqrt{\Delta})/2]$ and p prime were generated for the 5 smallest values of p such that $\gcd(p, \Delta) = 1$ and $4p \mid b^2 - \Delta$. Finally, for each ideal class $[\mathfrak{p}]$, the order of $[\mathfrak{p}]$, its corresponding prime factorization, and whether it lead to a successful factorization of the integer N were recorded. This data set is used throughout the computations in the following subsections and is denoted $\mathcal{D}$.

Subsection 8.3.1 uses data about the prime power factorization of the order of ideal

classes to determine values for each e_i in the exponent E used by the exponentiation stage. In Subsection 8.3.2, we justify a strategy of reusing a known multiple of the order of an ideal class to search for an ambiguous class starting with several ideal classes from the same class group. We then provide evidence in Subsection 8.3.3, at least for the integer range studied, that certain multipliers are more likely to lead to a factorization of the input integer. Finally, Subsection 8.3.4 shows that once a multiple of the order of an ideal class is known, that in expectation we only need to try $c \approx 2$ ideal classes before successfully factoring the input integer. However, this is tested empirically and we see that larger values of c perform better in practice.

8.3.1 Prime Power Bound

SPAR (Section 4.1) exponentiates an ideal class $[\mathbf{a}]$ to an exponent E where $E = \prod_{i=2}^t p_i^{e_i}$, p_i are prime, and $e_i = \max\{v : p_i^v \leq p_t^2\}$ for some appropriately chosen prime p_t [53, p.290]. In practice however, Schnorr and Lenstra recommend using smaller exponents e_i such that $e_i = \max\{v : p_i^v \leq p_t\}$ [53, p.293]. Doing so means that the exponentiation stage uses fewer group operations (and so takes less time), at the risk that the order of the resulting ideal class $[\mathbf{b}] = [\mathbf{a}]^E$ may still have small prime factors. Here we consider bounds for the exponents e_i for use with our implementation of SuperSPAR, assuming the range in which it is a competitive integer factoring tool.

The exponentiation stage of SuperSPAR computes $[\mathbf{b}] = [\mathbf{a}]^{2^\ell E}$ with $\ell = \left\lfloor \log_2 \sqrt{|\Delta|} \right\rfloor$. The search stage then computes baby steps $[\mathbf{b}]^i$ for i coprime to some primorial P_w . However, if $\text{ord}([\mathbf{b}])$ and P_w have a common factor p , then p is not coprime to P_w and $[\mathbf{b}]^p$ will not be added to the lookup table used by the coprime baby steps. If this is the case, the search phase is guaranteed to fail since it cannot find any $[\mathbf{b}]^{2js}$ such that $2js \equiv 0 \pmod{p}$ in the lookup table. In order to ensure that the order of $[\mathbf{b}]$ is coprime to E , the exponents in E would have to be chosen such that $e_i = \left\lfloor \log_{p_i} \sqrt{|\Delta|} \right\rfloor$, where $\sqrt{|\Delta|}$ is a bound on h_Δ (see Section 2.2). However, in practice it is more efficient to choose smaller values of e_i at the

risk that additional class groups are tried. For this reason, the exponent E is chosen to remove, with high probability, the factors from the order of $[\mathfrak{a}]$ that are common with the primorial P_w used by the search stage. In this subsection, we assume that the largest prime p_t dividing E in the exponentiation stage is larger than the largest prime p_w dividing P_w in the search stage. Here we are only concerned with determining bounds on e_i , the exponent of each prime factor of E .

For each ideal in our data set $\mathcal{D}$, let the factorization of the order be represented by $\text{ord}([\mathfrak{a}]) = \prod p_i^{e_i}$ for p_i prime. For each prime power factor 3^{e_2} , 5^{e_3} , 7^{e_4} , 11^{e_5} , 13^{e_6} , and 17^{e_7} , more than 99% of the ideals studied had either $e_2 \leq 4$, $e_3 \leq 2$, $e_4 \leq 2$, $e_5 \leq 1$, $e_6 \leq 1$, or $e_7 \leq 1$. This is captured by Table 8.2. This is not to say that more than 99% of the ideals in the data set were such that $\text{ord}([\mathfrak{a}]^{3^4 5^2 7^2 11^1 13^1 17^1})$ is coprime to $3 \times 5 \times 7 \times 11 \times 13 \times 17$, but only that $\text{ord}([\mathfrak{a}]^{3^4})$ is coprime to 3, and so on.

n	$e_2 = 4$	$e_3 = 2$	$e_4 = 2$	$e_5 = 1$	$e_6 = 1$	$e_7 = 1$
32	0.99598	0.99195	0.99713	0.99177	0.99419	0.99659
40	0.99594	0.99193	0.99709	0.99166	0.99404	0.99660
48	0.99593	0.99193	0.99713	0.99162	0.99406	0.99655
56	0.99593	0.99194	0.99709	0.99170	0.99404	0.99648
64	0.99584	0.99211	0.99713	0.99170	0.99412	0.99652
72	0.99586	0.99201	0.99707	0.99163	0.99415	0.99650
80	0.99580	0.99190	0.99719	0.99169	0.99404	0.99645

Table 8.2: The probability that $\text{ord}([\mathfrak{a}]^{p_i^{e_i}})$ is coprime to p_i .

On the other hand, Table 8.3 represents the probability that $\text{ord}([\mathfrak{a}]) = \prod p_i^{e_i}$ has $e_i \leq B_i$ for $2 \leq i \leq 5$, $e_i = 1$ for all $i > 5$, and e_1 is left unbounded. In other words, this table represents the probability that choosing some exponent E with the above constraints implies that the ideal class $[\mathfrak{b}] = [\mathfrak{a}]^E$ will have order $\text{ord}([\mathfrak{b}])$ coprime to the exponent E .

While almost any bounds on e_i work in practice, the purpose of this chapter is to optimize the performance of our implementation of SuperSPAR. To this end, let B denote a prime power bound such that $e_i = \max \{ \lfloor \log_{p_i} B \rfloor, 1 \}$ for all odd primes $p_i \leq B$ and $e_i = 1$ for all primes $p_i > B$; let $e_1 = \lfloor \log_2 \sqrt{|\Delta|} \rfloor$. We then iterate the bound B over increasing values

B_2	B_3	B_4	B_5	32 bits	40 bits	48 bits	56 bits	64 bits	72 bits	80 bits
1	1	1	1	0.40826	0.40693	0.40649	0.40598	0.40638	0.40624	0.40664
2	1	1	1	0.80574	0.80473	0.80484	0.80438	0.80500	0.80436	0.80502
2	2	1	1	0.90863	0.90739	0.90720	0.90718	0.90722	0.90714	0.90738
3	2	1	1	0.93187	0.93061	0.93071	0.93067	0.93057	0.93066	0.93051
3	2	2	1	0.94869	0.94735	0.94720	0.94724	0.94720	0.94734	0.94722
4	2	2	1	0.95669	0.95525	0.95511	0.95510	0.95518	0.95520	0.95499
4	2	2	2	0.96418	0.96256	0.96246	0.96238	0.96243	0.96255	0.96228
4	3	2	2	0.97062	0.96887	0.96871	0.96866	0.96858	0.96877	0.96856

Table 8.3: The probability that the order of an ideal class $[\mathbf{b}] = [\mathbf{a}]^E$ is coprime to the product $E = \prod p_i^{e_i}$ for primes p_i where $e_i \leq B_i$ for $2 \leq i \leq 5$; let $e_i = 1$ for all $i > 5$ and let e_1 be unbound.

such that exactly one of the e_i increases by just 1. Table 8.4 shows the average time to factor semiprime integers when the exponents e_i of the prime powers $p_i^{e_i}$ are bound accordingly. The largest prime p_t in the exponent E and the search stage bound mP_w are fixed for each input size, and were chosen based on the theoretically optimal values from Section 4.2. The sample set consists of 100,000 semiprime integers of each bit size, such that each integer was the product of two primes of equal size. Having determined bounds for the exponents e_i for the prime powers $p_i^{e_i}$, determining the exponent $E = \prod p_i^{e_i}$ for use with each bit range is simpler since it only involves bounding the largest prime p_t used. This is done in Section 8.6.

8.3.2 Difference between the Order of Two Ideal Classes

The purpose of this subsection is to justify a strategy where once h' , a multiple of the order of an ideal class $[\mathbf{a}_1]^{2^\ell E}$ for $[\mathbf{a}_1] \in Cl_\Delta$, is known, our implementation then attempts to find an ambiguous ideal by repeated squaring (up to at most $\ell = \left\lfloor \log_2 \sqrt{|\Delta|} \right\rfloor$ times) of $[\mathbf{a}_i]^{Eh'}$ for several other ideal classes $[\mathbf{a}_i] \in Cl_\Delta$. If both stages of the algorithm fail to find h' , then a different multiplier k for the discriminant of the class group is tried. Intuitively, the hope is that the factorization of the order of each ideal class $[\mathbf{a}_i]$ consists of one large prime and several small primes, and that the large prime is the same among all the ideal classes.

Power bound B	32 bits	40 bits	48 bits	56 bits	64 bits	72 bits	80 bits
9 = 3^2	0.04483	0.09538	0.24576	0.74423	2.13834	5.14941	11.81556
25 = 5^2	0.04469	0.09332	0.23505	0.70051	2.00290	4.82805	11.09770
27 = 3^3	<u>0.04453</u>	<u>0.08980</u>	0.22673	0.67292	1.90503	4.58491	10.51262
49 = 7^2	0.04550	0.09202	0.21968	0.65496	1.84011	4.42733	10.08651
81 = 3^4	0.04794	0.09219	<u>0.21840</u>	0.64810	1.80854	4.32226	9.88501
121 = 11^2	0.04988	0.09499	0.22151	0.64957	1.79549	4.27838	9.81882
125 = 5^3	0.04861	0.09505	0.22084	0.64985	1.79609	4.28010	9.80192
169 = 13^2	0.05103	0.09783	0.22195	0.64568	1.76646	4.21067	9.60217
243 = 3^5	0.05126	0.09871	0.22307	<u>0.64527</u>	<u>1.75882</u>	4.17222	9.50603
343 = 7^3	0.05415	0.10083	0.22761	0.64718	1.76139	4.17266	9.47648
625 = 5^4	0.05966	0.10620	0.23601	0.65786	1.76363	4.13652	9.31960
729 = 3^6	0.06031	0.11030	0.23750	0.65964	1.76242	<u>4.12565</u>	9.28392
1331 = 11^3	0.06028	0.10836	0.24391	0.66904	1.78810	4.13952	<u>9.28274</u>
2187 = 3^7	0.06274	0.11096	0.25596	0.69223	1.82186	4.18447	9.34884

Table 8.4: Average time (in milliseconds) to factor semiprime integers using the exponent $E = \prod_{p_i > 2} p_i^{e_i}$ with $e_i = \lfloor \log_{p_i} B \rfloor$ when $p_i \leq B$ and $e_i = 1$ otherwise. The lowest time for each integer range is underlined.

The exponentiation stage computes $[\mathbf{b}_1] = [\mathbf{a}_1]^{2^\ell E}$ for some exponent E chosen so that the factorization of the order of $[\mathbf{b}_1]$ is likely to be free of small primes. In this case, either the search stage successfully computes h' , which is hopefully also a multiple of the order of some other ideal class $[\mathbf{a}_i]^{2^\ell E}$, or the search stage was unsuccessful in determining the order of $[\mathbf{b}_1]$ because of the large prime factor dividing its order, and for this reason, the search stage will be unlikely to determine the order of some other ideal class $[\mathbf{a}_i]^{2^\ell E}$.

Suppose the search stage of SuperSPAR determines h' , a multiple of the order of an ideal class $[\mathbf{b}_1] = [\mathbf{a}_1]^{2^\ell E}$ for $\ell = \lfloor \log_2 \sqrt{|\Delta|} \rfloor$. Then for some other ideal class $[\mathbf{a}_i]$ in the same group, there exists a set of primes $\mathcal{P}$ such that the order of $[\mathbf{a}_i]^{2^\ell E}$ divides $h' \prod_{p \in \mathcal{P}} p$, where the product of the empty set is taken to be 1. Also suppose that an exponent E was chosen for the exponentiation stage such that $\text{ord}([\mathbf{a}]^E)$ is coprime to $2 \times 3 \times 5 \times 7 \times 11$. We chose 11 as the largest prime here since for over 99% of the ideal classes in our set $\mathcal{D}$, 11^2 does not divide the order (see Table 8.2). Also, notice that $2 \times 3 \times 5 \times 7 \times 11 \mid E$ but that E may be considerably larger than $2 \times 3 \times 5 \times 7 \times 11$.

Assuming the above constraints, for two ideal classes in the same group $[\mathbf{u}], [\mathbf{v}] \in Cl_\Delta$,

n	$ \mathcal{P} = 0$	$ \mathcal{P} = 1$	$ \mathcal{P} = 2$	$ \mathcal{P} = 3$
32	0.97804	0.02177	0.00020	0.00000
40	0.97791	0.02183	0.00026	0.00000
48	0.97785	0.02187	0.00028	0.00000
56	0.97776	0.02196	0.00028	0.00000
64	0.97779	0.02193	0.00028	0.00000
72	0.97782	0.02190	0.00027	0.00000
80	0.97781	0.02191	0.00028	0.00000

Table 8.5: If h' , a multiple of the order of an ideal class $[\mathbf{u}]$ is known, this table shows the probability that the order of the ideal class $[\mathbf{v}]^{h'}$, for some other ideal class $[\mathbf{v}]$ in the same group, has exactly $|\mathcal{P}|$ prime factors larger than 11.

Table 8.5 shows the probability that the smallest set of primes $\mathcal{P} = \{p : p \text{ is prime}, p > 11\}$ with $\text{ord}([\mathbf{v}]^{2^\ell E})$ dividing $\text{ord}([\mathbf{u}]^{2^\ell E}) \times \prod_{p \in \mathcal{P}} p$ is of a given size. Of the ideal class groups studied and assuming the above constraints, more than 97.7% of the time, if $2^\ell E h'$ is a multiple of the order of an ideal class, then it is also a multiple of the order of some other ideal class in the same group. For SuperSPAR, this means that once a multiple of the order of an ideal class is discovered, if this does not lead to a factor of N , then with high probability, it is also a multiple of the order of some other ideal class in the same group. Rather than starting over with a different ideal class, let h' be the odd part of the multiple of the order and compute $[\mathbf{a}_2]^{h'}$ for some other ideal class $[\mathbf{a}_2] \in Cl_\Delta$. SuperSPAR then tries to find an ambiguous class by repeated squaring of $[\mathbf{a}_2]^{h'}$ and, if successful, attempts to factor N . This process may be repeated for several different ideal classes in the same group. However, if the algorithm is unsuccessful for more than a fixed number of ideal classes within the same class group, Subsection 8.3.4 shows that changing class groups by changing multipliers is beneficial.

8.3.3 Best Multipliers

As in the previous subsection, assume that an exponent E is chosen for the exponentiation stage such that $\text{ord}([\mathbf{a}_1]^{2^\ell E})$ is coprime to E . Also, suppose that the search stage is successful in determining h' , a multiple of the order of $[\mathbf{a}_1]^{2^\ell E} \in Cl_\Delta$. Our implementation of

SuperSPAR will first attempt to factor the integer N associated with the discriminant Δ by searching for an ambiguous class via repeated squaring of the ideal class $[\mathfrak{a}_1]^{Eh'}$. Failing this, the algorithm tries again by repeated squaring of the ideal classes $[\mathfrak{a}_2]^{Eh'}$, $[\mathfrak{a}_3]^{Eh'}$, ... and so on in sequence. After trying a fixed number of ideal classes, our implementation starts over with a different multiplier of the input integer. This subsection uses the data set $\mathcal{D}$ to determine the expected number of ideal classes to try before a successful factoring of the integer N .

The discriminant Δ associated with an integer N and multiplier k is chosen to be either $\Delta = -kN$ when $-kN \equiv 0, 1 \pmod{4}$, or $\Delta = -4kN$. With this in mind, the data set $\mathcal{D}$ is separated into subsets for $N \equiv 1 \pmod{4}$ or $N \equiv 3 \pmod{4}$, and again for each multiplier $k \in \{1, 2, 3, 5, 6, 7, 10\}$. Figure 8.3 shows the expected number of ideal classes to try before factoring N , assuming that an appropriate exponent E for the exponentiation stage is chosen and that a multiple of the order h' was found during the search stage.

Given this separation of the integer N and the multiplier k in our data set $\mathcal{D}$, the class groups associated with discriminants $\Delta = -kN$ or $\Delta = -4kN$, as appropriate, appear to be separate with respect to the probability that an ideal class in the group will lead to a factoring of the integer N . Using this result, our implementation of SuperSPAR chooses multipliers of N sequentially such that when $N \equiv 1 \pmod{4}$, $k = 6, 10, 3, 1, 7, 2, 5$, and when $N \equiv 3 \pmod{4}$, $k = 1, 10, 3, 2, 5, 7, 6$. In the event that the algorithm exhausts this list of multipliers without successfully factoring N , square-free multipliers $k > 10$ are selected in increasing order.

8.3.4 Expected Number of Ideal Classes

Figure 8.3 indicates that for most multipliers, if the search stage of the algorithm is successful in finding a multiple of the order of an ideal class, then in expectation, roughly 2 ideal classes need to be tried before determining a factor of N . Although switching multipliers after at most two ideal classes does work in practice, our implementation of SuperSPAR is more

efficient if many ideal classes are tried before switching class groups. This is possibly due to the cost of the search phase of the algorithm, which is executed for each class group, in contrast to the exponentiation stage, which is executed for each ideal class tried.

Classes/Group	32 bits	40 bits	48 bits	56 bits	64 bits	72 bits	80 bits
1	0.05570	0.12454	0.31630	0.92448	2.56007	6.55736	14.64756
2	0.04462	0.09425	0.23631	0.68485	1.92037	4.88625	11.07386
3	0.04181	0.08708	0.21465	0.62268	1.74317	4.39471	10.01309
4	0.04104	0.08418	0.20621	0.59624	1.66741	4.19481	9.55292
5	<u>0.04080</u>	0.08306	0.20254	0.58463	1.63373	4.09556	9.31992
6	0.04081	0.08270	0.20113	0.57836	1.61780	4.03869	9.18897
7	0.04098	<u>0.08262</u>	0.20026	0.57618	1.60919	4.01559	9.09742
8	0.04113	0.08266	0.20008	0.57406	1.60498	3.99565	9.05229
9	0.04130	0.08284	<u>0.20005</u>	<u>0.57365</u>	1.60082	3.98321	9.02378
10	0.04146	0.08311	0.20028	0.57390	1.60011	3.97691	9.00030
11	0.04175	0.08327	0.20067	0.57404	<u>1.59738</u>	3.97021	8.98048
12	0.04180	0.08348	0.20092	0.57470	1.59855	<u>3.96747</u>	8.97534
13	0.04200	0.08367	0.20129	0.57520	1.59944	3.96789	<u>8.97226</u>
14	0.04221	0.08391	0.20161	0.57583	1.60174	3.96836	8.97280
15	0.04231	0.08407	0.20208	0.57680	1.60335	3.97339	8.97848
16	0.04251	0.08431	0.20248	0.57780	1.60844	3.97845	8.97971

Table 8.6: The average time (in milliseconds) to factor an integer N using no more than a fixed number of ideal classes for each class group. The lowest time for each integer range is underlined.

Table 8.6 shows the average time to factor integers given an upper bound on the number of ideal classes tried before switching multipliers. The times here are for the same set of 100,000 semiprime integers as in Table 8.4. Again, the largest prime in the exponent for the exponentiation stage, and the multiple of a primorial for the search stage where chosen according to the theoretically optimal values of Section 4.2. Our implementation of SuperSPAR bounds the number of ideal classes tried before switching class groups to the lowest time for each integer range in the table given. For integer sizes not appearing in the table, they are rounded up to the nearest value appearing in the table, or for integer sizes greater than 80, they are linearly extrapolated by 1 class/group for each 8 bits.

8.4 SuperSPAR Algorithm in Practice

This section describes and gives pseudo-code for our implementation of SuperSPAR. We include this here as our implementation makes use of the results of the previous sections.

Section 8.1 gave an algorithm to compute a list of coprime deltas for a primorial P_w . Since the baby-step portion of the search phase steps coprime up to a multiple of some primorial, we precompute deltas between neighbouring coprime values. Table 8.1 showed the rate of growth of the first 12 primorials, and the number of integers coprime to each primorial. The 10th primorial has over a billion coprime values less than P_{10} , while the 11th primorial has over 30 billion coprime values less than P_{11} . For this reason, we limit our precomputation of the delta lists to the first 10 primorials. These lists were generated using Algorithm 8.1 and then made available as global data to our implementation.

Section 8.2 discussed a method to simultaneously hash an ideal class and its inverse. Briefly, our implementation from Chapter 6 represents an ideal class using the triple (a, b, c) . The inverse is given as $(a, -b, c)$. Since the value of $|b|$ can be determined from the values a , c , and Δ , we compute a hash using only the values a and c . Since two ideals may still hash to the same value, we use open addressing (see Subsection 8.2.2) for collision resolution.

Our implementation of SuperSPAR uses many results from Section 8.3. The prime power bounds e_i for the exponent $E = \prod p_i^{e_i}$ in the exponentiation stage were given in Subsection 8.3.1 and in particular, Table 8.4. If either stage determines a multiple of the order of an ideal class, but this does not lead to a factor of the input integer N , then our implementation reuses the multiple of the order for several ideal classes. However, if either stage fails to determine a multiple of the order of an ideal class, the algorithm tries again using a different square-free multiplier. This strategy was justified in Subsection 8.3.2. In addition, our implementation uses the results from Subsection 8.3.4 to limit the number of ideal classes tried before switching multipliers, specifically, we use the results of Table 8.6. Lastly, Subsection 8.3.3 indicates that for the set of ideals tested, some multipliers are more

likely to be successful at factoring the input integer. For this reason, the list of square-free multipliers provided to our implementation is ordered according to these results.

The pseudo-code for our implementation of SuperSPAR is separated into three listings. Algorithm 8.2 attempts to find an ambiguous ideal class by repeated squaring of an input ideal class up to at most ℓ times. Bounding the number of times we square is necessary since the order of the input ideal class could be odd. If the algorithm finds an ambiguous ideal, it attempts to factor the input integer N . Details were given in Subsection 4.1.1, but we briefly restate them here. For an ideal class representative $\mathbf{a} = [a, (b + \sqrt{\Delta})/2]$, let $c = (b^2 - \Delta)/4a$. An ideal class is an ambiguous class when either $b = 0$, $a = b$, or $a = c$ (see [53, p.303]). Since the discriminant is $\Delta = b^2 - 4ac$, given an ambiguous form, either $\Delta = 4ac$ when $b = 0$, $\Delta = b(b - 4c)$ when $a = b$, or $\Delta = (b - 2a)(b + 2a)$ when $a = c$. Once an ambiguous class is found, we then compute $d = \gcd(a, N)$ if $b = 0$ or $a = b$, and $d = \gcd(b - 2a, N)$ otherwise. We compute the GCD since $\Delta = -kN$ or $\Delta = -4kN$ and the ambiguous form leads to a factor of Δ . Again, see Subsection 4.1.1 for details.

Algorithm 8.2 Try to find an ambiguous ideal class by repeated squaring (Subsection 4.1.1).

```

1: procedure REPEATEDLYSQUARE( $[\mathbf{b}]$ ,  $\ell$ )
2:   for  $i$  from 1 to  $\ell$  while  $[\mathbf{b}] \neq [\mathcal{O}_\Delta]$  do
3:     if  $[\mathbf{b}]$  is an ambiguous ideal class then
4:       if  $[\mathbf{b}]$  leads to a non-trivial factor of  $N$  then
5:          $d \leftarrow$  a non-trivial factor of  $N$ 
6:         return  $([\mathbf{b}], d)$ 
7:       else
8:         return  $([\mathbf{b}], 1)$ 
9:      $[\mathbf{b}] \leftarrow [\mathbf{b}]^2$ 
10:  return  $([\mathbf{b}], 1)$ 
```

Once a multiple of the order of an ideal class is known, Algorithm 8.3 is used to try to find an ambiguous ideal class from at most c ideal classes in the same class group. If the algorithm is successful, a non-trivial factor of the input integer N is returned, otherwise 1 is returned to indicate failure. This algorithm makes use of Algorithm 8.2 as a sub-algorithm and is itself a sub-algorithm of the main algorithm given by Algorithm 8.4.

Algorithm 8.3 Try to factor N using several ideal classes and a multiple of the order of a single ideal class.

```

1: procedure TRYMANYIDEALCLASSES( $\Delta, h, \ell, c$ )
2:   for  $2 \leq i \leq c$  do
3:     let  $[\mathfrak{p}_i]$  be the  $i^{\text{th}}$  smallest prime ideal class in  $Cl_\Delta$ 
4:      $([\mathfrak{b}], d) \leftarrow \text{REPEATEDLYSQUARE}([\mathfrak{p}_i]^h, \ell)$   $\{[\mathfrak{b}] \text{ is ignored}\}$ 
5:     if  $d \neq 1$  then return  $d$ 
6:   return 1

```

Algorithm 8.4 is the main factoring algorithm. It takes as input an odd composite positive integer N , and the configuration parameters. The configuration parameters are determined by the size of the input integer N and are looked up from a skeleton function that invokes the main algorithm. The previous sections in this chapter constrain the majority of the configuration parameters with the exception of the largest prime p_t used in the exponent $E = \prod_{i=2}^t p_i^{e_i}$ for the exponentiation stage, and the multiple of a primorial mP_w used for the search stage. We give pseudo-code for our implementation of SuperSPAR here, since these two parameters are empirically determined in Section 8.6 by running the main algorithm on sets of semiprime integers for various values of E and mP_w , by using Algorithm 8.7 to search for a local minimum in 2 dimensions.

The main algorithm first computes $\ell = \left\lceil \log_2 \sqrt{|\Delta|} \right\rceil$ where $\sqrt{|\Delta|}$ is a bound on the class number given in Section 2.2, and then sets the square-free multiplier index k to 1. The algorithm then computes a discriminant Δ for N and the multiplier corresponding to the index k , and then begins the exponentiation stage.

The exponentiation stage first finds an ideal class $[\mathfrak{p}_1] \in Cl_\Delta$ such that $\mathfrak{p}_1 = [p, (b+\sqrt{\Delta})/2]$ for the smallest prime p with $\gcd(p, \Delta) = 1$ and $4p \mid b^2 - \Delta$. Next, the algorithm computes $[\mathfrak{b}'] \leftarrow [\mathfrak{p}_1]^E$. In our implementation, E is given as a 2,3 representation precomputed using a left-to-right best approximations method of Section 7.6, and we use Algorithm 3.3 to perform the exponentiation. The last step of the exponentiation stage is to use Algorithm 8.2 to repeatedly square $[\mathfrak{b}']$, test for an ambiguous form, and attempt to factor the input integer N . Failing this, the exponentiation stage has computed $[\mathfrak{b}] = [\mathfrak{p}_1]^{2^\ell E}$ and the algorithm

continues with the search stage.

The search stage begins by precomputing each $[\mathbf{b}]^{\delta_i}$ and caches these in $[\mathfrak{d}_{\delta_i}]$. These are used to move from one coprime baby step to the next. Only even deltas are computed since only odd exponents are computed by the baby steps. Lines 12 through 16 compute baby steps for $[\mathbf{b}]^i$ with $1 \leq i \leq s$ and $\gcd(i, P_w) = 1$, but do so by utilizing δ_i and the cached steps $[\mathfrak{d}_{\delta_i}]$. If any $[\mathbf{b}]^i = [\mathcal{O}_\Delta]$, then the algorithm records i as the order of $[\mathbf{b}]$ and jumps to line 28, where we attempt to find an ambiguous ideal and then factor N . The loop exits after the last baby step with $i = s + 1$, at which point, line 17 effectively computes $[\mathbf{c}] = [\mathbf{b}]^{2s}$ where $[\mathbf{c}]$ is the first giant step. We choose to compute this by multiplying $[\mathbf{c}]$ with the inverse of $[\mathbf{b}]$ and then squaring. An alternative method is to not compute the last baby step such that $i < s$, however, this requires us to then compute $[\mathbf{c}] \leftarrow [\mathbf{c}] \cdot [\mathbf{b}]^{s-i}$ where $s - i$ is guaranteed to be odd. To speed this up, we can compute $[\mathbf{c}] \leftarrow [\mathbf{c}] \cdot [\mathbf{b}]^{s-i-1} \cdot [\mathbf{b}]$ since $[\mathbf{b}]^{s-i-1}$ is even and will be cached by $[\mathfrak{d}_{s-i-1}]$, however, this is the same number of operations (not counting inversions, which are essentially free) as the method employed in our implementation. Next, the algorithm performs an equal number of giant steps by computing $[\mathbf{c}] \leftarrow [\mathbf{c}] \cdot [\mathfrak{d}]$ for each iteration where $[\mathfrak{d}] = [\mathbf{b}]^{2s}$ from line 18. If any $[\mathbf{c}]$ is in the table, then either $2js - i$ or $2js + i$ is a multiple of the order of $[\mathbf{b}]$ and we continue at line 25 where we attempt to find an ambiguous ideal class for both values. If either stage fails, we start over with the next square-free multiplier from the list.

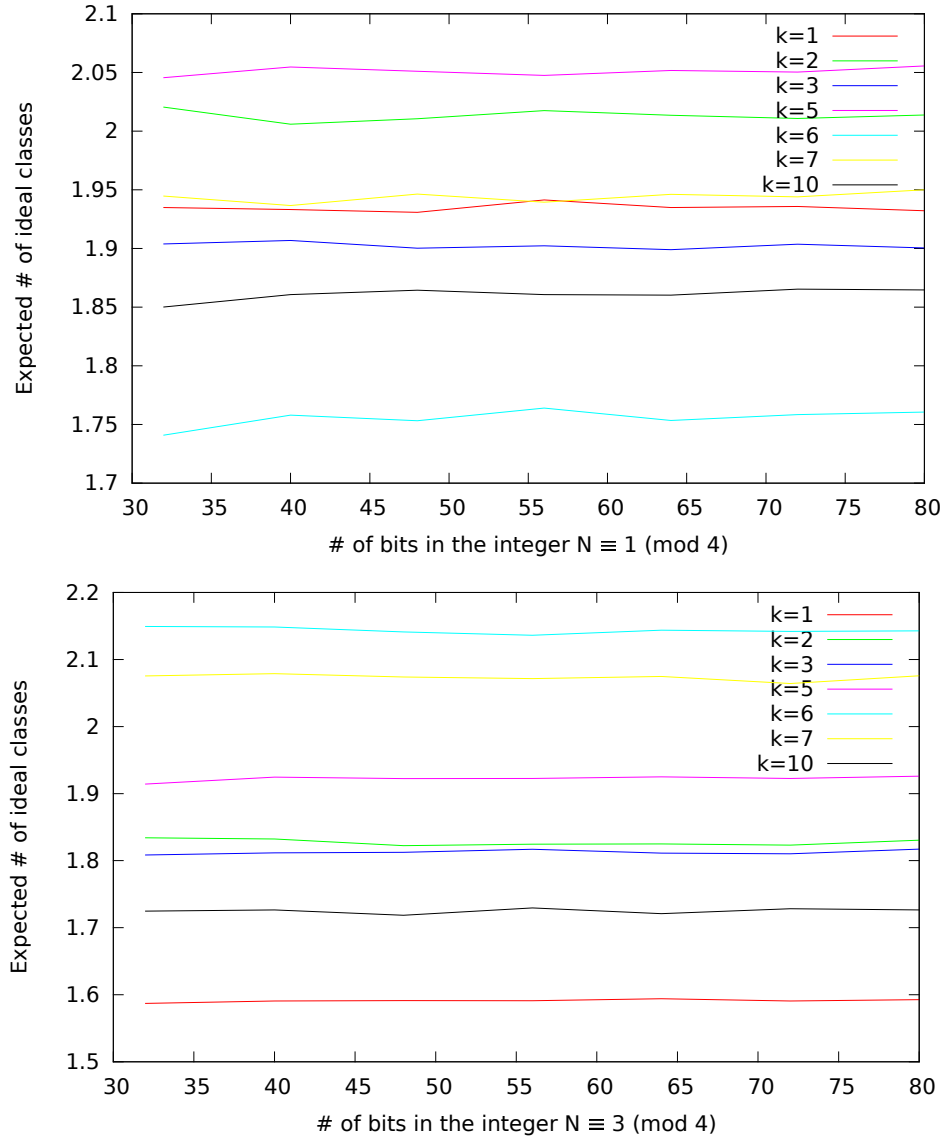


Figure 8.3: Attempts $N \equiv 1 \pmod{4}$

Algorithm 8.4 SuperSPAR Integer Factoring Algorithm.

Input: $N \in \mathbb{Z}_{\geq 0}$ odd and composite, $E = \prod_{i=2}^t p_i^{e_i}$ for the exponentiation phase, $s = mP_w$ a multiple of a primorial for the search phase and $\phi(P_w)$, $c \in \mathbb{Z}_{>0}$ the number of ideal classes to try before switching multipliers, a coprime delta list $\mathcal{D}_w = \delta_1, \delta_2, \dots, \delta_{\phi(P_w)}$ for the primorial P_w , $\delta_{\max} = \max \mathcal{D}_w$, and square-free multipliers $\kappa_1, \kappa_2, \dots$

- 1: $\ell = \left\lfloor \log_2 \sqrt{|\Delta|} \right\rfloor$ {bound for repeated squaring}
- 2: $k \leftarrow 1$ {square-free multiplier index}
- 3: $\Delta \leftarrow -\kappa_k N$
- 4: **if** $\Delta \not\equiv 0, 1 \pmod{4}$ **then** $\Delta \leftarrow 4\Delta$
{– exponentiation stage –}
- 5: let $[\mathfrak{p}_1]$ be the smallest prime ideal class in Cl_Δ
- 6: $[\mathfrak{b}'] \leftarrow [\mathfrak{p}_1]^E$
- 7: $([\mathfrak{b}], d) \leftarrow \text{REPEATEDLYSQUARE}([\mathfrak{b}'], \ell)$
- 8: **if** $d \neq 1$ **then return** d
- 9: **if** $[\mathfrak{b}]$ is an ambiguous ideal class **then** $h \leftarrow 1$, and go to line 30
{– search stage –}
- 10: compute $[\mathfrak{d}_2] = [\mathfrak{b}]^2$ and $[\mathfrak{d}_{2i}] = [\mathfrak{d}_{2(i-1)}] \cdot [\mathfrak{d}_2]$ for $2 \leq i \leq \delta_{\max}/2$
- 11: $i \leftarrow 1, j \leftarrow 1, [\mathfrak{c}] \leftarrow [\mathfrak{b}]$
- 12: **while** $i \leq s$ **do** {coprime baby-steps}
- 13: store $[\mathfrak{c}] \mapsto i$ in the lookup table
- 14: **if** $[\mathfrak{c}] = [\mathcal{O}_\Delta]$ **then** $h \leftarrow i$, and go to line 28
- 15: $[\mathfrak{c}] \leftarrow [\mathfrak{c}] \cdot [\mathfrak{d}_{\delta_j}], i \leftarrow i + \delta_j$
- 16: $j \leftarrow (j + 1) \bmod \phi(P_w)$
- 17: $[\mathfrak{c}] \leftarrow ([\mathfrak{c}] \cdot [\mathfrak{b}]^{-1})^2$ {last baby-step was $s + 1$, first giant-steps is $2s$ }
- 18: $[\mathfrak{d}] \leftarrow [\mathfrak{c}]$
- 19: **for** $1 \leq j \leq m\phi(P_w)$ **do** {giant-steps}
- 20: **if** $[\mathfrak{c}]$ is in the table, first lookup i **then**
- 21: $h \leftarrow$ odd part of $(2js - i)$, $h_2 \leftarrow$ odd part of $(2js + i)$
- 22: go to line 25
- 23: $[\mathfrak{c}] \leftarrow [\mathfrak{c}] \cdot [\mathfrak{d}]$
- 24: go to line 32 {failed to find an ambiguous class}
{– found a multiple of the order –}
- 25: $([\mathfrak{b}], d) \leftarrow \text{REPEATEDLYSQUARE}([\mathfrak{b}]^{h_2}, \ell)$
- 26: **if** $d \neq 1$ **then return** d
- 27: **if** $[\mathfrak{b}] = [\mathcal{O}_\Delta]$ **then** $h \leftarrow h_2$, and go to line 30.
- 28: $([\mathfrak{b}], d) \leftarrow \text{REPEATEDLYSQUARE}([\mathfrak{b}]^h, \ell)$ { $[\mathfrak{b}]$ is ignored}
- 29: **if** $d \neq 1$ **then return** d
- 30: $d \leftarrow \text{TRYMANYIDEALCLASSES}(\Delta, E \cdot h, \ell, c)$
- 31: **if** $d \neq 1$ **then return** d
- 32: $k \leftarrow k + 1$
- 33: start over at line 3

Before we move on, we reiterate some of the results from previous chapters that are used in our implementation of SuperSPAR. Since SuperSPAR is based on ideal class arithmetic, we use the results from Chapter 6. In particular, when the discriminant Δ is less than 60 bits, we use our 64-bit implementation of ideal class arithmetic, otherwise, when Δ is less than 119 bits, we use our 128-bit implementation, and when Δ is larger than 118 bits, we use our reference implementation based on GMP for multiple precision arithmetic.

Again, our 64-bit and 128-bit implementations of ideal class arithmetic rely heavily on the results of the extended GCD experiments in Chapter 7. In particular, when the inputs are guaranteed to be less than 32 bits, we use our 32-bit implementation of the extended Euclidean Algorithm (see Section 5.1). Otherwise, when inputs are guaranteed to be less than 64 bits, we use our 64-bit implementation of our simplified left-to-right binary extended GCD (see Section 5.3). Failing this, we use our 128-bit implementation of our simplified left-to-right binary extended GCD for inputs less than 118 bits, and our 128-bit implementation of Lehmer's extended GCD using our simplified left-to-right binary extended GCD for 64-bit machine words (see Section 5.4) when inputs are at least 119 bits. These methods were selected since Section 5.7 shows that each method performs the fastest on average for their given input range. For each of these extended GCD methods, we also implemented a left-only and partial version and use them as applicable for ideal class arithmetic. For larger inputs, we use GMP for the extended GCD and left-only extended GCD, and we implemented our own partial extended Euclidean GCD using GMP arithmetic.

8.5 Search Bounds

This section introduces the method we use to generate a list of candidate search bounds. Each search bound is a multiple of some primorial mP_w such that the search stage of SuperSPAR takes $m\phi(P_w)$ steps for values coprime up to mP_w , and then takes another $m\phi(P_w)$ giant steps up to $2m^2\phi(P_w)P_w$. The idea is to generate a list of pairs (m, w) such that

for each pair in the list, there does not exist a pair (m', w') with a larger final giant step requiring the same or fewer total number of steps. This list is useful since in Section 8.6, we empirically determine search bounds that work well on average for inputs of a given size. These empirically determined search bounds use a different number of total steps than the theoretically optimal number predicted in Section 4.2. This list may also be useful for other algorithms deriving from the bounded primorial steps algorithm (see Subsection 4.2.1) where the search bound is determined empirically. We describe the technique here as it does not appear in the literature.

The search stage will successfully compute the order of an ideal class if the order is no larger than the largest exponent $2m^2\phi(P_w)P_w$ generated by the giant steps – we will refer to this as the *search range*. The idea here is to generate a set $\mathcal{S}$ of primorial and multiplier pairs used as candidates for the baby step bound s . For each pair, let δ be a function from some multiple m of the primorial P_w to the total number of baby steps and giant steps taken during the search stage of the algorithm,

$$\delta(m, w) = 2m\phi(P_w). \quad (8.1)$$

Notice we take $m\phi(P_w)$ baby steps plus $m\phi(P_w)$ giant steps. Each giant step is of size $2mP_w$, so let θ be a function that computes the exponent of the final giant step, i.e. the search range,

$$\theta(m, w) = 2m^2\phi(P_w)P_w. \quad (8.2)$$

For a given pair (m, w) representing a search bound, the total number of steps taken and the search range are unique. This idea is more formally expressed by the following theorem.

Theorem 8.5.1. For the pair (m_i, w_i) , there does not exist a pair (m_j, w_j) with $m_i \neq m_j$ or $w_i \neq w_j$ such that both $\delta(m_i, w_i) = \delta(m_j, w_j)$ and $\theta(m_i, w_i) = \theta(m_j, w_j)$.

Proof. By contradiction, suppose there exists (m_i, w_i) and (m_j, w_j) with $m_i \neq m_j$ or $w_i \neq w_j$ such that $\delta(m_i, w_i) = \delta(m_j, w_j)$ and $\theta(m_i, w_i) = \theta(m_j, w_j)$. Let $w_i < w_j$. By Equation (8.1)

$$2m_i\phi(P_{w_i}) = 2m_j\phi(P_{w_j}) \quad (8.3)$$

which implies

$$\frac{m_i}{m_j} = \frac{\phi(P_{w_j})}{\phi(P_{w_i})}. \quad (8.4)$$

Then by Equation (8.2)

$$2m_i^2\phi(P_{w_i})P_{w_i} = 2m_j^2\phi(P_{w_j})P_{w_j}.$$

Substituting from Equation (8.3) above gives

$$2m_j\phi(P_{w_j})m_iP_{w_i} = 2m_j^2\phi(P_{w_j})P_{w_j}.$$

Cancelling $2m_j\phi(P_{w_j})$ from both sides gives

$$m_iP_{w_i} = m_jP_{w_j}$$

which implies

$$\frac{m_i}{m_j} = \frac{P_{w_j}}{P_{w_i}}. \quad (8.5)$$

Then by Equations (8.4) and (8.5), we have

$$\frac{\phi(P_{w_j})}{\phi(P_{w_i})} = \frac{P_{w_j}}{P_{w_i}}.$$

This expands to

$$(p_{w_j} - 1)(p_{w_{j-1}} - 1) \cdots (p_{w_{i+1}} - 1) = p_{w_j}p_{w_{j-1}} \cdots p_{w_{i+1}}.$$

Since each term on the left hand side is 1 less than the corresponding term on the right hand side, this is only possible when $w_i = w_j$ and then for $\delta(m_i, w_i) = \delta(m_j, w_j)$ and $\theta(m_i, w_i) = \theta(m_j, w_j)$ to be true, $m_i = m_j$ must also be true. $\square$

Thus, the set $\mathcal{S}$ consists of pairs (m, w) such that for all (m', w') either

$$\theta(m', w') < \theta(m, w) \text{ or } \delta(m', w') > \delta(m, w).$$

In other words, the set $\mathcal{S}$ consists of the pairs (m, w) such that for all other pairs (m', w') , either the search range $\theta(m', w')$ is not as large, or the total number of steps $\delta(m', w')$ is greater. The set $\mathcal{S}$ is defined as

$$\mathcal{S} = \{(m, w) : \text{for all } (m', w'), \text{ either } \theta(m', w') < \theta(m, w) \text{ or } \delta(m', w') > \delta(m, w)\} \quad (8.6)$$

and is infinite. When a bound B on the total number of steps $\delta(m, w)$ is given, a subset $\mathcal{S}_B \subset \mathcal{S}$ is defined as

$$\mathcal{S}_B = \{(m, w) : (m, w) \in \mathcal{S} \text{ and } \delta(m, w) \leq B\} \quad (8.7)$$

and Algorithm 8.5 can be used to generate such a set.

Algorithm 8.5 Compute baby step bound candidates.

Input: A bound B on the total number of steps $\delta(m, w)$.

Output: The set of candidate pairs $\mathcal{S}$ (Equation (8.6)).

- 1: generate the set $\mathcal{T} = \{(m, w) : \delta(m, w) \leq B\}$
- 2: define an ordering on the set $\mathcal{T}$ using

$$(m_i, w_i) < (m_j, w_j) \Leftrightarrow \begin{cases} \delta(m_i, w_i) < \delta(m_j, w_j) \\ \delta(m_i, w_i) = \delta(m_j, w_j) \text{ and } \theta(m_i, w_i) > \theta(m_j, w_j). \end{cases}$$

- 3: let $(m_1, w_1), (m_2, w_2), \dots, (m_n, w_n)$ be the list of elements generated by sorting the set $\mathcal{T}$ in ascending order
 - 4: $\mathcal{S}_B \leftarrow \{\}$
 - 5: $(m, w) \leftarrow (m_1, w_1)$
 - 6: **for** $i = 2$ to n **do**
 - 7: **if** $\theta(m_i, w_i) > \theta(m, w)$ **then**
 - 8: $\mathcal{S}_B \leftarrow \mathcal{S}_B \cup \{(m, w)\}$
 - 9: $(m, w) \leftarrow (m_i, w_i)$
 - 10: $\mathcal{S}_B \leftarrow \mathcal{S}_B \cup \{(m, w)\}$
 - 11: **return** $\mathcal{S}_B$
-

First the algorithm generates all pairs (m, w) such that $\delta(m, w) \leq B$. There is a finite number of these pairs, since there is a finite number of primorials no larger than B , and for a

given w there are $\lfloor B/2\phi(P_w) \rfloor$ possible values of m . The algorithm then sorts these pairs in ascending order by the total number of steps $\delta(m, w)$, breaking ties by sorting in descending order of the search range $\theta(m, w)$. The first pair in the list is chosen as a candidate (m, w) for the set $\mathcal{S}_B$. This pair is always in the solution set since by Equation (8.6) and the ordering imposed on the list, all other pairs either take more steps or an equal number of steps, but have a smaller search range. The algorithm then iterates over the remaining pairs, and the search range of each pair is compared with that of the candidate pair. If the search range is greater, then the candidate pair is output to the set $\mathcal{S}_B$ and the current pair becomes the next candidate pair. The correctness of this follows from the fact that pairs in the list are monotonically increasing by the total number of steps, and successive candidate pairs are strictly increasing in their search range. Therefore, when the candidate pair (m, w) is compared with the pair (m_i, w_i) from the list, if the search range of the list pair is less than that of the candidate pair, the list pair uses at least as many steps, but does not search as far, and therefore is not a member of the set $\mathcal{S}_B$. Otherwise, when the list pair has a larger search range, this implies that it uses more steps (since ties are broken by sorting in descending order of search range), and the candidate pair is guaranteed to be a member of the set $\mathcal{S}_B$. This holds since all the pairs remaining in the list use more steps, and all the previous pairs from the list either had a smaller search range, or were rejected because they required more steps.

As an example, the set $\mathcal{S}_{512}$ restricted to pairs using no more than 512 steps, and ordered according to Algorithm 8.5, is

$$\begin{aligned} \mathcal{S}_{512} = \{ & (1, 1), (1, 2), (3, 1), (2, 2), (5, 1), (3, 2), (1, 3), (5, 2), (6, 2), (7, 2), (2, 3), (9, 2), \\ & (10, 2), (11, 2), (3, 3), (14, 2), (15, 2), (4, 3), (18, 2), (19, 2), (5, 3), (23, 2), (1, 4), \\ & (7, 3), (8, 3), (9, 3), (10, 3), (11, 3), (2, 4), (13, 3), (14, 3), (15, 3), (16, 3), (17, 3), \\ & (3, 4), (20, 3), (21, 3), (22, 3), (23, 3), (4, 4), (26, 3), (27, 3), (28, 3), (29, 3), (5, 4) \}. \end{aligned}$$

The number of steps for each corresponding pair is

2, 4, 6, 8, 10, 12, 16, 20, 24, 28, 32, 36, 40, 44, 48, 56, 60, 64, 72, 76, 80, 92, 96, 112, 128, 144,
160, 176, 192, 208, 224, 240, 256, 272, 288, 320, 336, 352, 368, 384, 416, 432, 448, 464, 480,

and the search range, i.e. the value of the largest exponent generated by the giant steps, for each corresponding pair is

4, 24, 36, 96, 100, 216, 480, 600, 864, 1176, 1920, 1944, 2400, 2904, 4320, 4704, 5400, 7680,
7776, 8664, 12000, 12696, 20160, 23520, 30720, 38880, 48000, 58080, 80640, 81120, 94080,
108000, 122880, 138720, 181440, 192000, 211680, 232320, 253920, 322560, 324480, 349920,
376320, 403680, 504000.

Table 8.7 provides several values predicted by the theory in Section 4.2 in combination with the results of this section for integers of various sizes. In theory, SuperSPAR takes the number of baby steps and giant steps proportional to the largest prime, p_t , used in the exponent of the exponentiation stage, such that $p_t \leq N^{1/2r}$, p_t is as large as possible, and $r = \sqrt{\ln N / \ln \ln N}$. In the table, the total number of steps, $2m\phi(P_w)$, is the smallest value greater than or equal to $\lfloor N^{1/2r} \rfloor$. The table also shows the search range given by the specified multiple m and primorial P_w . Since we did not expect our implementation of SuperSPAR to be competitive for integers larger than 100 bits, we point out that for integers 112 bits in size, the theory predicts that the search stage should require around 9802 steps. For this reason, in Section 8.6, we restrict the search bound list so that each candidate takes no more than 16384 steps. We choose 16384 since it is larger than 9802 and provides some padding. Another item of interest is that even for integers of size 128 bits, the largest exponent generated during the search stage, as predicted by the theory, is 1173110400, which fits within 31 bits. This justifies our use of 32 bit values in the lookup table described in Section 8.2.

$\log_2 N$	$r = \sqrt{\ln N / \ln \ln N}$	$\lfloor N^{1/2r} \rfloor$	$2m\phi(P_w)$	$m^2\phi(P_w)P_w$	m	w
16	2.14693	13	16	480	1	3
32	2.67523	63	64	7680	4	3
48	3.08112	221	224	94080	14	3
64	3.42016	655	656	806880	41	3
80	3.71609	1738	1824	7277760	19	4
96	3.98139	4258	4320	40824000	45	4
112	4.22355	9802	10080	222264000	105	4
128	4.44745	21473	22080	1173110400	23	5

Table 8.7: The theoretically optimal number of group operations for each stage is given as $O(N^{1/2r})$. A search bound mP_w is selected so that the total number of steps, $2m\phi(P_w)$, is the smallest integer greater than or equal to the theoretically optimal value. The search range, $m^2\phi(P_w)P_w$, is given for the corresponding search bound.

This technique of generating candidate bounds for the search phase of SuperSPAR is used in the next section where we search for an exponent to use in the exponentiation stage and a corresponding bound for the search stage that perform well in practice for integers of various sizes.

8.6 Empirical Search for Exponent and Step Count

In order to factor integers of a fixed size, two parameters remain to be determined for our implementation of SuperSPAR. The first is the largest prime used in the exponent $E = \prod p_i^{e_i}$ for the exponentiation stage, and the second is a multiple of a primorial, mP_w , for use with the search stage. This section begins by displaying the average time to factor semiprime integers of a given size when the largest prime in the exponent for the exponentiation stage or the bound on the search stage is iterated. The purpose of the first part is not to find parameters that work well on average, but only to visualize the parameter search space. In Subsection 8.6.1, we propose using a 2-dimensional ternary-search in order to find values that perform well on average.

In selecting the exponent E for the exponentiation stage, our implementation uses the results of Chapter 7. Section 7.6 introduced a left-to-right best approximations technique

for generating 2,3 representations of an integer, and the results in Section 7.10 showed that this method performs best in practice when exponentiating an ideal class to the product of many prime numbers. This is the method used in our implementation of SuperSPAR. In practice, for a given exponent, the bound L was chosen to be $L = 2^k$ for some value k . The bound was repeatedly doubled and a left-to-right best approximation was computed for a given exponent until the cost of exponentiating an ideal class to that particular exponent did not change between iterations. For the exponents tested in this section, the bound was $L \leq 2048$. Representations for the exponents used here were computed beforehand and available in memory to the running application.

When selecting a multiple of a primorial for the search stage, candidates were selected from the list generated by Algorithm 8.5 with a bound of 16384 total steps. Since we did not expect our implementation of SuperSPAR to be competitive for integers larger than 100 bits, 16384 provides some padding over the theoretically optimal number of steps for integers this size, i.e. the search stage for 100 bit integers requires roughly 9802 steps (see Table 8.7).

To indicate the effect of iterating the largest prime in the exponent E and the search bounds $(m, w) \in S_{16384}$, five sets of 1000 random semiprime integers $N = p \cdot q$ were generated for p and q prime and half the size of N . The five sets consist of integers of 48 bits, 56 bits, 64 bits, 72 bits, and 80 bits. We then performed a 2-dimensional iteration of both the largest prime used in the exponent E for the exponentiation stage and the search bound $(m, w) \in S_{16384}$ over the theoretically optimal values. The results are visualized in Figures 8.4 through 8.8.

We hoped that as we iterated the largest prime in the exponent E and the search bounds $(m, w) \in S_{16384}$ that the average time to factor integers of a given size would indicate a local minimum that could be easily discovered. However, the purpose here is not to find values for the exponent E and search bound mP_w that perform well on average, but only to see the shape of the search space. We chose these sets of integer sizes as they are relatively

efficient to work with. In Subsection 8.6.1, we propose using a modified 2-dimensional ternary search in order to find parameters that work well on average. By applying a ternary search technique rather than the linear search used here, we are able to empirically test larger sets of semiprime integers in order to get a better representation of parameters that work well on average. We are also able to do this for every even integer size in the range 16 bits to 100 bits.

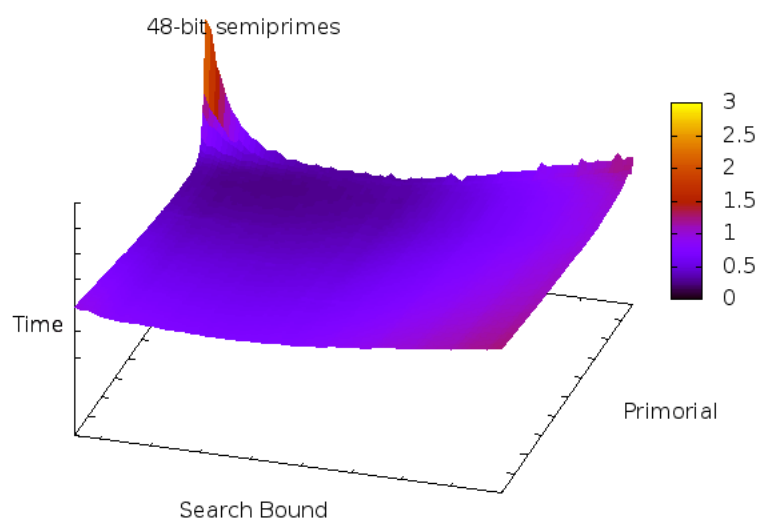


Figure 8.4: Average time to factor 48 bit integers.

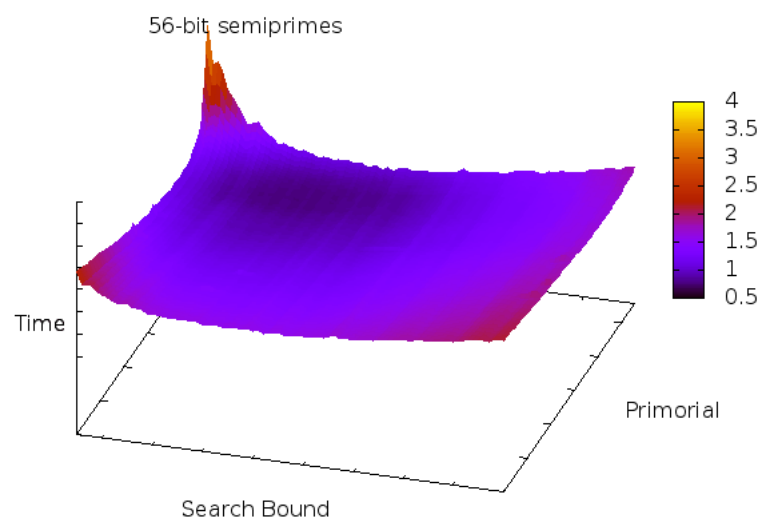


Figure 8.5: Average time to factor 56 bit integers.

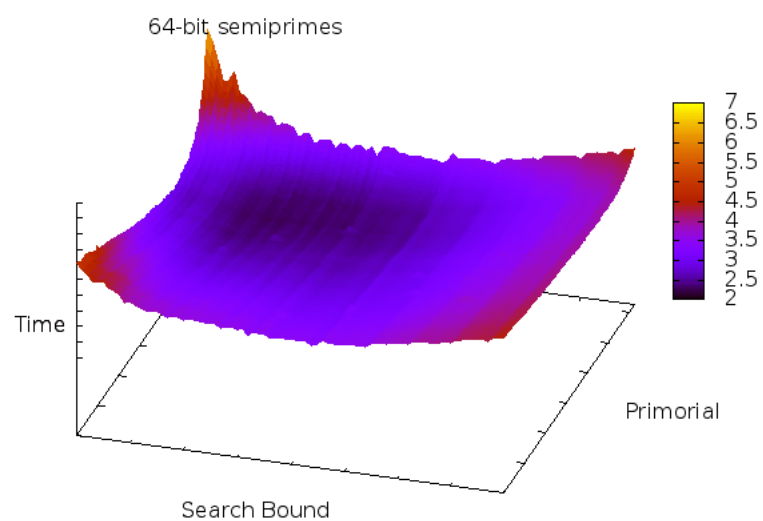


Figure 8.6: Average time to factor 64 bit integers.

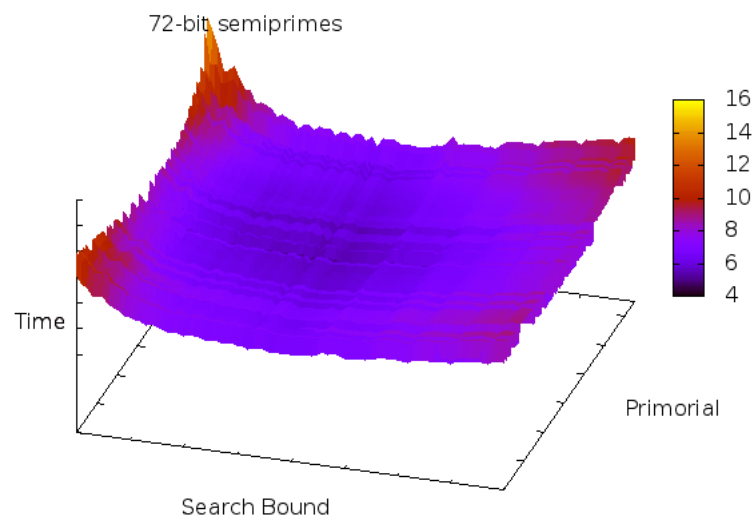


Figure 8.7: Average time to factor 72 bit integers.

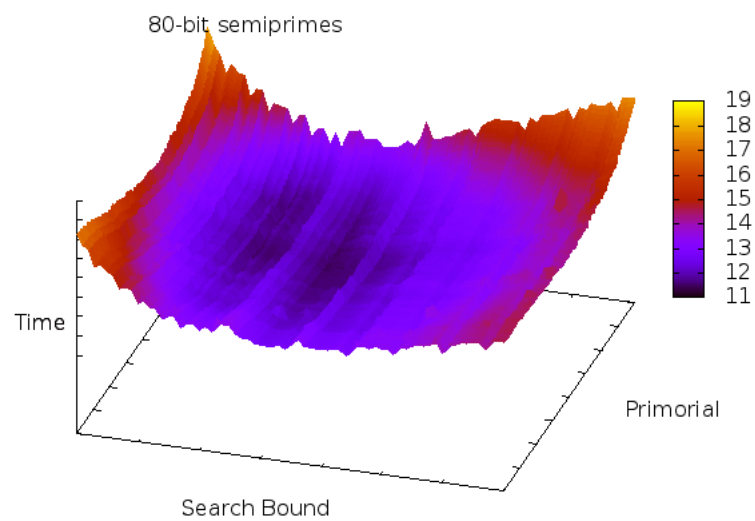


Figure 8.8: Average time to factor 80 bit integers.

The above results were generated on a 3Ghz Intel Xeon 5160 CPU with 16Gb of memory. The CPU has four cores, only one of which was used for all timing tests. For the three sets of 1000 semiprime integers, generating all the timing results took approximately one week. The timing data is noisy, which may be partly due to the inaccuracies inherent in timing very short operations. Even so, these figures indicate that as these two variables are iterated, the search space appears to have a minimum. In the next subsection, we propose a modified 2-dimensional ternary search in order to take advantage of the apparent minimum. Since a ternary search is more efficient than the linear search performed above, this allows us to use larger sets of semiprime integers so that the minimum discovered better represents parameters that work well on average. We are also able to run this method on sets of semiprime integers for every even bit size from 16 to 100 in substantially less time.

8.6.1 Two-Dimensional Ternary Search of a Noisy Surface

To find values for the exponentiation stage and search stage that work well in practice, one could iterate all suitable exponents E and multiples of primorials mP_w , but this would conceivably take a long time for even a small sample set of integers. The figures in the previous section suggest a more efficient approach, namely a 2-dimensional ternary search. Using a more efficient approach also allows us to use a larger sample of integers than would not be possible otherwise.

The idea is to perform a 2-dimensional search on the variables E and mP_w by nesting two 1-dimensional ternary searches, but switching to a linear search when the search range is smaller than a specified size. Switching to a linear search when the range is small helps to deal with the inherent noise of the timing data. The 1-dimensional search is given by Algorithm 8.6, which is a subroutine of the n -dimensional search given by Algorithm 8.7.

First, recall that the results of Subsection 8.3.1 fix the prime powers of the exponent E and so only the largest prime p_t is free. Let x (with subscripts) denote indices into a list of candidate exponents ordered by increasing largest prime p_t . Candidate search bounds

mP_w are chosen from the list of search bounds $\mathcal{S}_{16384}$ generated by Algorithm 8.5 such that no candidate uses more than 16384 steps. Let y (with subscripts) denote indices into the candidate list. Finally, let $\mathcal{T}(x, y)$ denote the average time to factor integers for the exponent E and search bound mP_w denoted by indices x and y .

The 1-dimensional search works as follows. Suppose the exponent E for the exponentiation stage is fixed and is indexed by x . Initially, let y_{lo} take the minimum search index and y_{hi} take the maximum search index. Then sample the time to factor integers at one third and two third the value between y_{lo} and y_{hi} . If the time associated with the one third mark is higher, the first third of the interval is thrown away and the algorithm recursively computes using the remaining two thirds. If the time associated with the two third mark is higher, the last third is thrown away and the algorithm recursively computes using the first two thirds. If we assume that by fixing the exponent E and iterating the search bound, that the timings will have a minimum, the third removed at each iteration should contain values generally larger than the minimum in the remaining two thirds. Since timings are noisy, when the interval $[y_{lo}, y_{hi}]$ is sufficiently small, the algorithm simply performs a linear search in order to find the $y \in [y_{lo}, y_{hi}]$ that gives the best time. This is not guaranteed to be the best time possible for the exponent E , however, this method works well in practice. We give pseudo-code in Algorithm 8.6.

Assuming that Algorithm 8.6 computes some search bound that is near minimal for a given exponent E for the exponentiation stage, we then nest applications of the ternary search in order to search for an exponent E , and corresponding bound mP_w that work well on average. In this case, we perform a ternary search on the index x and invoke Algorithm 8.6 to determine a near minimum time for the fixed exponent associated with the index x . Once the search range is too small, we then perform a linear search on the index x in the same manner. Pseudo-code for the 2-dimensional ternary search is given by Algorithm 8.7.

Table 8.8 shows the values used for the exponentiation and search stage in practice for

Algorithm 8.6 Determine some y such that $\mathcal{T}(x, y)$ is near minimal.

Input: A function $\mathcal{T}(x, y)$ corresponding to the average time to factor an integer for an exponent E associated with an index x and a search bound mP_w associated with an index y , a search range $[y_{\min}, y_{\max}]$, a bound k within which to linear search, and the index x of a fixed exponent E to be used by the exponentiation stage.

Output: $y \in [y_{\min}, y_{\max}]$ such that $\mathcal{T}(x, y)$ is near minimal for fixed E .

```

1: procedure SEARCH1( $\mathcal{T}, x, y_{\min}, y_{\max}, k$ )
   {– ternary search –}
2:    $(y_{\text{lo}}, y_{\text{hi}}) \leftarrow (y_{\min}, y_{\max})$ 
3:   while  $y_{\text{hi}} - y_{\text{lo}} > k$  do
4:      $y_{\ell} \leftarrow \lfloor (y_{\text{hi}} - y_{\text{lo}})/3 \rfloor + y_{\text{lo}}$ 
5:      $y_h \leftarrow \lfloor 2(y_{\text{hi}} - y_{\text{lo}})/3 \rfloor + y_{\text{lo}}$ 
6:     if  $\mathcal{T}(x, y_{\ell}) \leq \mathcal{T}(x, y_h)$  then
7:        $y_{\text{hi}} \leftarrow y_h$ 
8:     else
9:        $y_{\text{lo}} \leftarrow y_{\ell}$ 
   {– linear search –}
10:   $(y, t) \leftarrow (y_{\text{lo}}, \mathcal{T}(x, y_{\text{lo}}))$             $\{(y, t) \text{ represent smallest } \mathcal{T}(x, y) \text{ encountered}\}$ 
11:   $y_{\text{lo}} \leftarrow y_{\text{lo}} + 1$ 
12:  while  $y_{\text{lo}} \leq y_{\text{hi}}$  do
13:    if  $\mathcal{T}(x, y_{\text{lo}}) < t$  then  $(y, t) \leftarrow (y_{\text{lo}}, \mathcal{T}(x, y_{\text{lo}}))$ 
14:     $y_{\text{lo}} \leftarrow y_{\text{lo}} + 1$ 
15:  return  $(y, t)$ 

```

various sized integers. For each bit size, 10,000 random semiprime integers $N = p \cdot q$ were generated. We emphasize that this set is different from the set of semiprime integers used in Section 8.8 so as to avoid training SuperSPAR on the same set of integers used for comparison with other factoring algorithms. The range of exponents for the exponentiation stage was from a largest prime of 11 in the exponent to a largest prime of 2753. The range for the search bounds was given by the set $\mathcal{S}_{16384}$. Algorithm 8.7 was configured to switch to a linear search when the size of the range was ≤ 8 . Notice that the largest primorial used for search bounds is P_5 , which justifies our approach to bounding the values e_i in the exponent $E = \prod p_i^{e_i}$ for values of $i \leq 5$ in Section 8.3.

Algorithm 8.7 Determine an exponent E for the exponentiation stage and a search bound mP_w that work well on average.

Input: A function $\mathcal{T}(x, y)$ corresponding to the average time to factor an integer for an exponent E associated with an index x and a search bound mP_w associated with an index y , two search ranges $[x_{\min}, x_{\max}]$ and $[y_{\min}, y_{\max}]$, and a bound k within which to linear search.

Output: $x \in [x_{\min}, x_{\max}]$ and $y \in [y_{\min}, y_{\max}]$ such that $\mathcal{T}(x, y)$ is near minimal for exponent E and search bound mP_w corresponding to indices x and y .

```

1: procedure SEARCH2( $\mathcal{T}, x_{\min}, x_{\max}, y_{\min}, y_{\max}, k$ )
   {– ternary search –}
2:    $(x_{\text{lo}}, x_{\text{hi}}) \leftarrow (x_{\min}, x_{\max})$ 
3:   while  $x_{\text{hi}} - x_{\text{lo}} > k$  do
4:      $x_{\ell} \leftarrow \lfloor (x_{\text{hi}} - x_{\text{lo}})/3 \rfloor + x_{\text{lo}}$ 
5:      $x_h \leftarrow \lfloor 2(x_{\text{hi}} - x_{\text{lo}})/3 \rfloor + x_{\text{lo}}$ 
6:      $(-, t_{\ell}) \leftarrow \text{SEARCH1}(\mathcal{T}, x_{\ell}, y_{\min}, y_{\max}, k)$ 
7:      $(-, t_h) \leftarrow \text{SEARCH1}(\mathcal{T}, x_h, y_{\min}, y_{\max}, k)$ 
8:     if  $t_{\ell} \leq t_h$  then
9:        $x_{\text{hi}} \leftarrow x_h$ 
10:    else
11:       $x_{\text{lo}} \leftarrow x_{\ell}$ 
   {– linear search –}
12:    $x \leftarrow x_{\text{lo}}$  { $(x, y, t)$  represent smallest  $\mathcal{T}(x, y)$  encountered}
13:    $(y, t) \leftarrow \text{SEARCH1}(\mathcal{T}, x, y_{\min}, y_{\max}, k)$ 
14:    $x_{\text{lo}} \leftarrow x_{\text{lo}} + 1$ 
15:   while  $x_{\text{lo}} \leq x_{\text{hi}}$  do
16:      $(y', t') \leftarrow \text{SEARCH1}(\mathcal{T}, x_{\text{lo}}, y_{\min}, y_{\max}, k)$ 
17:     if  $t' < t$  then  $(x, y, t) \leftarrow (x_{\text{lo}}, y', t')$ 
18:      $x_{\text{lo}} \leftarrow x_{\text{lo}} + 1$ 
19:   return  $(x, y, t)$ 

```

Bit Size	Largest Prime in $E = \prod p_i^{e_i}$	Search Bound $s = mP_w$	Total Steps	Average Time (milliseconds)
16	11	$120 = 4P_3$	62	0.17992
20	11	$36 = 6P_2$	22	0.21529
24	11	$36 = 6P_2$	22	0.01728
28	11	$90 = 3P_3$	46	0.02461
32	11	$150 = 5P_3$	78	0.03475
36	17	$240 = 8P_3$	126	0.05341
40	17	$420 = 2P_4$	190	0.07951
44	23	$630 = 3P_4$	286	0.12786
48	47	$630 = 3P_4$	286	0.19711
52	53	$1050 = 5P_4$	478	0.32283
56	89	$840 = 4P_4$	382	0.55058
60	149	$1260 = 6P_4$	574	0.93604
64	173	$1260 = 6P_4$	574	1.51579
68	263	$2310 = 1P_5$	958	2.36646
72	347	$1680 = 8P_4$	766	3.52871
76	467	$2310 = 1P_5$	958	5.42861
80	727	$3360 = 16P_4$	1534	8.53706
84	859	$4620 = 2P_5$	1918	11.97196
88	1033	$4620 = 2P_5$	1918	18.00593
92	1597	$6090 = 29P_4$	2782	29.14281
96	1861	$9240 = 4P_5$	3838	40.02095
100	2753	$7140 = 34P_4$	3262	57.97719

Table 8.8: Values for the largest prime used in the exponent $E = \prod p_i^{e_i}$ for the exponentiation phase, and the baby step bound mP_w used for the search stage.

8.7 Reference Implementation of SPAR

This section compares each of the practical performance improvements from the previous sections to our reference implementation of SPAR [12]. Our reference implementation uses all of the improvements to arithmetic in the ideal class group from Chapters 5 and 6, as well as the improvements to exponentiation by power primorials from Chapter 7. We use the theoretically optimal parameters as described by Schnorr and Lenstra in [53]. The search stage uses a Pollard-Brent recursion [17] as described in [53, p.296]. We briefly describe the search stage using this algorithm here. The exponentiation stage computes an ideal class $[\mathbf{b}] = [\mathbf{a}]^{2^\ell E}$ with $\ell = \left\lfloor \log_2 \sqrt{|\Delta|} \right\rfloor$. The search stage in SPAR first computes $[\mathbf{q}_i] = [\mathbf{b}]^{r_i}$ for random $r_i \in [p_t, p_t^2]$ and $0 \leq i \leq 15$. Let $[\mathbf{c}_0] = [\mathbf{b}]$ and $h_0 = 0$. We then iterate computing

$$[\mathbf{c}_{i+1}] = [\mathbf{c}_i] \cdot [\mathbf{q}_{(i \bmod 16)}]$$

$$h_{i+1} = h_i + r_{(i \bmod 16)}$$

until we find some $[\mathbf{c}_i] = [\mathbf{c}_j]$ with $j < i$. Then $h' = h_i - h_j$ is a multiple of the order of the ideal class $[\mathbf{b}]$. Following the recommendation of Schnorr and Lenstra [53, p.296], we do not store every $[\mathbf{c}_i]$ computed, but instead maintain a circular queue of at most 7 of these ideal classes. If n items have been previously added to the queue, the next item is added after $[\mathbf{c}_i]$ is computed for $i = 1.1^n$.

The experiments in this section were performed on a 2.7GHz Intel Core i7-2620M CPU with 8Gb of RAM and four cores. Only one of the cores was used during performance testing. All source code was written for the GNU C compiler version 4.7.2 with x64 assembly language used for 128 bit arithmetic. The times given are the average time to factor semiprime integers $N = p \cdot q$, with N of the specified size and p and q roughly half the size. For each bit range, we factor a set of 10,000 integers. This is the same set of integers as in Section 8.8.

Figure 8.9 shows the average time to factor integers using our reference implementation of SPAR. The first practical performance improvement is shown in Figure 8.10. In this fig-

ure, we limit the number of ideal classes tried before switching class groups as determined by Subsection 8.3.4. Figure 8.11 shows the practical performance improvements when we switch from the Pollar-Brent recursion described above to a bounded primorial steps algorithm as described in Subsection 4.2.1. Since this line is quite flat relative to our reference implementation of SPAR, Figure 8.12 zooms in on this line. By bounding the exponents of the prime powers e_i in the exponent $E = \prod p_i^{e_i}$ in the exponentiation stage, we obtain a further practical improvement. Figure 8.13 shows the results when the bounds are selected according to Subsection 8.3.1. Figure 8.14 shows the improvement when the largest prime p_t in the exponent $E = \prod p_i^{e_i}$ and the search bound mP_w are chosen according to the approach outlined in Section 8.6. Finally, Figure 8.15 shows the benefit of only performing the search stage once per ideal class group. When we take all these improvements together, the practical improvement is quite substantial and is depicted by Figure 8.16.

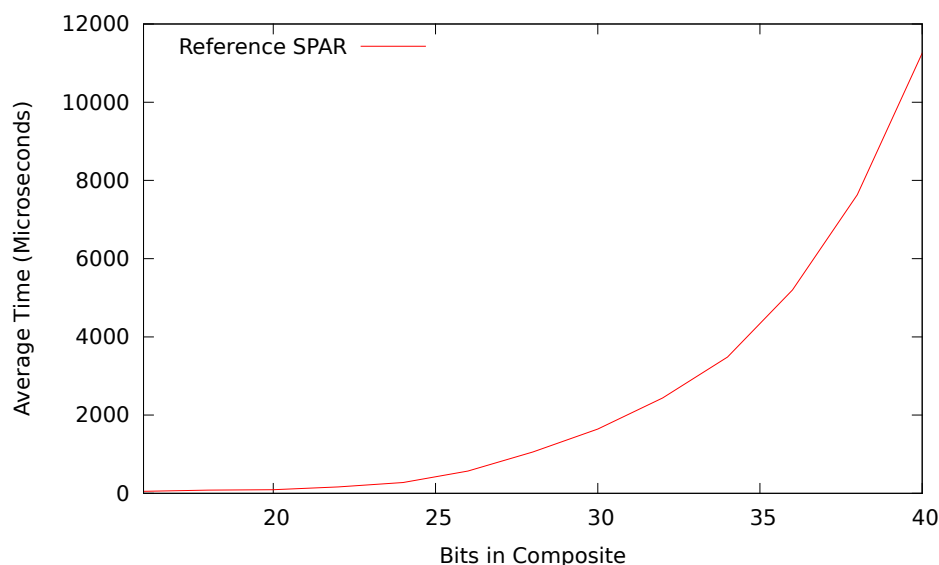


Figure 8.9: Reference Implementation of SPAR. This uses improvements to arithmetic in the ideal class group as well as improvements to power primorial exponentiation.

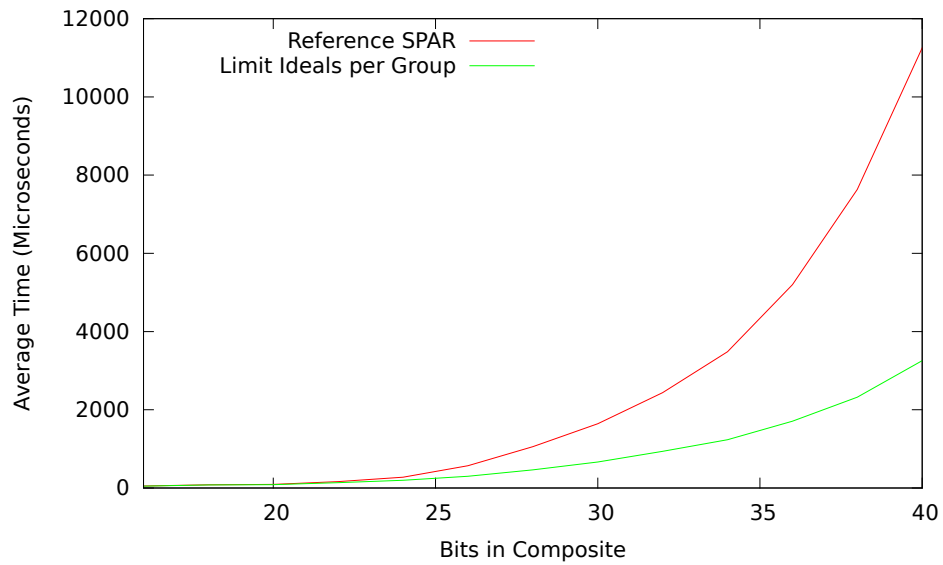


Figure 8.10: Improvement in the average time by limiting the number of ideals tried per class group as determined by Subsection 8.3.4.

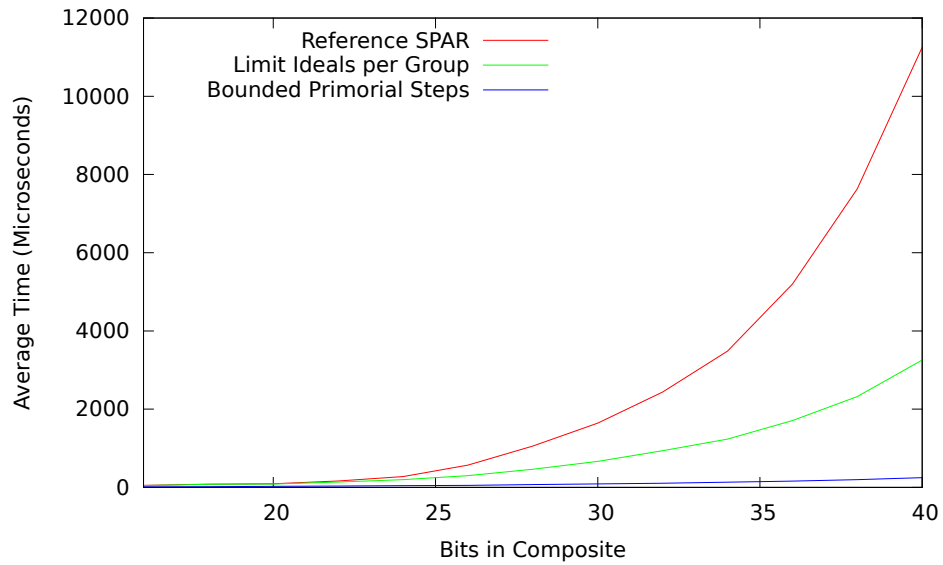


Figure 8.11: Simply using a bounded primorial steps search instead of a Pollard-Brent recursion.

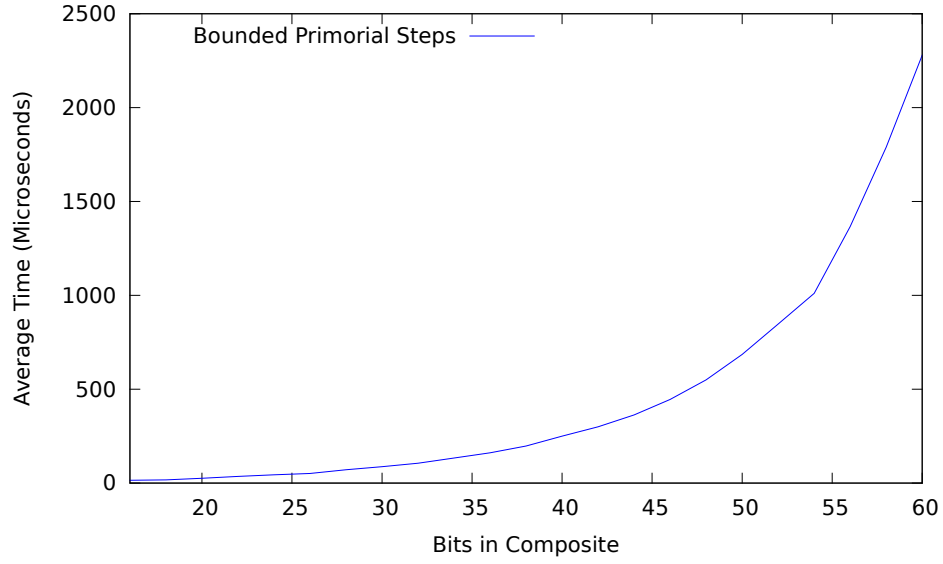


Figure 8.12: Same as Figure 8.11 only zoomed in on “Bounded Primorial Steps”.

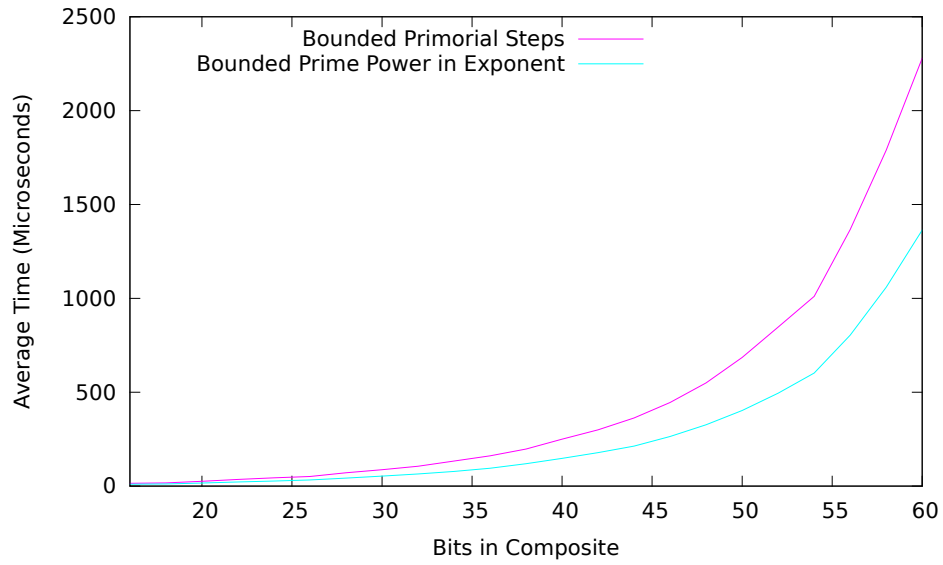


Figure 8.13: Bounding the exponent of the prime powers e_i in the exponent $E = \prod p_i^{e_i}$ as determined by Subsection 8.3.1.

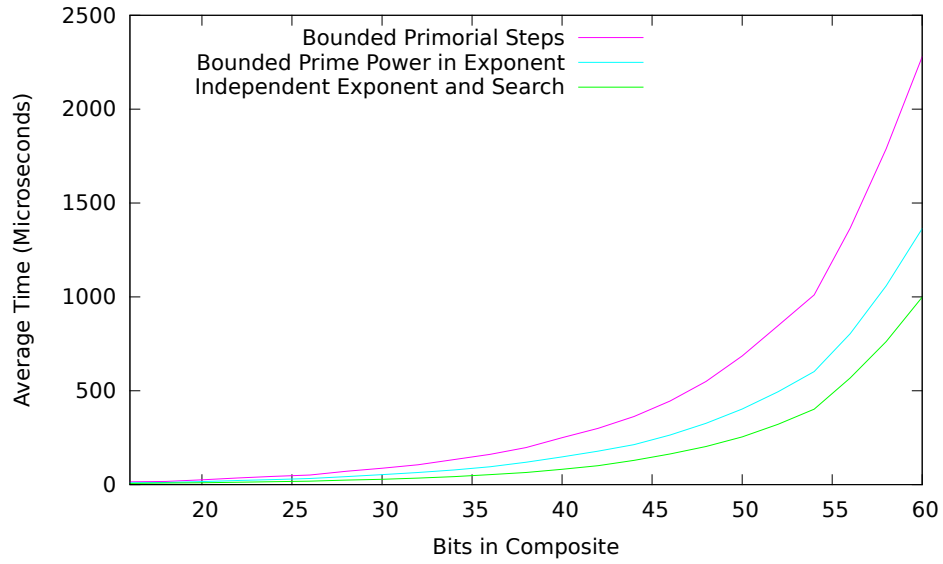


Figure 8.14: Using independently chosen exponent E and search bound mP_w as determined in Section 8.6.

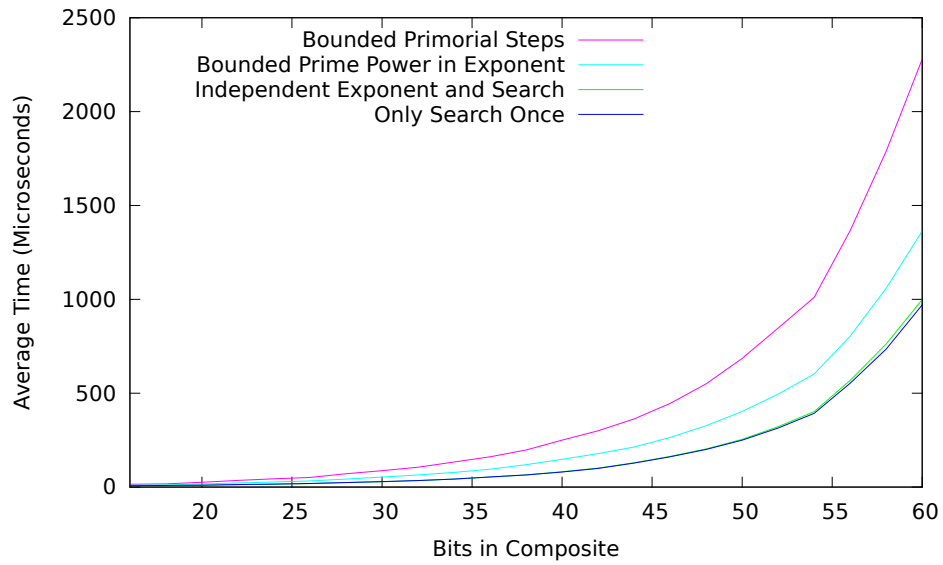


Figure 8.15: Only perform the search stage once per class group.

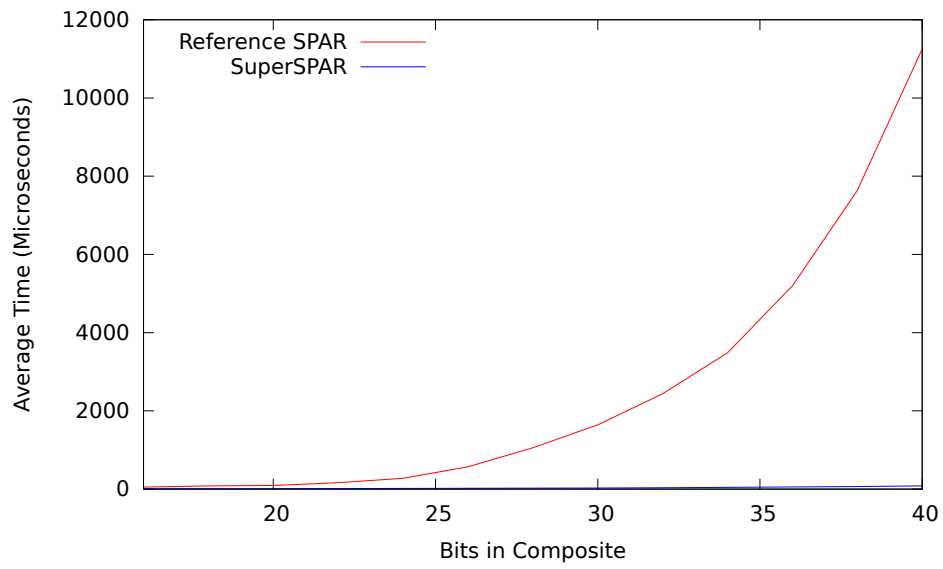


Figure 8.16: Comparison of our reference implementation of SPAR to our final implementation of SuperSPAR.

8.8 Comparison to Other Factoring Implementations

To demonstrate the performance of our implementation of SuperSPAR, several factoring implementations were compared, namely Pari/GP [11], Msieve [10], GMP-ECM [2], YAFU [14], Flint [1], Maple [8], and a custom implementation of SQUFOF [32] adapted from the source code of Pari/GP.

For each bit size 16, 18, ..., 100, sets of 10,000 random semiprime integers $N = p \cdot q$ for p and q prime and half the size of N were generated and written to disk. The file format was ASCII with one integer per line. In the case of Pari/GP and Flint, C programming libraries were provided, so we wrote a program to load the set of integers and use the library directly. For our implementations of SuperSPAR and SQUFOF, we were also able to load the set of integers and invoke the factoring function directly. Unfortunately, we were not able to directly interface with the factoring functions of Msieve, GMP-ECM, YAFU, or Maple. Our solution to time these implementations was to convert the text file to an implementation specific script and then invoke the application to factor the set.

The hardware platform used for timing is a 2.7GHz Intel Core i7-2620M CPU with 8Gb of memory. The CPU has four cores, only one of which was used during timing experiments. We observed the CPU load during experimentation to verify that at most a single CPU core was being used by the other factoring implementations. The operating system was 64 bit Ubuntu 12.10. When possible, the most recent version of each factoring implementation was used and built from source using the GNU C compiler version 4.7.2. Timings include forking the process, reading and parsing the test set, and factoring all the integers in the set. The timings do not include the generation of the semiprime integers or the conversion to the implementation specific batch file.

Figures 8.17 through 8.21 were chosen to emphasize different features of the performance of SuperSPAR. Figure 8.17 shows the performance of each factoring implementation for the complete range of integer sizes timed – visually, SuperSPAR is not particularly competitive

for integers much larger than 90 bits. Figure 8.18 zooms in on the range of integers 44 bits to 72 bits. This shows that SQUFOF quickly becomes impractical for integers larger than 70 bits, which is what we would expect from its $O(N^{1/4})$ runtime complexity (see [32, Theorem 4.22]). Maple, Msieve, and GMP-ECM do not perform as efficiently as the other implementations for integers in the range 44 bits to 72 bits. This also shows that SuperSPAR is highly competitive below 64 bits. Figure 8.19 shows the 5 best performing implementations for integers in the range of 44 bits to 68 bits. Figure 8.20 zooms in on the left of this image for integers in the range 44 bits to 58 bits. This shows the same 5 implementations. For integers between 44 bits and around 49 bits, the custom implementation of SQUFOF is the fastest performing, but as the integer size grows, SQUFOF takes longer more quickly. For integers around 49 bits in size, SuperSPAR is typically the fastest performing implementation of the implementations tested. Finally, Figure 8.21 zooms in on the right part of Figure 8.19. This shows the 3 best performing implementations for this range, and the point at which the performance of SuperSPAR decreases and YAFU is better performing. This occurs for integers around 62 bits in size.

Table 8.9 shows the average time in milliseconds to factor integers for each bit size given a particular implementation. The best performing implementation for integers of a given size is underlined. Timings were only recorded for implementations and integer sizes that took less than 100 milliseconds on average. In the case of SQUFOF, this was for integers 74 bits in size and less. Inexplicably, for integers of size 24 bits, 28 bits, and 42 bits, Flint did not complete the test set before crashing. Similarly, Msieve crashed for all sets of integers larger than 84 bits. This table shows that for integers in the range of 49 bits to 62 bits, SuperSPAR performs the fastest on average.

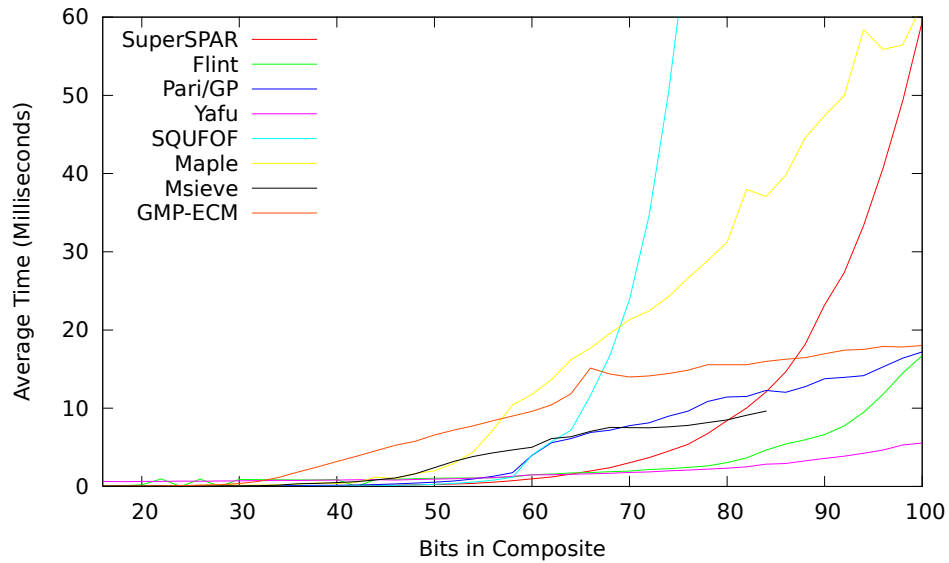


Figure 8.17: The performance of each factoring implementation for integers 16 bits to 100 bits. SuperSPAR appears to be competitive at less than 70 bits.

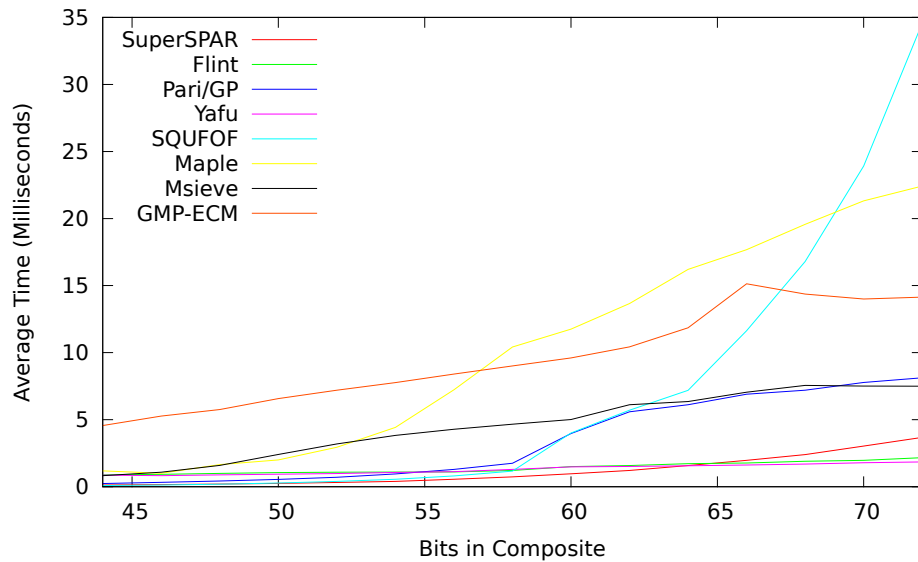


Figure 8.18: The performance of each factoring implementation for integers 44 bits to 72 bits. In this range, Maple, Msieve, and GMP-ECM do not appear to be competitive.

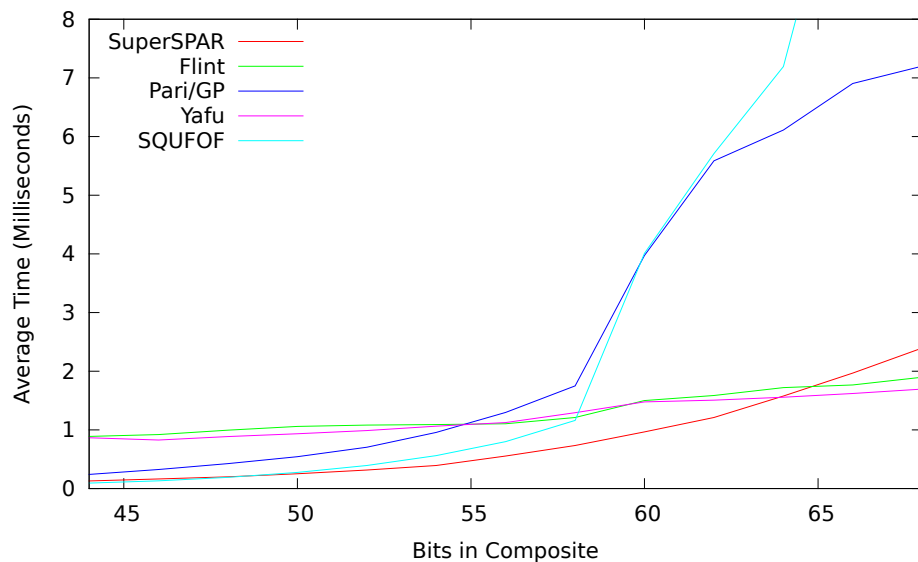


Figure 8.19: The 5 best performing factoring implementations for integers in the range 44 bits to 68 bits. This range shows that SuperSPAR is the fastest performing implementation for some integer sizes.

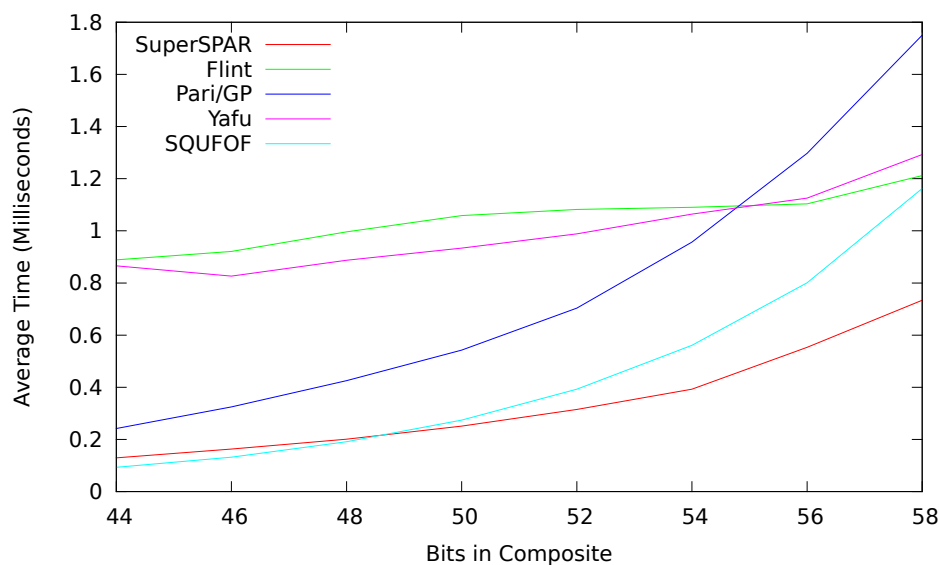


Figure 8.20: The 5 best performing factoring implementations for integers in the range 44 bits to 58 bits. This focuses the range on the lower half of Figure 8.19. Here SQUFOF grows faster than SuperSPAR and at integers around 49 bits in size, SuperSPAR is the best performing implementation.

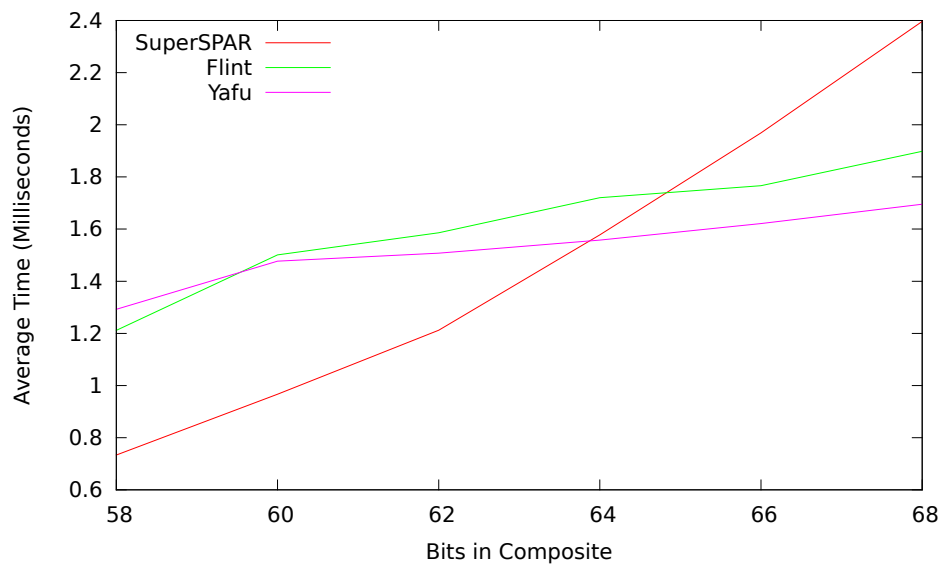


Figure 8.21: The 3 best performing factoring implementations for integers in the range 58 bits to 68 bits. This focuses the range on the upper half of Figure 8.19 and demonstrates that around integers 64 bits in size, YAFU performs better than SuperSPAR.

Bits	ECM	Flint	Maple	Msieve	Pari	SQUFOF	SSPAR	Yafu
16	0.0270	0.0378	0.0505	0.0799	0.0032	<u>0.0008</u>	0.0073	0.6524
18	0.0337	0.0774	0.0920	0.0815	0.0048	<u>0.0010</u>	0.0085	0.6082
20	0.0473	0.1994	0.0637	0.0812	0.0055	<u>0.0018</u>	0.0103	0.6492
22	0.0660	0.9614	0.0795	0.0820	0.0064	<u>0.0021</u>	0.0134	0.6469
24	0.0946	failed	0.1205	0.0829	0.0090	<u>0.0028</u>	0.0159	0.6779
26	0.1740	0.9406	0.1340	0.0832	0.0145	<u>0.0040</u>	0.0190	0.6569
28	0.2235	failed	0.2380	0.0883	0.0235	<u>0.0056</u>	0.0241	0.6975
30	0.3783	0.8982	0.1544	0.0914	0.0677	<u>0.0077</u>	0.0288	0.7193
32	0.6537	0.8725	0.1641	0.1003	0.0782	<u>0.0110</u>	0.0350	0.7511
34	1.1481	0.8591	0.2044	0.1642	0.0981	<u>0.0158</u>	0.0429	0.7827
36	1.8327	0.8310	0.2822	0.3351	0.1128	<u>0.0222</u>	0.0537	0.7661
38	2.4831	0.8327	0.4612	0.4004	0.1371	<u>0.0313</u>	0.0646	0.8022
40	3.1906	0.8796	0.3906	0.4883	0.1607	<u>0.0461</u>	0.0805	0.8167
42	3.8457	failed	0.5226	0.6174	0.1942	<u>0.0648</u>	0.0995	0.8439
44	4.5650	0.8890	1.1850	0.8271	0.2419	<u>0.0928</u>	0.1278	0.8655
46	5.2680	0.9210	0.9994	1.0838	0.3247	<u>0.1320</u>	0.1609	0.8263
48	5.7561	0.9963	1.6464	1.5946	0.4254	<u>0.1911</u>	0.1999	0.8873
50	6.5731	1.0584	2.0015	2.4111	0.5424	0.2741	<u>0.2497</u>	0.9338
52	7.2033	1.0819	2.9428	3.1965	0.7036	0.3929	<u>0.3145</u>	0.9886
54	7.7635	1.0901	4.4271	3.8279	0.9566	0.5608	<u>0.3924</u>	1.0640
56	8.4054	1.1036	7.2331	4.2903	1.2972	0.8010	<u>0.5532</u>	1.1253
58	9.0041	1.2117	10.4179	4.6656	1.7506	1.1622	<u>0.7347</u>	1.2925
60	9.6012	1.5008	11.7576	5.0153	3.9717	4.0100	<u>0.9703</u>	1.4769
62	10.4339	1.5859	13.6654	6.1142	5.5854	5.7092	<u>1.2190</u>	1.5075
64	11.8548	1.7206	16.2116	6.3498	6.1125	7.1933	1.5839	<u>1.5577</u>
66	15.1356	1.7662	17.6785	7.0451	6.9054	11.6313	1.9822	<u>1.6214</u>
68	14.3670	1.8983	19.5723	7.5541	7.1997	16.7912	2.4150	<u>1.6952</u>
70	13.9973	1.9687	21.3196	7.5087	7.7758	23.9112	3.0654	<u>1.7895</u>
72	14.1343	2.1680	22.4251	7.4943	8.1268	34.5667	3.7090	<u>1.8536</u>
74	14.4356	2.2741	24.2755	7.6182	8.9852	50.3791	4.5423	<u>2.0003</u>
76	14.8602	2.4442	26.6470	7.8022	9.6272	70.0198	5.4594	<u>2.0998</u>
78	15.5668	2.6173	28.8623	8.1577	10.8526	–	6.7997	<u>2.2202</u>
80	15.5548	3.0537	31.2528	8.5141	11.4426	–	8.5390	<u>2.3407</u>
82	15.5577	3.6557	37.9693	9.0961	11.5011	–	10.0478	<u>2.5122</u>
84	15.9824	4.6584	37.0573	9.6379	12.2647	–	12.1013	<u>2.8428</u>
86	16.2545	5.4168	39.7901	failed	12.0382	–	14.6628	<u>2.9365</u>
88	16.4600	5.9574	44.5510	failed	12.7549	–	18.1816	<u>3.2695</u>
90	16.9574	6.6372	47.3801	failed	13.7731	–	23.4825	<u>3.5713</u>
92	17.4196	7.7319	49.9509	failed	13.9487	–	27.8960	<u>3.8687</u>
94	17.5219	9.4851	58.3808	failed	14.1614	–	34.0354	<u>4.2393</u>
96	17.9056	11.7830	55.8692	failed	15.2958	–	41.4511	<u>4.6538</u>
98	17.8311	14.4835	56.4315	failed	16.3924	–	50.2681	<u>5.3058</u>
100	18.0168	16.7292	61.6214	failed	17.2042	–	59.9777	<u>5.5532</u>

Table 8.9: The average time (in milliseconds) to factor integers of a given size for a particular implementation.

Chapter 9

Conclusions and Future Work

The work of this thesis is to improve arithmetic and exponentiation by power primorials in the ideal class group of imaginary quadratic number fields, with an application to integer factoring. To this end, the results of Chapter 5 show an improvement to the average performance for computing the extended GCD for inputs smaller than 128 bits over the implementations provided by Pari 2.5.3 [11] and GMP 5.1.1[3], which we used as reference implementations. Chapter 6 demonstrates results that our implementations of 64 bit and 128 bit ideal class arithmetic are significantly faster than that provided by Pari, which we again used as a reference implementation. Furthermore, our reference implementation of ideal class arithmetic using GMP for multiple precision arithmetic is often faster on average than the implementation provided by Pari, and especially when the size of the discriminant is too large to work with either our 64-bit implementation or our 128-bit implementation, i.e. when the discriminant is larger than 118 bits. In Chapter 7, we discussed several algorithms for computing 2,3 representations of exponents, some of which optimize representations based on the average cost to multiply, square, and cube in a group. For large exponents that are the product of several primes, we demonstrated that a left-to-right best approximations technique generates representations that lead to the fastest on average exponentiations of elements within the ideal class group for our implementations of ideal class arithmetic. Finally, Chapter 8 shows that when all of these improvements are applied to the SuperSPAR factoring algorithm, the result is an implementation of an integer factoring algorithm that is the fastest on average of all implementations tested for semiprime integers no smaller than 49 bits and no larger than 62 bits.

To enable collaboration with others, the source code for the libraries used throughout

this thesis is available online. An optimized implementation of 128 bit arithmetic and each of the implementations of the extended GCD are available as `liboptarith` [4]. Our implementations of ideal class arithmetic is available as `libqform` [5]. This library exposes several programming interfaces. One can specifically target the 64 bit, 128 bit, or GMP implementation of ideal class arithmetic, or we also expose a generic interface that will automatically select the implementation best suited for a given discriminant size. Finally, our implementation of the SuperSPAR factoring library is available as `libsspar` [6]. We point out that this library is currently configured to give the best average time performance. A function is provided where one gives the integer to be factored and the library automatically selects the best parameters for the average case. An additional function is exposed where one can manually provide the parameters as well. Future work will likely include an entry point optimized for the median case. By making each of these projects publicly available, the hope is for eventual inclusion into common mathematical packages such as Pari/GP [11] and Sage [13].

The work presented in this thesis is by no means exhaustive. Sections 9.1, 9.2, and 9.3 contain suggestions for future work in ideal class arithmetic, exponentiation, and integer factoring respectively.

9.1 Ideal Arithmetic

The practical improvements to arithmetic in the ideal class group within this thesis are based on the algorithms for computing reduced (or almost reduced) representatives for multiplication (Subsection 2.3.3), squaring (Subsection 2.3.4), and cubing (Subsection 2.3.5). While these algorithms are certainly faster as the size of the discriminant grows, future work should include a comparison of the implementation in this thesis with an implementation of the basic ideal class multiplication presented in Subsection 2.3.2. Since this algorithm is simpler, it might be faster when operands are small. The idea is to optimize an implementation of

the straightforward ideal multiplication algorithm, as well as possible improvements to the ideal reduction algorithm from Subsection 2.3.1. Since ideal reduction is effectively the same as computing the extended GCD, future work could provide optimized implementations of ideal reduction for small discriminants by adapting several of the approaches of Chapter 5 for computing the extended GCD.

Furthermore, additional techniques for computing the extended GCD should be explored. One possible idea is to apply windowing to the left-to-right binary GCD algorithms of Section 5.3. Also useful is a more thorough study of each of the extended GCD techniques discussed in Chapter 5 in the context of ideal class arithmetic. What is proposed here is to time the performance of ideal class arithmetic for each of the different techniques mentioned, this includes each technique applied to a full, left-only, and partial extended GCD in the context of ideal class arithmetic.

This thesis compares our implementations of ideal class arithmetic to that of Pari. Future improvements could include comparisons with additional implementations as well, such as with Magma [7]. Improving the performance of ideal class arithmetic for imaginary quadratic number fields is useful in extending the class group tabulation of Ramachandran [50]. Furthermore, one could study practical improvements to other types of ideal arithmetic, such as the class group of real quadratic number fields and hyperelliptic curves. In the case of real quadratic number fields with positive discriminants of size smaller than 128 bits, the majority of our implementation could be reused. An optimized implementation of this would be useful for tabulating the class number and class group structure of such fields.

9.2 Exponentiation

The ideal class exponentiation experiments of Chapter 7 assume that the exponents are power primorials and that representations are computed beforehand and then easily retrieved for use. Future experiments could also study the behaviour of exponentiating using the

techniques for computing 2,3 representations on different types of exponents, such as random integers or large primes.

Other possibilities are to study the time to generate 2,3 representations against the time to exponentiate using such representations. Our expectation is that representations that lead to faster exponentiations also take longer to generate, and that the time to generate better representations grows faster than the time saved by the better representations. Along this line, several of the techniques discussed in Chapter 7 remain to be rigorously analysed, namely several of the pruned trees from Section 7.5, the left-to-right best approximations technique proposed in Section 7.6, the strictly chained partition techniques proposed in Section 7.7, and finally, the method of incrementally searching for unchained 2,3 representations in Section 7.8.

The left-to-right best approximations technique of Section 7.6 generates 2,3 representations for large primorials that work well in practice. This algorithm iterates on the *L smallest* partial representations. Future work could consider other heuristics on which to prune candidates, such as approximating the cost of the complete chain based on the partial chain and retaining the *L* best approximate costs.

The results of Chapter 7 show general trends as the size of the primorial used for the exponent grows. Let $\mathcal{T}(N)$ be the cost to exponentiate a random element to the exponent N given a 2,3 representation of N generated by a particular technique. For many of the techniques used to generate 2,3 representations, when $\mathcal{T}(N)$ is compared with $\mathcal{T}(N + 1)$, the cost can vary considerably. Interesting and useful work is to provide upper bounds (as a function of N) on how much the cost can vary as the exponent increases by a fixed amount for any of the techniques described in this thesis for generating 2,3 representations.

Finally, in this thesis, we only consider double base number systems using bases 2 and 3. Future work should consider other bases. This would involve additional research with ideal arithmetic to determine the benefit of direct methods for computing $[\mathbf{a}]^5$, $[\mathbf{a}]^7$, and so on, for

an ideal class $[\mathfrak{a}]$. Other open areas are multiple base number systems, i.e. number systems where an integer is expressed as the sum and difference of powers of several pairwise coprime bases $N = \sum_i \left(s_i \prod_j p_j^{e_{i,j}} \right)$.

9.3 Super²SPAR

Practical improvements in this thesis to ideal class arithmetic and to exponentiation also lead to improvements in the SuperSPAR integer factoring algorithm. Future work in either area will naturally contribute to future improvements to SuperSPAR. There are, however, many open research areas, some of which lead to direct practical improvements for SuperSPAR.

Each of the parameters of our implementation were chosen to work well in practice. Often many class groups are tried before a factor of the input integer N is obtained. Future work should include a study of the number of multipliers of N used, perhaps rigorously bounding the number of multipliers needed in expectation. Subsection 8.3.3 showed that the expected number of ideal classes to try before factoring N differed for each multiplier used. Studying the behaviour of multipliers might be useful in the parallelization of SuperSPAR – simply put, the algorithm could be implemented to run on several multipliers in parallel. One possible highly parallel platform for consideration would be an implementation of SuperSPAR for GPUs.

There are many open questions surrounding the running time of SuperSPAR. Brief investigations show that the majority of integers only require a single multiplier before a successful factorization of the integer is produced, however, a small number of integers require a very large number of multipliers. These few *hard* numbers tend to skew the average running time. Future work should include quartile timings and 5-number summaries of samples of integers to be factored. Although the results of Lenstra and Pomerance [45, §11] preclude a sub-exponential worst case running time for SuperSPAR, additional work should include a rigorous analysis of the *median* asymptotic complexity. This analysis would likely follow

Sutherland’s discussion of the median complexity of his order finding algorithm (see [58, §5.4]). The hope is that an exponential running time is limited to a few hard numbers and that a median case analysis might give a sub-exponential result. Not necessarily related to SuperSPAR would be a theoretical analysis of the factorization of the order of ideal classes, such as the probabilities of certain divisors of the order. This would be a theoretical extension of the work in Section 8.3.

Naturally, future improvements to SuperSPAR should involve comparisons with the factoring implementations used in this thesis and others that are available. Our recommendation for future comparisons is to interface directly with the factoring implementation when possible so that timing operations are limited to only the integer factoring operation. Several of the factoring implementations compared in this thesis, namely GMP-ECM [2], MSieve [10], YAFU [14], are open source, and future work should modify these sources so that the integer factoring routine can be better timed in isolation. Lastly, future work would inspect these sources in order to determine the factoring algorithm used for integers of the sizes studied in this thesis.

Bibliography

- [1] Flint 2.3. <http://www.flintlib.org/>. Accessed: 2013-04-10.
- [2] GMP-ECM 6.4.4. <http://ecm.gforge.inria.fr/>. Accessed: 2013-04-10.
- [3] GNU Multiple Precision (GMP) Library 5.1.1. <http://gmplib.org/>. Accessed: 2013-04-10.
- [4] liboptarith. <https://github.com/maxwellsayles/liboptarith>. Accessed: 2013-04-26.
- [5] libqform. <https://github.com/maxwellsayles/libqform>. Accessed: 2013-04-26.
- [6] libsspar. <https://github.com/maxwellsayles/libsspar>. Accessed: 2013-04-26.
- [7] Magma computational algebra system. <http://magma.maths.usyd.edu.au/magma/>. Accessed: 2013-04-27.
- [8] Maple 13. <http://www.maplesoft.com/>. Accessed: 2013-04-10.
- [9] MPIR 2.6.0. <http://www.mpir.org/>. Accessed: 2013-16-10.
- [10] Msieve 1.5.1. <http://msieve.sourceforge.net/>. Accessed: 2013-04-10.
- [11] Pari/GP 2.5.3. <http://pari.math.u-bordeaux.fr/>. Accessed: 2013-04-10.
- [12] Reference implementation of SPAR. <https://github.com/maxwellsayles/spar>. Accessed: 2013-05-28.
- [13] Sage. <http://sagemath.org/>. Accessed: 2013-04-26.
- [14] YAFU (Yet Another Factoring Utility) 1.33. <http://yafu.sourceforge.net/>. Accessed: 2013-04-10.

- [15] E. Bach and J.O. Shallit. *Algorithmic Number Theory, Volume I: Efficient Algorithms*. Mit Press, 1996.
- [16] V. Berthé and L. Imbert. Diophantine approximation, Ostrowski numeration and the double-base number system. *Discrete Mathematics and Theoretical Computer Science*, 11(1):153–172, 2009.
- [17] R.P. Brent. An improved Monte Carlo factorization algorithm. *BIT*, 20(2):176–184, 1980.
- [18] M. Ciet, M. Joye, K. Lauter, and P.L. Montgomery. Trading inversions for multiplications in elliptic curve cryptography. *Designs, Codes and Cryptography*, 39(2):189–206, May 2006.
- [19] M. Ciet and F. Sica. An analysis of double base number systems and a sublinear scalar multiplication algorithm. *Progress in Cryptology–Mycrypt*, 3715:171–182, 2005.
- [20] H. Cohen and G. Frey, editors. *Handbook of Elliptic and Hyperelliptic Curve Cryptography*. Chapman & Hall/CRC, 2006.
- [21] H. Cohn. *Advanced Number Theory*. Dover Books on Mathematics. Dover Publications, 1980.
- [22] T.H. Cormen, C. Stein, R.L. Rivest, and C.E. Leiserson. *Introduction to Algorithms*. McGraw-Hill Higher Education, 2nd edition, 2001.
- [23] R.E. Crandall and C. Pomerance. *Prime Numbers: A Computational Perspective*. Springer, 2001.
- [24] V. Dimitrov and T. Cooklev. Hybrid algorithm for the computation of the matrix polynomial $I + A + \dots + A^{N-1}$. *IEEE Transactions on Circuits and Systems – I: Fundamental Theory and Applications*, 42(7):377–380, July 1995.

- [25] V. Dimitrov and T. Cooklev. Two algorithms for modular exponentiation using nonstandard arithmetics. *IEICE Transactions on Fundamentals of Electronics*, E78-A(1):82–87, January 1995.
- [26] V. Dimitrov, L. Imbert, and P.K. Mishra. Fast elliptic curve point multiplication using double-base chains. *IACR Cryptology ePrint Archive*, 2005:69, 2005.
- [27] V. Dimitrov, K.U. Järvinen, M.J. Jacobson, Jr., W.F. Chan, and Z. Huang. Provably sublinear point multiplication on Koblitz curves and its hardware implementation. *IEEE Transaction on Computers*, 57(11):1469–1481, November 2008.
- [28] C. Doche and L. Habsieger. A tree-based approach for computing double-base chains. In *ACISP '08 Proceedings of the 13th Australasian conference on Information Security and Privacy*, pages 433–446, 2008.
- [29] C. Doche and L. Imbert. Extended double-base number system with applications to elliptic curve cryptography. In *INDOCRYPT*, pages 335–348, 2006.
- [30] A. Fröhlich and M. Taylor. *Algebraic Number Theory (Cambridge Studies in Advanced Mathematics)*, volume 27. Cambridge University Press, 1993.
- [31] C.F. Gauß. *Disquisitiones Arithmeticae*. Springer Verlag, 1986.
- [32] J.E. Gower and S.S. Wagstaff Jr. Square form factorization. *Mathematics of Computation*, 77(261):551–588, 2008.
- [33] T. Granlund and P.L. Montgomery. Division by invariant integers using multiplication. In *Proceedings of the SIGPLAN '94 Conference on Programming Language Design and Implementation*, pages 61–72, 1994.
- [34] L.K. Hua and P. Shiu. *Introduction to Number Theory*. Springer London, Limited, 2012.

- [35] L. Imbert, M.J. Jacobson, Jr., and A. Schmidt. Fast ideal cubing in imaginary quadratic number and function fields. *Advanced in Mathematics of Communications*, 4(2):237–260, 2010.
- [36] L. Imbert and F. Philippe. Strictly chained (p,q) -ary partitions. *Contributions to Discrete Mathematics*, 5(2):119–136, 2010.
- [37] K. Ireland and M.I. Rosen. *A Classical Introduction to Modern Number Theory*. Graduate Texts in Mathematics. Springer, 1990.
- [38] M.J. Jacobson, Jr. *Subexponential Class Group Computation in Quadratic Orders*. PhD thesis, Technische Universitt Darmstadt, Darmstadt, Germany, 1999.
- [39] M.J. Jacobson, Jr., R.E. Sawilla, and H.C. Williams. Efficient ideal reduction in quadratic fields. *International Journal of Mathematics and Computer Science*, 1:83–116, 2006.
- [40] M.J. Jacobson, Jr. and H.C. Williams. *Solving the Pell Equation*. CMS books in mathematics. Springer, 2009.
- [41] Tudor Jebelean. Improving the multiprecision Euclidean algorithm. In A. Miola, editor, *Design and Implementation of Symbolic Computation Systems*, volume 722 of *Lecture Notes in Computer Science*, pages 45–58. Springer Berlin Heidelberg, 1993.
- [42] D.E. Knuth. *The Art of Computer Programming, volume 3: (2nd ed.) Sorting and Searching*. Addison Wesley Longman Publishing Co., Inc., Redwood City, CA, USA, 1998.
- [43] A. Kuchling. Python’s dictionary implementation: Being all things to all people. In Andy Oram and Greg Wilson, editors, *Beautiful Code: Leading Programmers Explain How They Think*, pages 293–318. O’Reilly Media, 2008.

- [44] D.H. Lehmer. Euclid’s algorithm for large numbers. *American Mathematical Monthly*, 45(4):227–233, April 1938.
- [45] H.W. Lenstra, Jr. and C. Pomerance. A rigorous time bound for factoring integers. *Journal of the American Mathematical Society*, 5(3):483–516, July 1992.
- [46] N. Méloni and M.A. Hasan. Elliptic curve scalar multiplication combining yao’s algorithm and double bases. In *Cryptographic Hardware and Embedded Systems - CHES 2009, 11th International Workshop, Lausanne, Switzerland, September 6-9, 2009, Proceedings*, volume 5747 of *Lecture Notes in Computer Science*, pages 304–316. Springer, 2009.
- [47] N. Möller and T. Granlund. Improved division by invariant integers. *IEEE Transactions on Computers*, 60(2):165–175, 2011.
- [48] R.A. Mollin. *Algebraic Number Theory*. Chapman & Hall CRC, New York, 1999.
- [49] J. M. Pollard. A Monte Carlo method for factorization. *BIT Numerical Mathematics*, 15:331–334, 1975.
- [50] S. Ramachandran. Numerical results on class groups of imaginary quadratic fields. Master’s thesis, University of Calgary, Canada, 2006.
- [51] G.W. Reitwiesner. Binary arithmetic. *Advances in Computers*, 1:231–308, 1960.
- [52] R. Sawilla. Fast ideal arithmetic in quadratic fields. Master’s thesis, University of Calgary, Canada, 2004.
- [53] C.P. Schnorr and H.W. Lenstra, Jr. A Monte Carlo factoring algorithm with linear storage. *Mathematics of Computation*, 43(167):289–311, July 1984.
- [54] J. Shallit and J. Sorenson. Analysis of a left-shift binary GCD algorithm. *Journal of Symbolic Computation*, 17:473–486, 1994.

- [55] D. Shanks. Class number, A theory of factorization and genera. In *Proceedings of Symposia in Pure Mathematics*, volume 20, pages 415–440, Providence, R.I., 1971. AMS.
- [56] D. Shanks. On Gauss and composition I, II. *Proc. NATO ASI on Number Theory and Applications*, pages 163–179, 1989.
- [57] J. Stein. Computational problems associated with Racah algebra. *Journal of Computational Physics*, 1(3):397–405, February 1967.
- [58] A.V. Sutherland. *Order computations in generic groups*. PhD thesis, M.I.T., 2007.
- [59] A.V. Sutherland. A generic approach to searching for Jacobians. *Mathematics of Computation*, 78(265):485–507, January 2009.
- [60] H.S. Warren. *Hacker’s Delight*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 2002.